

Rust

IN ACTION

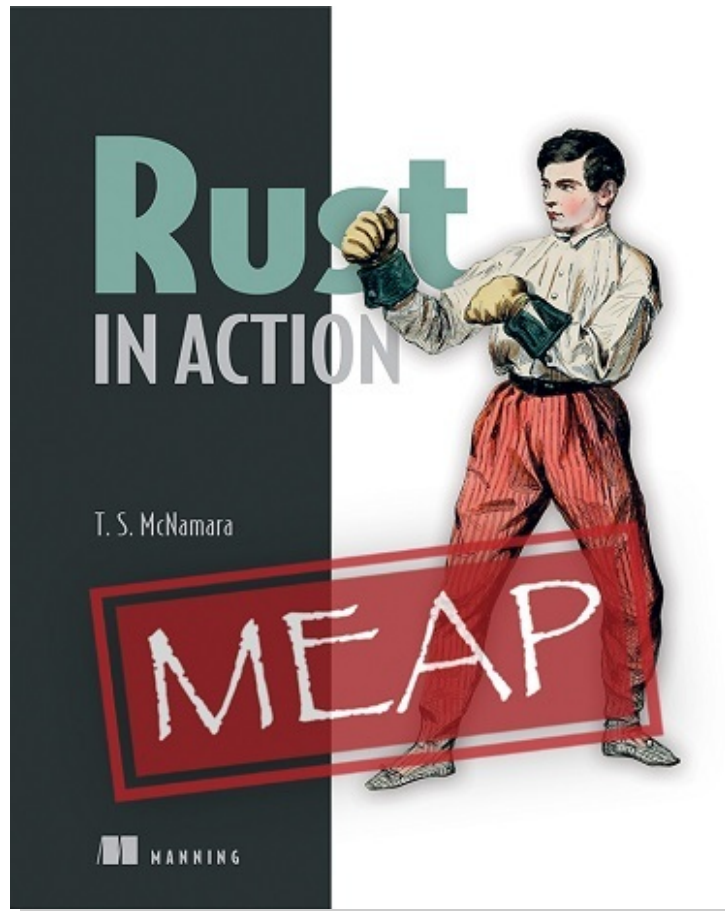
T. S. McNamara

MEAP



MANNING





MEAP VERSION 16

 MANNING PUBLICATIONS

ABOUT THIS MEAP

You can download the most up-to-date version of your electronic books from your Manning Account at account.manning.com. For customer support write to support@manning.com.

© Manning Publications Co. We welcome reader comments about anything in the manuscript - other than typos and other simple mistakes. These will be cleaned up during production of the book by copyeditors and proofreaders.

<https://livebook.manning.com/#!/book/rust-in-action/discussion>

WELCOME

Dear Reader,

Thanks for taking a chance and buying this early release book on the Rust programming language and the internals of computer systems. I hope that you'll be rewarded with a fun, informative read!

Part 1: Rust Language Distinctives will provide a quick-fire introduction to the Rust language by working through projects that begin to introduce concepts that are expanded upon later in the book, such as implementing a File API.

Part 2: Systems Programming from the Ground Up (Almost) will shift your focus towards the computer. You will learn how data is represented at various levels between the application, operating system and hardware. Rust language features will be introduced as required.

A note about Chapter 6: This chapter should be considered a work in progress. It covers lots of ground. Readers progress from learning what a pointer is to benchmarking memory allocations with system utilities for Linux. Some of the explanations are terse... perhaps too terse.

We believe that the content is interesting and relevant to professional programmers, but are considering how it should be expanded and refactored to be most relevant to the book's readers. The author and the editorial team are very interested in your views about what works and what needs more work.

Part 3: Concurrent and Parallel Programming will walk through the tools that computers offer to get work done in parallel. Its primary goal will be to explain how multiple threads can cooperate and coordinate. A large example—building a graph that can be distributed in multiple ways—will be extended throughout several chapters.

Finally, Part 4: Extending Applications Up and Out will show Rust being used to build up software components to extend large software projects. These projects demonstrate systems programming “with the system” rather than systems programming “of the system”. Part 4 will deal with the complexities of handling multiple CPU/OS pairs from the same code base.

As a MEAP reader, you have a unique opportunity to contribute to the book's development. All feedback is gratefully received, but please remember that there will be a human receiving the message. The best channel is the [Manning liveBook's Discussion Forum](#). I'm also active on [Twitter \(@timClicks\)](#), [Reddit \(/u/timclicks\)](#) and the Rust Discourse forum (<https://users.rust-lang.org/u/timclicks>).

Thank you once again for choosing to purchase the book. It's a privilege to ride a short way with you along your computing journey.

—Tim McNamara



BRIEF TABLE OF CONTENTS

1 [Introducing Rust](#)

Part 1: Rust Language Distinctives

2 [Language Foundations](#)

3 [Compound Data Types](#)

4 [Lifetimes, Ownership and Borrowing](#)

Part 2: Demystifying Systems Programming

5 [Data in Depth](#)

6 [Memory](#)

7 [Files & Storage](#)

8 [Networking](#)

9 [Time and Time Keeping](#)

10 [Processes, Threads and Containers](#)

11 [Kernel](#)

12 [Signals, Interrupts and Exceptions](#)

1 INTRODUCING RUST

This chapter covers:

- Introducing Rust’s features and goals
- Exposing Rust’s syntax
- Discussing where to use Rust and when to avoid it
- Building your first Rust program
- Explaining how Rust compares to object-oriented and wider languages

Welcome to Rust—the empowering programming language. Once you scratch its surface, you will not only find a programming language with unparalleled speed and safety, but one that is still comfortable to use. Your investment in learning Rust will be rewarded!

When you begin to program in Rust, it’s likely that you will want to continue to do so. And this book, *Rust in Action*, will build your confidence as a Rust programmer. But, it will not teach you how to program from the beginning. This book is intended to be read by people who are considering Rust as their next language, and for those who enjoy implementing practical working examples. Here is a list of some of the larger examples this book covers:

- Mandelbrot set renderer
- A grep clone
- CPU emulator
- Generative art
- A database
- HTTP, NTP, and hexdump clients
- LOGO language interpreter
- Operating system kernel

As you may gather from scanning through that list, reading this book will teach you more than Rust. It also introduces you to *systems programming* and *low-level programming*. Learning some of the concepts from these areas will benefit your Rust programming journey because much of the Rust community assumes that you already have that knowledge. As you work through *Rust in Action*, you'll learn about the role of an operating system (OS), how a CPU works, how computers keep time, what pointers are, and what a data type is. You will gain an understanding of how the computer's internal systems interoperate. Learning more than syntax, you will also see why Rust was created and the challenges that it addresses.

1.1 WHERE IS RUST USED?

Rust has won the “most loved programming language” award in Stack Overflow's annual developer survey in 2016, 2017, 2018, 2019, and 2020. Perhaps that's why large technology leaders such as the following have adopted Rust:

- Amazon Web Services (AWS) runs its serverless computing offerings, AWS Lambda and AWS Fargate, with Rust. [\[1\]](#)
- Cloudflare develops many of its services, including its public DNS, serverless computing, and packet inspection offerings with Rust. [\[2\]](#)
- Dropbox rebuilt its backend warehouse, which manages exabytes of storage, with Rust. [\[3\]](#)
- Google develops portions of the Chrome OS and Fuchsia operating systems in Rust. [\[4\]](#)
- Facebook uses Rust to power Facebook's web, mobile, and API services, as well as parts of HHVM, the HipHop virtual machine used by the Hack programming language. [\[5\]](#)
- Microsoft writes components of its Azure platform including a

- security daemon for its Internet of Things (IoT) service in Rust. [6]
- Mozilla uses Rust to enhance the Firefox web browser, which contains 15 million lines of code. Mozilla's first two Rust-in-Firefox projects, its MP4 metadata parser and text encoder/decoder, led to overall performance and stability improvements.
 - GitHub's npm, Inc., uses Rust to deliver "upwards of 1.3 billion package downloads per day." [7]
 - Oracle developed a container run time with Rust to overcome problems with the Go reference implementation. [8]
 - Samsung, via its subsidiary SmartThings, uses Rust in its "Hub," which is the firmware backend for its Internet of Things (IoT) service.

Rust is also productive enough for fast-moving startups to deploy it. Here's a few examples:

- Sourcegraph uses Rust to serve syntax highlighting across all of its languages. [9]
- Figma employs Rust in the performance-critical components of its multi-player server. [10]
- Parity develops its client to the Ethereum blockchain with Rust. [11]

As well, Rust supports WebAssembly (Wasm), a standard for deploying apps as a first-class citizen into browsers without JavaScript. Wasm allows you to compile and deploy your Rust project to a server, IoT devices, mobile devices, and a browser.

[1] See "Firecracker: Secure and fast microVMs for serverless computing," <https://firecracker-microvm.github.io/>

[2] See “Rust at Cloudflare,” <https://news.ycombinator.com/item?id=17077358>

[3] See “The Epic Story of Dropbox’s Exodus From the Amazon Cloud Empire,” <https://www.wired.com/2016/03/epic-story-dropboxs-exodus-amazon-cloud-empire/>

[4] See “crosvm—The Chrome OS Virtual Machine Monitor,” <https://chromium.googlesource.com/chromiumos/platform/crosvm/> and “Fuchsia Rust Crates,” <https://fuchsia.googlesource.com/fuchsia/+/master/docs/development/languages/rust/crates.md>

[5] See “HHVM 4.20.0 and 4.20.1,” <https://hhvm.com/blog/2019/08/27/hhvm-4.20.0.html>

[6] See <https://github.com/Azure/iotedge/tree/master/edgelet>

[7] See “Rust Case Study: Community makes Rust an easy choice for npm,” <https://www.rust-lang.org/static/pdfs/Rust-npm-Whitepaper.pdf>

[8] See “Building a Container Runtime in Rust,” <https://blogs.oracle.com/developers/building-a-container-runtime-in-rust>

[9] See “HTTP code syntax highlighting server written in Rust,” https://github.com/sourcegraph/syntect_server

[10] See “Rust in Production at Figma,” <https://www.figma.com/blog/rust-in-production-at-figma/>

[11] See “The fast, light, and robust EVM and WASM client,” <https://github.com/paritytech/parity-ethereum>

1.2 ADVOCATING FOR RUST AT WORK

What is it like to advocate for Rust at work? This 2017 snippet discusses advocating for Rust at Google within the Chrome OS:

Excerpt from a Hacker News comment thread [\[12\]](#)

indy on Sept 27, 2017 Is Rust an officially sanctioned language at Google?

zaxcellent on Sept 27, 2017

Author here: Rust is not officially sanctioned at Google, but there are pockets of folks using it here. The trick with using Rust in this component was convincing my coworkers that no other language was right for job, which I believe to be the case in this instance.

That being said, there was a ton of work getting Rust to play nice within the Chrome OS build environment. The Rust folks have been super helpful in answering my questions though.

ekidd on Sept 27, 2017

> The trick with using Rust in this component was convincing my coworkers that no other language was right for job, which I believe to be the case in this instance.

I ran into a similar use case in one of my own projects—a vobsub subtitle decoder, which parses complicated binary data, and which I someday want to run as web service. So obviously, I want to ensure that there are no vulnerabilities in my code.

I wrote the code in Rust, and then I used 'cargo fuzz' to try and find vulnerabilities. After running a billion(!) fuzz iterations, I found 5 bugs (see the 'vobsub' section of the trophy case for a list <https://github.com/rust-fuzz/trophy-case>).

Happily, not one of those bugs could actually be escalated into an actual exploit. In each case, Rust's various runtime checks successfully caught the problem and turned it into a controlled panic. (In practice, this would restart the web server cleanly.)

So my takeaway from this was that whenever I want a language (1) with no GC, but (2) which I can trust in a security-critical context, Rust is an excellent choice. The fact that I can statically link Linux binaries (like with Go) is a nice plus.

From this excerpt, we can see that language adoption has been “bottom up” by engineers looking to overcome technical challenges. Skepticism with using Rust for internal side-projects from colleagues has been overcome, thus gaining in

first-hand experience and measurable performance data. And in the time since late 2017, Rust has continued to mature and strengthen. It has become an accepted part of Google’s technology landscape; it is now an officially sanctioned language within the Fuchsia OS project.

[12] See “Chrome OS KVM—A component written in Rust,” <https://news.ycombinator.com/item?id=15346557>

1.3 A TASTE OF THE LANGUAGE

This section gives you a chance to experience Rust firsthand. We start by understanding how to use the compiler and then move on to writing a quick program next and full projects in later chapters.



Note

To install Rust, use the official installers provided at <https://rustup.rs/>.

1.3.1 Cheating your way to “Hello, world!”

The first thing that most programmers do when they reach for a new programming language is to learn how to print “Hello, world!” to the console. You’ll do that too, but with flair. You’ll verify that everything is in working order *before* you encounter annoying syntax errors.

If you use Windows, open the Rust command prompt that is available in the Start menu after installing Rust. Then execute this command:

```
C:\> cd %TMP%
```

If you are running Linux or macOS, please open a Terminal window. Once open, please enter the following:

```
$ cd $TMP
```

From this point forward, the commands for all operating systems should be the same. If you installed Rust correctly, the following three commands will display “Hello, world!” on the screen (as well as a bunch of other output):

```
$ cargo new hello
$ cd hello
$ cargo run
```

Here is an example of what the entire code looks like for Windows:

```
C:\> cd %TMP%

C:\Users\Tim\AppData\Local\Temp> cargo new hello
    Created binary (application) `hello` project

C:\Users\Tim\AppData\Local\Temp> cd hello

C:\Users\Tim\AppData\Local\Temp\hello> cargo run
    Compiling hello v0.1.0 (file:///C:/Users/Tim/AppData/Local/Temp/hello)
    Finished dev [unoptimized + debuginfo] target(s) in 0.32s
    Running `target\debug\hello.exe`
Hello, world!
```

And on Linux, the code would look like this:

```
$ cd $TMP

$ cargo new hello
    Created binary (application) `hello` package

$ cd hello

$ cargo run
    Compiling hello v0.1.0 (/home/tsm/hello)
    Finished dev [unoptimized + debuginfo] target(s) in 0.26s
    Running `target/debug/hello`
Hello, world!
```

If you have reached this far, fantastic! You have run your first Rust code without needing to write any Rust. Let’s take a look at what’s just happened.

cargo is a tool that provides both a build system and a package manager. That is, cargo knows how to execute rustc (the Rust compiler) to convert your Rust code into executable binaries or shared libraries. cargo new creates a project for you that follows a standard template. Your directory structure will have been changed

to something that looks like this:

```
$ tree hello
hello
├── Cargo.toml
└── src
    └── main.rs

1 directory, 2 files
```

All Rust crates begin with the same structure. In the base directory, a file called `Cargo.toml` describes the project’s metadata, such as the project’s name, its version, and its dependencies. Source code appears in the `src` directory. Rust source code files use the `.rs` filename extension.

The next command that you executed was `cargo run`. This line is much simpler to grasp, but `cargo` actually did much more work than you realized. You asked `cargo` to run the project. As there was nothing to actually run when you invoked the command, it decided to compile the code in debug mode on your behalf to provide maximal error information. As it happens, the `src/main.rs` file always includes a “Hello, world!” stub. The result of that compilation was a file called `hello` (or `hello.exe`). The `hello` file was executed, and the result printed to your screen.

For the curious, our project’s directory structure has changed a great deal. We now have a `Cargo.lock` file in the base of our project and a `target/` directory. Both that file and the directory are managed by `cargo`. Because these are artifacts of the compilation process, we won’t need to touch these. `Cargo.lock` is a file that specifies the exact version numbers of all the dependencies so that future builds are reliably built the same way until `Cargo.toml` is modified.

```
$ tree --dirsfirst hello
hello
├── src
│   └── main.rs
├── target
│   ├── build
│   ├── deps
│   ├── examples
│   ├── native
│   └── hello
```

```
|— Cargo.lock
|— Cargo.toml
```

For getting things up and running, well done! Now that we’ve cheated our way to “Hello, World!”, let’s get there via the long way.

1.3.2 Your first Rust program

For our first program, we want to write something that outputs the following text:

```
Hello, world!
Grüß Gott!
```

You have probably seen the first line in your travels. The other two are there to highlight few of Rust’s features: easy iteration and built-in support for Unicode.

For this program, we will use cargo as in the previous section. To start, open a console window and move into a temporary directory (`cd /tmp/` or `cd %TMP%` on Windows). After that, type the following at the command prompt.

```
$ cargo new hello2
$ cd hello2
```

Now open the file `hello2/src/main.rs` in a text editor. Replace the text in that file with the text in the following listing. You’ll find the source code for this listing in the file `ch1/ch1-hello2.rs` (see section 1.4 for access to this book’s source files).

Listing 1.1. “Hello World!” in three languages

```
fn greet_world() {
    println!("Hello, world!");    ❶

    let southern_germany = "Grüß Gott!";    ❷
    let japan = "";    ❸

    let regions = [southern_germany, japan];    ❹

    for region in regions.iter() {    ❺
```

```

        println!("{}", &region);           ⑥
    }
}

fn main() {
    greet_world();                          ⑦
}

```

- ① The exclamation mark indicates the use of a macro, which we'll discuss shortly.
- ② Assignment in Rust, more properly called variable binding, uses the `let` keyword.
- ③ Unicode support is provided out of the box.
- ④ Array literals use square brackets.
- ⑤ Many types can have an `iter()` method to return an iterator.
- ⑥ The ampersand (`&`) borrows the `region` value.
- ⑦ Calls a function. Note that parentheses follow the function name.

Now that our code is updated, execute `cargo run` from the `hello2/` directory. You should see three greetings. Let's take a few moments to touch on some of the elements in listing 1.1.

One of the first things that you are likely to notice is that strings in Rust are able to include a wide range of characters. Strings are UTF-8. This means that you can use non-English languages with relative ease.

The one character that might look out of place is the exclamation mark after `println`. If you have programmed in Ruby, you may be used to thinking that it is used to signal a destructive operation. In Rust, it signals the use of a *macro*. Macros can be thought of as sort of fancy functions for now. These offer the ability to avoid boilerplate code. In the case of `println!`, there is a lot of type detection going on under the hood so that arbitrary data types can be printed to the screen.

1.4 DOWNLOADING THE BOOK'S SOURCE CODE

In order to follow along with the examples in this book, you might want to access the source code for the listings. For your convenience, source code for every example is available from two sources:

- <https://manning.com/books/rust-in-action>
- <https://github.com/rust-in-action/code>

1.5 WHAT DOES RUST LOOK AND FEEL LIKE?

Rust is the programming language that allows Haskell and Java programmers to get along. Rust comes close to the high-level, expressive feel of dynamic languages like Haskell and Java while achieving bare-metal performance. We looked at a “Hello, world!” example in section 1.3, so let’s try something slightly more complex to get a better feel for Rust’s features. [Listing 1.2](#) provides a quick look at what Rust can do for basic text processing. Some features to notice include

- *Common control flow mechanisms*—This includes `for` loops and the `continue` keyword.
- *Method syntax*—Although Rust is not object-oriented as it does not support inheritance, it carries over this feature of object-oriented languages.
- *Higher-order programming*—Functions can both accept and return functions. For example, line 19 includes a closure, also known as an anonymous function or λ (lambda) function.
- *Type annotations*—Although relatively rare, these are occasionally required as a hint to the compiler (for example, line 28).
- *Conditional compilation*—In the listing, lines 22-24 are not included in release builds of the program.
- *Implicit return*—Rust provides a `return` keyword, but it’s usually omitted. Rust is an *expression-based language*.

Note

The source code for this listing is in the `ch1/ch1-penguins/src/main.rs` file.

Listing 1.2. Example of Rust code showing some basic processing of CSV data

```
fn main() {                                ①
    let penguin_data = "\
common name,length (cm)
Little penguin,33
Yellow-eyed penguin,65
Fiordland penguin,60
Invalid,data
";

    let records = penguin_data.lines();

    for (i, record) in records.enumerate() {
        if i == 0 || record.trim().len() == 0 { ②
            continue;
        }

        let fields: Vec<_> = record              ③
            .split(',')                          ④
            .map(|field| field.trim())           ⑤
            .collect();                          ⑥

        if cfg!(debug_assertions) {              ⑦
            // following line should not exceed 55 chars; please add a line break
            // and move annotation marker as needed
            eprintln!("debug: {:?} -> {:?}", record, fields); ⑧
        }

        let name = fields[0];                    ⑨

        // following line should not exceed 55 chars; please add a line break
        // and move annotation marker as needed
        let maybe_length: Result<f32, _> = fields[1].parse(); ⑩

        if maybe_length.is_err() {                ⑪
            continue;
        }

        let length = maybe_length.unwrap();       ⑫

        println!("{}", {}, {}cm", name, length); ⑬
    }
}
```

- ① Executable projects require a `main()` function.
- ② Skips header row and lines with only whitespace
- ③ A `Vec` type is shorthand for vector. Vectors are arrays that dynamically expand when needed. The underscore asks the compiler to infer the vector's element type.

- ④ Splits the record into substrings
- ⑤ For loops, you can use higher-order functions. This line trims the whitespace from every field.
- ⑥ “Collects” the results of an iterator into a vector
- ⑦ For debugging, include this code block. The ! indicates a macro invocation. Macros, similar to functions, return code rather than values and are often used to simplify common patterns.
- ⑧ Prints to standard error. The {:?} syntax requests the default debugging representation for the two types as output.
- ⑨ Rust supports indexing collections with integers.
- ⑩ Parses strings into other types, using the type information provided on the left side. This either returns a value or an error value wrapped in a Result. The underscore requests the compiler to infer the error type itself.
- ⑪ Skips any invalid data
- ⑫ Unwraps the f32 value from the Result
- ⑬ Prints to stdout. The {} syntax indicates that Rust should use a programmer-defined method to represent the value as a string, rather than its debug representation available with {:?}.

Converting source code into an executable file is called *compiling*. To compile Rust code, we need to install the Rust compiler and run it against the source code. To compile listing 1.2, follow these steps:

1. Open a console prompt (e.g., cmd.exe, PowerShell, Terminal, or Alacritty)
2. Move to the ch1/ch1-penguins directory (not ch1/ch1-penguins/src) of the source code you downloaded in section 1.4
3. Execute `cargo run`

Listing 1.3. Output from listing 1.2 after running `cargo run`

```
$ cargo run
// code lines without annotations are limited to 76 chars in order to fit the
printed page. Following line needs a line break
  Compiling ch1-penguins v0.1.0 (/path/to/rust-in-action/code/ch1/ch1-penguins)
    Finished dev [unoptimized + debuginfo] target(s) in 0.40s
    Running `target/debug/ch1-penguins`
debug: "  Little penguin,33" -> ["Little penguin", "33"]
Little penguin, 33cm
```

```
debug: "  Yellow-eyed penguin,65" -> ["Yellow-eyed penguin", "65"]
Yellow-eyed penguin, 65cm
debug: "  Fiordland penguin,60" -> ["Fiordland penguin", "60"]
Fiordland penguin, 60cm
debug: "  Invalid,data" -> ["Invalid", "data"]
```

You probably noticed the distracting lines starting with `debug:`. We can eliminate these by compiling a release build using cargo's `--release` flag. This conditional compilation functionality is provided by the `cfg!` (debug_assertions) { ... } block within lines 22-24 of the following listing.

Listing 1.4. Output from creating a release build of listing 1.2

```
$ cargo run --release
  Compiling ch1-penguins v0.1.0 (/path/to/code/ch1/ch1-penguins)
  Finished release [optimized] target(s) in 0.34s
  Running `target/release/ch1-penguins`
Little penguin, 33cm
Yellow-eyed penguin, 65cm
Fiordland penguin, 60cm
```

It's possible to further reduce the output by adding the `-q` flag (`-q` is shorthand for “quiet”). The following snippet shows the syntax:

```
$ cargo run -q --release
Little penguin, 33cm
Yellow-eyed penguin, 65cm
Fiordland penguin, 60cm
```

Should I use cargo or rustc to compile code?

In a strict sense, `rustc` is the Rust compiler. The preceding examples don't use it though. Instead, the examples interact with cargo. cargo provides a simple interface, invoking `rustc` on our behalf. To see what cargo passes to `rustc`, add the `--verbose` flag:

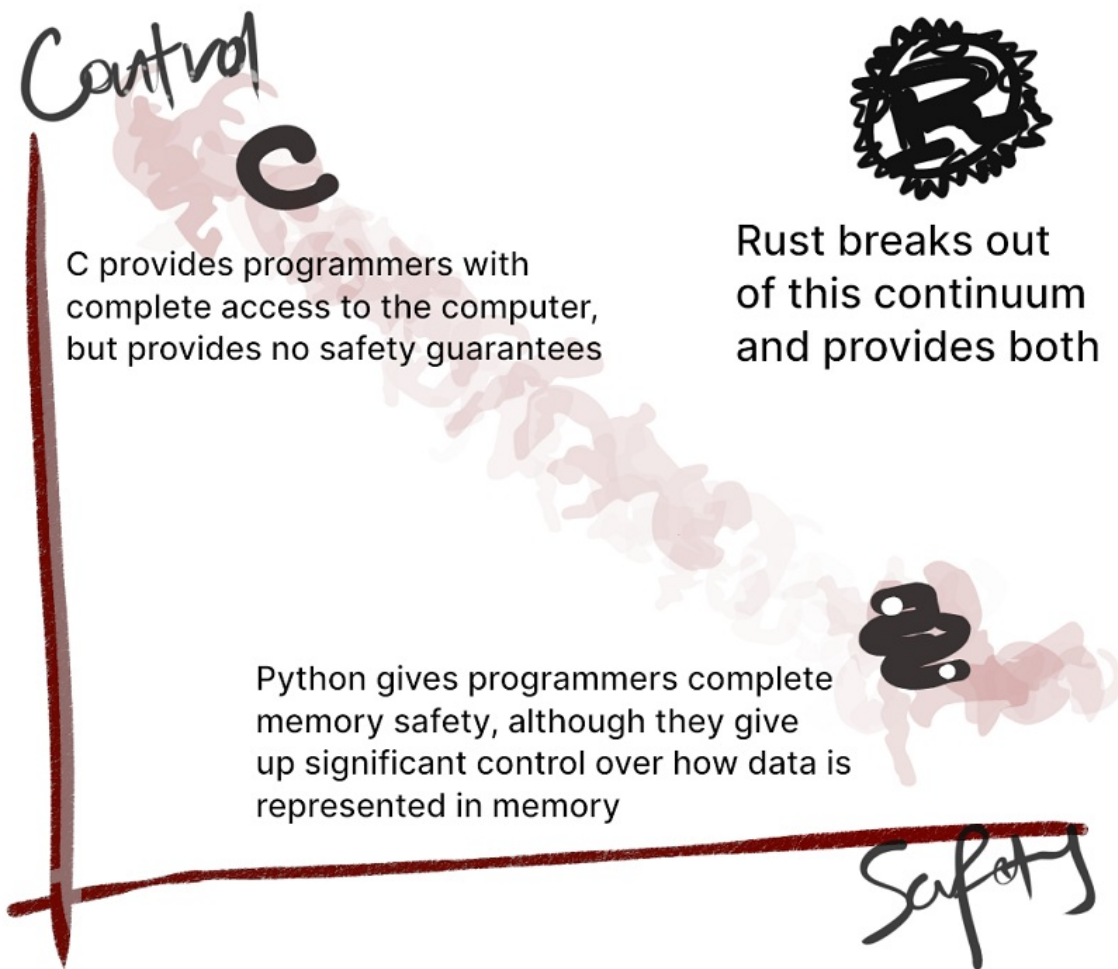
```
$ cd code/ch1/ch1-penguins
$ cargo run --verbose
  Compiling ch1-penguins v0.1.0 (rust-in-action/code/ch1/ch1-penguins)
// line breaks needed in following; 76 max characters including spaces per
each line
  Running `rustc --crate-name ch1_penguins --edition=2018 src/main.rs
--error-format=json --json=diagnostic-rendered-ansi --crate-type bin
--emit=dep-info,link -Cbitcode-in-rlib=no -C debuginfo=2 -C
metadata=390c3b75f851c687 -C extra-filename=-390c3b75f851c687
--out-dir rust-in-action/code/ch1/ch1-penguins/target/debug/deps`
```

```
-C incremental=rust-in-action/code/ch1/ch1-penguins/target/debug
/incremental
-L dependency=rust-in-action/code/ch1/ch1-penguins/target/debug/deps`
Finished dev [unoptimized + debuginfo] target(s) in 0.43s
Running `target/debug/ch1-penguins`
debug: " Little penguin,33" -> ["Little penguin", "33"]
Little penguin, 33cm
debug: " Yellow-eyed penguin,65" -> ["Yellow-eyed penguin", "65"]
Yellow-eyed penguin, 65cm
debug: " Fiordland penguin,60" -> ["Fiordland penguin", "60"]
Fiordland penguin, 60cm
debug: " Invalid,data" -> ["Invalid", "data"]
```

[Listing 1.1](#) and listing 1.2 were chosen to pack as many representative features of Rust into the examples as possible. Hopefully these demonstrated that Rust programs have a high-level feel, paired with a low-level performance. Let's take a step back from specific language features now and consider some of the thinking behind the language and where it fits within the programming language ecosystem.

1.6 WHAT IS RUST?

Rust's distinguishing feature as a programming language is its ability to prevent invalid data access at *compile time*. Research projects by Microsoft's Security Response Center and the Chromium browser project both suggest that issues relating to invalid data access account for approximately 70% of serious security bugs. [\[13\]](#) Rust eliminates that class of bugs. It guarantees that your program is *memory safe* without imposing any *run time* costs. Other languages can provide this level of safety, but these require adding checks that execute while your program is running, thus slowing it down.



Rust’s distinguishing feature—as the professional community acknowledges—is its willingness to explicitly include values into its decision-making process. This ethos of inclusion is pervasive. Public messaging is welcoming. All interactions within the Rust community are governed by its code of conduct. Even the Rust compiler’s error messages are ridiculously helpful.

Until late 2018, visitors to the Rust homepage were greeted with the (technically heavy) message, “Rust is a systems programming language that runs blazingly fast, prevents segfaults and guarantees thread safety.” At that point, the community implemented a change to its wording to put its users (and its potential users) at the center (table 1.1).

Table 1.1. Rust slogans over time. As Rust has developed its confidence, it has increasingly embraced the idea of acting as a facilitator and supporter of everyone wanting to achieve their programming aspirations.

Until late 2018	From that point onward
“Rust is a systems programming language that runs blazingly fast, prevents segfaults and guarantees thread safety.”	“Empowering everyone to build reliable and efficient software.”

Rust is labelled as a *systems programming language*, which tends to be seen as quite a specialized, almost esoteric, branch of programming. However, many Rust programmers have discovered that the language is applicable to many other domains. Safety, productivity, and control are useful in all software engineering projects. Moreover, the Rust community’s inclusiveness means that the language benefits from a steady stream of new voices with diverse interests.

Let’s flesh out those three goals: safety, productivity, and control. What are these and why do these matter?

1.6.1 Goal of Rust: Safety

Rust programs are free from

- *Dangling pointers*—Live references to data that has become invalid over the course of the program (see listing 1.5)
- *Data races*—Inability to determine how a program will behave from run to run because external factors are changing (see listing 1.6)
- *Buffer overflow*—Attempt to access the 12th element of an array with only 6 elements (see listing 1.7)
- *Iterator invalidation*—An issue caused by something that is iterated over after being altered mid-way through (see listing 1.8)

When programs are compiled in debug mode, Rust also protects against integer

overflow. What is integer overflow? Well, integers can only represent a finite set of numbers; these have a fixed-width in memory. Integer overflow is what happens when the integers hit their limit and flow over to the beginning again.

The following listing shows a dangling pointer. Note that you'll find this source code in the `ch1/ch1-cereals/src/main.rs` file.

Listing 1.5. Attempting to create a dangling pointer

```
#[derive(Debug)]    ①
enum Cereal {        ②
    Barley, Millet, Rice,
    Rye, Spelt, Wheat,
}

fn main() {
    let mut grains: Vec<Cereal> = vec![];    ③
    grains.push(Cereal::Rye);                ④
    drop(grains);                            ⑤

    println!("{:?}", grains);                ⑥
}
```

- ① Allows the `println!` macro to print the `Cereal` enum
- ② An enum (enumeration) is a type with a fixed number of legal variants.
- ③ Initializes an empty vector of `Cereal`
- ④ Adds one item to the `grains` vector
- ⑤ Deletes `grains` and its contents
- ⑥ Attempts to access the deleted value

[Listing 1.5](#) contains a pointer within `grains`. In the listing, `Vec<Cereal>` is implemented with an internal pointer to an underlying array. But the listing does not compile. An attempt to do so triggers an error message that complains about attempting to “borrow” a “moved” value. Learning how to interpret that error message and to fix the underlying error are topics for the pages to come. Here’s the compiled code:

```
$ cargo run
  Compiling ch1-cereals v0.1.0 (/rust-in-action/code/ch1/ch1-cereals)
error[E0382]: borrow of moved value: `grains`
  --> src/main.rs:12:22
```

```

8 |     let mut grains: Vec<Cereal> = vec![];
// following line needs a line break (76 char max)
|     ----- move occurs because `grains` has type `std::vec::Vec<Cereal>`,
|     which does not implement the `Copy` trait
9 |     grains.push(Cereal::Rye);
10 |     drop(grains);
|     ----- value moved here
11 |
12 |     println!("{:?}", grains);
|                               ^^^^^^^ value borrowed here after move

error: aborting due to previous error

For more information about this error, try `rustc --explain E0382`.
error: could not compile `ch1-cereals`.

```

[Listing 1.6](#) shows an example of a data race condition. If you remember, this condition results from the inability to determine how a program behaves from run to run due to external factors changing. You’ll find this code in the `ch1/ch1-race/src/main.rs` file.

Listing 1.6. Example of how Rust prevents a data race condition

```

use std::thread;                                ①

fn main() {
    let mut data = 100;

    thread::spawn(|| { data = 500; });           ②
    thread::spawn(|| { data = 1000; });          ②

    println!("{}", data);
}

```

① Brings the standard library’s threading implementation into local scope.

② `thread::spawn()` takes a closure as an argument.

If you are unfamiliar with the term “thread,” the upshot is that this code is not deterministic. It’s impossible to know what data will be like when `main()` exits. In the listing, the closure for `thread::spawn()` is denoted by vertical bars and curly braces (e.g., `|| { ... }`), limiting the returned data calls to 500 and 1,000, respectively.

Attempting to compile listing 1.6 results in a large stampede of error messages.

Rust does not allow multiple places in an application to have write access to data. [Listing 1.6](#) attempts to allow this in three places: once within the main thread running `main()` and once in each child thread created by `thread::spawn()`. Here's the compiler message:

```
$ cargo run
  Compiling ch1-race v0.1.0 (rust-in-action/code/ch1/ch1-race)
// add line break to following (76 char max)
error[E0373]: closure may outlive the current function, but it borrows `data`,
which is owned by the current function
--> src/main.rs:6:19
|
6 |     thread::spawn(|| { data = 500; });
|                        ^^      ---- `data` is borrowed here
|                        |
|                        may outlive borrowed value `data`
|
note: function requires argument type to outlive `static`
--> src/main.rs:6:5
|
6 |     thread::spawn(|| { data = 500; });
|     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
// add line break to following (76 char max)
help: to force the closure to take ownership of `data` (and any other referenced
variables), use the `move` keyword
|
6 |     thread::spawn(move || { data = 500; });
|                    ^^^^^^^^^
... ①

error: aborting due to 4 previous errors

Some errors have detailed explanations: E0373, E0499, E0502.
For more information about an error, try `rustc --explain E0373`.
error: could not compile `ch1-race`.
```

① Three other errors omitted.

[Listing 1.7](#) provides an example of a buffer overflow (attempting to access the 12th element of an array of only 6 elements, for example). The source code for this listing is in the file `ch1/ch1-fruit/src/main.rs`.

Listing 1.7. Example of invoking a panic via a buffer overflow

```
fn main() {
    let fruit = vec!['', '', ''];
    let buffer_overflow = fruit[4];    ①
```



```
    assert_eq!(buffer_overflow, '')    ②
}
// added extra line spaces above for longish annotations
```

① This code will “panic” rather than assign `buffer_overflow` to an invalid memory location.

② `assert_eq!()` allows assertions, but the two sides of `assert_eq!()` must equal or the program panics.

When listing 1.7 is compiled and executed, you’ll encounter this error message:

```
$ cargo run
  Compiling ch1-fruit v0.1.0 (/rust-in-action/code/ch1/ch1-fruit)
  Finished dev [unoptimized + debuginfo] target(s) in 0.31s
  Running `target/debug/ch1-fruit`
// the following two lines need line breaks (76 char max per line)
thread 'main' panicked at 'index out of bounds: the len is 3 but the index is 4',
src/main.rs:3:25
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

[Listing 1.8](#) shows an example of iterator invalidation, where an issue is caused by something that’s iterated over after being altered mid-way through. The source code for this listing is in `ch1/ch1-letters/src/main.rs`.

Listing 1.8. Attempting to modify an iterator while iterating over it

```
fn main() {
    let mut letters = vec![                ①
        "a", "b", "b"
    ];

    for letter in letters {
        println!("{}", letter);
        letters.push(letter.clone());    ②
    }
}
```

① Initializes a vector and allows mutation (alteration or modification)

② Creates a new copy of letter and appends it to the end of letters

[Listing 1.8](#) fails to compile because Rust will not allow `letters` to be modified within the iteration block. Here’s the error message:

```

$ cargo run
  Compiling ch1-letters v0.1.0 (/rust-in-action/code/ch1/ch1-letters)
error[E0382]: borrow of moved value: `letters`
  --> src/main.rs:8:7
   |
2 |   let mut letters = vec![
// add line break below (76 char max)
   |   ----- move occurs because `letters` has type `std::vec::Vec<&str>`,
   |   which does not implement the `Copy` trait
...
6 |   for letter in letters {
   |               ^^^^^^^
   |               |
   |               value moved here
// add line breaks below (55 char max for line with <1>)
   |               help: consider borrowing to avoid moving into the for loop:
   |   `&letters` ①
7 |       println!("{}", letter);
8 |       letters.push(letter.clone());
   |       ^^^^^^^^ value borrowed here after move

error: aborting due to previous error

For more information about this error, try `rustc --explain E0382`.
error: could not compile `ch1-letters`.

To learn more, run the command again with --verbose.

```

① In this case, the suggestion doesn't help. It recommends a read-only reference (e.g., `&letters`). Using a read/write reference (e.g., `&mut letters`) also fails to compile.

Knowing that a language is safe provides programmers with a degree of liberty. Because they know their program won't implode, they become much more willing to experiment. Within the Rust community, this liberty has spawned the saying "fearless concurrency."

1.6.2 Goal of Rust: Productivity

When given a choice, Rust prefers the option that is easiest for the developer. Many of its more subtle features are productivity boosts. But programmer productivity is a difficult concept to demonstrate with an example in a book. Let's start with something that can snag beginners: using assignment (`=`) within an expression that should use an equality (`==`) test.

```
fn main() {
```

```

let a = 10;

if a = 10 {
    println!("a equals ten");
}

```

In Rust, the preceding code would fail to compile. The Rust compiler generates the following message:

```

error[E0308]: mismatched types
--> src/main.rs:4:8
4 |         if a = 10 {
  |             ^^^^^^
  |             |
  |             expected `bool`, found `()`
  |             help: try comparing for equality: `a == 10`

error: aborting due to previous error

For more information about this error, try `rustc --explain E0308`.
error: could not compile `playground`.

To learn more, run the command again with --verbose.

```

At first, “mismatched types” might feel like a strange error message to encounter. Surely we can test variables for equality against integers.

After some thought, it becomes apparent why the `if` test receives the wrong type. The `if` is not receiving an integer. It’s receiving the result of an assignment. In Rust, this is the blank type: `()`. `()` is pronounced “unit.” [\[14\]](#)

When there is no other meaningful return value, expressions return `()`. As the following shows, adding a second equals sign on line 4 results in a working program that prints `a equals ten`.

```

fn main() {
    let a = 10;

    if a == 10 {          ①
        println!("a equals ten");
    }
}

```

① Using a valid assignment operator (==) allows the program to compile.

Rust has many ergonomic features. It offers generics, sophisticated data types, pattern matching, and closures. [\[15\]](#) Those who have worked with other ahead-of-time compilation languages are likely to appreciate Rust's build system and its comprehensive package manager: cargo. At first glance, we see that cargo is a front-end for rustc, the compiler, but cargo provides these commands for Rust programmers:

- `cargo init` creates a skeleton Rust project.
- `cargo build` downloads dependencies and compiles the code.
- `cargo run` does the same as `cargo build` but also runs the resulting executable file.
- `cargo doc` builds HTML documentation for every dependency in the current project.

1.6.3 Goal of Rust: Control

Control over memory access, memory layout, and specific CPU instructions is important when squeezing the best performance out of code. At times, it is imperative to manage how something is operating. It might matter that data is stored in the *stack* rather than on the *heap*. At times, it might make sense to add *reference counting* to a shared value. Often, it makes sense to pass references to functions. Occasionally, it might be useful to create one's own type of pointer for a particular access pattern. Most of the time though, there are other things to worry about. And for the majority, Rust uses sensible defaults that align with its "zero cost abstractions" philosophy.

Note

If the terms such as stack, heap, and reference counting are new, don't put the book down! We'll spend lots of time explaining these and how they work together throughout the rest of the book.

The following code snippet prints the line a: 10, b: 20, c: 30, d: Mutex {

data: 40 }. Each representation is another way to store an integer. As we progress through the next few chapters, the trade-offs related to each level become apparent. For the moment, the important thing to remember is that the menu is comprehensive. You are welcome to choose exactly what's right for your specific use case.

[Listing 1.9](#) demonstrates multiple mechanisms for creating integer values. Each form provides differing semantics and run-time characteristics. But programmers retain full control of the trade-offs that they want to make.

Listing 1.9. Multiple mechanisms for creating integer values

```
use std::rc::Rc;
use std::sync::{Arc, Mutex};

fn main() {
    let a = 10;                                ①

    let b = Box::new(20);                      ②

    let c = Rc::new(Box::new(30));             ③

    let d = Arc::new(Mutex::new(40));          ④

    println!("a: {:?}, b: {:?}, c: {:?}, d: {:?}", a, b, c, d);
}
```

- ① Integer on the stack
- ② Integer on the heap, also known as a boxed integer
- ③ Boxed integer wrapped within a reference counter
- ④ Integer wrapped in an atomic reference counter and protected by a mutual exclusion lock

To understand why Rust is doing something the way it is, it can be helpful to refer back to these three principles:

- Data within Rust is immutable by default.
- The language's first priority is safety.
- Compile-time checks are strongly preferred. Safety should be a “zero-cost abstraction.”

[13] See the articles “We need a safer systems programming language,” <https://msrc-blog.microsoft.com/2019/07/18/we-need-a-safer-systems-programming-language/> and “Memory safety,” <https://www.chromium.org/Home/chromium-security/memory-safety> for more information.

[14] The name “unit” reveals some of Rust’s heritage as a descendant of the ML family of programming languages. Theoretically, a unit type only has a single value, itself. Compare this with Boolean types that have two values, truth or falsehood.

[15] If these terms are unfamiliar, do keep reading. These are explained throughout the book.

1.7 RUST’S BIG FEATURES

Our tools shape what we believe we can create. Rust enables you to build the software that you want to make, but were too scared to try. So what kind of tool is Rust? Flowing from the three principles discussed in the last section are three overarching features of the language:

- Performance
- Concurrency
- Memory efficiency

1.7.1 Performance

Rust offers you all of your computer’s available performance. Famously, Rust does not rely on a garbage collector to provide its memory safety.

There is, unfortunately, a problem with promising you faster programs: the speed of your CPU is fixed. Thus, for software to run faster, it needs to do less. Yet, the language is large. To resolve this conflict, Rust pushes the burden onto the compiler.

The Rust community prefers a bigger language with a compiler that does more, rather than a simpler language where the compiler does less. The Rust compiler aggressively optimizes both the size and speed of your program. Rust also has some less obvious tricks:

- *Cache-friendly data structures are provided by default.* Arrays usually hold data within Rust programs rather than deeply nested tree structures that are created by pointers. This is referred to as *data-oriented programming*.
- *The availability of a modern package manager (cargo) makes it easy to benefit from the world's smartest programmers.* C and C++ have much less consistency here, and building large projects with many dependencies is typically difficult.
- *Methods are always dispatched statically unless you explicitly request dynamic dispatch.* This enables the compiler to heavily optimize code, sometimes to the point of eliminating the cost of a function call entirely.

1.7.2 Concurrency

Asking a computer to do more than one thing at the same time has proven difficult for software engineers. As far as an operating system is concerned, two independent threads of execution are at liberty to destroy each other if a programmer makes a serious mistake. Yet Rust has spawned the saying “fearless concurrency.” Its emphasis on safety crosses the bounds of independent threads. There is no global interpreter lock (GIL) to constrain a thread's speed. We explore some of the implications of this in part 2.

1.7.3 Memory efficiency

Rust enables you to create programs that require minimal memory. When needed, you can use fixed-size structures and know exactly how every byte is managed. High-level constructs, such as iteration and generic types, incur minimal run-time overhead.

1.8 DOWNSIDES OF RUST

It's easy to talk about this language as if it is the panacea for all software engineering. For example

- “A high-level syntax with low-level performance!”
- “Concurrency without crashes!”
- “C with perfect safety!”

These slogans (sometimes overstated) are great. But for all of its merits, Rust does have some disadvantages.

1.8.1 Cyclic data structures

It is difficult to model cyclic data like an arbitrary graph structure in Rust. Implementing a doubly-linked list is an undergraduate-level computer science problem. Yet Rust’s safety checks do hamper progress here. If you’re learning the language, avoid implementing these sorts of data structures until you’re more familiar with Rust.

1.8.2 Compile times

Rust is slower at compiling code than its peer languages. It has a complex compiler toolchain that receives multiple intermediate representations and sends lots of code to the LLVM compiler. The “unit of compilation” for a Rust program is not an individual file but a whole package (known affectionately as a “crate”). As crates can include multiple modules, these can be very large units to compile. Although this enables whole-of-crate optimization, it requires whole-of-crate compilation as well.

1.8.3 Strictness

It’s impossible—well, difficult—to be lazy when programming with Rust. Programs won’t compile until everything is just right. The compiler is strict, but helpful.

Over time, it’s likely that you’ll come to appreciate this feature. If you’ve ever programmed in a dynamic language, then you may have encountered the frustration of your program crashing because of a misnamed variable. Rust brings that frustration forward so that your users don’t have to experience the

frustration of things crashing.

1.8.4 Size of the language

The language is large! It has a complex type system with multiple ways to gain access to values and an ownership system that is paired with enforced object lifetimes. The following snippet, shown before as listing 1.9, probably causes a degree of anxiety. It's fairly overwhelming.

Listing 1.10. Demonstrating several ways to wrap an integer

```
use std::rc::Rc;
use std::sync::{Arc, Mutex};

fn main() {
    let a = 10;                                ①

    let b = Box::new(20);                      ②

    let c = Rc::new(Box::new(30));             ③

    let d = Arc::new(Mutex::new(40));          ④

    println!("a: {:?}, b: {:?}, c: {:?}, d: {:?}", a, b, c, d);
}
```

- ① Integer on the stack
- ② Integer on the heap, also known as a boxed integer
- ③ Boxed integer wrapped within a reference counter
- ④ Integer wrapped in an atomic reference counter and protected by a mutual exclusion lock

The downside of allowing full control is that programmers have the burden of choice. Exactly which integer type you should choose depends on the problem at hand.

1.8.5 Hype

The Rust community is wary of growing too quickly and being consumed by hype. Yet, a number of software projects have encountered this question in their

Inbox: “Have you considered rewriting this in Rust?” Or they issue a tracker recommending a complete rewrite in a new language.

Software written in Rust is not immune to security problems. By 2015, as Rust gained prominence, implementations of SSL/TLS (namely, OpenSSL and Apple’s own fork) were found to have serious security holes. Known informally as “Heartbleed” and “goto fail,” both exploits provided opportunities to test Rust’s claims of memory safety. Rust is likely to have helped in both cases, but it is still possible to write Rust code that suffers from similar issues.

HEARTBLEED

Heartbleed was caused by re-using a buffer incorrectly. A *buffer* is a space set aside in memory for receiving input. Data can leak from one read to the next if the buffer’s contents are not cleared between writes.

Why does this situation occur? Programmers hunt for performance. Buffers are reused to minimize how often memory applications ask for memory from the OS.

Imagine that we want to process some secret information from multiple users. We decide, for whatever reason, to reuse a single buffer through the course of the program. If we don’t reset this buffer once we use it, information from earlier calls will leak to the latter ones. Here is a précis of a program that would encounter this error:

```
let buffer = &mut[0u8; 1024]; ①  
  
read_secrets(&user1, buffer); ②  
store_secrets(buffer);  
  
read_secrets(&user2, buffer); ③  
store_secrets(buffer);
```

- ① Binds a reference (&) to a mutable (mut) array ([...]) that contains 1,024 unsigned 8-bit integers (u8) initialized to 0 to the variable buffer.
- ② Fills buffer with bytes from the user1 object
- ③ buffer still contains data from user1 that may or may not be overwritten by

user2.

Rust does not protect you from logical errors. Rust ensures that your data is never able to be written in two places at the same time. It does not ensure that your program is free from all security issues.

GOTO FAIL

The “goto fail” bug is caused by programmer error coupled with C design issues (and potentially its compiler for not pointing out the flaw). A function that was designed to verify a cryptographic key pair ended up skipping all checks. Here is a selected extract from the function with a fair amount of obfuscatory syntax retained:

```
static OSStatus
// line break OK below? original exceeded 76 char limit
SSLVerifySignedServerKeyExchange(SSLContext *ctx, bool isRsa,
                                SSLBuffer signedParams,
                                uint8_t *signature, UInt16 signatureLen)
{
    OSStatus      err; ①

    ...

    // need line break below at 55 char max (because line contains annotation)
    if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0) ②
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
        goto fail;
    goto fail; ③
    if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
        goto fail;

    err = sslRawVerify(ctx,
                      ctx->peerPubKey,
                      dataToSign,          /* plaintext */
                      dataToSignLen,        /* plaintext length */
                      signature,
                      signatureLen);

    if(err) {
        sslErrorLog("SSLDecodeSignedServerKeyExchange: sslRawVerify "
                    "returned %d\n", (int)err);
        goto fail;
    }
}

fail:
    SSLFreeBuffer(&signedHashes);
    SSLFreeBuffer(&hashCtx);
    return err; ④
```

```
}
```

- ① Initializes OSStatus with a pass value (e.g., 0)
- ② A series of defensive programming checks
- ③ Unconditional goto skips SSLHashSHA1.final() and the (significant) call to sslRawVerify().
- ④ Returns the pass value of 0, even for inputs that should have failed the verification test

In the example code, the issue lies between lines 12 and 14. In C, logical tests do not require curly braces. C compilers interpret those three lines like this:

```
if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0) {  
    goto fail;  
}  
goto fail;
```

Would Rust have helped? Probably. In this specific case, Rust's grammar would have caught the bug. It does not allow logical tests without curly braces. Rust also issues a warning when code is unreachable. But that doesn't mean the error is made impossible in Rust. Stressed programmers under tight deadlines make mistakes. In general, similar code would compile and run.



Tip

Code with caution.

1.9 WHERE DOES RUST FIT BEST?

Although it was designed as a systems programming language, Rust is a general-purpose language. It has been successfully deployed in many areas, which we discuss next.

1.9.1 Command-line utilities

Rust offers three main advantages for programmers creating command-line

utilities: minimal startup time, low memory use, and easy deployment. Programs start their work quickly because Rust does not need to initialize an interpreter (Python, Ruby, ...) or virtual machine (Java, C#, ...).

As a bare-metal language, Rust produces memory-efficient programs. ^[16] As you'll encounter throughout the book, many types are zero-sized. That is, these only exist as hints to the compiler and take up no memory at all in the running program.

Utilities written in Rust are compiled as *static binaries* by default. This compilation method avoids depending on shared libraries that you must install before the program can run. Creating programs that can run without installation steps makes them easy to distribute.

1.9.2 Data processing

Rust excels at text processing and other forms of data wrangling. Programmers benefit from control over memory use and fast startup times. As of mid-2017, Rust touts the world's fastest regular expression engine. In 2019, the Apache Arrow data-processing project—foundational to the Python and R data science ecosystems—accepted the Rust-based DataFusion project.

Rust also underlies the implementation of multiple search engines, data-processing engines, and log-parsing systems. Its type system and memory control provide you with the ability to create high-throughput data pipelines with a low and stable memory footprint. Small filter programs can be easily embedded into the larger framework via Apache Storm, Apache Kafka, or Apache Hadoop streaming.

1.9.3 Extending applications

Rust is well suited for extending programs written in a dynamic language. This enables JNI (Java Native Interface) extensions, C extensions, or Erlang/Elixir NIFs (native implemented functions) in Rust. C extensions are typically a scary

proposition. These tend to be quite tightly integrated with the run time. Make a mistake and you could be looking at runaway memory consumption due to a memory leak or a complete crash. Rust takes away a lot of this anxiety.

- Sentry, a company that processes application errors, finds that Rust is an excellent candidate for rewriting CPU-intensive components of their Python system. [\[17\]](#)
- Dropbox used Rust to rewrite the file synchronization engine of its client-side application: “More than performance, [Rust’s] ergonomics and focus on correctness has helped us tame sync’s complexity.” [\[18\]](#)

1.9.4 Resource-constrained environments

C has occupied the domain of microcontrollers for decades. Yet, the Internet of Things (IoT) is coming. That could mean many billions of insecure devices exposed to the network. Any input parsing code will be routinely probed for weaknesses. Given how infrequently firmware updates for these devices occur, it’s critical that these are as secure as possible from the outset. Rust can play an important role here by adding a layer of safety without imposing run-time costs.

1.9.5 Server-side applications

Most applications written in Rust live on the server. These sit between the OS and your application. Databases, servers, search .

- The npm package registry for the JavaScript and node.js communities is written in Rust. [\[19\]](#)
- sled, an embedded database, can process a workload of 1 billion operations that includes 5% writes in less than a minute on a 16-core machine. [\[20\]](#)

1.9.6 Desktop applications

There is nothing inherent in Rust's design that prevents it from being deployed to develop user-facing software. Servo, the web browser engine that acted as an incubator for Rust's early development, is a user-facing application. Naturally, so are games.

1.9.7 Desktop

There is still a large need to write applications that live on people's computers. Desktop applications are often complex, difficult to engineer, and hard to support. With Rust's ergonomic approach to deployment and its rigor, it is likely to become the secret sauce for many applications. To start, these will be built by small, independent developers. As Rust matures, so will the ecosystem.

1.9.8 Mobile

Android, iOS, and other smartphone operating systems generally provide a blessed path for developers. In the case of Android, that path is Java. In the case of macOS, developers generally program in Swift. There is however, another way.

Both platforms provide the ability for “native applications” to run on the system. This is generally intended for applications written in C++, such as games, to be able to be deployed to people's phones. Rust is able to talk to the phone via the same interface with no additional run-time cost.

1.9.9 Web

As you are probably aware, JavaScript is the language of the web. Over time though, this will change. Browser vendors are developing a standard called WebAssembly (Wasm) that promises to be a compiler target for many languages. Rust is one of the first. Porting a Rust project to the browser requires only two additional command-line commands. Several companies are exploring the use of

Rust in the browser via Wasm, notably CloudFlare and Fastly.

1.9.10 Systems programming

In some sense, systems programming is Rust's *raison d'être*. Many large programs have been implemented in Rust, including compilers (Rust itself), video game engines, and operating systems. The Rust community includes writers of parser generators, databases, and file formats. Rust has proven to be a productive environment for programmers who share Rust's goals. Three standout projects in this area include

- Google is sponsoring the development of Fuchsia OS, an operating system for devices. [\[21\]](#)
- Microsoft is actively exploring writing low-level components in Rust for Windows. [\[22\]](#)
- Amazon Web Services (AWS) is building Bottlerocket, a bespoke OS for hosting containers in the cloud. [\[23\]](#)

[\[16\]](#) The joke goes that Rust is as close to bare metal as possible.

[\[17\]](#) See "Fixing Python Performance with Rust," <https://blog.sentry.io/2016/10/19/fixing-python-performance-with-rust>

[\[18\]](#) See "Rewriting the heart of our sync engine," <https://dropbox.tech/infrastructure/rewriting-the-heart-of-our-sync-engine>

[\[19\]](#) See "Community makes Rust an easy choice for npm: The npm Registry uses Rust for its CPU-bound bottlenecks," <https://www.rust-lang.org/static/pdfs/Rust-npm-Whitepaper.pdf>

[\[20\]](#) See <https://github.com/spacejam/sled>

[\[21\]](#) See "Welcome to Fuchsia!," <https://fuchsia.dev/>

[22] See “Using Rust in Windows,” <https://msrc-blog.microsoft.com/2019/11/07/using-rust-in-windows/>

[23] See “Bottlerocket: Linux-based operating system purpose-built to run containers,” <https://aws.amazon.com/bottlerocket/>

1.10 RUST’S HIDDEN FEATURE: ITS COMMUNITY

It takes more than software to grow a programming language. One of the things that the Rust team has done extraordinarily well is to foster a positive and welcoming community around the language. Everywhere you go within the Rust world, you’ll find that you’ll be treated with courtesy and respect.

1.11 RUST PHRASE BOOK



What do members of the Rust community mean when they use these terms?

- *Empowering everyone*—All programmers regardless of ability or background are welcome to participate. Programming and particularly systems programming should not be restricted to a blessed few.

- *Blazingly fast*—Rust is a fast programming language. You'll be able to write programs that match or exceed the performance of its peer languages, but you will have more safety guarantees.
- *Fearless concurrency*—Concurrent and parallel programming has always been seen as difficult. Rust frees you from whole classes of errors that have plagued its peer languages.
- *No Rust 2.0*—Rust code written today will always compile with a future Rust compiler. Rust is intended to be a reliable programming language that can be depended upon for decades to come. In accordance with *semantic versioning*, Rust is never backward incompatible, so it will never release a new major version.
- *Zero-cost abstractions*—The features you gain from Rust impose no run-time cost. When you program in Rust, safety does not sacrifice speed.

1.12 SUMMARY

- Many companies have successfully built large software projects in Rust.
- Software written in Rust can be compiled for the PC, the browser, the server, as well as mobile and IoT devices.
- The Rust language is well loved by software developers. It has repeatedly won Stack Overflow's "most loved programming language" title.
- Rust allows you to experiment without fear. It provides correctness guarantees that other tools are unable to provide without imposing run-time costs.
- With Rust, there are three main `command_line` tools to learn:
 - `cargo`, which manages a whole crate
 - `rustup`, which manages Rust installations
 - `rustc`, which manages compilation of Rust source code

Rust projects are not immune from all bugs.

2 LANGUAGE FOUNDATIONS

This chapter covers:

- Coming to grips with the Rust syntax
- Learning fundamental types and data structures
- Building command-line utilities
- Compiling programs

This chapter introduces you to the fundamentals of Rust programming. By the end of the chapter, you will be able to create command-line utilities and should be able to get the gist of most Rust programs. We'll work through most of the language's syntax, but defer much of the detail about *why* things are how they are for later in the book.



Note

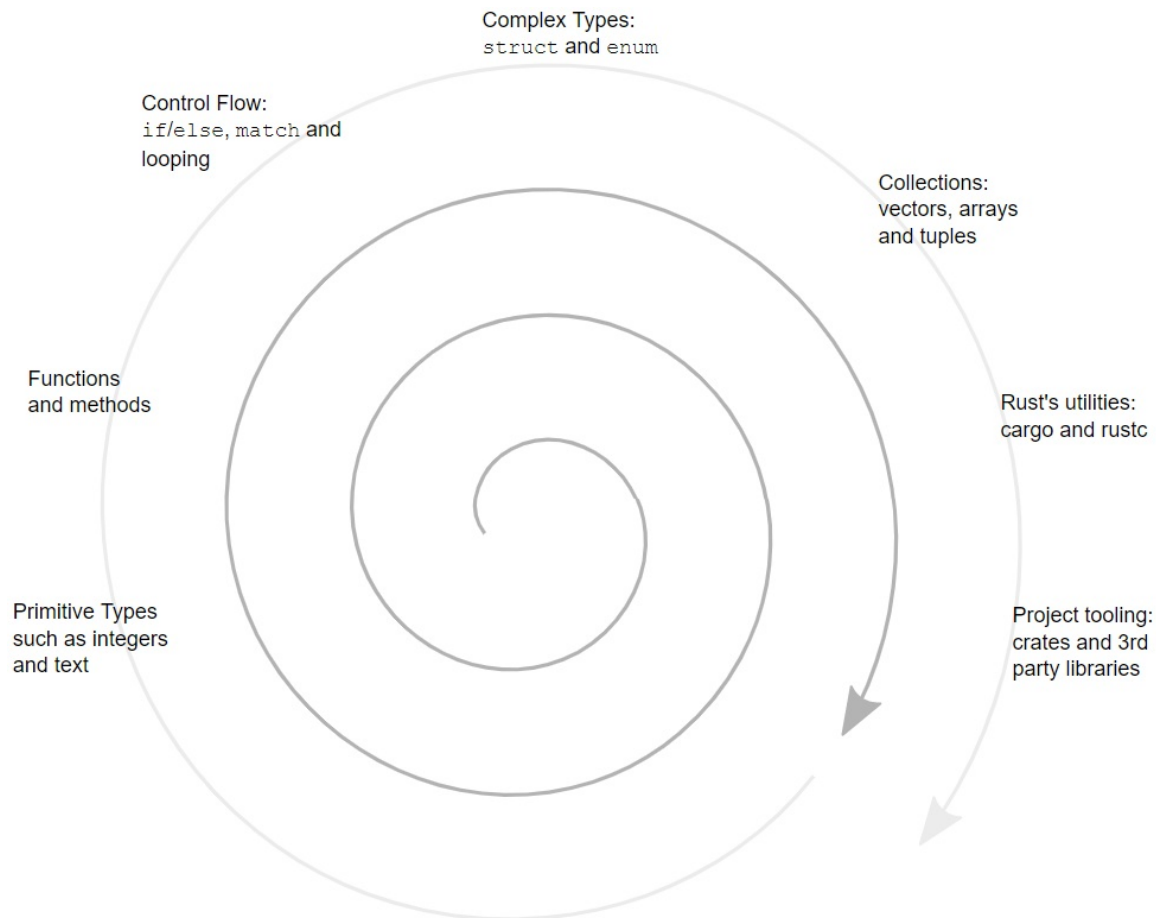
Programmers who have experience with another programming language will benefit the most from this chapter. If you are an experienced Rust programmer, feel free to skim through it.

Beginners are welcomed. Rust's community strives to be responsive to newcomers. At times, you may strike a mental pothole when you encounter terms such as “lifetime elision,” “hygienic macros,” “move semantics,” and “algebraic data types” without context. Don't be afraid to ask for help. The community is much more welcoming than these helpful, yet opaque, terms may suggest.

In this chapter, we will build `grep-lite`, a greatly stripped-down version of the ubiquitous `grep` utility. Our `grep-lite` program will look for patterns within text and print lines that match. This simple program allows us to focus on the unique features of Rust.

The chapter takes a spiral approach to learning. A few concepts will be discussed multiple times. With each iteration, you will find yourself learning more. [Figure 2.1](#) shows a completely unscientific map of the chapter.

Figure 2.1. Chapter topic outline. Starting with primitive types, the chapter progresses through several concepts with increasing levels of depth.



2.1 CREATING A RUNNING PROGRAM

Every plain text file has a hidden superpower: when it includes the right symbols, it can be converted into something that can be interpreted by a CPU. We'll explore this statement in the following sections.

It's highly recommended that you follow along with the examples in this book. To access or download the source code for the listings, use either of these two

sources:

- <https://manning.com/books/rust-in-action>
- <https://github.com/rust-in-action/code>

2.1.1 Compiling single files

[Listing 2.1](#) is a short, yet complete Rust program. To translate it into a working program, we use software called a *compiler*. The compiler's role is to translate the source code into machine code, as well as take care of lots of bookkeeping to satisfy the operating system (OS) and CPU that it is a runnable program. The Rust compiler is called *rustc*. Note that you'll find the source code for this listing in the file `ch2/ok.rs`.

Listing 2.1. Almost the shortest valid Rust program

```
fn main() {  
    println!("ok")  
}
```

To compile a single file created with Rust into a working program:

- Make sure that the file includes a `main()` function
- Open a shell window such as `cmd.exe`, Powershell, Git Bash, or Terminal
- Execute the command `rustc <file>`, where `<file>` is the file you want to compile

When compilation succeeds, `rustc` is silent. Even so, `rustc` dutifully reports OK whenever you execute the newly created file. The following listing provides a short demonstration of the process.

Listing 2.2. Compiling listing 2.1 on Linux and executing the resulting binary

```
$ rustc ch2-ok.rs  
$ ./ch2-ok ①  
ok
```

① For Windows, include the .exe filename extension (e.g., ch2-ok.exe)

2.1.2 Compiling larger projects

We normally compile projects larger than a single file with a higher-level tool called cargo. cargo understands how to compile multi-file projects (and much more). It executes the simple compiler rustc on your behalf.

Note

If you're ever curious about what cargo is doing under the hood, add the verbose flag (-v) to your code. This instructs cargo to report the commands that it executes on your behalf.

2.2 A GLANCE AT RUST'S SYNTAX

You might appreciate that Rust is boring and predictable where it can be. It has variables, numbers, functions, and other familiar things that you have seen in other languages. For example, it delimits blocks with curly brackets ({ and }), it uses a single equals sign as its assignment operator (=), and it is whitespace agnostic.

2.2.1 Defining and calling functions

To better understand Rust, let's look at another short listing. It prints `a + b = 30` to the console after we define some functions and variables. The source code for this listing is in the `ch2-first-steps.rs` file.

Listing 2.3. Adding integers using variables and declaring types

```
fn main() {           ①
    let a = 10;        ②
    let b: i32 = 20;    ③
    let c = 30i32;      ④
    let d = 30_i32;     ⑤
    let e = add(add(a, b), add(c, d));
    println!("( a + b ) + ( c + d ) = {}", e);
```

```

}
fn add(i: i32, j: i32) -> i32 {    ⑥
    i + j                          ⑦
}

```

- ① Rust is flexible with the location of the `main()` function.
- ② Types can be inferred by the compiler ...
- ③ ... or declared by the programmer when creating variables.
- ④ Numeric types can include a type annotation in their literal form.
- ⑤ Numbers can include underscores, which are intended to increase readability and have no functional impact.
- ⑥ Type declarations are required when defining functions.
- ⑦ Functions return the last expression's result so that `return` is not required.

In the listing, be careful about adding a semicolon to the `add()` function declaration. This changes the semantics, returning `()` rather than `i32`).

Although only 12 lines of code, there is quite a lot packed into listing 2.2. Here's a brief description that should provide the gist of what's going on. We will cover the details in the rest of the chapter.

In line 1 (`fn main() {}`), the `fn` keyword begins a function definition. The entry point to all Rust programs is `main()`. It takes no arguments and returns no value. [\[24\]](#) Code blocks, also known as *lexical scopes*, are defined with curly braces: `{` and `}`.

In line 2 (`let a = 10;`), we use `let` to declare *variable bindings*. Variables are *immutable* by default, meaning that variables are read-only rather than read-write. And finally, statements are delimited with semicolons `;`).

In line 3 (`let b: i32 = 20;`), you can designate a specific data type for the compiler. At times, this will be required as the compiler will be unable to deduce a unique type on your behalf.

In line 4 (`let c = 30i32;`), you'll note that Rust's numeric literals can include

types annotations. This can be helpful when navigating complex numerical expressions. And in line 5 (`let c = 30_i32;`), you'll see that Rust permits the use of underscores within numeric literals. These increase readability but are insignificant to the compiler. In line 6 (`let e = add(add(a, b), add(c, d));`), it should be easy to see that calling functions looks like what you've experienced in most other programming languages.

In line 7 (`println!("( a + b ) + ( c + d ) = {}", e);`),`println!()` is a *macro*, which are function-like but return code rather than a value. With printing, every data type has its own way of being converted to a string, but `println!()` takes care of figuring out the exact methods to call on its arguments. Strings use double quotes (") rather than single quotes ('). Rust uses single quotes for single characters, which are a distinct type, `char`. And with Rust, string formatting uses `{}` as a placeholder, rather than the C-like `printf` style of `%s` or other variants.

Finally, in line 10 (`fn add(...) -> i32 {}`) of listing 2.2, you can see that Rust's syntax for defining functions is similar to those programming languages that use explicit type declarations. Commas delimit parameters and type declarations follow variable names. The dagger (`->`) or thin arrow syntax indicates the return type.

[\[24\]](#) This isn't technically correct, but is accurate enough for now. If you're an experienced Rust programmer skimming through this chapter, you'll know that `main()` returns "unit" by default and can also return a `Result`.

2.3 NUMBERS

Computers have been associated with numbers for longer than you have been able to say "formula translator." This section discusses how to create numeric types in Rust and how to perform operations on these.

2.3.1 Integers and decimal (floating-point) numbers

Rust uses a relatively conventional syntax for creating integers (1, 2, ...) and floating-point numbers (1.0, 1.1, ...). Operations on numbers use *infix notation*, meaning that numeric expressions look like those that you're used to. To operate on multiple types, Rust also allows the same token (+) for addition. This is called *operator overloading*. Some notable differences from other languages include

- *Rust includes a large number of numeric types.* You will become used to declaring the size in bytes, which affects how many numbers the type can represent and whether your type is able to represent negative numbers.
- *Conversions between types are always explicit.* Rust does not automatically convert your 16-bit integer into a 32-bit integer.
- *Rust's numbers can have methods.* For example, to round 24.5 to the nearest integer, Rust programmers use `24.5_f32.round()` rather than `(round(24.5_f32))`. Here, the type suffix is required because a concrete type is required.

To start, let's consider a small example. You'll find the code in `ch2-intro-to-numbers.rs` in the examples for this book. [Listing 2.4](#) prints these few lines to the console:

```
20 + 21 + 22 = 63
10000000000000
42
```

Listing 2.4. Numeric literals and basic operations on numbers in Rust

```
fn main() {
    let twenty = 20;                                ①
    let twenty_one: i32 = 21;                        ②
    let twenty_two = 22i32;                          ③

    let addition = twenty + twenty_one + twenty_two;
    println!("{}", twenty, twenty_one, twenty_two, addition);

    let one_million: i64 = 1_000_000;                ④
    println!("{}", one_million.pow(2));              ⑤

    let forty_twos = [                               ⑥
        42.0,                                         ⑦
    ]
```

```

    42f32,           ⑧
    42.0_f32,       ⑨
];

println!("{:02}", forty_twos[0]); ⑩
}

```

- ① Rust infers a type on your behalf if you don't supply one ...
- ② ... which is done by adding type annotations (i32) ...
- ③ ... or type suffixes.
- ④ Underscores increase readability and are ignored by the compiler.
- ⑤ Numbers have methods.
- ⑥ Creates an array of numbers, which must all be the same type, by surrounding those with square brackets.
- ⑦ Floating-point literals without an explicit type annotation become 32-bit or 64-bit, depending on context.
- ⑧ Floating-point literals can also have type suffixes ...
- ⑨ ... and optional underscores.
- ⑩ Elements within arrays can be indexed numerically, starting at 0.

2.3.2 Integers with base 2, base 8, and base 16 notation

Rust also has built-in support for numeric literals that allow you to define integers in base 2 (binary), base 8 (octal), and base 16 (hexadecimal). This notation is also available within the formatting macros like `println!`. [Listing 2.5](#) demonstrates the three styles and produces the following output. You can find the source code for this listing in `ch2-non-base2.rs`.

```

base 10: 3 30 300
base 2:  11 11110 100101100
base 8:  3 36 454
base 16: 3 1e 12c

```

Listing 2.5. Using base 2, base 8, and base 16 numeric literals

```
fn main() {
```

```

let three = 0b11;           ①
let thirty = 0o36;          ②
let three_hundred = 0x12C;  ③

println!("base 10: {} {} {}", three, thirty, three_hundred);
println!("base 2:  {:b} {:b} {:b}", three, thirty, three_hundred);
println!("base 8:  {:o} {:o} {:o}", three, thirty, three_hundred);
println!("base 16: {:x} {:x} {:x}", three, thirty, three_hundred);
}

```

- ① The 0b prefix indicates binary (base 2) numerals.
- ② The 0o prefix indicates octal (base 8) numerals.
- ③ The 0x prefix indicates hexadecimal (base 16) numerals.

In binary (base 2) numerals, 0b11 equals 3 because $3 = 2 \times 1 + 1 \times 1$. With octal (base 8) numerals, 0o36 equals 30 because $30 = 8 \times 3 + 1 \times 6$. And with hexadecimal (base 16) numerals, 0x12C equals 300 because $300 = 256 \times 1 + 16 \times 2 + 1 \times 12$. Table 2.1 shows the types that represent scalar numbers.

Table 2.1. Rust types for representing scalar (single) numbers

i8, i16, i32, i64	Signed integers ranging from 8 bit to 64 bit
u8, u16, u32, u64	Unsigned integers ranging from 8 bit to 64 bit
f32, f64	Floating-point numbers in 32-bit and 64-bit variants
isize, usize	Integers that assume the CPU's "native" width (e.g., in 64-bit CPUs, usize and isize will be 64-bits wide)

Rust contains a full complement of numeric types. The types are grouped into a few families:

- Signed integers (i) represent negative as well as positive integers.
- Unsigned integers (u) only represent positive integers but can go twice as high as their signed counterparts
- Floating-point integers (f) represent real numbers, with special values for infinity, negative infinity, and "not a number"

Integer width is the number of bits that the type uses in RAM and in the CPU. Types that take up more space, such as u32 vs. i8, represent more numbers. But this adds the expense of needing to store extra zeros for smaller numbers as the

following table shows.

Number	Type	Bit pattern in memory
20	u32	000000000000000000000000000010100
20	i8	00010100

Although we’ve only touched on numbers, we nearly have enough exposure to Rust to create a prototype of our pattern-matching program. But let’s look at comparing numbers before we create our program.

2.3.3. Comparing numbers

Rust’s numeric types support a large suite of comparisons that you’re probably familiar with. Enabling support for these comparisons is provided by a feature that you have not encountered yet. It is called *traits*. [\[25\]](#) Table 2.2 summarizes the comparison types.

Table 2.2. Comparisons supported by Rust’s numeric types

Comparison	Operator	Example
Less than	<	1.0 < 2.0
Greater than	>	2.0 > 1.0
Equal to	==	1.0 == 1.0
Unequal to	!=	1.0 != 2.0
Less than or equal to	<=	1.0 <= 2.0
Greater than or equal to	>=	2.0 >= 1.0

That support does include a few caveats. We'll look at these conditions in the rest of this section.

IMPOSSIBLE TO COMPARE DIFFERENT TYPES

Rust's type safety requirements prevent comparisons between types. For example, this code does not compile:

```
fn main() {
    let a: i32 = 10;
    let b: u16 = 100;

    if a < b {
        println!("Ten is less than one hundred.");
    }
}
```

To appease the compiler, we need to use an `as` operator to cast one of the operands to the other's type. The following code shows this type cast: `b as i32`.

```
fn main() {
    let a: i32 = 10;
    let b: u16 = 100;

    if a < (b as i32) {
        println!("Ten is less than one hundred.");
    }
}
```

It is safest to cast the smaller type to a larger one (e.g., a 16-bit type to a 32-bit type). This is sometimes referred to as *promotion*. In this case, we could have demoted `a` down to a `u16`, but such a move is more generally risky.



Warning

Using type casts carelessly will cause your program to behave unexpectedly. For example, the expression `300_i32 as i8` returns 44.

In some cases, using the `as` keyword is too restrictive. It's possible to regain fuller control over the type conversion process via some bureaucracy. The following listing shows a Rust method to use instead of the `as` keyword when the conversion might fail.

Listing 2.6. The `try_into()` method converts between types

```
use std::convert::TryInto;                                ①

fn main() {
    let a: i32 = 10;
    let b: u16 = 100;

    if a < b.try_into().unwrap() {                          ②
        println!("Ten is less than one hundred.");
    }
}
```

```
}  
}
```

- ① Effectively adds `try_into()` to the `u16` type
- ② `b.try_into()` returns an `i32` value wrapped within a `Result` (discussed in the next chapter). For now, it's enough to know that `unwrap()` returns the wrapped `i32`.

In the listing, `TryInto` is a *trait*. Traits from other crates can extend the types that we use, particularly if those types are defined within the standard library. And making use of `try_into()` involves a lot of Rust you haven't encountered yet. Feel free to stick with `as` for the moment if you like. Once you have read through chapter 3, you will feel much more comfortable with what is happening.

FLOATING-POINT HAZARDS

Floating-point types (`f32` and `f64`, for example) are a special case for comparisons. There are (at least) two reasons for this:

- *These often approximate the numbers that they're representing.* Floating-point types are implemented in base 2, but we often want to calculate numbers in base 10. This mismatch creates ambiguity.
- *These can represent values that have unintuitive semantics.* Unlike integers, floating-point types have some values that do not play well with others. Formally, these only have a *partial equivalence relation*.

Floating-point numbers that are impossible to represent in binary

Floating-point numbers are implemented inside computing systems that use binary mathematics, but often we use these to represent decimal values. This poses a problem because many values we care about, such as 0.1, have no exact representation in binary notation. [\[26\]](#) Yet because Rust aims to be pragmatic, the following code compiles and runs without error:

```
fn main() {  
    assert!(0.1 + 0.2 == 0.3);  
}
```

```
}
```



Warning

The `assert!` macro will crash a program when the expression it is evaluating does not evaluate to `true`.

Rust includes some tolerances to allow comparisons between floating-point values. These tolerances are defined as `f32::EPSILON` and `f64::EPSILON`. To be more precise, it's possible to get closer to how Rust is behaving under the hood as the following shows. What's actually happening is interesting but mostly irrelevant here. The Rust compiler actually delegates the comparison to the CPU. [\[27\]](#) Whereas floating-point operations are implemented in the hardware.

```
fn main() {  
    let result: f32 = 0.1 + 0.1;  
    let desired: f32 = 0.2;  
    let absolute_difference = (desired - result).abs();  
    assert!(absolute_difference <= f32::EPSILON);  
}
```

Comparisons where the result is mathematically undefined

Floating-point types include “not a number” values (typically represented as `NaN` but within Rust as `NAN`). These values represent the result of numerical operations that are undefined, such as taking the square root of a negative number. By definition, one `NAN` value is never equal to other `NAN` values. Let's look at an example.

```
fn main() {  
    assert_eq!(f32::NAN == f32::NAN, false);  
}
```



Warning

The `assert_eq!` macro will crash the program when the expressions it evaluates do not equal each other.

2.3.4 Rational, complex numbers, and other

numeric types

Rust's standard library is comparatively slim. It excludes numeric types that are often available within other languages, such as:

- Many mathematical objects for working with rational numbers and complex numbers
- Arbitrary size integers and arbitrary precision floating-point numbers for working with very large or very small numbers
- Fixed-point decimal numbers for working with currencies

To access items in this list of specialized numeric types, you can use the format `num crate`. *Crates* are installable packages that extend the standard library. We'll learn about these in detail later this chapter in section 1.11. For now, it will suffice to know that these are *not* available by default.

[Listing 2.7](#) demonstrates how to add two complex numbers. It calculates $(2.1 + -1.2i) + (11.1 + 22.22i)$. Running the code should produce the following output: `13.2 + 21.02_i_`.

[Listing 2.7](#) introduces some new commands. It demonstrates two forms of initializing non-primitive data types. One is a *literal* syntax provided by the Rust language (line 4). The other is the `new()` static method, which many numeric types implement for added convenience (line 5). A *static method* is a function that's available for a type, but it's not an instance of that type. [\[28\]](#)

[Listing 2.7](#) demonstrates accessing a third-party crate (`num`), calling a type's `new()` method, and accessing fields with the dot operator (`.`). The source code for this file is available in `ch2/ch2-complex/src/main.rs`.

Listing 2.7. Calculating values with complex numbers

```
use num::complex::Complex;           ①

fn main() {

    let a = Complex { re: 2.1, im: -1.2 }; ②
```



```

let b = Complex::new(11.1, 22.2);           ③

let result = a + b;

println!("{}", result.re, result.im)      ④
}

```

- ① The `use` keyword pulls crates into local scope, and the namespace operator (`::`) restricts what's included. Here, only a single type is required: `Complex`.
- ② Types don't require the creation of constructors. Initialize types by using the type name (`Complex`) and assigning their fields (`re`, `im`) values (2.1, -1.2) within curly braces (`{, }`).
- ③ Many types implement a `new()` method for simplicity, but this convention is not part of the Rust language.
- ④ `num::complex::Complex` type has two fields: `re` represents the real part, and `im` represents the imaginary part. Both are accessible with the dot operator (`.`).

To compile the listing, the following provides the recommended workflow. Listing 2.7 shows

1. Open a shell session
2. Execute the following commands:
 - `cd $TMP` (`cd %TMP%` on MS Windows)
 - `cargo new ch2-complex`
 - `cd ch2-complex`

Add the `num` crate into the `[dependencies]` section of `Cargo.toml`

Add the source code from listing 2.6 to `src/main.rs`

Execute `cargo run`

Listing 2.8. Example session following the recommended workflow to compile listing 2.6

```
$ cargo new ch2-complex
```

```

    Created binary (application) `ch2-complex` package
$ cd ch2-complex
$ # Update Cargo.toml
$ # Write code from Listing 2.6 into src/main.rs
$ cargo run
... ①
    Compiling ch2-complex v0.1.0 (/path/to/ch2-complex)
    Finished dev [unoptimized + debuginfo] target(s) in 2.51s
    Running `target/debug/ch2-complex`
13.2 + 21.02i

```

① Lines skipped for readability.

Shortcut for adding a third-party dependency to a project with cargo

I recommend that you install the cargo-edit crate to enable the `cargo add` command. You can do this with the following:

```

$ cargo install cargo-edit
    Updating crates.io index
    Installing cargo-edit v0.6.0
...
    Installed package `cargo-edit v0.6.0` (executables `cargo-add`, `cargo-rm`,
    `cargo-upgrade`)

```

Up until this point, we have manually added dependencies to Cargo.toml. The `cargo add` command simplifies this process by editing the file correctly on your behalf:

```

$ cargo add regex
    Updating https://github.com/rust-lang/crates.io-index index
    Adding regex v1.3.9 to dependencies

```

We’ve now addressed how to access built-in numeric types and types available from third-party libraries. Let’s take a moment to discuss one of Rust’s more novel features: the `match` keyword. We’ll combine that with the knowledge we’ve gained so far to create our first non-trivial program.

[25] For the curious and eager, the traits involved here are `std::cmp::PartialOrd` and `std::cmp::PartialEq`.

[26] If this is confusing to think about, consider that many values like 1/3 (one third) have no exact representation within a decimal number system.

[27] Illegal or undefined operations trigger a CPU exception. You will read about those later in chapter 12.

[28] Although Rust is not object-oriented (it's impossible to create a subclass, for example), Rust makes use of some terminology from that domain. It's common to hear of Rust programmers discussing instances, methods, and objects.

2.4 ITERATION

Iterating through collections of things, including iterating over collections that may have infinitely many values, is easy in Rust. Here is a snippet of the core syntax:

```
for item in collection {  
    // ...  
}
```

Let's put that syntax to use. [Listing 2.9](#) searches for a number (the `needle` defined on line 2) within an array of numbers (the `haystack` defined on line 3). The code then prints 42 to the console when compiled. The code for this listing is in `ch2/ch2-needle-in-haystack.rs`.

The ampersand (&) prefix to `haystack` on line 5 might not look familiar. The ampersand acts as a *unary operator* that returns a reference to `haystack`. For any type `T`, the syntax `&T` returns a read-only reference to `T`. A useful quirk of references to arrays is that with `for` loops you can iterate over references to the elements of the array. In Rust terminology, when we use `&T`, we “borrow” `T`. A borrowed value outlives the scope that it is used in. The concept of borrowing and its terminology are explained in detail in chapter 4.

Listing 2.9. Searching for an integer in an array of integers

```
fn main() {  
    let needle = 0o52;
```

```

let haystack = [1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862];

for item in &haystack {           ①
    if *item == needle {         ②
        println!("{}", item);
    }
}

```

- ① Iterates over references to elements within haystack. Each iteration changes the value of `item` to refer to the next item.
- ② `*item` dereferences `item` to return its referent. On the first iteration, `*item` returns 1; and on the last, it returns 4862.

2.4.1 Creating iterators that support for loops

[Listing 2.9](#) allowed us to create an iterator over an array by taking a reference to it. Concretely, `&haystack` let us iterate over items within `haystack`.

Unfortunately, not all types support this syntax. [Listing 2.10](#) provides an alternative, where calling `haystack.iter()` returns an iterator. You'll find the code for this listing in `ch2/ch2-bell-numbers.rs`.

Listing 2.10. Using the `iter()` method to return an iterator

```

fn main() {
    let needle = 0xCB;
    let haystack = [1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147];

    for item in haystack.iter() {
        if *item == needle {
            println!("{}", item);
        }
    }
}

```

We'll learn more about this approach when we learn to define our own types. In the meantime, it can be a useful trick to remember while you're learning.

Some types also provide `iter_mut()` and `into_iter()` methods. These are more complicated and most readers can probably skip them for now. The `iter_mut()` method allows you to mutate (to change) values as you are iterating over those.

The `into_iter()` method is closer to `iter()`, but rather than using references, it returns elements *by value*. As a consequence, this prevents other parts of the code from accessing those values. The following listing provides an example of this. The source code for this listing is in the `ch2/ch2-catalan-numbers.rs` file.

Listing 2.11. Using `into_iter()` to return an iterator to get elements by value rather than by reference

```
fn main() {
    let needle = 0o204;
    let haystack = vec![1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796];

    for item in haystack.into_iter() {
        if item == needle {
            println!("{}", item);
        }
    }
}
```

① Dereferencing `item` is not required as it's already returned by value.



Note

Hidden under the syntactic differences between `iter()` and `into_iter()` lies a subtle semantic difference that will become clearer once you've read through chapter 4. In Rust terminology, `iter()` *borrow*s the `haystack`, and `into_iter()` takes *ownership* of `haystack`.

2.5 FLOW CONTROL

Programs execute from top to bottom, except when you don't want that. Rust has a useful set *flow control* mechanisms to facilitate this. This section provides a brief tour of the fundamentals.

2.5.1 `for`: The central pillar of iteration

We have seen `for` loops several times already. The basic form is

```
for item in container {
    // ...
}
```

```
}
```

The local variable `item` is available within the block. Attempting to access it after the block finishes, even within the bounds of a function, your program will fail to compile.

REFERENCES ARE USUALLY WHAT YOU WANT

When you want to reuse `container` later in your program, use a reference. For reasons that are explained in chapter 4, when a reference is omitted, Rust assumes that `container` is no longer needed. To add a reference to the container, prefix it with an ampersand (`&`) as this example shows:

```
for item in &container {  
    // ...  
}
```

ANONYMOUS LOOPS

When a local variable is not used within a block, by convention, you'll use an underscore (`_`). Using this pattern in conjunction with the `_exclusive` range syntax `_ (n..m)` and the *inclusive range syntax* `(n..=m)` makes it clear that the intent is to perform a loop for a fixed number of times. Here's an example:

```
for _ in 0..10 {  
    // ...  
}
```

AVOID MANAGING AN INDEX VARIABLE

In many programming languages, it's common to loop through things by using a temporary variable that's incremented at the end of each iteration.

Conventionally, this variable is named `i` (for index). A Rust version of that pattern is

```
let collection = [1, 2, 3, 4, 5];  
for i in 0..collection.len() {  
    let item = collection[i];
```

```
// ...  
}
```

This is legal Rust. It's also essential in cases when iterating directly over `collection` (for `item` in `collection`) is impossible. However, it is generally discouraged. The manual approach introduces two problems with this:

- *Performance*—Indexing values with the `collection[index]` syntax incurs run-time costs for *bounds checking*. That is, Rust checks that `index` currently exists within `collection` as valid data. Those checks are not necessary when iterating directly over `collection`. The compiler can use compile-time analysis to prove that illegal access is impossible.
- *Safety*—Periodically accessing `collection` over time introduces the possibility that it has changed. Using a `for` loop over `collection` directly allows Rust to guarantee that the `collection` remains untouched by other parts of the program.

2.5.2 `continue`: Skipping the rest of the current iteration

The `continue` keyword operates as you would expect. Here's an example:

```
for n in 0..10 {  
    if n % 2 == 0 {  
        continue;  
    }  
    // ...  
}
```

2.5.3 `while`: Looping until a condition changes its state

The `while` loop proceeds as long as a condition holds. The condition, formally known as a *predicate*, can be any expression that evaluates to `true` or `false`. This (non-functioning) snippet takes air quality samples, checking to avoid anomalies:

```

let mut samples = vec![];

while samples.len() < 10 {
    let sample = take_sample();
    if is_outlier(sample) {
        continue;
    }

    samples.push(sample);
}

```

USING WHILE TO STOP ITERATING ONCE A DURATION IS REACHED

[Listing 2.12](#) provides a working example of `while`. It isn't an ideal method for implementing benchmarks, but can be a useful tool to have in your toolbox. In the listing, `while` continues to execute a block when a time limit is not reached. The source code for this listing is in the file `ch2/ch2-while-true-incr-count.rs`.

Listing 2.12. Testing how fast your computer can increment a counter

```

use std::time::{Duration, Instant};           ①

fn main() {
    let mut count = 0;
    let time_limit = Duration::new(1,0);       ②
    let start = Instant::now();                 ③

    while (Instant::now() - start) < time_limit { ④
        count += 1;
    }
    println!("{}", count);
}

```

- ① This form of an import hasn't been seen before. It brings the `Duration` and `Instant` types from `std::time` into local scope.
- ② Creates a `Duration` that represents 1 second
- ③ Accesses time from the system's clock
- ④ An `Instant` minus an `Instant` returns a `Duration`.

AVOID WHILE WHEN ENDLESSLY LOOPING

Most Rust programmers avoid the following idiom to express looping “forever”. The preferred alternative is to use the `loop` keyword, explained shortly in the next section.

```
while true {  
    println!("Are we there yet?");  
}
```

2.5.4 `loop`: The basis for Rust’s looping constructs

Rust contains a `loop` keyword for providing more control than `for` and `while`. `loop` executes a code block again and again, never stopping for a tea (or coffee) break. `loop` continues to execute until a `break` keyword is encountered or the program is terminated from the outside. Here’s an example showing the `loop` syntax:

```
loop {  
    // ...  
}
```

`loop` is often seen when implementing long-running servers as the following example shows:

```
loop {  
    let requester, request = accept_request();  
    let result = process_request(request);  
    send_response(requester, result);  
}
```

2.5.5 `break`: Aborting a loop

The `break` keyword breaks out of a loop. Rust’s generally operates as you are used to.

```
for (x, y) in (0..).zip(0..) {  
    if x + y > 100 {  
        break;  
    }  
}
```

```
// ...  
}
```

BREAK FROM NESTED LOOPS

You can break out of a nested loop with loop labels. [\[29\]](#) A *loop label* is an identifier prefixed with an apostrophe (') like this example shows:

```
'outer: for x in 0.. {  
    for y in 0.. {  
        for z in 0.. {  
            if x + y + z > 1000 {  
                break 'outer;  
            }  
        }  
        // ...  
    }  
}  
}
```

Rust does not include the `goto` keyword, which provides the ability to jump to other parts of the program. The `goto` keyword can make control flow confusing, and its use is generally discouraged. One place where it is still commonly used though is to jump to and clean up a section of a function when an error condition is detected. Use loop labels to enable that pattern.

2.5.6 if, if else, and else: Condition testing

So far, we've indulged in the exciting pursuit of looking for numbers within lists of numbers. Our tests have involved utilizing the `if` keyword. For example

```
if item == 42 {  
    // ...  
}
```

`if` accepts any expression that evaluates to a Boolean value (e.g., `true` or `false`). Rust has no concept of “truthy” or “falsey” types. Other languages allow special values such as `0` or an empty string to stand in for `false` and for other values to represent `true`, but Rust doesn't allow this. The only value that can be used for `true` is `true`, and for `false`, use `false`.

```
if item == 42 {  
    // ...  
} else if item == 132 {  
    // ...  
} else {  
    // ...  
}
```

CONDITIONAL TESTS RETURN VALUES

Rust's conditionals include one difference from many languages—Rust is an *expression-based language*. One characteristic of languages from this heritage is that all expressions return values. This enables two patterns: concise helper functions and variable assignment directly from the return value of a conditional expression.

Here's an example of a helper function:

```
fn is_even(a: i32) -> bool {  
    if a % 2 == 0 {  
        true  
    } else {  
        false  
    }  
}
```

Rust allows you to use the result of a conditional expression as a variable. The following gives an example of this usage:

```
let description = if is_even(123456) {  
    "even"  
} else {  
    "odd"  
};
```

2.5.7 match: Type-aware pattern matching

While it's possible to use `if/else` blocks in Rust, `match` provides a safer alternative. `match` warns you if you haven't considered a relevant alternative. It is also elegant and concise:

```
match item {
    0      => {},    ①
    10 ..= 20 => {},    ②
    40 | 80 => {},    ③
    _      => {},    ④
}
```

- ① To match a single value, provide the value. No operator is required.
- ② The `..=` syntax matches an inclusive range. Supported by types that
- ③ The vertical bar (`|`) matches values on either side of it.
- ④ The underscore (`_`) matches every value. Failing to provide a branch for every value triggers a compiler error.

Rust's `match` keyword is analogous to the `switch` keyword in other languages. Unlike C's `switch` however, `match` guarantees that all possible options for a type are explicitly handled. Additionally, a match does not “fall through” to the next option by default. Instead, `match` returns immediately when a match is found.

[Listing 2.13](#) demonstrates a working example of `match`. The source code for this listing is in `ch2-match-needles.rs`. The code prints these two lines to the screen:

```
42: hit!
132: hit!
```

Listing 2.13. Using `match` to match multiple values

```
fn main() {
    let needle = 42;                ①
    let haystack = [1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862];

    for item in &haystack {
        let result = match item {    ②
            42 | 132 => "hit!",        ③
            _ => "miss",              ④
        };

        if result == "hit!" {
            println!("{}", item, result);
        }
    }
}
```

- ① The variable `needle` is now redundant.
- ② This match expression returns a value that can be bound to a variable.
- ③ Success! `42 | 132` matches both 42 and 132.
- ④ A wildcard pattern that matches everything

The `match` keyword plays an important role within the Rust language. Many control structures like looping are defined in terms of `match` under the hood. These really shine when combined with the `Option` type that's discussed in depth in the next chapter.

Now that we have taken a good look at defining numbers and working with some of Rust's control-flow mechanisms, let's move on to adding structure to programs with functions.

[\[29\]](#) This functionality is also available with `continue`, but it's less common.

2.6 DEFINING FUNCTIONS

Looking back to where the chapter begins, the snippet in listing 2.2 (repeated in listing 2.13) contained a small function, `add()`. As the listing shows, `add` takes two `i32` values and returns another.

Listing 2.14. Extract of listing 2.2

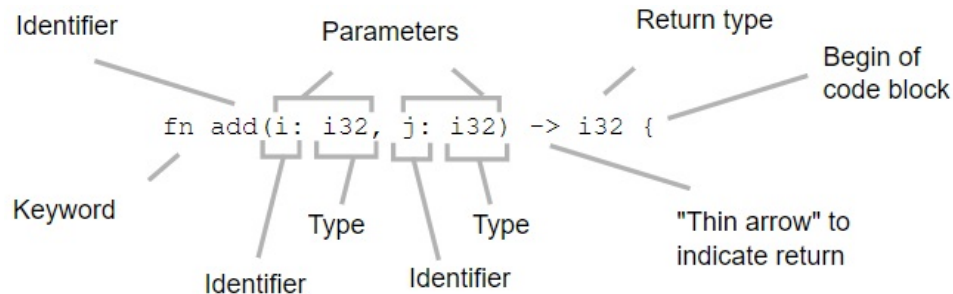
```
fn add(i: i32, j: i32) -> i32 {    ①
    i + j
}
```

- ① `add` takes two integer parameters and returns an integer. The two arguments are bound to the local variables `i` and `j`.

For the moment, let's concentrate on the syntax of each of the elements in listing 2.13. [Figure 2.2](#) provides a visual picture of each of the pieces. Anyone who has programmed in a strongly-typed programming language should be able to squint

their way through the diagram.

Figure 2.2. Rust's function definition syntax



Rust's functions require that you specify your parameters' types and the function's return type. This is the foundational knowledge that we'll need for the majority of our work with Rust. Let's put it to use with our first non-trivial program.

2.7 PROJECT: RENDERING THE MANDELBROT SET

So far, we haven't learned much Rust, but we already have the tools to create some interesting pictures of fractals. So let's do that. In a terminal window, execute the following commands to create a project that can render the Mandelbrot set.

```
cd $TMP
cargo new mandelbrot --vcs none
cd mandelbrot
cargo add num
```

Replace `src/main.rs` with the code in listing 2.14, which you'll find in `ch2/ch2-mandelbrot/src/main.rs`. Then type `cargo run`. The following shows the output produced by listing 2.14:

```
.....*.....
.....*.....
.....*.....
.....*%+*.....
.....*%+%*.....
.....*%+%*.....
.....*%+%*.....
.....*%+%*.....
```



```

    for i in 0..max_iters {
        if z.norm() > 2.0 {
            return i;
        }
        z = z * z + c;
    }
    return max_iters;
}

fn render_mandelbrot(escape_vals: Vec<Vec<usize>>) {
    for row in escape_vals {
        let mut line = String::with_capacity(row.len());
        for column in row {
            let val = match column {
                0..=2 => ' ',
                2..=5 => '.',
                5..=10 => '.',
                11..=30 => '*',
                30..=100 => '+',
                100..=200 => 'x',
                200..=400 => '$',
                400..=700 => '#',
                _ => '%',
            };

            line.push(val);
        }
        println!("{}", line);
    }
}

fn main() {
    let mandelbrot = calculate_mandelbrot(1000, -2.0, 1.0, -1.0, 1.0, 100, 24);

    render_mandelbrot(mandelbrot);
}

```

- ① Imports the Complex number type from num crate and its complex submodule
- ② Converts between the output space (a grid of rows and columns) and a range that surrounds the Mandelbrot set (a continuous region near (0,0))
- ③ If a value has not escaped before reaching the maximum number of iterations, it's considered to be within the Mandelbrot set.
- ④ Parameters that specify the space we're searching for to look for members of the set
- ⑤ Parameters that represent the size of the output in pixels
- ⑥ Creates a container to house the data from each row
- ⑦ Iterates row by row, allowing us to print the output line by line
- ⑧ Calculates the proportion of the space covered in our output and converts

that to points within the search space

- ⑨ cx and cy are the real and imaginary parts of a complex number.
- ⑩ The function called at every pixel (e.g., for every row and column that's printed to stdout).
- ⑪ Initializes a complex number at the origin with real (re) and imaginary (im) parts at 0.0
- ⑫ Initializes a complex number from the coordinates provided as function arguments
- ⑬ Checks the escape condition and calculates the distance from the origin (0, 0), an absolute value of a complex number
- ⑭ Repeatedly mutates z to check whether c lies within the Mandelbrot set
- ⑮ As i is no longer in scope, we fall back to max_iters.

So far in this section, we've put the basics of Rust into practice. Let's continue to explore more function definitions and types.

2.8 ADVANCED FUNCTION DEFINITIONS

Rust's functions can get somewhat scarier than `add(i: i32, j: i32) -> i32` as shown in listing 2.13. To assist those who are reading more Rust source code than writing it, the following sections provide some extra content.

2.8.1 Explicit lifetime annotations

As a bit of forewarning, allow me to introduce some more complicated notation. As you look through Rust code developed by others, you might encounter definitions that are hard to decipher because these look like hieroglyphs from an ancient civilizations. The following listing provides an extract from listing 2.16 that shows one such example.

Listing 2.16. Demonstrating a function signature with explicit lifetime annotations

```
// following needs line break (55 char limit)
add_with_lifetimes<'a, 'b>(i: &'a i32, j: &'b i32) -> i32 ①
```

① `<'a, 'b>` introduces the lifetimes `'a` and `'b` within the scope of `add_with_lifetimes()`. `&'a i32` reads as “reference to an `i32` with lifetime `a`.”

In general terms, this extra information provides more specific information to the Rust compiler about data that lives outside the function. Functions that use references—denoted by the ampersands preceding the types—have data that exists outside of their scope. Rust wants to know whether the referent data should outlive the function call or, perhaps, be cleaned up when the function returns.

Objects that live outside of a function are checked to make sure that accessing these is valid throughout that function. That is, Rust checks to make sure that all input data lives at least as long as the function that needs it. The details of what is happening is explained through the next few chapters, but here are some broad principles.

Underpinning Rust’s safety checks is a *lifetime* system that verifies that all attempts to access data are valid. The lifetime system usually works unaided. Although present for every parameter, these checks are typically invisible as the compiler can infer these itself. [\[30\]](#) But the compiler needs assistance in difficult cases, such as when multiple references are accepted as arguments or when a reference is returned from a function.

When lifetime parameters are required, these appear within angle brackets (`<`, `>`) between the function’s name and its parameter list. Their labels (e.g., `'a` and `'b` as in listing 2.15) are arbitrary and local to the function that is being defined. Using different labels allows for the lifetimes of the two arguments to be distinct, but Rust does not require that.

Moving to the parameter list in listing 2.15 (`(i: &'a i32, j: &'b i32)`), we see lots of noise. `i: &'a i32` reads as “`i` is a reference to an `i32` with a lifetime `'a`”, and `j: &'b i32` reads as “`j` is a reference to an `i32` with a lifetime `'b`.” At this stage of the function definition, we are describing to Rust where the lifetime parameters introduced in the angle brackets are actually used.

No lifetime annotations are required when calling a function. When used in a complete example as in the following listing, you can see lifetime annotations at the function definition (line 1), but not when it's used (line 8). The source code for the listing is in `ch2-add-with-lifetimes.rs`.

Listing 2.17. Type signature of a function with lifetime-explicit annotations

```
fn add_with_lifetimes<'a, 'b>(i: &'a i32, j: &'b i32) -> i32 {  
    *i + *j    ①  
}  
  
fn main() {  
    let a = 10;  
    let b = 20;  
    let res = add_with_lifetimes(&a, &b);    ②  
  
    println!("{}", res);  
}
```

- ① Adds the values referred to by `i` and `j` rather than adding the references directly
- ② `&10` and `&20` mean “reference to 10” and “reference to 20” respectively. No lifetime notation is required when calling a function.

On line 2, `*i + *j` adds together the referent values held by the `i` and `j` variables. It's common to see lifetime parameters when using references. While Rust can infer lifetimes in other cases, references require the programmer to specify the intent. Using two lifetime parameters (`'a` and `b`) indicates that the lifetimes of `i` and `j` are decoupled.



Note

Lifetime parameters are a way of providing control to the programmer while maintaining high-level code.

2.8.2 Generic functions

Another special case of function syntax appears when programmers write Rust functions to handle many possible input types. So far, we have seen functions

that accept 32-bit integers (`i32`). The following listing shows a function signature that can be called by many input types, as long as these are all the same.

Listing 2.18. Type signature of a generic function

```
fn add<T>(i: T, j: T) -> T {    ①
    i + j
}
```

① The type variable `T` is introduced with angle brackets (`<T>`). This function takes two arguments of the same type and returns a value of that type.

Capital letters in place of a type indicate a *generic* type. Conventionally, the variables `T`, `U` and `V` are used as placeholder values, but this is arbitrary. `E` is often used to denote an error type. We'll look at error handling in detail in chapter 3. Generics enable significant code reuse and can greatly increase the usability of a strongly-typed language.

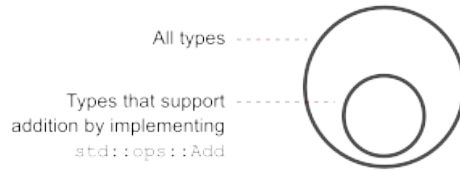
Unfortunately, listing 2.17 doesn't compile as is. The Rust compiler complains that it cannot add two values of any type `T` together. The following shows the output produced when attempting to compile listing 2.17:

```
error[E0369]: cannot add `T` to `T`
--> add.rs:2:5
  |
2 |   i + j
  |   ^ - T
  |   |
  |   T
  |
help: consider restricting type parameter `T`
  |
1 | fn add<T: std::ops::Add<Output = T>>(i: T, j: T) -> T {
  |           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
error: aborting due to previous error

For more information about this error, try `rustc --explain E0369`.
```

This issue arises because `T` really means any type at all, even types where addition is not supported.

Figure 2.3. Only a subset of types have implement operators. When creating generic functions that include such an operator, that operation’s trait must be included as a trait bound.



How do we specify that type `T` must implement addition? Answering this requires introducing some new terminology.

All of Rust’s operators, including addition, are defined within traits. To require that type `T` must support addition, we include a *trait bound* alongside the type variable in the function’s definition. The following listing gives an example of this syntax.

Listing 2.19. Type signature of a generic function with trait bounds

```
// please add line break to following (55 char limit)
fn add<T: std::ops::Add<Output = T>>(i: T, j: T) -> T {    ①
    i + j
}
```

① The fragment `<T: std::ops::Add<Output = T>>` requires that `T` must implement `Add` and that the resulting output must be the same type.

What is a trait? A *trait* is a language feature that is analogous to an interface, protocol, or contract. If you have a background in object-oriented programming, consider a trait to be an abstract base class. If you have a background in functional programming, Rust’s traits are close to Haskell’s type classes. For now, it’s enough to say that traits enable types to advertise that they are using common behavior. All of Rust’s operations are defined with traits. For example, the addition operator (`+`) is defined as the `std::ops::Add` trait. Traits are properly introduced in chapter 3 and are progressively explained in depth during

the course of the book.

To reiterate: all of Rust's operators are syntactic sugar for a trait's methods. Rust supports operator overloading this way. During the compilation process, `a + b` is converted to `a.add(b)`.

[Listing 2.20](#) is a full example that demonstrates that generic functions can be called by multiple types. The listing prints these three lines to the console:

```
4.6
30
15s
```

Listing 2.20. Defining a generic function with a type variable and trait bounds

```
use std::ops::{Add};           ①
use std::time::{Duration};     ②

fn add<T: Add<Output = T>>(i: T, j: T) -> T { ③
    i + j
}

fn main() {
    let floats = add(1.2, 3.4);    ④
    let ints = add(10, 20);        ⑤
    let durations = add(Duration::new(5, 0), Duration::new(10, 0)); ⑥
    // above line needs a line break (55 char max)

    println!("{}", floats);
    println!("{}", ints);
    println!("{:?}", durations); ⑦
}
```

- ① Brings the `Add` trait from `std::ops` into local scope
- ② Brings the `Duration` type from `std::time` into local scope
- ③ The arguments to `add()` can accept any type that implements `std::ops::Add`.
- ④ Calls `add()` with floating-point values
- ⑤ Calls `add()` with integer values
- ⑥ Calls `add()` with `Duration` values, representing a duration between two points in time

Because `std::time::Duration` does not implement the `std::fmt::Display` trait,

⑦ we can fall back to requesting `std::fmt::Debug`.

As you can see, function signatures can become somewhat convoluted. Interpreting these can take some patience. Hopefully, you now have the tools to break the pieces apart in case you get stuck down the track. Here are a few principles that should assist you when reading Rust code:

- Terms in lowercase (`i`, `j`) denote variables.
- Single uppercase letters (`T`) denote generic type variables.
- Terms beginning with uppercase (`Add`) are either traits or concrete types, such as `String` or `Duration`.
- Labels (`'a`) denote lifetime parameters.

[30] Within the Rust community, the ability to request that the compiler should infer lifetimes on its own is known as *lifetime elision*.

2.9 CREATING GREP-LITE

We've spent most of the chapter discussing numbers. It's time for another practical example. We'll use it to learn a little bit about how Rust handles text.

[Listing 2.21](#) is our first iteration of `grep-lite`. The code for this program is in the `ch2-str-simple-pattern.rs` file. Its hard-coded parameters restrict flexibility somewhat, but these are useful illustrations of string literals. The code prints a line to the console:

```
dark square is a picture feverishly turned--in search of what?
```

Listing 2.21. Searching for a simple pattern within lines of a string

```
fn main() {  
    let search_term = "picture";  
    let quote = "\nEvery face, every shop, bedroom window, public-house, and  
dark square is a picture feverishly turned--in search of what?  
// please add a line break to the following (55 char limit)  
It is the same with books. What do we seek through millions of pages?";    ①
```

```

for line in quote.lines() {           ②
    if line.contains(search_term) {
        println!("{}", line);
    }
}
}

```

① Multilined strings do not require special syntax. The `\` character on line 4 escapes the new line.

② `lines()` returns an iterator over `quote` where each iteration is a line of text. Rust uses each operating system’s conventions on what constitutes a new line.

As you can see, Rust’s strings can do quite a lot by themselves. Some features of listing 2.20 that are worth highlighting include the following. From here, we’ll expand the functionality of our proto-application.

- Line 7 (`quote.lines()`) demonstrates iterating line-by-line in a platform-independent manner.
- Line 8 (`line.contains()`) demonstrates searching for text using the method syntax.

Navigating Rust’s rich collection of string types

Strings are complicated for newcomers to Rust. Implementation details tend to bubble up from below and make comprehension difficult. How computers represent text is complicated, and Rust chooses to expose some of that complexity. This enables programmers to have full control but does place a burden on those learning the language.

`String` and `&str` both represent text, yet are distinct types. Interacting with values from both types can be an annoying exercise at first as different methods are required to perform similar actions. Prepare yourself for irritating type errors as your intuition develops. Until that intuition develops, you will usually have fewer issues if you convert your data to the `String` type.

A `String` is (probably) closest to what you know as a string type from other languages. It supports familiar operations such as concatenation (joining two strings together), appending new text on to an existing string, and trimming whitespace.

`str` is a high-performance, relatively feature-poor type. Once created, `str` values cannot expand or shrink. In this sense, these are similar to interacting with a raw memory array. Unlike a raw memory array though, `str` values are guaranteed to be valid UTF-8 characters.

`str` is usually seen in this form: `&str`. A `&str` (pronounced “string slice”) is a small type that contains a reference to `str` data and a length. Attempting to assign a variable to type `str` will fail. The Rust compiler

wants to create fixed-sized variables within a function's stack frame. As `str` values can be of arbitrary length, these can only be stored as local variables by reference.

For those readers that have prior experience with systems programming, `String` uses dynamic memory allocation to store the text that it represents. Creating `&str` values avoids a memory allocation.

`String` is an *owned* type. Ownership has a particular meaning within Rust. An *owner* is able to make any changes to the data and is responsible for deleting values that it owns when it leaves scope (this is fully explained in chapter 3). A `&str` is a *borrowed* type. In practical terms, this means that `&str` can be thought of as read-only data, whereas `String` is read/write.

`String` literals (e.g., "Rust in Action") have the type `&str`. The full type signature including the lifetime parameter is `&'static str`. The `'static` lifetime is somewhat special. It too owes its name to implementation details. Executable programs can contain a section of memory that is hard-coded with values. That section is known as *static memory* because it is read-only during execution.

Some other types may be encountered in your travels. Here's a short list: [\[31\]](#)

- `char`—A single character, encoded as 4 bytes. The internal representation of `char` is equivalent to UCS-4/UTF-32. This differs from `&str` and `String`, which encodes single characters as UTF-8. Conversion does impose a penalty, but means that `char` values are of fixed-width and are, therefore, easier for the compiler to reason about. Characters encoded as UTF-8 can span 1 to 4 bytes.
- `[u8]`—A slice of raw bytes, usually found when dealing with streams of binary data.
- `Vec<u8>`—A vector of raw bytes, usually created when consuming `[u8]` data. `String` is to `Vec<u8>` as `str` is to `[u8]`.
- `std::ffi::OSString`—A platform-native string. It's behavior is close to `String` but without a guarantee that it's encoded as UTF-8 and that it won't contain the zero byte (`0x00`).
- `std::path::Path`—A string-like type that is dedicated to handling file system paths.

Fully understanding the distinction between `String` and `&str` requires knowledge of arrays and vectors. Textual data is similar to these two types with added convenience methods applied over the top.

Let's start adding functionality to `grep-lite` by printing the line number, along with the match. This is equivalent to the `-n` option within the POSIX.1-2008 standard for the `grep` utility

(<http://pubs.opengroup.org/onlinepubs/9699919799/utilities/grep.html>).

Adding a few lines to our previous example, we now see the following line printed to the screen. Listing [Listing 2.22](#) shows the code that adds this functionality, which you'll find in `ch2-simple-with-linenums.rs`.

```
2: dark square is a picture feverishly turned--in search of what?
```

Listing 2.22. Searching for a pattern and printing line numbers by

manually incrementing a variable

```
fn main() {
    let search_term = "picture";
    let quote = "\
Every face, every shop, bedroom window, public-house, and
dark square is a picture feverishly turned--in search of what?
It is the same with books. What do we seek through millions of pages?";
    let mut line_num: usize = 1;                                ①

    for line in quote.lines() {
        if line.contains(search_term) {
            println!("{}", line_num, line);                    ②
        }
        line_num += 1;                                          ③
    }
}
```

- ① Declares `line_num` as mutable via `let mut` and initializes it with 1
- ② Updates the `println!` macro to allow for both values to be printed
- ③ Increments `line_num` in place

Listing 2.22 shows another approach to achieving this and demonstrates more of Rust's syntax. The output is the same, but here the code makes use of the `enumerate()` method and method chaining. `enumerate()` takes an iterator `I`, returning another `(N, I)`, where `N` is a number that starts at 0 and increments by 1 each iteration. The source code for this listing can be found in `ch2-simple-with-enumerate.rs`.

Listing 2.23. Searching for a pattern and printing line numbers using `enumerate()`

```
fn main() {
    let search_term = "picture";
    let quote = "\
// "\ OK in above line? (there's another in listing 2.25)
Every face, every shop, bedroom window, public-house, and
dark square is a picture feverishly turned--in search of what?
It is the same with books. What do we seek through millions of pages?";

    for (i, line) in quote.lines().enumerate() {                ①
        if line.contains(search_term) {
            let line_num = i + 1;                                ②
            println!("{}", line_num, line);
        }
    }
}
```

```
}
```

- ① Because `lines()` returns an iterator, it can be chained with `enumerate()`.
- ② Performs addition to calculate the line number, avoiding calculations at every step

Another feature of `grep` that is extremely useful is to print some context before and after the line that matches. In the GNU `grep` implementation, this is the `-c NUM` switch. To add support for that feature in `grep-lite`, we need to be able to create lists.

[31] Unfortunately, this is not an exclusive list. Specific use cases sometimes require special handling.

2.10 MAKING LISTS OF THINGS WITH ARRAYS, SLICES, AND VECTORS

Lists of things are incredibly common. The two types that you will be working with most often are arrays and vectors. Arrays are fixed-width and extremely lightweight. Vectors are growable, but incur a small run-time penalty because of the extra bookkeeping that these do. To understand the underlying mechanisms with text data in Rust, it helps to have a cursory understanding of what is happening.

The goal of this section is to support printing out n lines of context that surround a match. To get there, we need to segue somewhat and explain what arrays, slices, and vectors are. The most useful type for this exercise is the vector. To learn about vectors though, we need to start by learning about its two simpler cousins: arrays and slices.

2.10.1 Arrays

An *array*, at least as far as Rust is concerned, is a tightly-packed collection of the same thing. It's possible to replace items within an array, but its size cannot change. Because variable-length types like `String` add a degree of complication,

we'll revert back to discussing numbers for a little while.

Creating arrays takes two forms. We can provide a comma-delimited list within square brackets (e.g., `[1, 2, 3]`) or a *repeat expression*, where you furnish two values delimited by a semicolon (e.g., `[0; 100]`). The value on the left (`0`) is repeated by the number of times on the right (`100`).

Listing 2.23 shows all four variations. The source code for this listing is in the `ch2-3arrays.rs` file. It prints these four lines to the console:

```
[1, 2, 3]:      1 + 10 = 11    2 + 10 = 12    3 + 10 = 13    (Σ[1, 2, 3] = 6)
[1, 2, 3]:      1 + 10 = 11    2 + 10 = 12    3 + 10 = 13    (Σ[1, 2, 3] = 6)
[0, 0, 0]:      0 + 10 = 10    0 + 10 = 10    0 + 10 = 10    (Σ[0, 0, 0] = 0)
[0, 0, 0]:      0 + 10 = 10    0 + 10 = 10    0 + 10 = 10    (Σ[0, 0, 0] = 0)
```

Listing 2.24. Defining arrays and iterating over their elements

```
fn main() {
    let one          = [1, 2, 3];    ❶

    let two: [u8; 3]  = [1, 2, 3];    ❷

    let blank1        = [0; 3];       ❸

    let blank2: [u8; 3] = [0; 3];      ❹

    let arrays = [one, two, blank1, blank2];

    for a in &arrays {                ❺
        print!("{:?}", a);
        for n in a.iter() {           ❻
            print!("\t{} + 10 = {}", n, n+10);
        }

        let mut sum = 0;
        for i in 0..a.len() {
            sum += a[i];                ❼
        }
        println!("\t(Σ{:?} = {})", a, sum);
    }
}
```

❶ `[1, 2, 3]` denotes an array literal; Rust infers its type.

❷ `[u8; 3]` explicitly declares array type: 3 elements of `u8`.

❸ This form, `[0; 3]` is known as a repeat expression, which expects a constant (`0`) to be repeated `n` times (`3`).

- ④ Type signatures are also supported for repeat expressions.
- ⑤ Taking a reference to an array returns a slice, which support iteration through arrays without needing to call `iter()`.
- ⑥ Arrays also have methods for iteration and manipulation.
- ⑦ Array indexing is bounds checked. Requesting an item that's out of bounds crashes (panics) the program.

Arrays are a simple data structure from the machine's point of view. These are a contiguous block of memory with elements of a uniform type. The simplicity is still somewhat deceptive. Arrays can cause a few learning difficulties for newcomers:

- *The notation can be confusing.* `[T; n]` describes an array's type, where `T` is the elements' type and `n` is a non-negative integer. `[f32; 12]` denotes an array of 12 32-bit floating point numbers. It's easy to get confused with slices `[T]`, which do not have a compile-time length.
- *`[u8; 3]` is a different type than `[u8; 4]`.* That is, the size of the array matters to the type system.
- *In practice, most interaction with arrays occurs via another type called a slice (`[T]`).* The slice is itself interacted with by reference (`&[T]`). And to add some linguistic confusion into the mix, both slices and references to slices are called *slices*.

2.10.2 Slices

Slices are dynamically sized array-like objects. The term *dynamically sized* means that their size is not known at compile time. Yet, like arrays, these don't expand or contract. The use of the word "dynamic" in dynamically sized is closer in meaning to "dynamic typing" rather than movement. The lack of compile-time knowledge explains the distinction in the type signature between an array (`[T; n]`) and a slice (`[T]`).

Slices are important because it's easier to implement traits for slices than arrays.

Traits are how Rust programmers add methods to objects. As `[T; 1]`, `[T; 2]`, ..., `[T; n]` are all different types, implementing traits for arrays can become unwieldy. Creating a slice from an array is easy and cheap because it doesn't need to be tied to any specific size.

Another important use for slices is their ability to act as a view on arrays (and other slices). The term “view” here is taken from database technology and means that slices can gain fast, read-only access of data without needing to copy anything around.

The rub with slices is that Rust wants to know the size of every object in your program, and slices are defined as not having a compile-time size. References to the rescue. As mentioned in the discussion about the use of the term “dynamically sized,” slice size is fixed in memory. These are made up of two size components (a pointer and a length). That's why you typically see slices referred to in their referenced form, `&[T]` (like string slices that take the notation `&str`).



Note

Don't worry too much about the distinctions between arrays and slices yet. In practice, it's not material. Each term is an artifact of implementation details. Those implementation details are important when dealing with performance-critical code, but not when learning the basics of the language.

2.10.3 Vectors

Vectors (`Vec<T>`) are growable lists of `T`. Using vectors is extremely common in Rust code. These incur a small run-time penalty compared to arrays because of the extra bookkeeping that must be done to enable their size to change over time. But vectors often make up for this with their added flexibility.

The task at hand is to expand the feature set of the `grep-lite` utility. Specifically, we want the ability to store n lines of context around a match. Naturally, there are many ways to implement such a feature.

To minimize code complexity, we'll use a two-pass strategy. In the first pass, we'll tag lines that match. During the second pass, we'll collect lines that are within n lines of each of the tags.

The code in listing 2.24 is the longest you've seen so far. Take your time to digest it. You'll find the source for this listing in `ch2-introducing-vec.rs`.

The most confusing syntax in the listing is probably `Vec<Vec<(usize, String)>>`, which appears on line 15. `Vec<Vec<(usize, String)>>` is a vector of vectors (e.g., `Vec<Vec<T>>`), where T is a pair of values of type `(usize, String)`. `(usize, String)` is a tuple that we'll use to store line numbers along with the text that's a near match. When the `needle` variable on line 4 is set to `"oo"`, the following text prints onscreen:

```
1: Every face, every shop,  
2: bedroom window, public-house, and  
3: dark square is a picture  
4: feverishly turned--in search of what?  
3: dark square is a picture  
4: feverishly turned--in search of what?  
5: It is the same with books.  
6: What do we seek  
7: through millions of pages?
```

Listing 2.25. Enabling context lines to be printed out with a `Vec<Vec<T>>`

```
fn main() {  
    let context_lines = 2;  
    let needle = "oo";  
    let haystack = "\nEvery face, every shop,  
bedroom window, public-house, and  
dark square is a picture  
feverishly turned--in search of what?  
It is the same with books.  
What do we seek  
through millions of pages?";  
  
    let mut tags: Vec<usize> = Vec::new();    ①  
    // following line needs a line break (55 chars max)  
    let mut ctx: Vec<Vec<(usize, String)>> = Vec::new();    ②  
  
    for (i, line) in haystack.lines().enumerate() {    ③  
        if line.contains(needle) {  
            tags.push(i);  
        }  
    }  
}
```

```

// following line needs a line break (55 chars max)
    let v = Vec::with_capacity(2*context_lines + 1);    ④
    ctx.push(v);
}
}

if tags.empty() {    ⑤
    return;
}

for (i, line) in haystack.lines().enumerate() {    ⑥
    for (j, tag) in tags.iter().enumerate() {
// following line needs a line break (55 chars max)
        let lower_bound = tag.saturating_sub(context_lines);    ⑦
        let upper_bound = tag + context_lines;

        if (i >= lower_bound) && (i <= upper_bound) {
            let line_as_string = String::from(line);    ⑧
            let local_ctx = (i, line_as_string);
            ctx[j].push(local_ctx);
        }
    }
}

for local_ctx in ctx.iter() {
    for &(i, ref line) in local_ctx.iter() {    ⑨
        let line_num = i + 1;
        println!("{}", line_num, line);
    }
}
}

```

- ① tags holds line numbers where matches occur.
- ② ctx contains a vector per match to hold the context lines.
- ③ Iterates through the lines, recording line numbers where matches are encountered
- ④ Vec::with_capacity(n) reserves space for n items. No explicit type signature is required as it can be inferred via the definition of ctx on line 15.
- ⑤ When there are no matches, exits early
- ⑥ For each tag, at every line, check to see if we are near a match. When we are, add that line to the relevant Vec<T> within ctx.
- ⑦ use.saturating_sub() is subtraction that returns 0 on integer underflow rather than crashing the program (CPUs don't like attempting to send use below zero).
- ⑧ Copies line into a new String and stores that locally for each match
- ⑨ ref line informs the compiler that we want to borrow this value rather than move it. These two terms are explained fully in later chapters.

`Vec<T>` performs best when you can provide it with a size hint via `Vec::with_capacity()`. Providing an estimate minimizes the number of times memory will need to be allocated from the operating system.



Note

When considering this approach in real text files, encodings can cause issues. `String` is guaranteed to be UTF-8. Naively reading in a text file to a `String` causes errors if invalid bytes are detected. A more robust approach is to read in data as `[u8]` (a slice of `u8` values), then decode those bytes with help from your domain knowledge.

2.11 INCLUDING THIRD-PARTY CODE

Incorporating third-party code is essential to productive Rust programming. Rust's standard library tends to lack many things that other languages provide; things like random number generators and regular expression support. To get your feet wet, you'll include the `regex` crate into our program.

Crates are the name the Rust community uses where others use terms such as package, distribution, or library. `regex` provide the ability to match for regular expressions rather than simply looking for exact matches.

To use third-party code, we'll rely on the `cargo` command-line tool. Follow these instructions:

1. Open a command prompt, using `cmd.exe`, `Bash`, `zsh` or your own personal favorite
2. Type `cd /tmp` (`cd %TMP%` on MS Windows)
3. Press Enter and then type `cargo new grep-lite`

You now should see the following line appear:

```
Created binary (application) `grep-lite` package
```

Now, open the file `/tmp/grep-lite/Cargo.toml` in a text editor. The section

[dependencies] should be updated to look like this:

Listing 2.26. grep-lite project metadata

```
[dependencies]
regex = "1"
```

We've now done all we need to do to enable cargo to pull down this code and all of its dependencies. This gives us a working copy for our project. Open your command prompt again and execute these commands:

```
cd /tmp/grep-lite (cd %TMP%\grep-lite on MS Windows) cargo build
```

You should see output fairly similar to the following begin to appear:

Listing 2.27. Output from cargo build

```
Updating crates.io index
Downloaded regex v1.3.6
Compiling lazy_static v1.4.0
Compiling regex-syntax v0.6.17
Compiling thread_local v1.0.1
Compiling aho-corasick v0.7.10
Compiling regex v1.3.6
Compiling grep-lite v0.1.0 (/tmp/grep-lite)
Finished dev [unoptimized + debuginfo] target(s) in 4.47s
```

Now that you have the crate installed and compiled, let's put it into action. First, we add support of regular expressions.

2.11.1 Adding support for regular expressions

Regular expressions add great flexibility to the patterns that we are able to search for. Here is a copy of an early example that we'll modify:

Listing 2.28. Matching on exact strings with the contains() method

```
fn main() {
    let search_term = "picture";
    let quote = "Every face, every shop, bedroom window, public-house, and
dark square is a picture feverishly turned--in search of what?
It is the same with books. What do we seek through millions of pages?";
```

```

for line in quote.lines() {
    if line.contains(search_term) {      ❶
        println!("{}", line);
    }
}
}

```

❶ Implements a `contains()` method that searches for a substring

Make sure that you have updated `grep-lite/Cargo.toml` to include `regex` as a dependency as described in the previous section. Now, open `grep-lite/src/main.rs` in a text editor and fill it in with the code in the following listing. The source code for this listing is available in `ch2/ch2-with-regex.rs`.

Listing 2.29. Searching for patterns with regular expressions

```

use regex::Regex;      ❶

fn main() {
    let re = Regex::new("picture").unwrap();      ❷

    let quote = "Every face, every shop, bedroom window, public-house, and
dark square is a picture feverishly turned--in search of what?
It is the same with books. What do we seek through millions of pages?";

    for line in quote.lines() {
        let contains_substring = re.find(line);
        match contains_substring {      ❸

            Some(_) => println!("{}", line),      ❹
            None => (),      ❺

        }
    }
}

```

❶ Brings the `Regex` type from the `regex` crate into local scope

❷ `unwrap()` unwraps a `Result`, crashing if an error occurs. Handling errors more robustly is discussed in depth later in the book.

❸ Replaces the `contains()` method from listing 2.26 with a `match` block that requires that we handle all possible cases

❹ `Some(T)` is the positive case of an `Option`, meaning that `re.find()` was successful: `_` matches all values.

❺ `None` is the negative case of an `Option`; `()` can be thought of as a null placeholder value here.

Open a command prompt and move to the root directory of your grep-lite project. Executing `cargo run` should produce output similar to this:

Listing 2.30. Output of `cargo run`

```
$ cargo run
  Compiling grep-lite v0.1.0 (file:///tmp/grep-lite)
    Finished dev [unoptimized + debuginfo] target(s) in 0.48s
    Running `target/debug/grep-lite`
dark square is a picture feverishly turned--in search of what?
```

Admittedly, the code within listing 2.27 hasn't taken significant advantage of its new-found regular expression capabilities. Hopefully, you'll have the confidence to be able to slot those into some of the more complex examples.

2.11.2 Generating the third-party crate documentation locally

Documentation for third-party crates is typically available online. Still, it can be useful to know how to generate a local copy in case the Internet fails you:

1. Move to the root of the project directory in a terminal: `/tmp/grep-lite` or `%TMP%\grep-lite`
2. Execute `cargo doc`.

Listing 2.31. Output of `cargo doc`

```
$ cargo doc
  Checking lazy_static v1.4.0
 Documenting lazy_static v1.4.0
  Checking regex-syntax v0.6.17
 Documenting regex-syntax v0.6.17
  Checking memchr v2.3.3
 Documenting memchr v2.3.3
  Checking thread_local v1.0.1
  Checking aho-corasick v0.7.10
 Documenting thread_local v1.0.1
 Documenting aho-corasick v0.7.10
  Checking regex v1.3.6
 Documenting regex v1.3.6
 Documenting grep-lite v0.1.0 (file:///tmp/grep-lite)
    Finished dev [unoptimized + debuginfo] target(s) in 3.43s
```

Congratulations. You have now created HTML documentation. By opening `/tmp/grep-lite/target/doc/grep_lite/index.html` in a web browser (also try `cargo doc --open` from the command line), you'll be able to view the documentation for all the crates that yours depend on. It's also possible to inspect the output directory to take a look at what is available to you; for example

```
$ tree -d -L 1 target/doc/  
target/doc/  
├── aho_corasick  
├── grep_lite  
├── implementors  
├── memchr  
├── regex  
├── regex_syntax  
├── src  
└── thread_local
```

2.11.3 Managing Rust toolchains with rustup

`rustup` is another handy command-line tool, along with `cargo`. Where `cargo` manages projects, `rustup` manages your Rust installation(s). `rustup` cares about Rust toolchains and enables you to move between versions of the compiler. This way it's possible to compile your projects for multiple platforms and experiment with nightly features of the compiler, while keeping the stable version nearby.

`rustup` also simplifies accessing Rust's documentation. Typing `rustup doc` opens your web browser to a local copy of Rust's standard library.

2.12 SUPPORTING COMMAND-LINE ARGUMENTS

Our program is rapidly increasing its feature count. Yet, there is no way for any options to be specified. To become an actual utility, `grep-lite` needs to be able to interact with the world.

Sadly though, Rust has a fairly tight standard library. As with regular expressions, another area with relatively minimalist support is handling command-line arguments. A nicer API is available through a third-party crate

(among others) called clap. Now that we've seen how to bring in third-party code, let's take advantage of that to enable users of grep-lite to choose their own pattern. (We'll get to choosing their own input source in the next section.) First, add clap as a dependency in your Cargo.toml file as the following listing shows.

Listing 2.32. grep-lite/Cargo.toml

```
[package]
name = "grep-lite"
version = "0.1.0"
authors = ["Tim McNamara <author@rustinaction.com>"]

[dependencies]
regex = "1"
clap = "2"
```

Now, adjust src/main.rs. The following listing shows how to do this.

Listing 2.33. grep-lite/src/main.rs

```
use regex::Regex;
use clap::{App, Arg};    ❶

fn main() {
    let args = App::new("grep-lite")    ❷
        .version("0.1")
        .about("searches for patterns")
        .arg(Arg::with_name("pattern")
            .help("The pattern to search for")
            .takes_value(true)
            .required(true))
        .get_matches();

    let pattern = args.value_of("pattern").unwrap();    ❸
    let re = Regex::new(pattern).unwrap();

    let quote = "Every face, every shop, bedroom window, public-house, and
dark square is a picture feverishly turned--in search of what?
It is the same with books. What do we seek through millions of pages?";

    for line in quote.lines() {
        match re.find(line) {
            Some(_) => println!("{}", line),
            None => (),
        }
    }
}
```

❶ Brings clap::App and clap::Arg objects into local scope

- ② Incrementally builds a command argument parser, where each argument takes an Arg. In our case, we only need one.
- ③ Extracts the pattern argument

Now running `cargo run` should set off a few lines in your console:

```
$ cargo run
  Finished dev [unoptimized + debuginfo] target(s) in 2.21 secs
  Running `target/debug/grep-lite`
error: The following required arguments were not provided:
  <pattern>

USAGE:
  grep-lite <pattern>

For more information try --help
```

The error is due to the fact that we haven't passed sufficient arguments through to our resulting executable. To pass arguments through, cargo supports some special syntax. Any arguments appearing after `--` are sent through to the build's result:

```
$ cargo run -- picture
  Finished dev [unoptimized + debuginfo] target(s) in 0.0 secs
  Running `target/debug/grep-lite picture`
dark square is a picture feverishly turned--in search of what?
```

But clap does more than provide parsing. It also generates usage documentation on your behalf. Running `grep-lite --help` provides an expanded view:

```
$ ./target/debug/grep-lite --help
grep-lite 0.1
searches for patterns

USAGE:
  grep-lite <pattern>

FLAGS:
  -h, --help          Prints help information
  -V, --version        Prints version information

ARGS:
  <pattern>    The pattern to search for
```

2.13 READING FROM FILES

Searching for text wouldn't be complete without being able to search within files. File I/O can be surprisingly finicky and so has been left until last.

Before adding this functionality to `grep-lite`, let's take a look at a standalone example in listing 2.30. The code for this listing is in the `ch2-read-file.rs` file.

The general pattern is to open a `File` object, then wrap that in a `BufReader`. `BufReader` takes care of providing *buffered I/O*, which can reduce system calls to the OS if the hard disk is congested.

Listing 2.34. Reading a file manually line by line

```
use std::fs::File;
use std::io::BufReader;
use std::io::prelude::*;

fn main() {
    let f = File::open("readme.md").unwrap();    ①
    let mut reader = BufReader::new(f);

    let mut line = String::new();    ②

    loop {    ③
        let len = reader.read_line(&mut line).unwrap();    ④
        if len == 0 {
            break
        }

        println!("{}", ({} bytes long)", line, len);

        line.truncate(0);    ⑤
    }
}
```

- ① Creates a `File` object that requires a path argument and error handling if the file does not exist. This program crashes if a `readme.md` is not present.
- ② Reuses a single `String` object over the lifetime of the program
- ③ Loops until the program encounters `return`, `break`, or panics
- ④ Because reading from disk can fail, we need to explicitly handle this. In our case, errors crash the program.
- ⑤ Shrinks the `String` back to length 0, preventing lines from persisting into the following ones

Manually looping through a file can be cumbersome, despite its usefulness in some cases. For the common case of iterating through lines, Rust provides a helper iterator as the following listing shows. The source code for this listing is in the file `ch2-bufreader-lines.rs`.

Listing 2.35. Reading a file line by line via `BufReader::lines()`

```
use std::fs::File;
use std::io::BufReader;
use std::io::prelude::*;

fn main() {
    let f = File::open("readme.md").unwrap();
    let reader = BufReader::new(f);

    for line_ in reader.lines() {      ①
        let line = line_.unwrap();    ②
        println!("{}", ({} bytes long)", line, line.len());
    }
}
```

- ① A subtle behavior change occurs here. `BufReader::lines()` removes the trailing newline character from each line.
- ② Unwraps the `Result`, but a possible error at each line is still required as with the manual version.

We're now in a position to add reading from a file into `grep-lite`'s feature list. The following listing creates a complete program that takes a regular expression pattern and an input file as arguments.

Listing 2.36. `grep-lite/src/main.rs`

```
use std::fs::File;
use std::io::BufReader;
use std::io::prelude::*;
use regex::Regex;
use clap::{App, Arg};

fn main() {
    let args = App::new("grep-lite")
        .version("0.1")
        .about("searches for patterns")
        .arg(Arg::with_name("pattern")
            .help("The pattern to search for"))
```

```

        .takes_value(true)
        .required(true)
        .arg(Arg::with_name("input")
            .help("File to search")
            .takes_value(true)
            .required(true))
        .get_matches();

let pattern = args.value_of("pattern").unwrap();
let re = Regex::new(pattern).unwrap();

let input = args.value_of("input").unwrap();
let f = File::open(input).unwrap();
let reader = BufReader::new(f);

for line_ in reader.lines() {
    let line = line_.unwrap();
    match re.find(&line) {
        Some(_) => println!("{}", line),
        None => (),
    }
}
}

```

① line is a String, but `re.find()` takes an `&str` as an argument.

2.14 READING FROM STDIN

A command-line utility wouldn't be complete if it wasn't able to read from STDIN. Unfortunately for those readers who skimmed over earlier parts of this chapter, some of the syntax on line 8 might look quite unfamiliar. In short, rather than duplicate code within `main()`, we'll use a generic function to abstract away the details of whether we are dealing with files or STDIN.

```

use std::fs::File;
use std::io;
use std::io::BufReader;
use std::io::prelude::*;
use regex::Regex;
use clap::{App, Arg};

fn process_lines<T: BufRead + Sized>(reader: T, re: Regex) {
    for line_ in reader.lines() {
        let line = line_.unwrap();
        match re.find(&line) {
            Some(_) => println!("{}", line),
            None => (),
        }
    }
}

```

```

fn main() {
    let args = App::new("grep-lite")
        .version("0.1")
        .about("searches for patterns")
        .arg(Arg::with_name("pattern")
            .help("The pattern to search for")
            .takes_value(true)
            .required(true))
        .arg(Arg::with_name("input")
            .help("File to search")
            .takes_value(true)
            .required(false))
        .get_matches();

    let pattern = args.value_of("pattern").unwrap();
    let re = Regex::new(pattern).unwrap();

    let input = args.value_of("input").unwrap_or("-");

    if input == "-" {
        let stdin = io::stdin();
        let reader = stdin.lock();
        process_lines(reader, re);
    } else {
        let f = File::open(input).unwrap();
        let reader = BufReader::new(f);
        process_lines(reader, re);
    }
}

```

2.15 SUMMARY

- Rust has full support for primitive types, such as integers and floating point.
- Functions are strongly typed and require both types for their parameters and return values.
- List-like types are tailored to specific use cases. You will typically reach for `Vec<T>` first.
- All Rust programs have a single entry function: `main()`.
- Every crate has a `Cargo.toml` file that specifies its metadata.
- The cargo tool is able to compile your code and fetch its dependencies.
- The rustup tool provides access to multiple compiler toolchains and to the language's documentation.

3 COMPOUND DATA TYPES

In this chapter:

- Composing data with structs
- Creating enumerated data types
- Add methods to types
- Handling errors in a type-safe manner
- Defining and implementing common behavior with traits
- Understanding how to keep implementation details private
- Using cargo to build documentation

Welcome to chapter 3. If we spent the last chapter looking at Rust's atoms, this chapter is focused more on its molecules.

This chapter focuses on two key building blocks for Rust programmers, `struct` and `enum`. Both are forms of *compound data types*. That is, both `struct` and `enum` can compose other types together to create something more useful than those other types would be alone. Consider how a 2D point (x, y) is composed from two numbers, x and y . We wouldn't want to maintain two variables x and y in our program. Instead, we would like to refer to the point as a whole entity.

We also discuss how to add methods to types with `impl` blocks. Lastly, we take a good look at *traits*, Rust's system for defining interfaces. [\[32\]](#)

Throughout this chapter, you'll be working through how to represent files in code. Although conceptually simple—if you're reading this book, it's highly likely you've interacted with a file through code before—there are enough edge cases to make things interesting.

Our strategy will be to create mock version of everything using our own

imaginary API. Towards the latter part of the chapter you'll learn how to interact with your actual operating system and its file system(s).

3.1 USING PLAIN FUNCTIONS TO EXPERIMENT WITH AN API

To start us off, let's see how far we can get by making use of the tools that we've already know. [Listing 3.1](#) lays out a few things that we would expect, such as opening and closing a "file". We'll use a rudimentary mock type to model one: a simple alias around `String` that holds a filename and little else.

To make things slightly more interesting than writing lots of boilerplate code, [Listing 3.1](#) does sprinkle in a few new concepts. They show you how to tame the compiler while you're experimenting with your design. Attributes are provided to relax compiler warnings and there the `read` function illustrates how to define a function that never returns. The code doesn't actually do anything, however. That will come very shortly.

Listing 3.1. Using type aliases to stub out a type (ch3-not-quite-file-1.rs)

```
#![allow(unused_variables)] ①

type File = String; ②

fn open(f: &mut File) -> bool {
    true ③
}

fn close(f: &mut File) -> bool {
    true ③
}

#[allow(dead_code)] ④
fn read(f: &mut File, save_to: &mut Vec<u8>) -> ! { ⑤
    unimplemented!() ⑥
}

fn main() {
    let mut f1 = File::from("f1.txt"); ⑦
    open(&mut f1);
    //read(f1, vec![]); ⑧
    close(&mut f1);
}
```

- ① Relax compiler warnings while working through ideas
- ② Create a type alias. The compiler won't distinguish between `String` & `File`, but your source code will.
- ③ Let's assume for the moment that these two functions will always succeed
- ④ Relaxes a compiler warning about an unused function
- ⑤ Using `!` as a return type indicates to the Rust compiler that this function never returns. `!` is known as the "Never" type. See the [Special return types in Rust](#) sidebar.
- ⑥ A macro that crashes the program if it is encountered
- ⑦ With the type declaration at line 3, `File` "inherits" all of `String`'s methods
- ⑧ There's little point in calling this method

There is *lots* that needs to be built upon from [Listing 3.1](#).

- We haven't created a persistent object that could represent a file (there's only so much that can be encoded in a string)
- There was no attempt at implementing `read()` (and if we did, how would we handle the failure case?)
- `open()` and `close()` return `bool`; perhaps there is a way to provide a more sophisticated result type that might be able to contain an error message if the operating system reports one?
- None of our functions are methods. From a stylistic point of view, it might be nice to call `f.open()` rather than `open(f)`.

Let's begin at the top and work our way through this list. Brace yourself for a few scenic detours along the way as we encounter a few learning side roads that will be profitable to explore.

Special return types in Rust

Some return types are difficult to interpret if you are new to the language. They're also especially difficult to search for, because they are made from symbols rather than words.

- `()` - Known as the "unit type". Formally is a zero-length tuple. It is used to express that a

function returns no value. Functions that appear to have no return type return (). Expressions that are terminated with semi-colon (;) return ().

Listing 3.2. Return the unit type implicitly

```
use std::fmt::Debug;

fn report<T: Debug>(item: T) {                                ①
    println!("{:?}", item);                                  ②
}
```

- ① `item` may be any type that implements `std::fmt::Debug`.
- ② The `{:?}` syntax directs the `println!` macro to use `std::fmt::Debug` trait to convert `item` to a printable string.

Listing 3.3. Return the unit type explicitly

```
fn clear(text: &mut String) -> () {
    *text = String::from("");
}
```

①

- ① Replace the string at text with an empty string.

The unit type often occurs in error messages. It's common to forget that the last expression of a function shouldn't end with a semi-colon.

- ! - Known as the “Never type”. Never is used to indicate that a function will never return.

Listing 3.4. When a function is guaranteed to crash, it will never return

```
fn dead_end() -> ! {
    panic!("you have reached a dead end"); ①
}
```

- ① The panic! macro causes the program to crash. That means the function is guaranteed never to return.

Listing 3.5. Creating an infinite loop prevents the function from returning

```
fn forever() -> ! {
  loop {
    //...
  };
}
```

①

① Unless it contains a `break`, loop will never finish. This prevents the function from returning.

Like with `unit`, `Never` sometimes occurs within error messages. The Rust compiler will complain about mismatched types when you forget to add a `break` in your loop block when you've indicated that the function returns a non-`Never` type.

[32] Or protocols, or contracts, or perhaps another term depending on your programming background.

3.2 MODELLING FILES WITH `STRUCT`

We need something to represent the thing we're trying to model. `struct` allows you to create a composite type made up of other types. Depending on your programming heritage, you may be more familiar with terms such as object or record.

We'll start with requiring that our files have a name and zero or more bytes of data.

[Listing 3.6](#) prints the following two lines to the console:

```
File { name: "f1.txt", data: [] }  
f1.txt is 0 bytes long
```

To represent data, the listing uses `Vec<u8>`. That is, a growable list of `u8` (single byte) values. The bulk of the `main()` function demonstrates usage, such as field access.

Listing 3.6. Defining and creating an instance of a `struct` to represent files (`ch3-mock-file.rs`)

```
#[derive(Debug)] ①  
struct File {  
    name: String,  
    data: Vec<u8>, ②  
}
```



```
fn main() {
    let f1 = File {
        name: String::from("f1.txt"), ③
        data: Vec::new(), ④
    };

    let f1_name = &f1.name; ⑤
    let f1_length = &f1.data.len(); ⑤

    println!("{:?}", f1);
    println!("{}", f1 is {} bytes long", f1_name, f1_length);
}
```

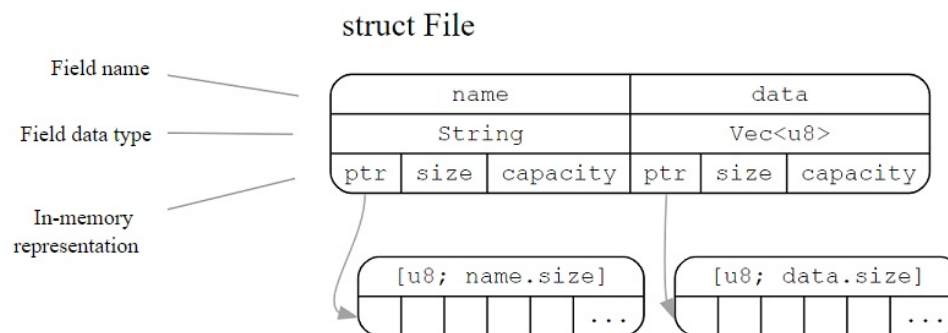
- ① Allows the `println!` macro to print `File`. The `std::fmt::Debug` trait works in conjunction with the `{:?}` mini-language within the macro to enable `File` to be represented as a printable string.
- ② Using `Vec<u8>` provides access to some useful conveniences, such as dynamic sizing so that it will be possible to simulate writing to a file
- ③ `String::from` allows owned strings to be generated from string literals (which are slices)
- ④ We'll use the `vec!` macro to simulate an empty file
- ⑤ Accessing fields uses the "dot operator" (`.`); Accessing fields by reference prevents use after move issues.

Here is a detailed overview of [Listing 3.6](#):

- Lines 1-5 define the `File` struct. Definitions include fields and their associated types. They also include each field's lifetimes, which happened to be elided here. Explicit lifetimes are required when a field is a reference to another object.
- Lines 8-11 create our first instance of `File`. We use a literal syntax here, but typically structs in the wild will be created via a convenience method. `String::from()` is one of those convenience methods. It takes a value of another type, in this case a string slice (`&str`) and returns a `String` instance. `Vec::new()` is the more common case.
- Lines 13-17 demonstrate accessing our new instance's fields. We prepend an ampersand, indicating that we wish to access this data by reference. In Rust parlance, this means that the variables `f1_name` and `f1_length` are borrowing the data they refer to.

You have probably noticed that our `File` struct doesn't actually store anything to disk at all. That's actually okay for now. If you're interested in its internals, you are welcome to review [Figure 3.1](#). Its two fields (`name` and `data`) are themselves both created by structs. If you're unfamiliar with the term pointer (`ptr`), think of it as an address to some memory that we don't know beforehand. This topic is discussed in the chapter "Data in Depth".

Figure 3.1. Inspecting the internals of the `File` struct



We'll leave interacting with the hard disk drive or other persistent storage until later in the chapter. For the meantime, let's re-create [Listing 3.1](#) with the `File` type that we have created.

The newtype pattern

Sometimes, the `type` keyword is all that you need. But what about when you need to compiler to treat your new "type" as a fully-fledged, distinct type, rather than just an alias? Enter "newtype".

The newtype pattern consists of wrapping a core type within a single-field struct or perhaps a tuple.

Listing 3.7. Using the newtype pattern to distinguish URLs from ordinary strings (ch3-newtype-pattern.rs)

```
#[derive(PartialEq)] ①
struct Hostname(String); ②

fn main() {
    let ordinary_string = String::from("localhost");
    let host = Hostname ( ordinary_string.clone() );
    if host == ordinary_string { ③
        println!("huh?"); ④
    };
}
```

- ① `PartialEq` enables types to be compared for equality. It's called partial to enable certain types to describe situations where equality is not valid, such as floating point's "Not a Number" value.
- ② `Hostname` is our newtype
- ③ This line won't compile because the compiler understands that `Hostname` and `String` are distinct
- ④ Will never happen

Here is the compiler output:

Listing 3.8. Output from `rustc` when attempting to compile [Listing 3.7](#)

```
error[E0308]: mismatched types
--> ch3-newtype-pattern.rs:7:16
|
7 |     if host == ordinary_string { ①
|                                ^^^^^^^^^^^^^^^^^ expected struct `Hostname`, found struct
|                                `std::string::String`
|
= note: expected type `Hostname`
       found type `std::string::String`

error: aborting due to previous error
```

We can now add a little bit of functionality to the first listing of the chapter. [Listing 3.9](#) adds the ability to "read" a file that has some data in it. All functions are assumed to always succeed and the code is still littered with hard-coded values. Still, the code finally prints something to the screen. Here is partially obscured output from the program:

```
File { name: "2.txt", data: [114, 117, 115, 116, 33] }
2.txt is 5 bytes long
***** ①
```

- ① Revealing this line would spoil all of the fun!

Listing 3.9. Using a `struct` to Mimic a File and Simulate Reading

its Contents. Converts opaque data into a string. (ch3-not-quite-file-2.rs)

```
#![allow(unused_variables)] ①

#[derive(Debug)] ②
struct File {
    name: String,
    data: Vec<u8>,
}

fn open(f: &mut File) -> bool { ③
    true
}

fn close(f: &mut File) -> bool { ③
    true
}

fn read(f: &File, save_to: &mut Vec<u8>) -> usize { ④
    let mut tmp = f.data.clone(); ⑤
    let read_length = tmp.len();
    save_to.reserve(read_length); ⑥
    save_to.append(&mut tmp); ⑦
    read_length
}

fn main() {
    let mut f2 = File {
        name: String::from("2.txt"),
        data: vec![114, 117, 115, 116, 33],
    };

    let mut buffer: Vec<u8> = vec![];

    open(&mut f2); ⑧
    let f2_length = read(&f2, &mut buffer); ⑧
    close(&mut f2); ⑧

    let text = String::from_utf8_lossy(&buffer); ⑨

    println!("{:?}", f2);
    println!("{}", f2.name, f2_length);
    println!("{}", text) ⑩
}
```

- ① Silences a warnings caused by open() and close() not making use of needing their argument
- ② This enables File to work with println! and its fmt! sibling macros, used at the bottom of the code listing
- ③ These two functions will remain inert for now
- ④ Return the "number of bytes read"

- ⑤ Make a copy of the data here, as `save_to.append()` will shrink the input `Vec<T>`
- ⑥ Not strictly necessary, but useful to know about. Ensures that there is sufficient space to fit the incoming data and minimizes allocations when data is inserted byte-by-byte
- ⑦ Allocate sufficient data in the `save_to` buffer to hold the contents of `f`
- ⑧ Do the hard work of interacting the file.
- ⑨ Convert `Vec<u8>` to `String`. Any bytes that are not valid UTF-8 are replaced with 
- ⑩ View the bytes 114, 117, 115, 116 & 33 as an actual word

The code so far has tackled 2 of the 4 issues raised at the end of [Listing 3.1](#). Our `File` struct is a *bona fide* type. `read()` is implemented, albeit in a memory-inefficient manner. The last two points remain.

3.3 ADDING METHODS TO A STRUCT WITH `impl`

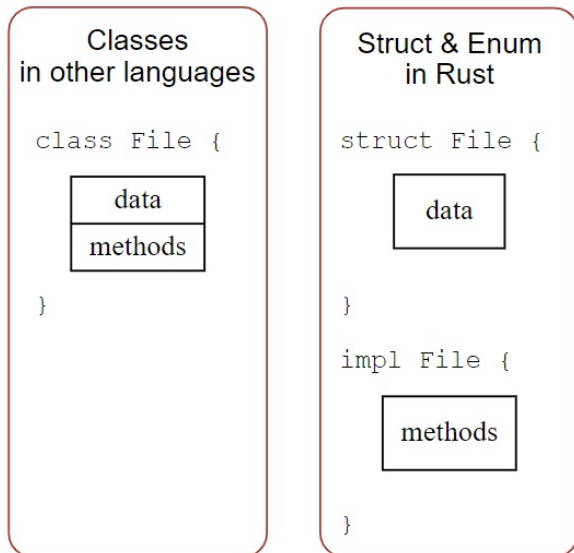
This section explains briefly what methods are and describes how to make use of them in Rust.

Methods are functions that are coupled to some object. From a syntactic point of view, they're just functions that don't need to specify one of their arguments. Rather than calling `open()` and passing a `File` object in as an argument (`read(f, buffer)`), methods allow the main object to be implicit in the function call (`f.read(buffer)`) using the dot operator. [\[33\]](#)

Rust is different than other languages that support methods: there is no `class` keyword. Types created with `struct`—and `enum`, which is described further on—feel like classes at times, but as they don't support inheritance it's probably a good thing that they're named something different.

To define methods, Rust programmers use an `impl` block that are physically distinct in source code from the `struct` (and `enum`) blocks that you have already encountered.

Figure 3.2. Illustrating syntactic differences between Rust and most object oriented languages. Within Rust, methods are defined separately from fields.



3.3.1 Simplifying object creation by implementing a `new()` method

Creating objects with reasonable defaults is done through the `new()` method.

Every struct can be instantiated through a literal syntax. This is handy for getting started, but leads to unnecessary verbosity in most code.

Using `new()` is a convention within the Rust community. Unlike other languages, `new` is not a keyword and isn't given some sort of blessed status above other methods.

Table 3.1. Comparing Rust's literal syntax for creating objects with the use of `new()` methods

Current	With <code>File::new()</code>
<pre>File { name: String::from("f1.txt"), data: Vec::new(), };</pre>	<pre>File::new("f1.txt", vec![]);</pre>

<pre>File { name: String::from("f2.txt"), data: vec![114, 117, 115, 116, 33], };</pre>	<pre>File::new("f2.txt", vec![114, 117, 115, 116, 33]);</pre>
--	---

To enable these changes, you make use of an `impl` block, shown below in [Listing 3.10](#). The resulting executable should print out the same message as [Listing 3.6](#), substituting "f3.txt" for the original's "f1.txt".

Listing 3.10. Using `impl` to add methods to a struct (ch3-defining-files-neatly.rs)

```
#[derive(Debug)]
struct File {
  name: String,
  data: Vec<u8>,
}

impl File {
  fn new(name: &str) -> File { ①
    File { ②
      name: String::from(name), ②
      data: Vec::new(), ②
    }
  }
}

fn main() {
  let f3 = File::new("f3.txt");

  let f3_name = &f3.name; ③
  let f3_length = f3.data.len();

  println!("{:?}", f3);
  println!("{}", f3_name, f3_length);
}
```

- ① As `File::new()` is a completely normal function—rather than something blessed by the language—we need to tell Rust that it will be returning a `File` from this function
- ② `File::new()` does little more than encapsulate the object creation syntax. This is normal
- ③ Fields are private by default, but can be accessed within the module that defines the struct. The module system is discussed further on in the chapter

Merging this new knowledge with the example that we already have, [Listing 3.11](#) is the result. It prints

```
File { name: "2.txt", data: [114, 117, 115, 116, 33] }  
2.txt is 5 bytes long  
***** ①
```

① Still hidden!

to the console.

Listing 3.11. Using `impl` to improve the ergonomics of `File` (ch3-defining-files-neatly.rs)

```
#![allow(unused_variables)]  
  
#[derive(Debug)]  
struct File {  
    name: String,  
    data: Vec<u8>,  
}  
  
impl File {  
    fn new(name: &str) -> File {  
        File {  
            name: String::from(name),  
            data: Vec::new(),  
        }  
    }  
  
    fn new_with_data(name: &str, data: &Vec<u8>) -> File { ①  
        let mut f = File::new(name);  
        f.data = data.clone();  
        f  
    }  
  
    fn read(self: &File, save_to: &mut Vec<u8>) -> usize { ②  
        let mut tmp = self.data.clone();  
        let read_length = tmp.len();  
        save_to.reserve(read_length);  
        save_to.append(&mut tmp);  
        read_length  
    }  
}  
  
fn open(f: &mut File) -> bool { ③  
    true  
}  
  
fn close(f: &mut File) -> bool { ③  
    true
```



```

}

fn main() {
    let f3_data: Vec<u8> = vec![114, 117, 115, 116, 33]; ④
    let mut f3 = File::new_with_data("2.txt", &f3_data);

    let mut buffer: Vec<u8> = vec![];

    open(&mut f3);
    let f3_length = f3.read(&mut buffer); ⑤
    close(&mut f3);

    let text = String::from_utf8_lossy(&buffer);

    println!("{:?}", f3);
    println!("{}", f3.name, f3_length);
    println!("{}", text);
}

```

- ① This method has snuck in to deal with cases where we want to simulate cases where a file has pre-existing data
- ② The `f` argument has been replaced with `self`
- ③ These two can remain as-is until we've looked at error handling
- ④ An explicit type needs to be provided, as `vec!` can't infer the necessary type through the function boundary
- ⑤ Here is the change in the calling code

[33] There are a number of theoretical differences between methods and functions relating to *purity*, but a detailed discussion of those differences is better placed elsewhere. Functions are regarded as *pure*, meaning their behavior is determined solely by their arguments. Methods are inherently impure, given that one of their arguments is effectively a side-effect. These are muddy waters though. Functions are perfectly capable of acting on side effects themselves. Moreover, methods are implemented with functions. And, to add an exception to an exception, objects are not always required for so called *static methods*.

3.4 RETURNING ERRORS

Early on in the chapter, two points were raised discussing dissatisfaction with being unable to properly signify errors:

- There was no attempt at implementing `read()` (and if we did, how would we handle the failure case?)
- `open()` and `close()` return `bool`; perhaps there is a way to provide a

more sophisticated result type that might be able to contain an error message if the operating system reports one?

The issue arises because dealing with hardware is unreliable. Even ignoring hardware faults, the disk may be full or the operating system may intervene and tell you that you don't have permission to delete a particular file. This section discusses different methods for signalling that an error has occurred, beginning with approaches common in other areas and finishing with idiomatic Rust.

3.4.1 Modifying a known global variable

One of the simplest methods for signalling that an error has occurred is by checking the value of a global variable. Although notoriously error prone, this is a common idiom in systems programming.

C programmers are used to checking the value of `errno` once system calls have returned. As an example, the `close()` system call closes a *file descriptor*—an integer representing a file, with numbers assigned by the operating system—and `errno` may be modified.

The section of POSIX standard discussing the `close()` system call includes this snippet:

If `close()` is interrupted by a signal that is to be caught, it shall return -1 with `errno` set to `EINTR` and the state of `fildes` [file descriptor] is unspecified. If an I/O error occurred while reading from or writing to the file system during `close()`, it may return -1 with `errno` set to `EIO`; if this error is returned, the state of `fildes` is unspecified.

-- close - close a file descriptor: The Open Group Base Specifications
Issue 7

<http://pubs.opengroup.org/onlinepubs/9699919799/functions/close.html>

Setting `errno` to either `EIO` or `EINTR` means to set it to some magical internal constant. The specific values are arbitrary and defined per operating system.

In Rust syntax, checking global variables would look something like this:

Listing 3.12. Rust-like code demonstrating how to check error codes from a global variable.

```
static mut ERROR: i32 = 0; ①

// ...

fn main() {
    let mut f = File::new("something.txt");

    read(f, buffer);
    unsafe {                ②
        if ERROR != 0 {    ③
            panic!("An error has occurred while reading the file ")
        }
    }

    close(f);
    unsafe {                ②
        if ERROR != 0 {    ③
            panic!("An error has occurred while closing the file ")
        }
    }
}
```

- ① `static mut`, spoken as "mutable static", is a global variable with the 'static' lifetime (that is, it's valid for the life of the program).
- ② Accessing and modifying `static mut` variables requires the use of an `unsafe` block. This is Rust's way of disclaiming all responsibility.
- ③ Checking the `ERROR` value happens here. Error checking relies on the convention that 0 means no error.

Here are the instructions to experiment with this pattern:

- Open a terminal
- `cd $TMP` (`cd %TMP%` on MS Windows)
- `cargo new --vcs none globalerror`
- `cd globalerror`
- `cargo add rand@0.7`
- Replace the contents of `src/main.rs` with the code in [Listing 3.13](#)
- Execute `cargo run`. You should see output like this:

```
$ cargo run
Compiling globalerror v0.1.0 (file:///path/to/globalerror)
Finished dev [unoptimized + debuginfo] target(s) in 0.74 secs
```

```
Running `target/debug/globalerror`
```

Most of the time, the program will not do anything. Occasionally—if the book has enough readers with sufficient motivation—it will print out a much louder message:

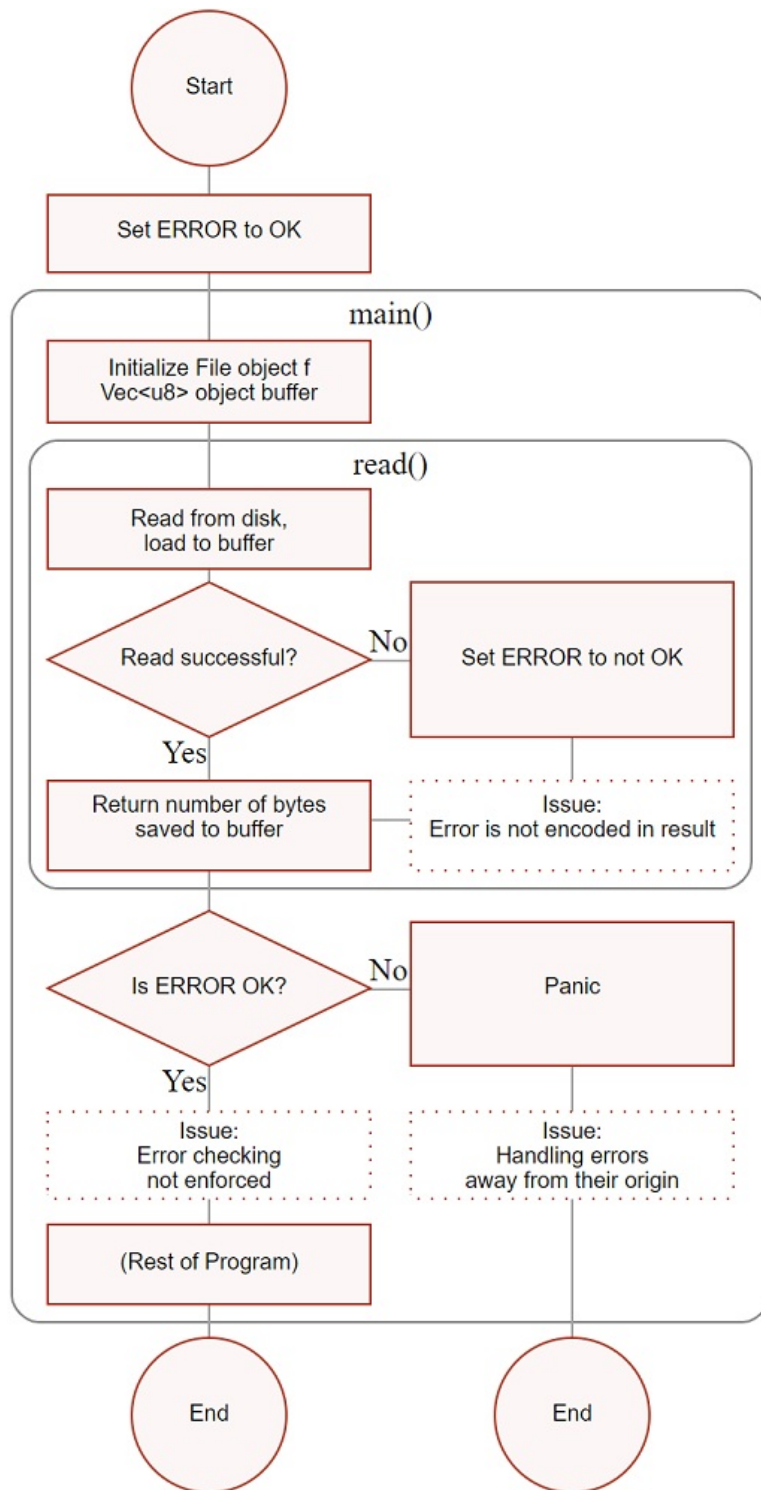
```
$ cargo run
thread 'main' panicked at 'An error has occurred!', src/main.rs:27:13
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

[Listing 3.13](#) introduces some new syntax. The most significant is probably the `unsafe` keyword. The significance of this keyword will be discussed later in the book. In the meantime, consider `unsafe` to be a warning sign rather than an indicator that you’re embarking on anything illegal. `Unsafe` means “the same level of safety offered by C at all times”.

There are also some other small additions to the Rust that you know already:

- Mutable global variables are denoted with `static mut`.
- By convention, global variables in Rust use ALL CAPS.
- Rust includes a `const` keyword as well for values that may never change.

Figure 3.3. A visual overview of [Listing 3.13](#), including explanations of problems with using global error codes



Listing 3.13. Using global variables to propagate error information through a program (ch3/globalerror/src/main.rs)

```
use rand::{random}; ①
```

```

static mut ERROR: isize = 0; ②

struct File; ③

#[allow(unused_variables)]
fn read(f: &File, save_to: &mut Vec<u8>) -> usize {
    if random() && random() && random() { ④
        unsafe {
            ERROR = 1; ⑤
        }
    }

    0 ⑥
}

#[allow(unused_mut)] ⑦
fn main() {
    let mut f = File;
    let mut buffer = vec![];

    read(&f, &mut buffer);
    unsafe { ⑧
        if ERROR != 0 {
            panic!("An error has occurred!")
        }
    }
}

```

- ① Bring the rand crate into local scope
- ② Initialize ERROR to 0.
- ③ Create a “zero-sized type” to stand in for a real struct while we’re experimenting
- ④ Return true 1/8 times this function is called
- ⑤ Set ERROR to 1, having the effect of notifying the rest of the system that an error has occurred
- ⑥ Always read() 0 bytes
- ⑦ Let’s keep buffer mutable for consistency with other code, even though it isn’t touched here
- ⑧ Accessing static mut variables is an unsafe operation

The difference between const and let

Variables defined with let are immutable anyway, so why does Rust include a const keyword? const creates compile-time constants. let values are created at runtime.

`let`, at the compiler level, relates more to *aliasing* than immutability. Aliasing, in compiler terminology, refers to having multiple references to the same location in memory at the same time. Values declared with `let` are shared. Values declared with `let mut` are guaranteed to be unique. This distinction allows the apparently contradictory notion of “interior mutability” to exist in Rust. Some types, such as `Rc`, present an immutable facade that still make internal changes to themselves when accessed. In the case of `Rc`, it maintains a count of references to itself.

Experienced programmers will know that using the global variable `errno` is commonly adjusted by the operating system during system calls.

This style of programming would typically be discouraged in Rust, as it omits both type safety (errors are encoded as plain integers) and can reward sloppy programmers with unstable programs when they forget to check the `errno` value. However, it’s an important style to be aware of because:

- Systems programmers may need to interact with operating system-defined global values
- Software that interacts with CPU registers and other low-level hardware will need to get used to inspecting flags to check that operations were completed successfully

3.4.2 Making use of the `Result` return type

Rust’s approach to error handling is to use a type that stands for both the standard case and the error case. This type is known as `Result`. `Result` has two states, `Ok` and `Err`. This multi-headed hydra is versatile and is put to work all through the standard library. We’ll consider *how* a single type can act as two later on. For the moment, let’s just investigate the mechanics of working with it.

The example below at [Listing 3.14](#) makes two major changes from previous iterations:

- Functions that interact with the file system return `Result<T, String>`, where `T` is the intended type. `String` will be used to report back error messages.
- When we call those functions, there is now an additional call to `unwrap()` which “unwraps” `Ok(T)` and produces `T`

There are some additional changes that may be useful to understand, but are not essential at this stage. `open()` and `close()` now take full *ownership* of their `File` arguments. While we'll defer a full explanation of the term ownership for now, it enables the `File` argument to be inserted into `Ok(T)` as `T` then returned.

Associated with this change is that the variable `f4` is now re-bound multiple times with the results of function calls. Without this, we would run into issues with using data that is no longer valid. In a sense, re-assigning the value from the result of the calls to `open()` and `close()` allows the variable `f4` to re-claim ownership of its `File` value.

To run the code, follow these steps:

- Open a terminal
- Move to a scratch directory, such as `/tmp/`
- Execute `cargo new --bin --vcs none fileresult`
- Ensure that the crate's `Cargo.toml` file specifies the 2018 edition and includes the `rand` crate as a dependency::

```
[package]
name = "fileresult"
version = "0.1.0"
authors = ["Tim McNamara <author@rustinaction.com>"]
edition = "2018"

[dependencies]
rand = "0.8"
```

- Replace the contents of `fileresult/src/main.rs` with the code in [Listing 3.14](#)
- Execute `cargo run`. You should see output like this:

```
$ cargo run
Compiling fileresult v0.1.0 (file:///path/to/fileresult)
Finished dev [unoptimized + debuginfo] target(s) in 1.04 secs
Running `target/debug/fileresult`
```

Listing 3.14. Using `Result` to mark functions that are susceptible to file-system errors (`ch3/fileresult/src/main.rs`)

```
use rand::prelude::*;
```

①


```

fn one_in(denominator: u32) -> bool { ②
    thread_rng().gen_ratio(1, denominator) ③
}

#[derive(Debug)]
struct File {
    name: String,
    data: Vec<u8>,
}

impl File {
    fn new(name: &str) -> File {
        File { name: String::from(name), data: Vec::new() } ④
    }

    fn new_with_data(name: &str, data: &Vec<u8>) -> File {
        let mut f = File::new(name);
        f.data = data.clone();
        f
    }

    fn read(self: &File, save_to: &mut Vec<u8>) -> Result<usize, String> { ⑤
        let mut tmp = self.data.clone();
        let read_length = tmp.len();
        save_to.reserve(read_length);
        save_to.append(&mut tmp);
        Ok(read_length) ⑥
    }
}

fn open(f: File) -> Result<File, String> {
    if one_in(10_000) { ⑦
        let err_msg = String::from("Permission denied");
        return Err(err_msg);
    }
    Ok(f)
}

fn close(f: File) -> Result<File, String> {
    if one_in(100_000) { ⑧
        let err_msg = String::from("Interrupted by signal!");
        return Err(err_msg);
    }
    Ok(f)
}

fn main() {
    let f4_data: Vec<u8> = vec![114, 117, 115, 116, 33];
    let mut f4 = File::new_with_data("4.txt", &f4_data);

    let mut buffer: Vec<u8> = vec![];

    f4 = open(f4).unwrap(); ⑨
    let f4_length = f4.read(&mut buffer).unwrap(); ⑨
    f4 = close(f4).unwrap(); ⑨

    let text = String::from_utf8_lossy(&buffer);

    println!("{:?}", f4);
    println!("{}", f4.name, f4_length);
}

```

```
println!("{}", text);  
}
```

- ① Bring common traits and types from the `rand` crate into this crate's scope.
`thread_rng`
- ② Helper function to enable us to trigger sporadic errors
- ③ `thread_rng()` creates a thread-local random-number generator. Its `gen_ratio(n, m)` method returns a Boolean value with probability n/m .
- ④ Stylistic change to shorten the code block
- ⑤ First appearance of `Result<T, E>`, where `T` is an integer of type `usize`, and `E` is `String`. Using `String` allows us to provide arbitrary error messages.
- ⑥ In this code, `read()` never fails. But we are still wrapping `read_length` in `Ok` because we're returning `Result`
- ⑦ Once in 10,000 executions, return an error.
- ⑧ Once in 100,000 executions, return an error.
- ⑨ "Unwrap" `T` from `Ok`, leaving `T`



Tip

Calling `.unwrap()` on a `Result` is often considered poor style. When called on an error type, the program will crash without a helpful error message.

As the chapter progresses, we'll encounter sophisticated mechanisms to handle errors.

Using `Result` provides you with compiler-assisted code correctness: your code won't compile unless you've taken the time to handle the edge cases. This program will fail on error, but at least we have made this explicit.

So, what is a `Result`? `Result` is an enum defined in Rust's standard library. It has the same status as any other type, but is tied together with the rest of the language through strong community conventions. You may be wondering: "Wait.. what is an enum?" I'm glad you asked. That's the topic of our next section.

3.5 DEFINING AND MAKING USE OF ENUM

An enum—enumeration in full—is a type that can represent multiple known variants. Classically, an enum would be used to represent several pre-defined known options, such as the suits of playing cards or planets in the solar system.

Listing 3.15. Defining an enum that represents the suits of a French deck of playing cards

```
enum Suit {  
    Clubs,  
    Spades,  
    Diamonds,  
    Hearts,  
}
```

If you haven't programmed in a language that makes use of enums, understanding their value takes some effort. As you program with them for a while, you're likely to experience a minor epiphany. Consider creating some code that is parsing event logs. Each event has a name, perhaps "UPDATE" or "DELETE". Rather than storing those values as strings in your application, which can lead to subtle bugs later on when string comparisons become unwieldy, enums allow you to give the compiler some knowledge of the event codes. Later on, you'll be given warnings such as "Hi there, I see that you have considered the UPDATE case, but it looks like you've forgotten the DELETE case. You should fix that."

[Listing 3.16](#) shows the beginnings of an application that parses text and emits structured data. When run, the program produces this output:

```
(Unknown, "BEGIN Transaction XK342")  
(Update, "234:LS/32231 {"price\": 31.00} -> {"price\": 40.00}")  
(Delete, "342:L0/22111")
```

Listing 3.16. Defining an enum and making use of it to parse an event log (ch3-parse-log.rs)

```
#[derive(Debug)] ⓘ
```

```

enum Event {
    Update, ②
    Delete, ②
    Unknown, ②
}

type Message = String; ③

fn parse_log(line: &'static str) -> (Event, Message) { ④
    let parts: Vec<&str> = line.splitn(2, ' ').collect(); ⑤
    if parts.len() == 1 { ⑥
        return (Event::Unknown, String::from(line))
    }

    let event = parts[0]; ⑦
    let rest = String::from(parts[1]); ⑦

    match event {
        "UPDATE" | "update" => (Event::Update, rest), ⑧
        "DELETE" | "delete" => (Event::Delete, rest), ⑧
        _ => (Event::Unknown, String::from(line)), ⑨
    }
}

fn main() {
    let log = "BEGIN Transaction XK342
UPDATE 234:LS/32231 {\"price\": 31.00} -> {\"price\": 40.00}
DELETE 342:LO/22111";

    for line in log.lines() {
        let parse_result = parse_log(line);
        println!("{:?}", parse_result);
    }
}

```

- ① Enable this enum to be printed to the screen via auto-generated code
- ② Create three variants of Event, including one value for unrecognized events
- ③ A convenient name for String for use in this crate's context
- ④ A function for parsing a line and converting it into semi-structured data
- ⑤ `collect()` consumes an iterator (returned from `line.splitn()`) and returns `Vec<T>`
- ⑥ If `line.splitn()` didn't split log into two parts, return an error
- ⑦ Assign each part of `parts` to a variable to ease future use
- ⑧ When we match a known event, return structured data
- ⑨ If we don't recognize the event type, return the whole line

Enums have a few tricks up their sleeves:

- They work together with Rust’s pattern matching capabilities to help you build robust, readable code. This is visible on lines 19-23 of [Listing 3.16](#).
- Like struct, they support methods via `impl` blocks
- Rust’s enums are more powerful than a set of constants. It’s possible to include data within variants, granting them a struct-like persona:

```
enum Suit {
    Clubs,
    Spades,
    Diamonds,
    Hearts, ①
}

enum Card {
    King(Suit), ②
    Queen(Suit), ②
    Jack(Suit), ②
    Ace(Suit), ②
    Pip(Suit, usize), ③
}
```

- ① The last element of enums also ends with a comma. This eases refactoring.
- ② Face cards have a suit
- ③ Pip cards have a suit and a rank

3.5.1 Using an enum to manage internal state

Now that you’ve seen how to define and use an enum, how would this be useful when applied to modelling files? We can expand our `File` type and allow it to change as it is opened and closed.

[Listing 3.17](#) produces code that prints out a short alert to the console:

```
Error checking is working
File { name: "5.txt", data: [], state: Closed }
5.txt is 0 bytes long
```

Listing 3.17. An enum that represents a File being open or closed (ch3-file-states.rs)

```

#[derive(Debug,PartialEq)]
enum FileState {
    Open,
    Closed,
}

#[derive(Debug)]
struct File {
    name: String,
    data: Vec<u8>,
    state: FileState,
}

impl File {
    fn new(name: &str) -> File {
        File { name: String::from(name), data: Vec::new(), state: FileState::Closed }
    }

    fn read(self: &File, save_to: &mut Vec<u8>) -> Result<usize, String> {
        if self.state != FileState::Open {
            return Err(String::from("File must be open for reading"));
        }
        let mut tmp = self.data.clone();
        let read_length = tmp.len();
        save_to.reserve(read_length);
        save_to.append(&mut tmp);
        Ok(read_length)
    }
}

fn open(mut f: File) -> Result<File, String> {
    f.state = FileState::Open;
    Ok(f)
}

fn close(mut f: File) -> Result<File, String> {
    f.state = FileState::Closed;
    Ok(f)
}

fn main() {
    let mut f5 = File::new("5.txt");

    let mut buffer: Vec<u8> = vec![];

    if f5.read(&mut buffer).is_err() {
        println!("Error checking is working");
    }

    f5 = open(f5).unwrap();
    let f5_length = f5.read(&mut buffer).unwrap();
    f5 = close(f5).unwrap();

    let text = String::from_utf8_lossy(&buffer);

    println!("{:?}", f5);
    println!("{}", f5.name, f5_length);
    println!("{}", text);
}

```

Enums can be a powerful aide in your quest to produce reliable, robust software. Consider it for your code whenever you discover yourself introducing stringly typed data such as messages codes.

3.6 DEFINING COMMON BEHAVIOR WITH TRAITS

A robust definition of the term “file” needs to be agnostic to storage medium. Files (at least) support *reading* and *writing*. Focusing on those two capabilities allows us to ignore where the reads and writes are actually taking place. They could be from a hard disk drive, an in-memory cache, over a network or via something more exotic.

Irrespective of whether a “file” is a network connection, a spinning metal platter or a superposition of an electron, it’s possible to define rules that say, "To call yourself a file, you must implement this". Traits have close relatives in other languages. They’re often named interfaces, protocols or perhaps contracts.

You have already seen traits in action several times. Every time you’ve used `#[derive(Debug)]` in a type definition, you’ve implemented the `Debug` trait for that type. Traits permeate the Rust language. Let’s see how to create one.

3.6.1 Creating a Read trait

Traits enable the compiler (and other humans) to know that multiple types are attempting to perform the same task. Types that use `#[derive(Debug)]` are all able to be printed to the console via the `println!` macro and its relatives. Allowing multiple types to implement a `Read` trait enables code re-use and enables the Rust compiler to perform its “zero cost abstraction” wizardry.

For the sake of brevity, [Listing 3.19](#) is a bare bones version of the code that we’ve already seen. When built with `rustc` and executed, [Listing 3.19](#) prints the following line to the console:

Listing 3.18. Output of [Listing 3.19](#)

```
0 byte(s) read from File
```

Listing 3.19. Defining the bare bones of a Read trait for File. Shows the distinction between the trait keyword which is used for definitions and the impl keyword for attaching a trait to a specific type. (ch3/ch3-skeleton-read-trait.rs)

```
#![allow(unused_variables)] ①

#[derive(Debug)]
struct File; ②

trait Read { ③
    fn read(self: &Self, save_to: &mut Vec<u8>) -> Result<usize, String>; ④
}

impl Read for File {
    fn read(self: &File, save_to: &mut Vec<u8>) -> Result<usize, String> {
        Ok(0) ⑤
    }
}

fn main() {
    let f = File{};
    let mut buffer = vec!();
    let n_bytes = f.read(&mut buffer).unwrap();
    println!("{}", n_bytes);
}
```

- ① Silence any warnings relating to unused variables within functions
- ② Define a stub File type
- ③ Provide a specific name for the trait
- ④ A trait block includes the type signatures of functions that implementors must comply with. The pseudo-type `Self` is a placeholder for the type that will eventually be implementing `Read`.
- ⑤ A simple stub value that complies with the type signature required

Defining a trait and implementing it in on the same page can feel quite drawn out in small examples such as this. `File` is spread across three code blocks within [Listing 3.19](#). The flip side of this is that many common traits become second nature as your experience grows. Once you've learned what the `PartialEq` trait does for one type, you'll understand it for every other one.

And what does `PartialEq` do for types? It enables comparisons with the `==` operator. “Partial” allows for cases where two values that match exactly should not be treated as equal, such as the floating point’s Not a Number value or SQL’s `NULL`.



Tip

If you’ve spent some time looking through the Rust community’s forums and documentation, you may noticed that they’ve formed their own variant of English grammar.

When you see a sentence with the following structure, “...`T` is `Debug`...”, then they’re saying that `T` implements the `Debug` trait.

3.6.2 Implementing `std::fmt::Display` for your own types

`println!` and a number of others live within a family of macros that all use the same underlying machinery. The macros `println!`, `print!`, `write!`, `writeln!` and `format!` all rely on the `Display` and `Debug` traits. That is, they rely on trait implementations provided by programmers to be able to convert from `{}` to what is printed to the console.

Looking a few pages back to [Listing 3.17](#), the `File` type was composed of a few fields and a custom subtype, `FileState`:

Listing 3.20. Snippet from [Listing 3.17](#) illustrating the use of the `Debug` trait

```
#[derive(Debug, PartialEq)]
enum FileState {
    Open,
    Closed,
}

#[derive(Debug)]
struct File {
    name: String,
    data: Vec<u8>,
}
```

```

    state: FileState,
}

fn main() {
    let f5 = File::new("f5.txt");
    //...
    println!("{:?}", f5); ①
}

```

① Debug relies on the question mark to work, which can be frustrating if other types implement `Display` and are happy to be printed via `{}`

It's possible to rely on the `Debug` trait auto-implementations as a crutch, but what should you do if you wanted to provide custom text? `Display` requires that types implement a `fmt` method that returns `fmt::Result`.

Listing 3.21. Implementing the `std::fmt::Display` trait for `File` and its associated type `FileState`

```

impl Display for FileState {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result { ①
        match *self {
            FileState::Open => write!(f, "OPEN"),
            FileState::Closed => write!(f, "CLOSED"),
        }
    }
}

impl Display for File {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result { ①
        write!(f, "<{} ({})>", self.name, self.state) ②
    }
}

```

① To implement `std::fmt::Display`, a single method `fmt`, must be defined for your type.

② It is common to defer to inner types' `Display` implementation via the `write!` macro.

Listing 3.22. Working code snippet to implement `Display` for a struct that includes fields that also need to implement `Display` (ch3/ch3-implementing-display.rs)

```

#![allow(dead_code)] ①

```

```

use std::fmt; ②
use std::fmt::{Display}; ③

#[derive(Debug, PartialEq)]
enum FileState {
    Open,
    Closed,
}

#[derive(Debug)]
struct File {
    name: String,
    data: Vec<u8>,
    state: FileState,
}

impl Display for FileState {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match *self {
            FileState::Open => write!(f, "OPEN"), ④
            FileState::Closed => write!(f, "CLOSED"), ④
        }
    }
}

impl Display for File {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "<{} ({})>", self.name, self.state) ⑤
    }
}

impl File {
    fn new(name: &str) -> File {
        File {
            name: String::from(name),
            data: Vec::new(),
            state: FileState::Closed,
        }
    }
}

fn main() {
    let f6 = File::new("f6.txt");
    //...
    println!("{:?}", f6); ⑥
    println!("{}", f6); ⑦
}

```

- ① Silence warnings related to FileState::Open not being used
- ② Bring the std::fmt crate into local scope, allowing us to make use of fmt::Result
- ③ Bring Display into local scope, avoiding the need for us to prefix it as fmt::Display in our code
- ④ Sneakily, we can make use of write! to do the grunt work for us. Strings

already implement `Display` themselves, so there's very little left for us to do.

- ⑤ We can rely on the `FileState Display` implementation in our own code
- ⑥ The `Debug` implementation prints out a familiar message, in common with all other implementors of `Debug`, `File { ... }`
- ⑦ Our `Display` implementation follows its own rules, displaying itself as `<f6.txt (CLOSED)>`

We'll see many uses of traits throughout the course of the book. They underlie Rust's generics system and the language's robust type checking. With a little bit of abuse, they can also support a form of inheritance of the form that's common in most object oriented languages. For now though, the thing to remember is that traits are common behavior that types opt into via the syntax: `impl Trait for Type`.

3.7 EXPOSING YOUR TYPES TO THE WORLD

Your crates will interact with others that you build up over time. You may wish to make that process easier for your future self by hiding internal details and documenting what's public. This section describes some of the tooling available within the language and cargo to make that process easier.

3.7.1 Protecting private data

Rust defaults to keeping things private. If you were to create a library with only the code that you have seen so far, importing your crate would provide no extra benefit. To remedy this, use the `pub` keyword to make things public.

[Listing 3.24](#) provides a few examples of prefixing types and methods with `pub`. Its output is not exciting:

Listing 3.23. Output of [Listing 3.24](#)

```
File { name: "f7.txt", data: [], state: Closed }
```

Listing 3.24. Making use of the `pub` keyword to mark the name

and state fields of File to be public

```
#[derive(Debug,PartialEq)]
pub enum FileState { ①
    Open,
    Closed,
}

#[derive(Debug)]
pub struct File {
    pub name: String,
    data: Vec<u8>, ②
    pub state: FileState,
}

impl File {
    pub fn new(name: &str) -> File { ③
        File {
            name: String::from(name),
            data: Vec::new(),
            state: FileState::Closed
        }
    }
}

fn main() {
    let f7 = File::new("f7.txt");
    //...
    println!("{:?}", f7);
}
```

- ① An enum's variants are assumed to be public if the overall type is made public
- ② File.data remains private if a third party were to import this crate via use
- ③ Even though the File struct is public, its methods must also be explicitly marked as such too

3.8 CREATING IN-LINE DOCUMENTATION

When software systems become larger, it becomes more important to document one's progress. This section walks through adding documentation to your code and generating HTML versions of that content.

Below at [Listing 3.25](#), you'll see the familiar code with some added lines beginning with `///` or `///!`. The first form is much more common. It generates documents that refer to the item that immediately follow. The second form refers

to the current item, as the compiler scans the code. By convention, it is only used to annotate the current module but is available for other places as well.

Listing 3.25. Adding Doc Comments to Code (ch3-file-doced.rs)

```
//! Simulating files one step at a time. ①

/// Represents a "file", which probably lives on a file system. ②
#[derive(Debug)]
pub struct File {
    name: String,
    data: Vec<u8>,
}

impl File {
    /// New files are assumed to be empty, but a name is required.
    pub fn new(name: &str) -> File {
        File {
            name: String::from(name),
            data: Vec::new(),
        }
    }

    /// Returns the file's length in bytes.
    pub fn len(&self) -> usize {
        self.data.len()
    }

    /// Returns the file's name.
    pub fn name(&self) -> String {
        self.name.clone()
    }
}

fn main() {
    let f1 = File::new("f1.txt");

    let f1_name = f1.name();
    let f1_length = f1.len();

    println!("{:?}", f1);
    println!("{}", f1 is {} bytes long", f1_name, f1_length);
}
```

① The form `//!` is used to refer to the “current” item, the module that’s just been entered by the compiler

② The form `///` annotates whatever is immediately following it

3.8.1 Using `rustdoc` to Render Docs For a Single Source File

You may not know it, but you installed a command-line tool called `rustdoc` when you installed Rust. `rustdoc` is like a special-purpose Rust compiler. Instead of producing executable code, it produces HTML versions of your in-line documentation.

Here is how to use it. Assuming that you have the code at [Listing 3.25](#) saved as `ch3-file-doced.rs`, follow these steps:

- Open a terminal
- Move to the location of your source file
- Execute `rustdoc ch3-file-doced.rs`
- Optional: Run `ls` (or `dir` on Windows).
You will notice that a directory (`doc/`) has been created for you. The documentation's entry point is actually within a sub-directory, `doc/ch3_file_doced/index.html`.
- Open a web browser, then navigate to `file:///path/to/doc/ch3_file_doced` and open a web browser (START /c <http://localhost> in Windows)

When your programs start to get larger and span multiple files, invoking `rustdoc` manually can become a bit of a pain. Thankfully, `cargo` can do the grunt work on your behalf. That's discussed later at [Section 3.8.2](#).

3.8.2 Using `cargo` to Render Docs for a Crate and its Dependencies

These comments can be rendered as rich text with `cargo`. `cargo` works with crates, rather than the individual files as we've been working with so far. To get around this, create a crate named `filebasics` by following these instructions:

- Open a terminal
- Move to a working directory, such as `/tmp/` (`cd %TEMP%` on MS Windows)
- Run `cargo new --bin filebasics`

You should end up with a project directory tree that looks like this:

```
filebasics
|--.gitignore
|--Cargo.toml
`--src
    |--main.rs ①
```

① This file is what you will be editing in the following steps

Save the source code from [Listing 3.25](#) into `filebasics/src/main.rs`, overwriting the hello world boilerplate code that is already there

To build an HTML version of a crate's documentation:

- Move to the project's root directory (`filebasics/`) which includes the `Cargo.toml` file
- Run `cargo doc --open` and Rust will compile an HTML version of the documentation for your code.

You should see output similar to the following appear:

```
Documenting files v0.1.0 (file:///C:/Users/Tim/AppData/Local/Temp/files)
  Finished dev [unoptimized + debuginfo] target(s) in 1.68 secs
   Opening
C:\Users\Tim\AppData\Local\Temp\files\target\doc\files\index.html
  Launching cmd /C
```

Your default web browser will now be open and the documentation should be visible.



Tip

If you have lots of dependencies in your crate, the build process may take a while.

A useful flag is `cargo doc --no-deps`. Adding `--no-deps` can significantly restrict the work `rustdoc` has to do.

rustdoc supports rich text rendering text written in Markdown. That allows you to add headings, lists and links within your documentation. Code snippets that are wrapped in grave accent gates (``...``) are given syntax highlighting.

```
//! Simulating files one step at a time.
```

```
impl File {  
    /// Creates a new, empty `File`.  
    ///  
    /// # Examples  
    ///  
    /// ```  
    /// let f = File::new("f1.txt");  
    /// ```  
    pub fn new(name: &str) -> File {  
        File {  
            name: String::from(name),  
            data: Vec::new(),  
        }  
    }  
}
```

3.9 SUMMARY

This chapter has packed a great deal of content into its pages. You have learned that:

- Struct is the foundational compound data type. Paired with traits, struct is the closest to “object” from other domains
- Enum is more powerful than a simple list. Enum’s strength lies in its ability to work with the compiler to consider all edge cases.
- Methods are added to types via `impl` blocks
- Global error codes can be used in Rust, but it can be cumbersome and would generally be frowned upon
- The `Result` type is the mechanism the Rust community prefers to use to signal the possibility of error
- Traits enable common behavior through Rust programs.
- Data and methods remain private until they’re declared public
- Cargo can be used to build the documentation of your crate and all of its dependencies

4 LIFETIMES, OWNERSHIP AND BORROWING

In this chapter:

- discovering what the term “lifetime” means in the context of Rust programming
- working with the borrow checker rather than against it
- multiple tactics for dealing with issues when they crop up
- understanding what the responsibilities of an “owner” are
- learning how to “borrow” values that are owned elsewhere

This chapter attempts to explain one of the concepts that trips up most newcomers to Rust: its “borrow checker”. The borrow checker checks that all access to data is legal. Checking to see that all data access is legal allows Rust to prevent safety issues.

Learning how this system works will—at the very least—speed up your development time, by avoiding run ins with the compiler. More significantly though, learning to work with the borrow checker allows you to build larger software systems with confidence. It underpins the term “fearless concurrency”.

To explain how this system operates—and learn how to comply with it when an error is discovered—is this chapter’s role. It uses the somewhat lofty example of simulating a satellite constellation to explain the trade offs relating to different ways to provide shared access to data.

The details of borrow checking are thoroughly explored within the chapter. However, a few bullet points might be useful for readers wanting a quick gist:

Borrow checking relies on three inter-related concepts: lifetimes, ownership and

borrowing.

- *ownership* is a stretched metaphor. There is no relationship to property rights. Within Rust, ownership relates to cleaning values up when they're no longer needed. For example, when a function returns, the memory holding its local variables needs to be freed. Owners cannot prevent other parts of the program from accessing their values or report data theft to some overarching Rust authority.
- A value's *lifetime* is the period when accessing that value is valid behavior. A function's local variables live until the function returns. Global variables might live for the life of the program.
- To *borrow* a value means to access it. This terminology is confusing, as there is no obligation to return the value back to its owner. It's used to emphasize that while values may have a single owner, it's possible for many parts of the program to share access to those values.

4.1 “IMPLEMENTING” A MOCK CUBESAT GROUND STATION

Our strategy for this chapter will be to use an example that compiles, then make a minor change that will trigger an error that appears to emerge without any adjustment to the program's flow. Working through the fixes to those issues should make the concepts more complete.

The learning example for the chapter is a CubeSat constellation. If you've never encountered that phrase before, here are some definitions:

- CubeSats are miniature artificial satellites that have increasingly increased the accessibility of space research compared to conventional satellite.
- A ground station is an intermediary between the operators and the satellites themselves. It's listening on the radio, checking on the status of every satellite in the constellation and transmitting messages to and fro. When introduced in our code, it will act as the

- gateway between the user and the satellites.
- A constellation is the collective noun for satellites in orbit.

Figure 4.1. CubeSats in Orbit



In [Figure 4.1](#), we have three CubeSats. To model this, we'll create a variable for each of them. They can happily be integers for the moment. We don't need to model the ground station explicitly yet, as we're not yet sending messages around the constellation, so we'll omit it for now.

```
let sat_a = 0;  
let sat_b = 1;  
let sat_c = 2;
```

To check on the status of each of our satellites, we'll use a stub function and an enum to represent potential status messages.

```
#[derive(Debug)]  
enum StatusMessage {  
    Ok, ①  
}  
  
fn check_status(sat_id: u64) -> StatusMessage {  
    StatusMessage::Ok ①  
}
```

① For now, all of our CubeSats function perfectly all of the time

The `check_status()` function would be extremely complicated in a production system. For our purposes though, returning the same value every time is perfectly sufficient. Pulling these two snippets into a whole program that "checks" our satellites twice, we end up with something like this:

Listing 4.1. Checking the status of our integer-based CubeSats (ch4/ch4-check-sats-1.rs)

```
#![allow(unused_variables)]

#[derive(Debug)]
enum StatusMessage {
    Ok,
}

fn check_status(sat_id: u64) -> StatusMessage {
    StatusMessage::Ok
}

fn main () {
    let sat_a = 0;
    let sat_b = 1;
    let sat_c = 2;

    let a_status = check_status(sat_a);
    let b_status = check_status(sat_b);
    let c_status = check_status(sat_c);
    println!("a: {:?}, b: {:?}, c: {:?}", a_status, b_status, c_status);

    // "waiting" ...
    let a_status = check_status(sat_a);
    let b_status = check_status(sat_b);
    let c_status = check_status(sat_c);
    println!("a: {:?}, b: {:?}, c: {:?}", a_status, b_status, c_status);
}
```

Running [Listing 4.1](#) should be fairly uneventful. The code compiles, albeit begrudgingly. We encounter the following output from our program:

Listing 4.2. Output of [Listing 4.1](#)

```
a: Ok, b: Ok, c: Ok
a: Ok, b: Ok, c: Ok
```

4.1.2 Encountering our first lifetime issue

Let's move closer to idiomatic Rust by introducing type safety. Instead of integers, let's create a type to model our satellites. A real implementation of a CubeSat type would probably include lots of information about its position, its RF frequency band and more. We'll stick with only recording an identifier.

Listing 4.3. Modelling a CubeSat as its own type

```
#[derive(Debug)]
struct CubeSat {
    id: u64,
}
```

Now that we have a struct definition, let's inject it into our code. [Listing 4.4](#) will not compile (yet). Understanding the details of why not is the task of much of this chapter.

Listing 4.4. Checking the status of our integer-based CubeSats (ch4/ch4-check-sats-2.rs)

```
#[derive(Debug)] ①
struct CubeSat {
    id: u64,
}

#[derive(Debug)]
enum StatusMessage {
    Ok,
}

fn check_status(sat_id: CubeSat) -> StatusMessage { ②
    StatusMessage::Ok
}

fn main() {
    let sat_a = CubeSat { id: 0 }; ③
    let sat_b = CubeSat { id: 1 }; ③
    let sat_c = CubeSat { id: 2 }; ③

    let a_status = check_status(sat_a);
    let b_status = check_status(sat_b);
    let c_status = check_status(sat_c);
    println!("a: {:?}, b: {:?}, c: {:?}", a_status, b_status, c_status);

    // "waiting" ...
    let a_status = check_status(sat_a);
    let b_status = check_status(sat_b);
    let c_status = check_status(sat_c);
    println!("a: {:?}, b: {:?}, c: {:?}", a_status, b_status, c_status);
}
```

- ① Modification 1: Add definition
- ② Modification 2: Use the new type within `check_status()`
- ③ Modification 3: Create three new instances

When you attempt to compile the code within [Listing 4.4](#), you will receive a message similar to the following (which has been edited for brevity):

Listing 4.5. Error Message Produced When Attempting to Compile [Listing 4.4](#)

```
error[E0382]: use of moved value: `sat_a`
--> code/ch4-check-sats-2.rs:26:31
   |
20 |     let a_status = check_status(sat_a);
   |                               ----- value moved here
...
26 |     let a_status = check_status(sat_a);
   |                               ^^^^^^ value used here after move
   |
   = note: move occurs because `sat_a` has type `CubeSat`,
   = note: which does not implement the `Copy` trait
...

error: aborting due to 3 previous errors
```

To trained eyes, the compiler's message is very helpful. It tells us exactly where the problem is and provides us with a recommendation on how to fix it. To less experienced eyes, it's significantly less useful. We are using a "moved" value and are fully advised to implement the `Copy` trait on `CubeSat`. Huh? Turns out, although it is written in English, the term "move" means something very specific within Rust. Nothing physically moves.

Movement within Rust code refers to movement of *ownership*, rather than movement of data. Ownership is a term used within the Rust community to refer to the compile-time process that checks that every use of a value is valid and that every value will be destroyed cleanly.

Every value in Rust is *owned*. In both [Listing 4.1](#) and [Listing 4.4](#), `sat_a`, `sat_b` and `sat_c` *own* the data that they refer to. When calls to `check_status()` are

made, ownership of the data moves from the variables in the scope of `main()` to the `sat_id` variable within the function. The significant difference is that the second example places that integer within a `CubeSat` struct. [\[34\]](#) This type change alters the semantics of how the program behaves.

Here is a stripped down version of the `main()` function from [Listing 4.4](#), focusing on `sat_a` and places where ownership moves:

Listing 4.6. Extract of [Listing 4.4](#), focusing on the `main()` function

```
fn main() {  
    let sat_a = CubeSat { id: 0 };           ①  
  
    let a_status = check_status(sat_a);      ②  
  
    let a_status = check_status(sat_a);      ③  
}
```

- ① Ownership originates here at the creation of the `CubeSat` object
- ② Ownership of the object moves to `check_status()`, but is not returned to `main()`
- ③ At this point, `sat_a` is no longer owner of the object, making access invalid

Rebinding is legal when values are not borrowed

If you have experience with programming languages such as JavaScript (from 2015 onwards), you may have been surprised to see that the variables for each of the `CubeSats` were redefined in [Listing 4.4](#).

On line 20, `a_status` is assigned to the result of the first call to `check_status(sat_a)`. On line 26, it is reassigned to the result of the second call. The original value is overwritten.

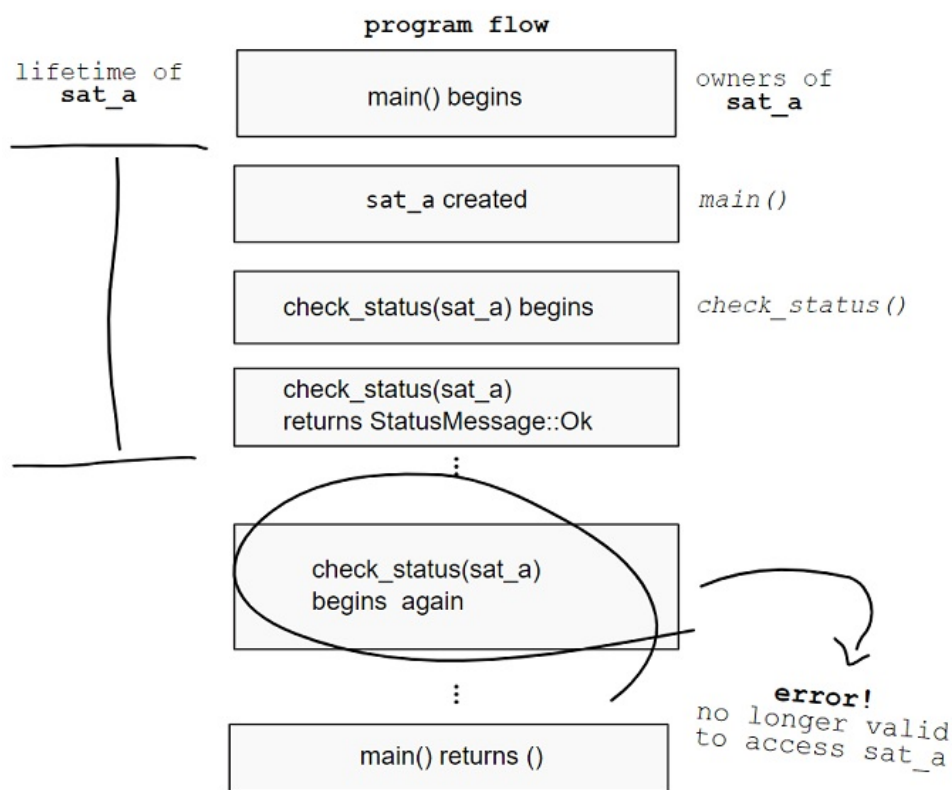
This is legal Rust code, but one must be aware of ownership issues and lifetime here too. It's possible in this context because there are no live borrows to contend with. Attempting to overwrite a value that's still available from elsewhere in the program will cause the compiler to refuse to compile your program.

[Figure 4.2](#) provides a visual walk-through of the inter-related processes of control flow, ownership and lifetimes. During the call to `check_status(sat_a)`, ownership *moves* to the `check_status()` function. When `check_status()` returns a `StatusMessage`, it drops the `sat_a` value. The *lifetime* of `sat_a` ends

here. Yet, `sat_a` remains in the local scope of `main()` after the first call to `check_status()`. Attempting to access that variable will incur the wrath of the borrow checker.

The distinction between a value's lifetime and its scope—which is what many programmers are trained to rely on—can make things difficult to disentangle. Avoiding and overcoming this type of issue makes up the bulk of the chapter.

Figure 4.2. Visual Explanation of Rust Ownership Movement



4.1.2 Special behavior of primitive types

Before carrying on, it may be wise to explain why the first code snippet, that is [Listing 4.1](#), compiled at all. Indeed, the only change that we make at [Listing 4.4](#) was to wrap our satellite variables in a custom type. As it happens, primitive types in Rust have special behavior. They implement the `Copy` trait.

Types implementing `Copy` are duplicated at times that would otherwise be illegal.

This provides some convenience day-to-day, at the expense of adding a trap for newcomers. As you grow out from toy programs using integers, your code suddenly breaks.

Formally, primitive types are said to possess *copy semantics*, whereas all other types have *move semantics*. Unfortunately for learners of Rust, that special case looks like the default case because they typically encounter primitive types first. [Listing 4.7](#) and [Listing 4.8](#) illustrate the difference between these two concepts. The first compiles and runs, the other does not. Their only difference is that they use different types.

Listing 4.7. Demonstrating the *copy semantics* of Rust's primitive types (and all types that implement `Copy`). Values may be accessed

```
fn use_value(_val: i32) {           ①
}

fn main() {
    let a = 123 ;
    use_value(a);

    println!("{}", a);             ②
}
```

- ① `use_value()` takes ownership of the `_val` argument. The `use_value()` function is generic, because it will be used in the next example.
- ② Perfectly legal to access `a` after `use_value()` has returned.

Listing 4.8. Demonstrating the *move semantics* of types that do not implement the `Copy` trait. When used as an argument to a function that takes ownership, values may not be accessed again from the outer scope.

```
fn use_value(_val: Demo) {         ①
}

struct Demo {
    a: i32,
}

fn main() {
    let demo = Demo { a: 123 };
}
```

```

use_value(demo);
println!("{}", demo.a);    ②
}

```

① `use_value()` takes ownership of `_val`.

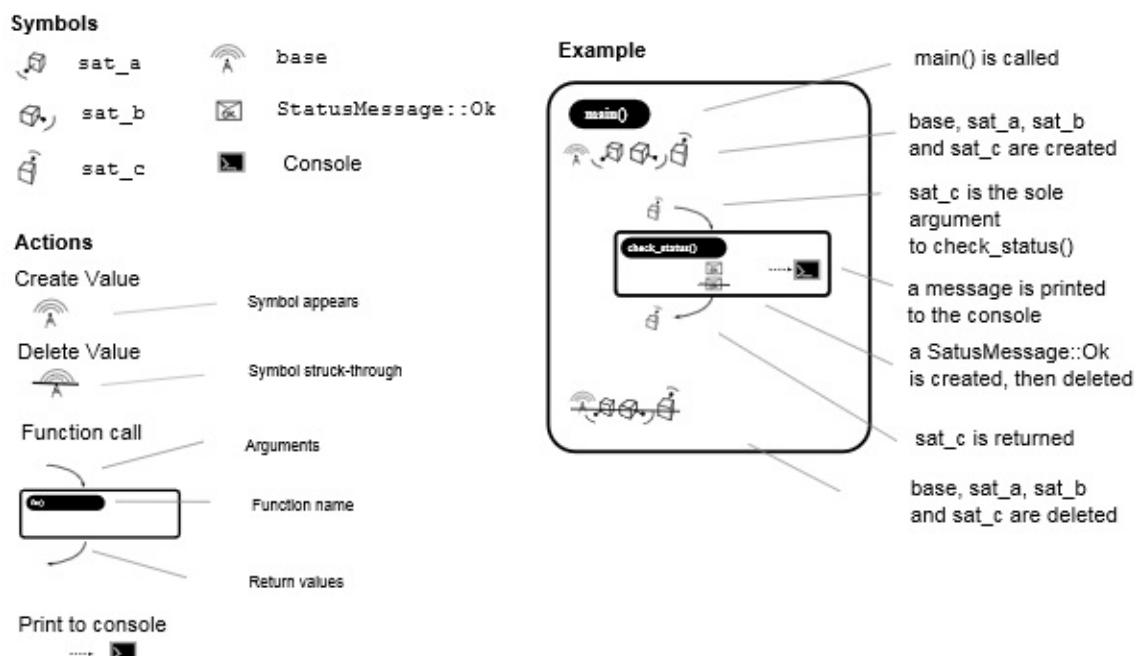
② Illegal to access `demo.a`, even after `use_value()` has returned.

[34] Remember the phrase zero-cost abstractions? One of the ways this is manifest is by not adding extra data around values within structs.

4.2 GUIDE TO THE FIGURES IN THIS CHAPTER

The figures used in this chapter use a bespoke notation to illustrate the three interrelated concepts of scope, lifetimes and ownership.

Figure 4.3. How to Interpret the Figures in This Chapter



4.3 WHAT IS AN OWNER? DOES IT HAVE ANY RESPONSIBILITIES?

In the world of Rust, the notion of *ownership* is rather limited: An owner cleans up when its values' lifetimes end.

When values go out of scope or their lifetimes end for some other reason, their destructors are called. A destructor is a function that removes traces of the value from the program by deleting references and freeing memory. You won't find a call to any destructors in most Rust code. The compiler injects that code itself as part of the process of tracking every value's lifetime.

To provide a custom destructor for a type, implement `Drop`. This will typically be needed in cases where you have used `unsafe` blocks to allocate memory. `Drop` has one method, `drop(&mut self)` that you can use to conduct any necessary wind up activities.

An implication of this system is that values may not outlive their owner. This kind of situation can make data structures built with references, such as trees and graphs, feel slightly bureaucratic. If the root node of a tree is the owner of the whole tree, it can't be removed without taking ownership into account.

Finally, unlike the Lockean notion of personal property, ownership does not imply control or sovereignty. In fact, the “owners” of values do not even have special access to their owned data. Nor do they have an ability to restrict others from trespassing. Owners don't get a say on other sections of code borrowing their values.

4.4 HOW OWNERSHIP MOVES

There are two main ways to shift ownership from one variable to another within a Rust program. The first is through assignment. ^[35] The second is by passing data through a function barrier, either as an argument or a return value.

Revisiting our original code from [Listing 4.4](#), we can see that `sat_a` starts its life with ownership over a `CubeSat` object.

```
fn main() {  
    let sat_a = CubeSat { id: 0 };  
}
```

```
// ...
```

The `CubeSat` object is then passed into `check_status()` as an argument, moving ownership to the local variable `sat_id`.

```
fn main() {  
    let sat_a = CubeSat { id: 0 };  
    // ...  
    let a_status = check_status(sat_a);  
    // ...  
}
```

Another possibility could have been that `sat_a` relinquishes its ownership within `main()` to another variable. That would look something like this:

```
fn main() {  
    let sat_a = CubeSat { id: 0 };  
    // ...  
    let new_sat_a = sat_a;  
    // ...  
}
```

Lastly, were there to be a change in the `check_status()` function signature, it too could pass ownership of the `CubeSat` to a variable within the calling scope.

Here is our original function,

```
fn check_status(sat_id: CubeSat) -> StatusMessage {  
    StatusMessage::Ok  
}
```

and here is an adjusted function that achieves its message notification through a side-effect.

```
fn check_status(sat_id: CubeSat) -> CubeSat {  
    println!("{:?}: {:?}", sat_id, StatusMessage::Ok); ①  
    sat_id ②  
}
```

① Use the `Debug` formatting syntax as our types have use `#[derive(Debug)]`

② Return a value by omitting the semi-colon at the end of the last line

When the adjusted `check_status()` function used in conjunction with a new `main()`, it's possible to see ownership of the `CubeSat` objects back to their original variables. The new code is:

Listing 4.9. Returning ownership of objects back to their original variables via functions' return values (ch4/ch4-check-sats-3.rs)

```
#![allow(unused_variables)]

#[derive(Debug)]
struct CubeSat {
    id: u64,
}

#[derive(Debug)]
enum StatusMessage {
    Ok,
}

fn check_status(sat_id: CubeSat) -> CubeSat {
    println!("{:?}: {:?}", sat_id, StatusMessage::Ok);
    sat_id
}

fn main () {
    let sat_a = CubeSat { id: 0 };
    let sat_b = CubeSat { id: 1 };
    let sat_c = CubeSat { id: 2 };

    let sat_a = check_status(sat_a); ①
    let sat_b = check_status(sat_b);
    let sat_c = check_status(sat_c);

    // "waiting" ...
    let sat_a = check_status(sat_a);
    let sat_b = check_status(sat_b);
    let sat_c = check_status(sat_c);
}
```

① Now that the return value of `check_status()` is the original `sat_a`, the new `let` binding is "reset"

Printing to the console changes, this responsibility has been pushed into `check_status()`. The output from the new `main()` function looks like this:

Listing 4.10. Output of [Listing 4.9](#)

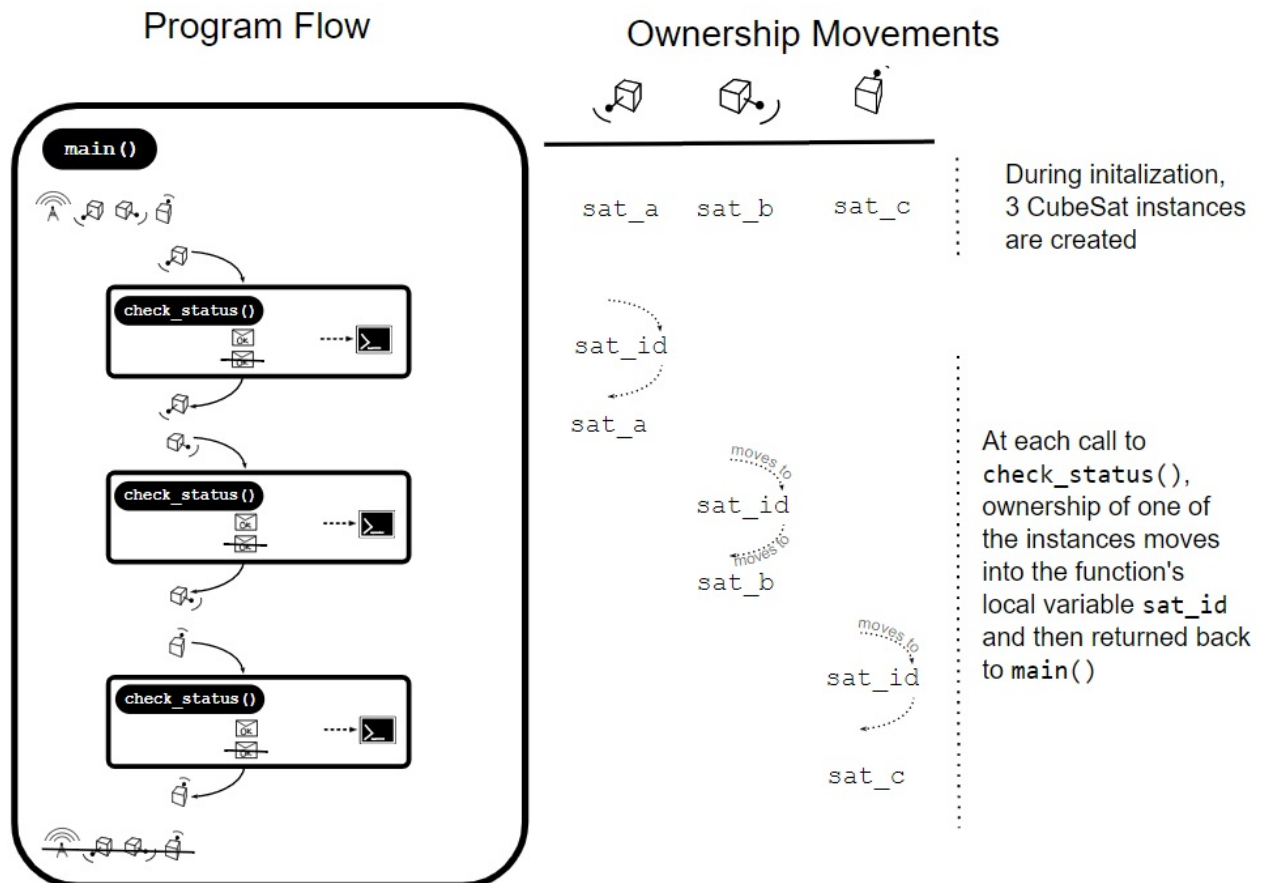
```
CubeSat { id: 0 }: Ok
```

```

CubeSat { id: 1 }: Ok
CubeSat { id: 2 }: Ok
CubeSat { id: 0 }: Ok
CubeSat { id: 1 }: Ok
CubeSat { id: 2 }: Ok

```

A visual overview of the ownership movements within [Listing 4.9](#) is provided below.



[35] Within Rust community, the term variable binding is preferred as is is more technically correct.

4.5 RESOLVING OWNERSHIP ISSUES

Rust's ownership system is excellent. It provides a route to memory safety without needing a garbage collector. But there is a but. The ownership system can trip you up if you don't understand what's happening. This is particularly the case when you bring the programming style from your past experience to a new

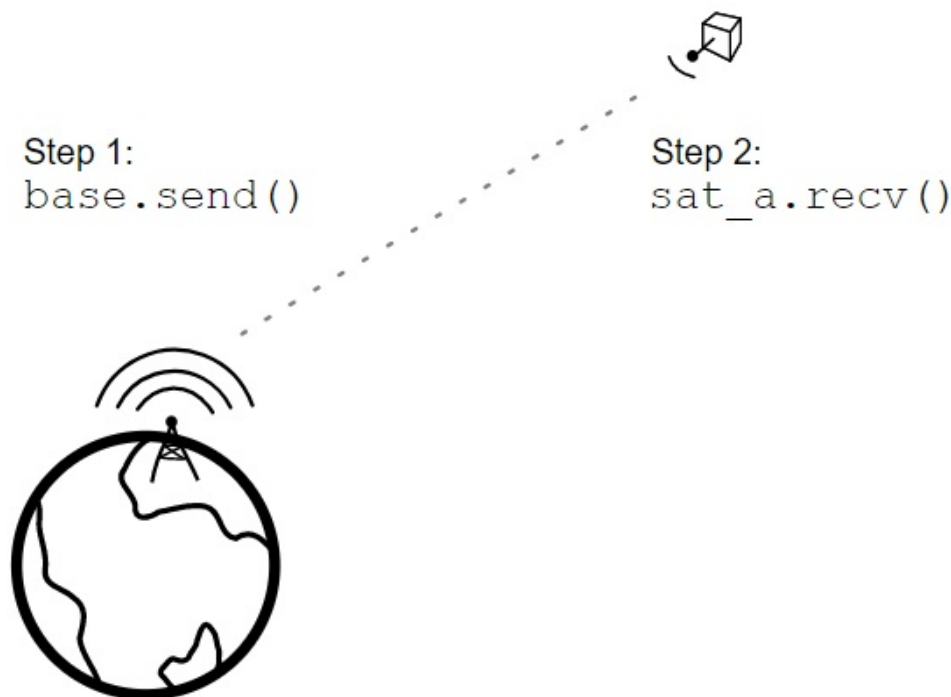
paradigm.

Four general strategies can help with ownership issues:

1. Use references where full ownership is not required
2. Duplicate the value
3. Refactoring code to reduce the number of long-lived objects
4. Wrap your data in a type designed to assist with movement issues

To examine each of these strategies, let's extend the capabilities of our satellite network. Let's give the ground station and our satellites ability to send and receive messages.

Figure 4.4. Game plan: Sending Messages to Our Satellites



Ignoring the details of implementing the methods, we want to avoid code that looks like this:

```
base.send(sat_a, "hello!"); ①  
sat_a.recv();
```


- ① Moving ownership of `sat_a` to a local variable in `base.send()` will end up hurting us. That value will no longer be accessible for the rest of `main()`.

To get to a toy implementation, we will need a few more types to help us out somewhat. In [Listing 4.11](#), a new field is added to `CubeSat`, `mailbox`. `CubeSat.mailbox` is a `Mailbox` struct that contains a vector of `Messages` within its `messages` field. We have aliased `String` to `Message`, giving us the functionality of the `String` type without needing to implement it ourselves.

Listing 4.11. Adding a Mailbox Type to our System

```
#[derive(Debug)]
struct CubeSat {
    id: u64,
    mailbox: Mailbox,
}

#[derive(Debug)]
enum StatusMessage {
    Ok,
}

#[derive(Debug)]
struct Mailbox {
    messages: Vec<Message>,
}

type Message = String;
```

Creating a `CubeSat` instance has become slightly more complicated. To create one, we also need to create its associated `Mailbox` and the mailbox's associated `Vec<Message>`.

Listing 4.12. Creating a new CubeSat with Mailbox

```
CubeSat { id: 100, mailbox: Mailbox { messages: vec![] } }
```

Another type to add is to represent the ground station itself. We will use a bare struct for the moment. That will allow us to add methods to it and give us the option of adding a mailbox as a field later on also.

Listing 4.13. Defining a struct to represent our ground station

```
struct GroundStation;
```

Creating an instance of `GroundStation` should be trivial for you now.

Listing 4.14. Creating a new ground station

```
GroundStation {};
```

Now that we have our new types in place, let's put them to work.

4.5.1 Use references where full ownership is not required

The most common change you will make to your code is to reduce the level of access you require. Instead of requesting ownership, use a borrow in your function definitions. For read-only access, use `& T`. For read/write access, use `&mut T`. Ownership might be needed in advanced cases, such as when functions wish to adjust the lifetime of their arguments.

Here is a comparison of the two different approaches.

Using ownership	Using a mutable reference
<pre>fn send(to: CubeSat, msg: Message) { ① to.mailbox.messages.push(msg); }</pre>	<pre>fn send(to: &mut CubeSat, msg: Message) { to.mailbox.messages.push(msg); }</pre>
① Moving ownership of the <code>to</code> variable.	

Sending messages will eventually be wrapped up in a method, but in essence functions implementing it must modify the internal mailbox of the `CubeSat`. For simplicity's sake, we'll return `()` and hope for the best in case of transmission difficulties caused by solar winds.

Here is the flow that we want to end up with. We send a message to `sat_a` and then receive it.

```
base.send(sat_a, "hello!".to_string());

let msg = sat_a.recv();
println!("sat_a received: {:?}", msg); // -> Option("hello!")
```

and implementations of those two methods.

Listing 4.15. Adding `GroundStation.send()` and `SubeSat.recv()` methods

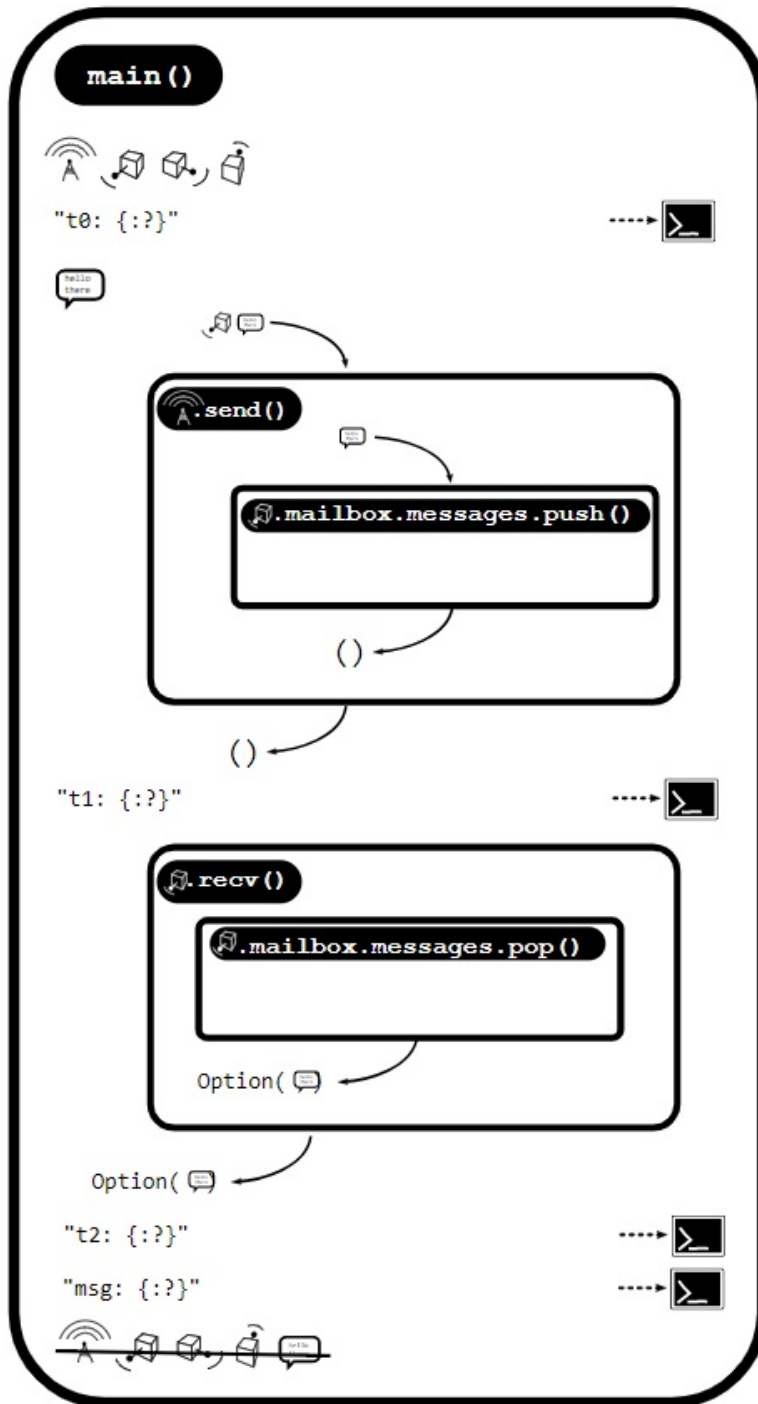
```
impl GroundStation {
    fn send(&self, to: &mut CubeSat, msg: Message) { ①
        to.mailbox.messages.push(msg);                ②
    }
}

impl CubeSat {
    fn recv(&mut self) -> Option<Message> {
        self.mailbox.messages.pop()
    }
}
```

- ① `&self` indicates that `GroundStation.send()` only requires a read-only reference to `self`. The recipient `to` is taking a mutable borrow (`&mut`) of the `CubeSat` instance. The `msg` takes full ownership of its `Message` instance
- ② Ownership of the `Message` instance transfers from `msg` into `messages.push()` as a local variable

Notice that both `GroundStation.send()` and `CubeSat.recv()` require mutable access to a `CubeSat` instance, as both methods are modifying the underlying `CubeSat.messages` vector. We move ownership of the message that we're sending into the `messages.push()`. This will provide us with some quality assurance later, notifying us if we were to access a message after it had already been sent.

Figure 4.5. Game plan: Pictorial View of "Avoiding Ownership Issues with References"



The code listing [Listing 4.16](#) brings together all of the code snippets in this section and prints out the following.

```
t0: CubeSat { id: 0, mailbox: Mailbox { messages: [] } } ①
t1: CubeSat { id: 0, mailbox: Mailbox { messages: ["hello there!"] } }
t2: CubeSat { id: 0, mailbox: Mailbox { messages: [] } }
msg: Some("hello there!")
```

- ① The output `t0 - t2` are added to assist your understanding of how data is flowing through the program.

Listing 4.16. Avoiding Ownership Issues with References (ch4/ch4-sat-mailbox.rs)

```
#[derive(Debug)]
struct CubeSat {
    id: u64,
    mailbox: Mailbox,
}

#[derive(Debug)]
struct Mailbox {
    messages: Vec<Message>,
}

type Message = String;

struct GroundStation;

impl GroundStation {
    fn send(&self, to: &mut CubeSat, msg: Message) {
        to.mailbox.messages.push(msg);
    }
}

impl CubeSat {
    fn recv(&mut self) -> Option<Message> {
        self.mailbox.messages.pop()
    }
}

fn main() {
    let base = GroundStation {};
    let mut sat_a = CubeSat { id: 0, mailbox: Mailbox { messages: vec![] } };

    println!("t0: {:?}", sat_a);
    base.send(&mut sat_a, Message::from("hello there!")); ①

    println!("t1: {:?}", sat_a);

    let msg = sat_a.recv();
    println!("t2: {:?}", sat_a);

    println!("msg: {:?}", msg);
}
```

- ① We don't have a completely ergonomic way to create `Message` instances yet. Instead, we'll take advantage of the `String::from()` method, which converts from `&str` to `String` (aka `Message`).

4.5.2 Use Fewer Long-Lived Values

If we have a large, long-standing object such as a global variable, it can be somewhat unwieldy to keep this around for every component of your program that needs it. Rather than using an approach of long-standing objects, consider making objects that are more discrete and ephemeral. Ownership issues can sometimes be resolved by considering the design of the overall program.

In our CubeSat case, we don't need to handle much complexity at all. Each of our four variables, `base`, `sat_a`, `sat_b` and `sat_c` live for the duration of `main()`. In a production system, there may be hundreds of different components and many thousands of interactions to manage. To increase the manageability of this kind of scenario, let's break things apart.

Figure 4.6. Game plan: Short-Lived Variables

main()



sat_ids()

Vec<u64>

for {

hello there

0

(then 1 & 2)

.connect()



(then &)



hello there

(...)

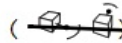
.send()

hello there

.mailbox.messages.push()

()

()



}

for {



(then &)

.recv()

.mailbox.messages.pop()

Option()

Option()

"msg: {:?}"

Option()

}



To implement this kind of strategy, we will create a function that returns CubeSat identifiers. That function is assumed to be a black box that's responsible for communicating with some store of identifiers, such as a database. Whenever we need to communicate with a satellite, we'll create a new object. In this way, there is no requirement for us to maintain live objects for the whole of the program's duration. It also has the dual benefit that we can afford to transfer ownership of our short-lived variables to other functions.

```
fn fetch_sat_ids() -> Vec<u64> { ①
    vec![1,2,3]
}
```

① Returns a vector of CubeSat IDs

We'll also create a method for GroundStation that allows us to create a CubeSat instance on demand once.

```
impl GroundStation {
    fn connect(&self, sat_id: u64) -> CubeSat {
        CubeSat { id: sat_id, mailbox: Mailbox { messages: vec![] } }
    }
}
```

Now we are now somewhat closer to our intended outcome. Our main function looks like the code snippet below. In effect, we've implemented the first half of [Figure 4.6](#).

```
fn main() {
    let base = GroundStation();

    let sat_ids = fetch_sat_ids();

    for sat_id in sat_ids {
        let mut sat = base.connect(sat_id);

        base.send(&mut sat, Message::from("hello"));
    }
}
```

But there's a problem. Our cubeSat instances die at the end of the for loop's

scope, along with any messages that base sends them. To carry on our design decision of short-lived variables, the messages need to live somewhere outside of the `CubeSat` instances. In a real system, they would live on the RAM of a device in zero gravity. In our not-really-a-simulator, let's put them in a buffer object that lives for the duration of our program.

Our message store will be a `Vec<Message>`, aka our `Mailbox` type that we defined in one of the first code examples of this chapter. We'll change the `Message` struct to add a sender and recipient field. That way our now-proxy `CubeSat` instances can match against their id to receive messages.

```
#[derive(Debug)]
struct Mailbox {
    messages: Vec<Message>,
}

#[derive(Debug)]
struct Message {
    to: u64,
    content: String,
}
```

We also need to re-implement sending and receiving messages. Up until now, `CubeSat` objects have had access to their own mailbox object. The central `GroundStation` also had the ability to sneak into those mailboxes to send messages. That needs to change now, as only one mutable borrow may exist per object.

In the modifications below, the `Mailbox` instance is given the ability to modify its own messages vector. When any of the satellites transmit messages, they take a mutable borrow to the mailbox. They then defer the delivery to the mailbox object. According to this API, our satellites are able to call `Mailbox` methods, they are just not allowed to touch any internal `Mailbox` data themselves.

Listing 4.17. Modifications to Mailbox and

```
impl GroundStation {
    fn send(&self, mailbox: &mut Mailbox, to: &CubeSat, msg: Message) { ①
        mailbox.post(to, msg);
    }
}
```

```

impl CubeSat {
    fn recv(&self, mailbox: &mut Mailbox) -> Option<Message> {           ②
        mailbox.deliver(&self)
    }
}

impl Mailbox {
    fn post(&mut self, msg: Message) {                                   ③
        self.messages.push(msg);
    }

    fn deliver(&mut self, recipient: &CubeSat) -> Option<Message> {      ④
        for i in 0..self.messages.len() {
            if self.messages[i].to == recipient.id {
                let msg = self.messages.remove(i);                       ⑤
                return Some(msg);                                         ⑥
            }
        }

        None                                                             ⑦
    }
}

```

- ① Sending messages becomes a call to `Mailbox.post()`, yielding ownership of a `Message`
- ② Receiving messages becomes a call to `Mailbox.deliver()`, gaining ownership of a `Message`
- ③ `Mailbox.post()` requires mutable access to itself and ownership over a `Message`
- ④ `Mailbox.deliver()` requires a shared reference to a `CubeSat`, to pull out its `id` field
- ⑤ Astute readers will notice a strong anti-pattern here: mutating a collection while it is being iterated over. This is legal here because of the return on the next line. The compiler can prove that another iteration will not occur and allows this to go through.
- ⑥ When we find a message, return early with the `Message` wrapped in `Some`, per the `Option` type.
- ⑦ When no messages are found, return `None`

With that groundwork in place, we're now able to fully implement the strategy laid out in [Figure 4.6](#). The output from a compiled version of [Listing 4.19](#) is below, followed by the listing.

Listing 4.18. Output of [Listing 4.19](#)

```
CubeSat { id: 1 }: Some(Message { to: 1, content: "hello" })
CubeSat { id: 2 }: Some(Message { to: 2, content: "hello" })
CubeSat { id: 3 }: Some(Message { to: 3, content: "hello" })
```

Listing 4.19. Implementing the Short-Lived Variables Strategy (ch4/ch4-short-lived-strategy.rs)

```
#![allow(unused_variables)]

#[derive(Debug)]
struct CubeSat {
    id: u64,
}

#[derive(Debug)]
struct Mailbox {
    messages: Vec<Message>,
}

#[derive(Debug)]
struct Message {
    to: u64,
    content: String,
}

struct GroundStation {}

impl Mailbox {
    fn post(&mut self, msg: Message) {
        self.messages.push(msg);
    }

    fn deliver(&mut self, recipient: &CubeSat) -> Option<Message> {
        for i in 0..self.messages.len() {
            if self.messages[i].to == recipient.id {
                let msg = self.messages.remove(i);
                return Some(msg);
            }
        }

        None
    }
}

impl GroundStation {
    fn connect(&self, sat_id: u64) -> CubeSat {
        CubeSat {
            id: sat_id,
        }
    }

    fn send(&self, mailbox: &mut Mailbox, msg: Message) {
        mailbox.post(msg);
    }
}
```

```

    }
}

impl CubeSat {
    fn recv(&self, mailbox: &mut Mailbox) -> Option<Message> {
        mailbox.deliver(&self)
    }
}

fn fetch_sat_ids() -> Vec<u64> {
    vec![1,2,3]
}

fn main() {
    let mut mail = Mailbox { messages: vec![] };

    let base = GroundStation {};

    let sat_ids = fetch_sat_ids();

    for sat_id in sat_ids {
        let sat = base.connect(sat_id);
        let msg = Message { to: sat_id, content: String::from("hello") };
        base.send(&mut mail, msg);
    }

    let sat_ids = fetch_sat_ids();

    for sat_id in sat_ids {
        let sat = base.connect(sat_id);

        let msg = sat.recv(&mut mail);
        println!("{:?:} {:?:}", sat, msg);
    }
}

```

4.5.3 Duplicate the value

Having a single owner for every object can mean significant up-front planning and/or refactoring of your software. As we saw in the previous section, it can be quite a lot of work to wriggle oneself out of an early design decision.

One alternative to refactoring is to simply copy values. Doing this often is typically frowned upon, but it can be useful in a pinch. Primitive types, such as integers, are a good example of that. Primitive types are cheap for a CPU to duplicate. So cheap in fact, that Rust always copies them if it would otherwise worry about ownership being moved.

Types can opt into two modes of being copied: `Clone` and `Copy`. `Copy` acts implicitly whenever ownership would otherwise be moved. The bits of object *a* are replicated to create object *b*. `Clone` acts explicitly. Types that implement `Clone` have a `.clone()` method that is permitted to do whatever it needs to create a new type.

Table 4.1. Distinguishing `copy` and `clone`

Copy	Clone
* always fast and cheap * implementation details are provided by the language—always a bit-for-bit exact copy of the value	* may potentially be expensive * implementation defined by the programmer in a type-specific manner

So why do Rust programmers not always use `Copy`? Well, to implement `Copy`, your types must consist of types that themselves implement `Copy`. Integers and floating point numbers implement `Copy`, but `String` and many other types such as `Vec<T>` do not. The reason `String` and friends do not implement `Copy` is that the types' internal pointers would be duplicated, not the data that those pointers refer to. That could lead to multiple values pointing to the same data, leading to compiler confusion about which one is responsible for that data.

IMPLEMENTING `COPY`

Let's go back to our original example ([Listing 4.4](#)) that caused the original movement issue. Here it is replicated for convenience, with `sat_b` and `sat_c` removed for brevity.

```
#[derive(Debug)]
struct CubeSat {
    id: u64,
}

#[derive(Debug)]
enum StatusMessage {
    Ok,
}

fn check_status(sat_id: CubeSat) -> StatusMessage {
    StatusMessage::Ok
}

fn main() {
```

```

let sat_a = CubeSat { id: 0 };

let a_status = check_status(sat_a);
println!("a: {:?}", a_status);

let a_status = check_status(sat_a); ①
println!("a: {:?}", a_status);
}

```

① The second call to `check_status(sat_a)` is the location of error

At this stage, our program consisted of types that contain types that themselves implement `Copy`. That's good, as it means implementing it ourselves is fairly straightforward:

Listing 4.20. Deriving `Copy` for Types That are Made Up of Types That Implement `Copy` Themselves

```

#[derive(Copy, Clone, Debug)] ①
struct CubeSat {
    id: u64,
}

#[derive(Copy, Clone, Debug)] ①
enum StatusMessage {
    Ok,
}

```

① Adding `#[derive(Copy)]` tells the compiler to add an implementation itself

It's also possible to implement `Copy` manually. The `impl` blocks are impressively terse.

Listing 4.21. Implementing the `Copy` trait manually

```

impl Copy for CubeSat { }

impl Copy for StatusMessage { }

impl Clone for CubeSat { ①
    fn clone(&self) -> Self {
        CubeSat { id: self.id } ②
    }
}

impl Clone for StatusMessage {

```

```
fn clone(&self) -> Self {
    *self ③
}
}
```

- ① Implementing Copy requires an implementation of Clone
- ② If desired, we can write out the creation of a new object ourselves..
- ③ ..but often we can simply dereference self

USING CLONE AND COPY

Now that we know how to implement them, let's put Clone and Copy to work.

We've discussed that Copy is implicit. Whenever ownership would otherwise move, such as during assignment and passing through function barriers, data is copied instead. Clone requires an explicit call to `.clone()`. That's a useful marker in non-trivial cases such as this example, because it warns the programmer that the process may be expensive.

Listing 4.22. Using clone and copy (ch4/ch4-check-sats-clone-and-copy-traits.rs)

```
#[derive(Debug, Clone, Copy)] ①
struct CubeSat {
    id: u64,
}

#[derive(Debug, Clone, Copy)] ①
enum StatusMessage {
    Ok,
}

fn check_status(sat_id: CubeSat) -> StatusMessage {
    StatusMessage::Ok
}

fn main () {
    let sat_a = CubeSat { id: 0 };

    let a_status = check_status(sat_a.clone()); ②
    println!("a: {:?}", a_status.clone());      ②

    let a_status = check_status(sat_a);          ③
    println!("a: {:?}", a_status);              ③
}
```

- ① Copy implies Clone, so we can use either trait later on
- ② Cloning each object is as easy as calling `.clone()`
- ③ Copy works as expected

4.5.4 Wrap Data Within Specialty Types

This chapter, we have been discussing Rust's ownership system and ways to navigate the constraints it imposes. A final strategy that is quite common is to use "wrapper" types that present a façade to the outside world of move semantics, but actually are doing something special under the hood.

Rust allows programmers to opt-in to runtime garbage collection. To explain this, we'll need to introduce some new notation. `Rc<T>` means a "reference counted type `T`". We could wrap a single instance of `GroundStation` in a `Rc`, providing shared access to each of the satellites. Setting up that scenario involves a call to the `Rc::new()` static method.

Listing 4.23. Wrapping a User-Defined Type in Rc (ch4/ch4-rc-groundstation.rs)

```
use std::rc::Rc; ①

#[derive(Debug)]
struct GroundStation {}

fn main() {
    let base: Rc<GroundStation> = Rc::new(GroundStation {}); ②

    println!("{:?}", base);
}
```

- ① The `use` keyword brings modules from the standard library into local scope
- ② "Wrapping" involves enclosing the `GroundStation` instance in a call to `Rc::new()`

`Rc<T>` implements `Clone`. Every call to `base.clone()` increments an internal counter. Every `Drop` decrements that counter. When the internal counter reaches zero, the original instance is freed.

`Rc<T>` does not allow mutation. To permit that, we need to wrap our wrapper. `Rc<RefCell<T>>` is a type that can be used to perform *interior mutability*. An object that has interior mutability presents an immutable façade, while internal values are being modified.

In the following example, the variable `base` is able to be modified despite being marked as an immutable variable. It's possible to visualize this by looking at the changes to the internal `base.radio_freq`.

```
base: RefCell { value: GroundStation { radio_freq: 87.65 } }  
base_2: GroundStation { radio_freq: 75.31 }  
base: RefCell { value: GroundStation { radio_freq: 75.31 } }  
base: RefCell { value: "<borrowed>" } ①  
base_3: GroundStation { radio_freq: 118.52000000000001 }
```

① `value: "<borrowed>"` indicates that `base` has been mutably borrowed by somewhere else, and is no longer generally accessible

An important consideration to remember is that Rust's guarantees remain. They become enforced at runtime. It is certainly possible to cause runtime panics if you ignore the ownership and borrowing rules.

Listing 4.24. Using `Rc<RefCell<T>>` to Permit Mutation Within an Object Marked Immutable

```
use std::rc::Rc;  
use std::cell::RefCell;  
  
#[derive(Debug)]  
struct GroundStation {  
    radio_freq: f64 // Mhz  
}  
  
fn main() {  
    let base: Rc<RefCell<GroundStation>> = Rc::new(RefCell::new(  
        GroundStation {  
            radio_freq: 87.65  
        })  
    );  
  
    println!("base: {:?}", base);  
  
    { // introduce a new scope  
        let mut base_2 = base.borrow_mut();  
        base_2.radio_freq -= 12.34;  
    }  
}
```

```

    println!("base_2: {:?}", base_2);
}

println!("base: {:?}", base);

let mut base_3 = base.borrow_mut();
base_3.radio_freq += 43.21;

println!("base: {:?}", base);
println!("base_3: {:?}", base_3);
}

```

Adding more functionality—for example, reference counting semantics rather than move semantics—from types reduces their runtime performance. `Rc<T>` can be handy when implementing `clone` would be prohibitively expensive.



Note

`Rc<T>` is not thread safe. Replace `Rc<T>` with `Arc<T>` and `Rc<RefCell<T>>` with `Arc<Mutex<T>>`. "Arc" stands for *atomic reference counter*.

4.6 SUMMARY

In this chapter you learned that:

- A value's *owner* is responsible for cleaning up after that value when its lifetime ends.
- A value's *lifetime* is the period when accessing that value is valid behavior. Attempting to access a value after its lifetime has expired will lead to code that won't compile.
- To *borrow* a value means to access that value
- Several tactics are available to you if you find that the borrow checker won't allow your program to compile. This often means that you will need to rethink the design of your program.
- Use shorter-lived values, rather than values that stick around for a long time.
- Borrows may be read-only or read/write. Only one read/write borrow may exist at any one time.
- Duplicating a value can be a pragmatic way to break an impasse

with the borrow checker. To duplicate a value, implement `Clone` or `Copy`.

- It's possible to opt-in to reference counting semantics through `Rc<T>`
- Rust supports a feature known as *interior mutability* that enables types to present themselves as immutable, even though their values can change over time

5 DATA IN DEPTH

In this chapter:

- Learn how data is represented in the computer
- Build a working CPU emulator
- Create your own numeric data type
- Understand the inner workings of floating point numbers

This chapter is all about understanding how zeroes and ones can be built up to become much larger objects such as text, images and sound. We will also touch upon how computers do computation. By the end of the chapter, you will have emulated a fully-functional computer with CPU, memory and user-defined functions. You will break apart floating point numbers to create a numeric data type of your own making that only takes up a single byte. The chapter introduces a number of terms that may not be familiar to programmers who have never done systems programming, such as *endianness* and *integer overflow*.

5.1 BIT PATTERNS AND TYPES

A small but important lesson is that a single bit pattern can mean different things. The type system of a higher-level language, such as Rust, is just an artificial abstraction over reality. Understanding this becomes important as you begin to unravel some of that abstraction and to gain a deeper understanding of how computers work.

[Listing 5.2](#) is an example that uses the same bit pattern to represent two different numbers. The type system—not the CPU—is what makes the distinction.

Listing 5.1. Output of [Listing 5.2](#)

```
a: 1100001111000011 50115
```

b: 1100001111000011 -15421

Listing 5.2. Demonstrating that the same sequence of bits can mean different things, depending on the type information attached with it. (ch5-int-vs-int.rs)

```
fn main() {
    let a: u16 = 50115;
    let b: i16 = -15421;

    println!("a: {:016b} {}", a, a); ①
    println!("b: {:016b} {}", b, b); ①
}
```

① Note that these two values have the same bit pattern, but different types

The different mapping between bit strings and numbers explains part of the distinction between “binary” files and “text” files. Text files are just binary files that happen to follow a consistent mapping between bit strings and characters. This mapping is called an encoding. Arbitrary files don’t describe their meaning to the outside world, which makes them very opaque.

We can take this process one step further. What happens if we ask Rust to treat a bit pattern produced by one type as another?

Listing 5.3. Inspecting a float’s bit string by interpreting its bits as an integer (ch5-f32-as-u32.rs)

```
fn main() {
    let a: f32 = 42.42;
    let frankentype: u32 = unsafe { std::mem::transmute(a) }; ①

    println!("{}", frankentype); ②
    println!("{:032b}", frankentype); ③

    let b: f32 = unsafe { std::mem::transmute(frankentype) }; ④
    println!("{}", b); ④
}
```

① Note, no semi-colon here. We want the result of this expression to feed into the outer scope.

② Use the bits from 42.42_f32 as an integer.

- ③ The `{:032b}` syntax means format as a binary—via the `std::fmt::Binary` trait—with zeros padded on the left to fill 32 places. Without `032`, leftmost bits with a value of 0 would be truncated in the output.
- ④ Confirm that the operation is symmetrical.

When compiled and run, the code from [Listing 5.3](#) produces the following output:

```
1110027796
01000010001010011010111000010100
42.42
```

To print a value as individual bits, the type of that value must implement `fmt::Binary`. `f32` doesn't implement `fmt::Binary`, but integer types do. There are two integer types guaranteed to take up the same number of bits as `f32`, `i32` and `u32`. The decision about which to choose is somewhat arbitrary. Let's use `u32`. So, now we need a way to treat an `f32` as an `u32`, without affecting any of the underlying bits. That's what `std::mem::transmute()` does. It allows us to tell the compiler that we know best and to treat one value's data as if it is actually another type.



Tip

The keyword `unsafe` does not imply that code is *inherently* dangerous. It indicates that the compiler is not able to guarantee that the program memory safe.



Warning

`unsafe` means that the programmer is fully responsible for maintaining the program's integrity. Some functionality allowed within `unsafe` blocks is more difficult to verify than others.

For example, the `std::mem::transmute()` function is one of the least safe in the language. It shreds all type safety. Investigate alternatives before using it in your own code.

Mixing data types underneath a program is inherently chaotic, so we need to wrap the operation within an `unsafe` block. `unsafe` tells the Rust compiler, “Stand back, I’ll take care of things from here. I’ve got this.”. Unsafe Rust is exactly the same language as safe Rust. It just does its work without the benefit of the compiler’s guarantees. As we peek at and poke individual bytes during the course of the book, we’ll be using `unsafe` sections of code.

5.2 LIFE OF AN INTEGER

During earlier chapters, we spent some time discussing what it means for an integer to be an `i32` or a `u8` or a `usize`.

Integers are like small, delicate fish. They do what they do remarkably well, but take them outside of their natural range and they die a quick, painful death.

Integers live within a fixed range. When represented inside the computer, they occupy a fixed number of bits per type. Unlike floating point numbers, they cannot sacrifice their precision to extend their bounds. Once those bits have been filled up with ones, the only way forward is back to all zeros.

A 16-bit integer can represent numbers between 0 and 65,535, inclusive. What happens when you want to count to 65,536? Let’s find out.

The technical term for the class of problem that we are investigating is *integer overflow*. One of the most innocuous ways of overflowing an integer is by incrementing forever. [Listing 5.4](#) is a trivial example of this.

Listing 5.4. Investigating the effect of incrementing an integer past its range (ch5/ch5-to-oblivion.rs)

```
fn main() {
    let mut i: u16 = 0;
    print!("{}", i);

    loop {
        i += 1000;
        print!("{}", i);
        if i % 10000 == 0 {
            print!("\n")
        }
    }
}
```

```
}  
}
```

When we try to run [Listing 5.4](#), things don't end well for our program:

```
$ rustc ch5-to-oblivion.rs && ./ch5-to-oblivion  
0..1000..2000..3000..4000..5000..6000..7000..8000..9000..10000..  
11000..12000..13000..14000..15000..16000..17000..18000..19000..20000..  
21000..22000..23000..24000..25000..26000..27000..28000..29000..30000..  
31000..32000..33000..34000..35000..36000..37000..38000..39000..40000..  
41000..42000..43000..44000..45000..46000..47000..48000..49000..50000..  
51000..52000..53000..54000..55000..56000..57000..58000..59000..60000..  
thread 'main' panicked at 'attempt to add with overflow', ch5-to-oblivion.rs:5:7  
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace  
61000..62000..63000..64000..65000..
```

A panicked program is a dead program. A panic means that the programmer has asked the program something that's impossible. It doesn't know what to do to proceed and shuts itself down.

To understand why this is such a critical class of bug, let's take a look at what's going on under the hood. Here is a program that prints out six numbers with their bit patterns laid out in literal form.



Tip

Try compiling the code with optimizations enabled via `rustc -O ch5-to-oblivion.rs` and running the resulting executable. The behavior is quite different.

When compiled, [Listing 5.5](#) should print out the following short line:

```
0, 1, 2, ..., 65533, 65534, 65535
```

Listing 5.5. Demonstrating how bit patterns in `u16` translate to a fixed number of integral values (`ch5/ch5-bit-patterns.rs`)

```
fn main() {  
    let zero: u16 = 0b0000_0000_0000_0000;  
    let one: u16 = 0b0000_0000_0000_0001;  
    let two: u16 = 0b0000_0000_0000_0010;  
    // ...  
    let sixtyfivethousand_533: u16 = 0b1111_1111_1111_1101;  
}
```



```

let sixtyfivethousand_534: u16 = 0b1111_1111_1111_1110;
let sixtyfivethousand_535: u16 = 0b1111_1111_1111_1111; ①

print!("{}", {}, {}, {}, ..., ", zero, one, two);
println!("{}", {}, {}, {}", sixty5_533, sixty5_534, sixty5_535);
}

```

① The problem we're interested in is what happens when there's no more space left. 65,536 cannot be represented by u16.

There is another easy way to kill a program using a similar technique. In this case, we ask Rust to fit 400 into an u8, which can only count up to 255 values.

Listing 5.6. Impossible addition (ch5/ch5-impossible-addition.rs)

```

#[allow(arithmetic_overflow)] ①

fn main() {
    let (a, b) = (200, 200);
    let c: u8 = a + b;          ②
    println!("200 + 200 = {}", c);
}

```

① This declaration is required because the Rust compiler can detect this obvious overflow situation.

② The type declaration is important here. Without it, Rust won't assume that you will be trying to create an impossible situation.

The code compiles, but one of two things happen:

- The program panics:

```

thread 'main' panicked at 'attempt to add with overflow', 5-impossible-add.rs:3:15
note: Run with `RUST_BACKTRACE=1` for a backtrace

```

This behavior can be invoked via executing `rustc` with its default options: `rustc ch5-impossible-add.rs && ch5-impossible-add`

- The program gives you the wrong answer:

```

200 + 200 = 144

```

This behavior can be invoked by executing `rustc` with the `-O` flag:
`rustc -O ch5-impossible-add.rs && ch5-impossible-add`

There are two small lessons here.

- It's important to understand the limitations of your types
- Despite Rust's strengths, programs written in Rust can still break

Developing strategies for preventing integer overflow is one of the ways that systems programmers are distinguished from others. Programmers who only have experience with dynamic languages are extremely unlikely to encounter an integer overflow. Dynamic languages typically check to see that the result of integer expressions will be able to fit. When they can't, the variable that's receiving the result will be promoted to a wider integer type.

When developing performance critical code, you get to choose which parameters to adjust. If you use fixed-sized types, you gain speed and you need to accept some risk. To mitigate that risk, you can check to see that overflow won't occur at runtime. Imposing those checks will slow you down. Another, much more common option, is to sacrifice space by using a large integer type, such as `i64`. To go higher still, you'll need to move to arbitrary size integers, which come with their own costs.

5.2.1 Understanding Endianness

CPUs argue about which way round the individual bytes that make up integers should be laid out. Some prefer byte sequences to be from left-to-right and others prefer them right-to-left. This preference is known as a CPU's *endianness*. This is one of the reasons why copying an executable file from one computer to another might not work.

Let's consider a 32-bit integer that is representing the number made up of four bytes: AA, BB, CC and DD. [Listing 5.7](#), with the help of our friend `sys::mem::transmute()`, demonstrates that byte order matters.

When compiled and executed, the code from [Listing 5.7](#) will print out one of two

things, depending on the endianness of your machine.

Most computers [\[36\]](#) that people run for day-to-day work these days will print out

```
-573785174 vs -1430532899
```

but more exotic hardware swaps the two numbers around

```
-1430532899 vs -573785174
```

Listing 5.7. Inspecting Endianness (ch5-endianness.rs)

```
use std::mem;

fn main() {
    let big_endian: [u8; 4] = [0xAA, 0xBB, 0xCC, 0xDD];
    let little_endian: [u8; 4] = [0xDD, 0xCC, 0xBB, 0xAA];

    let a: i32 = unsafe { mem::transmute(big_endian) }; ①
    let b: i32 = unsafe { mem::transmute(little_endian) }; ①

    println!("{}", a vs b);
}
```

① The `mem::transmute()` function instructs the compiler to interpret its argument as the type on the left (`i32`).

The terminology comes from the "significance" of the bytes in the sequence. To take you back to when you learned addition, we can factor the number 123 into three parts:

100×1	100
10×2	20
1×3	3

Summing all of these parts gets us back to our original number. The first part, 100, is labelled as the "most significant". When written out in the conventional way, 123 as 123, we are writing in *big endian* format. Were we to invert that

ordering, i.e. by writing 123 as 321, we would be writing in *little endian* format. Binary numbers work in a very similar way. Each number part is a power of 2 ($2^0, 2^1, 2^2, \dots, 2^n$), rather than a power of 10 ($10^0, 10^1, 10^2, \dots, 10^n$).

Before the late-1990s, endianness was a very big issue, especially in the server market. Glossing over the fact that a number of processors can support bidirectional endianness, Sun Microsystems, Cray, Motorola and SGI went one way. ARM decided to hedge its bets and develop a “bi-endian” architecture. Intel went the other way. The other way won. Integers are almost certainly stored in *little endian* format.

In addition to multi-byte sequences, there is a related problem within a byte. Should a `u8` that represents 3 look like `0000_0011` or should it look like `1100_0000`? The computer’s preference for layout of individual bits is known as its *bit numbering* or *bit endianness*. It’s very unlikely that this internal ordering will affect your day-to-day programming. To investigate further, look for your platform’s details on which end its *most significant bit* lies



Tip

The abbreviation MSB can be deceptive. Authors use the same abbreviation to refer to both most significant *bit* (bit numbering) and most significant *byte* (endianness).

[\[36\]](#) In 2020, the x86-64/AMD64 CPU architecture is dominant.

5.3 DECIMAL NUMBERS

One of the claims made at the start of this chapter was that understanding more about bit patterns should enable you to compress your data. Let’s put that into practice. In this section, you will learn how to pull bits out of a floating point number and inject them into a single byte format of your own creation.

Here is some context for the problem at hand: Machine learning practitioners often need to store and distribute large models. A model, for our purposes here, is just a large array of numbers. The numbers within those models often fall

within the ranges $0 \dots 1$ or $-1 \dots 1$, depending on the application. Given we don't need the whole range that `f32` or `f64` support, why use all of their bytes? Let's see how far we can get with 1. Because there is a known limited range, it's possible to create a decimal number format that can model that range compactly.

To start, we're going to need to learn about how decimal numbers are represented inside today's computers. We're going to need to learn about the internals of floating point.

5.3.1 About Floating Point

Each floating point number is laid out in memory as scientific notation. If you're unfamiliar with scientific notation, here is a very quick primer. Scientists will describe the mass of Jupiter as 1.898×10^{27} kg and the mass of an ant as 3.801×10^{-4} kg. The key insight is that the same number of characters are used to describe vastly different scales. Computer scientists have taken advantage of that insight to create a fixed-width format that encodes a very wide range of numbers.

Each position within a number in scientific notation is given a role:

- a sign, which is implied in our two examples, would be present for negative numbers (-)
- the *mantissa*, also known as the *significand*, can be thought of as being the value in question (1.898 and 3.801)
- the *radix*, also known as the *base*, is the value that is raised to the power of the exponent (10 in both of our examples)
- the *exponent*, which describes the scale of the value (27 and -4)

This crosses over to floating point quite neatly. A floating point value is a container with three fields:

- a sign bit
- an exponent
- a mantissa

Where is the radix? The standard defines it as 2 for all floating point types. This definition allows the radix to be omitted from the bit pattern itself.

To convert the bits within a floating point number to a number, there are three tasks. They're implemented in [Listing 5.9](#). The first is extracting the bits of those values from the container (see `deconstruct_f32()`). The second is to decode each value from its raw bit pattern to its actual value (see `decode_f32_parts()`). The third is to perform the arithmetic to convert from scientific notation to an ordinary number (see `f32_from_parts()`).

Our first task is to extract each field from its container. This is slightly tricky, as they are not aligned at the edge of a byte. Therefore, we'll need to learn some bit manipulation techniques to get at them.

The three main operations that are new are:

- “Right shift” ($n \gg m$) where n and m are both integers. This operation moves bits towards the right and pads the now-empty left bits with zeros. The new value can now be interpreted as an integer in its own right. This technique is used to isolate the sign bit and the mantissa.
- “AND mask” ($n \& m$). This operation is used as a filter. It allows you to selectively choose which bits to retain by adjusting m to suit.
- “Left shift” ($n \ll m$), with n typically being 1. This operation is used to create the m value for a subsequent AND mask. That is, we will be dynamically creating filters to isolate individual bits as the program progresses.

Those three operations are expanded upon after the code listing.

When [Listing 5.9](#) is run, its output is a visual marker of the process that it has undertaken:

Listing 5.8. Output from [Listing 5.9](#)

```
42.42 -> [sign:0, exponent:32, mantissa:1.325625] -> 42.42
```

Listing 5.9. Deconstructing a floating point value with bit manipulation techniques. `deconstruct_f32()` extracts each field of a floating point value with bit manipulation techniques. `decode_f32_parts()` demonstrates how to convert those fields to the relevant number. `f32_from_parts()` combines them to create a single decimal number. (ch5/ch5-visualizing-f32.rs)

```
const BIAS: i32 = 127; ①
const RADIX: f32 = 2.0; ①

fn main() { ②
    let n: f32 = 42.42;

    let (signbit, exponent, fraction) = deconstruct_f32(n); ③
    let (sign, exponent, mantissa) = decode_f32_parts(signbit, exponent, fraction); ④
    let reconstituted_n = f32_from_parts(sign, exponent, mantissa); ⑤

    println!("{}", n, signbit, exponent, mantissa, reconstituted_n);
}

fn deconstruct_f32(n: f32) -> (u32, u32, u32) {
    let n_: u32 = unsafe { std::mem::transmute(n) };

    let sign      = (n_ >> 31) & 1; ⑥
    let exponent  = (n_ >> 23) & 0xff; ⑦
    let fraction  = 0b00000000_01111111_11111111_11111111 & n_; ⑧

    (sign, exponent, fraction) ⑨
}

fn decode_f32_parts(sign: u32, exponent: u32, fraction: u32) -> (f32, f32, f32) {
    let signed_1 = (-1.0_f32).powf(sign as f32); ⑩

    let exponent = (exponent as i32) - BIAS; ⑪
    let exponent = RADIX.powf(exponent as f32); ⑪

    let mut mantissa: f32 = 1.0; ⑫
    for i in 0..23_u32 { ⑬
        let one_at_bit_i = 1 << i; ⑭
        if (one_at_bit_i & fraction) != 0 { ⑮
            mantissa += 2_f32.powf((i as f32) - 23.0); ⑯
        }
    }

    (signed_1, exponent, mantissa)
}

fn f32_from_parts(sign: f32, exponent: f32, mantissa: f32) -> f32 { ⑰
    sign * exponent * mantissa
}
```

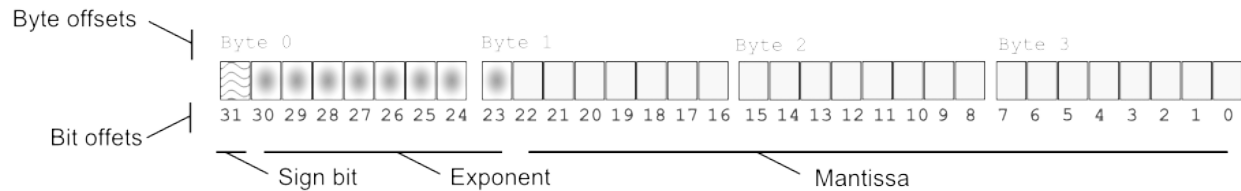
① Similar constants are accessible via the `std::f32` module

- ② `main()` can live happily at the beginning of a file
- ③ Here the three components of `n` are extracted, with each one being an untyped sequence of bits
- ④ Each component is interpreted according to the standard
- ⑤ The original value is produced from the three decoded components
- ⑥ Strip 31 unwanted bits away by shifting them into nowhere, leaving only the sign bit
- ⑦ Filter out the top bit with a logical AND mask, then strip 23 unwanted bits away
- ⑧ Only retain the 23 “least significant” bits via an AND mask
- ⑨ The mantissa part is called a fraction here, as it becomes the mantissa once it’s decoded
- ⑩ Convert the sign bit to 1.0 or -1.0. Parentheses are required around `-1.0_f32` to clarify operator precedence as method calls rank higher than unary minus.
- ⑪ We need to do a bit of a type dance here. `exponent` must become an `i32` in case subtracting the BIAS results in a negative number. Then it needs to be cast as a `f32`, so that it can be used for exponentiation.
- ⑫ We start by assuming that the implicit 24th bit is set. That has the upshot of defaulting the mantissa’s value as 1.
- ⑬ We provide a concrete type here to ensure that the bit patterns that are generated by the mask are defined
- ⑭ At each iteration, create an AND mask of a single bit in the position that we’re currently interested in.
- ⑮ Any non-zero result means that the bit is present within `fraction`
- ⑯ To arrive at the decimal value of the bit at `i`, we find 2^{i-23} . `-23` means that the result gets smaller when `i` is close to 0, as desired.
- ⑰ This code cheats a bit by using `f32` values in intermediate steps. Hopefully it is a forgivable offense.

Understanding how to unpack bits from bytes will mean that you’ll be in a much stronger position when you’re faced with interpreting untyped bytes flying in from the network.

5.3.2 Looking inside an f32

Figure 5.1. An overview of the three components encoded within the bits of a floating point number. The layout corresponds to the `f32` type in Rust, called `binary32` within the IEEE 754-2019 and IEEE 754-2008 standards and `single` by their predecessor IEEE 754-1985.



The value 42.42 is encoded as `f32` with the bit pattern `01000010001010011010111000010100`. That bit pattern is more compactly represented as `0x4229AE14`. Here are the values of each of the three fields and what they represent:

Table 5.1. The components of 42.42 represented by the bit pattern `0x4229AE14` as a `f32` type

Component name (Variable in Equation 5.1)	Component in binary	Component as base-10 (u32)	Decoded Value
Mantissa/Significand (<i>m</i>)	01010011010111000010100	2,731,540	1.325625
Sign bit (<i>s</i>)	0	0	1
Exponent (<i>t</i>)	10000100	132	5

See lines 32-38 of [Listing 5.9](#) and the explanation provided in [Section 5.3.5](#) to learn how `01010011010111000010100` represents 1.325625. | Base/Radix ||| 2 |
Exponent bias ||| 127

$$: (n, =, 1^{\text{sign_bit}}, , tt^{\text{mantissa}}, , \text{Radix}^{(tt^{\text{exponent}} - \text{Bias}))) ,$$

$$(n, =, 1^{\text{sign_bit}}, , tt^{\text{mantissa}}, , 2^{(tt^{\text{exponent}} - 127)}),$$

$$(42.42, =, 1^0, , 1.325625, , 2^{(132 - 127)}),$$

$$(42.42, =, 1, , 1.325625, , 2^5),$$

$$(42.42, =, 1, , 1.325625, , 32) :$$

Equation 5.1. Decoding the fields of a floating point number into a single number. Variables from the standard (Radix, Bias) are presented in Title Case. Variables from the bit pattern (sign_bit, mantissa, exponent) are presented in lower-case and monospace.

5.3.3 About the Sign Bit



The *sign bit* is a single bit that indicates which side of zero the decimal number lies on.

- Notes - + 1 represents negative numbers and 0 positive ones.
- Special Cases - +-0 and 0 have different bit patterns in memory, but compare as equal.

How to Isolate the Sign Bit

Shift the other bits out of the way. For f32, this involves a right shift 31 places (>> 31). Steps:

1. Cast f32 as u32 to allow for bit manipulation.

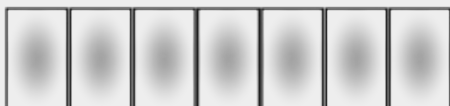
```
let n: u32 = unsafe { std::mem::transmute(42.42_f32) };
```



The issue that needs to be resolved is the sign bit's position. Treated naïvely, it represents 4,294,967,296 (2^{32}) or 0, rather than 1 (2^0) or 0.

2. Shift the bits within n 31 places to the right.

```
let sign_bit = n >> 31;
```



5.3.4 About the Exponent

- Notes - To decode the exponent, treat bits as an integer then subtract 127 from the result. 127 is known as the “bias”.
- Special Cases
 - 0×00 (0b00000000) indicates that the mantissa should be treated as a “subnormal number”. This increases how many decimal numbers near zero that can be represented.
 - $0 \times FF$ (0b11111111) indicates that the decimal number is infinity (∞), negative infinity ($-\infty$) or “Not a Number”. Not a Number values indicate special cases where the numeric result is mathematically undefined, such as $0 \div 0$, or that are otherwise invalid. Operations involving Not a Number values are often counter-intuitive. For example, testing whether two values are equal is always false, even when the two bit patterns are exactly the same.

How to Isolate the Exponent

Shift bits 31-22 rightwards to bit 0. Then strip away the sign bit with a AND mask. Interpret the remaining 8 bits as an integer, then remove the bias (127) to produce the actual exponent.

Steps:

1. Cast f32 as u32 to allow for bit manipulation

```
let n: u32 = unsafe { std::mem::transmute(42.42_f32) };
```



2. Shift bits to the right

```
let exponent_: n >> 23;
```



3. Filter the sign bit away with an AND mask. Only non-zero bits in the mask can pass through.

```
let exponent_: exponent_ & 0b00000000_00000000_00000000_11111111;
```



4. Subtract the exponent bias, as defined by the standard.

```
let exponent: (exponent_ as i32) - 127; ①
```

① Casting the exponent bits to an i32 allows a negative value to be represented.

5.3.5 About the Mantissa

Each bit represents a known value defined by the floating point standard. To calculate the mantissa, iterate through each bit i , adding 2^{i-23} to a temporary variable when i th bit equals 1. The bits represent 0.5 (2^{-1}), 0.25 (2^{-2}), through to $0.00000011920928955078125$ (2^{-23}). An implicit 24th bit that represents 1.0 (2^0) is always on, except when special cases are triggered.

- Special Cases - The state of the exponent can trigger special treatment of the mantissa.
 - When the exponent is 255 (0b11111111), infinity (∞) is represented by 0 in the mantissa and every other bit pattern represents Not a Number (f32::NAN).
 - When the exponent is 0 (0b00000000), zero is represented by 0 in the mantissa. Every non-zero bit pattern in the mantissa switches the implicit 24th to 0.0.

How to Isolate the Mantissa

Iterate through the fractional bits of the mantissa, adding their value to a variable mantissa initialised with the value of the implicit 24th bit (1.0). Special cases are not handled in the steps below, but involve returning early with sentinel values for infinity/-infinity/Not a Number or subtracting 1.0 from the mantissa variable.

Steps:

1. Cast f32 as u32 to allow for bit manipulation

```
let n: u32 = unsafe { std::mem::transmute(42.42_f32) };
```

2. Create a mutable f32 value initialized to 1.0 (2^{-0}).

```
let mut mantissa: f32 = 1.0;
```

3. Iterate through the fractional bits of the mantissa, adding that bit's defined value to the mantissa variable.

```
for i in 0..23_u32 { ①
  let one_at_bit_i = 1 << i; ②
  if (one_at_bit_i & fraction) != 0 {
    let i_ = i as f32;
    mantissa += 2_f32.powf( i_ - 23.0 );
  }
}
```

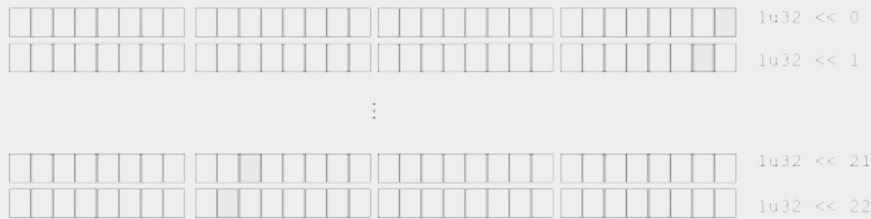
① Specifying the type (u32) allows the bit shifting machinery to work by clarifying the intended number of bits.

② The syntax `1 << i` creates a bit pattern with a 1 at position `i` and zero

everywhere else. When `i` equals 5, the bit pattern is
`0b00000000_00000000_00000000_00100000`

Figure 5.2. Illustrating how left shift works to isolate individual bits

This figure shows the effect of left shifts on the integer 1 encoded as `u32`. White cells represent 0 bits, grey cells represent 1 bits. This technique enables the programmer to set any desired bit to be set to 1, independently of all others.



Tip

Rust's numbers have methods. While convenient, this can cause issues. Unary minus has lower precedence than method calls, meaning unexpected mathematical errors can occur.

The correct way to calculate -1^0 is to use parentheses to make your intent clear to the compiler like

```
(-1.0_f32).powf(0.0)
```

rather than

```
-1.0_f32.powf(0.0)
```

which is interpreted as $-(1^0)$.

5.3.6 Representing decimal numbers in a single byte with a fixed-point number format

To represent numbers within a single byte, we'll use the "Q format", developed by Texas Instruments for embedded computing devices. The Q format, is a fixed-point number. [\[37\]](#) Unlike floating point numbers, the decimal place does

not move to dynamically accommodate different ranges.

The specific version of the Q format that we will be implementing is called Q7. This indicates that there is 7 bits available for the represented number, plus 1 sign bit. We'll disguise the decimal nature of the type by hiding the 7 bits within an i8. That will mean that the Rust compiler will be able to assist us to keep track of the value's sign. We will also be able to derive traits such as `PartialEq` and `Eq`, which provides comparison operators for our type for free.

Listing 5.10. Definition of the Q7 format (ch5/ch5-q/src/lib.rs)

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct Q7(i8); ①
```

① This form of a struct is known as a “tuple struct” as it only has anonymous fields

Q7 is intended as a storage and data transfer type only. Its most important role is to be able to be converted to and from floating point types. Here is the conversion to f64:

Listing 5.11. Converting between f64 and Q7 with the `std::convert::From` trait. (ch5/ch5-q/src/lib.rs)

```
impl From<f64> for Q7 { ①
    fn from (n: f64) -> Self {
        // assert!(n >= -1.0);           ②
        // assert!(n <= 1.0);           ②
        if n >= 1.0 { ③
            Q7(127)
        } else if n <= -1.0 {
            Q7(-128)
        } else {
            Q7((n * 128.0) as i8)
        }
    }
}

impl From<Q7> for f64 { ④
    fn from(n: Q7) -> f64 {
        (n.0 as f64) * 2f64.powf(-7.0) ⑤
    }
}
```

- ① Converting from f64 to Q7. From, or std::convert::From more fully, is implicitly included in every Rust program unless you opt-out of that behavior.
- ② One option available to you is to crash the program if the conversion receives an out-of-bounds input.
- ③ Out of bounds numbers are coerced to the maximum of the Q7 range. This should never happen for the use cases that we are interested in.
- ④ Converting from Q7 to f64
- ⑤ This is mathematically equivalent to iterating through each of the bits and multiplying it to its weight, as was carried out earlier when decoding the floating point mantissa in the previous section

As there are also f32 values to convert between, let's make use of Rust's own machinery to do the hard work on our behalf:

Listing 5.12. Converting between f32 and q7 via f64 values (ch5/ch5-q/src/lib.rs)

```
impl From<f32> for Q7 {  
    fn from (n: f32) -> Self {  
        Q7::from(n as f64)    ①  
    }  
}  
  
impl From<Q7> for f32 {  
    fn from(n: Q7) -> f32 {  
        f64::from(n) as f32    ②  
    }  
}
```

- ① It's safe (by design) to convert from f32 to f64. If a number can be represented in 32 bits, it can be represented in 64 bits.
- ② Converting an f64 into a f32 risks a loss of precision in the general case. However, in this application that risk doesn't apply as we only have numbers between -1 and 1 to convert from.

Now both floating point types are covered. But how do we know that the code that what we've written actually does what we intended? How do we test what we've written? As it happens, Rust has excellent support for unit testing via

cargo.

The Q7 code that you've seen is available as a complete listing at [Listing 5.14](#). To test the code, enter the root directory of the crate and run `cargo test`.

Listing 5.13. Output of `cargo test` on code from [Listing 5.14](#)

```
Compiling ch5-q v0.1.0 (file:///path/to/ch5/ch5-q)
Finished dev [unoptimized + debuginfo] target(s) in 2.86 s
Running target\debug\deps\ch5_q-013c963f84b21f92

running 3 tests
test tests::f32_to_q7 ... ok
test tests::out_of_bounds ... ok
test tests::q7_to_f32 ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

Doc-tests ch5-q

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

Listing 5.14. Full code implementing the Q7 format and its conversion to and from `f32` and `f64` types (`ch5/ch5-q/src/lib.rs`)

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct Q7(i8);

impl From<f64> for Q7 {
    fn from (n: f64) -> Self {
        if n >= 1.0 {
            Q7(127)
        } else if n <= -1.0 {
            Q7(-128)
        } else {
            Q7((n * 128.0) as i8)
        }
    }
}

impl From<Q7> for f64 {
    fn from(n: Q7) -> f64 {
        (n.0 as f64) * 2f64.powf(-7.0)
    }
}

impl From<f32> for Q7 {
    fn from (n: f32) -> Self {
        Q7::from(n as f64)
    }
}
```

```

}

impl From<Q7> for f32 {
    fn from(n: Q7) -> f32 {
        f64::from(n) as f32
    }
}

#[cfg(test)]
mod tests {
    use super::*; ①

    #[test]
    fn out_of_bounds() {
        assert_eq!(Q7::from(10.), Q7::from(1.));
        assert_eq!(Q7::from(-10.), Q7::from(-1.));
    }

    #[test]
    fn f32_to_q7() {
        let n1: f32 = 0.7;
        let q1 = Q7::from(n1);

        let n2 = -0.4;
        let q2 = Q7::from(n2);

        let n3 = 123.0;
        let q3 = Q7::from(n3);

        assert_eq!(q1, Q7(89));
        assert_eq!(q2, Q7(-51));
        assert_eq!(q3, Q7(127));
    }

    #[test]
    fn q7_to_f32() {
        let q1 = Q7::from(0.7);
        let n1 = f32::from(q1);
        assert_eq!(n1, 0.6953125);

        let q2 = Q7::from(n1);
        let n2 = f32::from(q2);
        assert_eq!(n1, n2);
    }
}

```

- ① Define a sub-module within this file.
- ② Bring the parent module within the sub-module's local scope. Items that are marked as pub are accessible here.

A very brief look at Rust's module system

Rust includes a powerful and ergonomic module system. To keep the examples simple however, this book does not make heavy use of it.

Here are some very basic guidelines.

- Modules are combined into *crates*
- Modules can be defined by a project's directory structure. Sub-directories under `src` become a module when that directory contains a `mod.rs` file
- Modules can also be defined within a file with the `mod` keyword
- Modules can be nested arbitrarily
- All members of a module, including its sub-modules, are *private* by default. Private items can be accessed within the module, and any of the module's descendants. Prefix things that you wish to make public with the `pub` keyword.
- The `pub` keyword has some specialized cases [\[38\]](#)
 - `pub(crate)` exposes an item to other modules within the crate
 - `pub(super)` exposes an item to the parent module
 - `pub(in path)` exposes an item to a module within *path*
- Bring items from other modules into local scope with the `use` keyword

[\[37\]](#) $\mathbb{Q}$ —often written as $\mathbb{Q}$ (the style is called “blackboard bold”)—is the mathematical symbol for the set of rational numbers. Rational numbers are numbers that can be represented as a fraction of two integers, such as $1/3$.

[\[38\]](#) For completeness, `pub(self)` also exists. Its behavior is to explicitly keep the item private.

5.4 GENERATING F32 VALUES BETWEEN 0 AND 1 FROM RANDOM BYTES

Here is an interesting exercise of the knowledge that you have developed over the preceding pages. Imagine that you have a source of random bytes and you want to convert them into floating point values between 0 and 1. Naively interpreting the incoming bytes as `f32/f64` via `mem::transmute` would result in massive variations in scale. As division is a slow operation, perhaps there is something faster than simply dividing by the largest value that a byte can represent, as per [Listing 5.15](#).

Listing 5.15. Generating a `f32` that lies between 0 and 1 from an

arbitrary input byte through division

```
fn mock_rand(n: u8) -> f32 {  
    (n as f32) / 255.0 ①  
}
```

① 255 is the maximum value that u8 can represent

Perhaps it's possible to assume a constant exponent value, then shift the incoming bits into the mantissa, such that they would form a range between 0 and 1? [Listing 5.16](#) is the best result that I could achieve. With an exponent of -1 (represented as 126/0b01111110), the source byte achieves a range of 0.5 to 0.998. That can be normalized to 0.0 to 0.996 with subtraction and multiplication. Is there a way to do better?

Listing 5.16. Shiftbyte

```
fn mock_rand(n: u8) -> f32 {  
    let base: u32 = 0b0_01111110_000000000000000000000000; ①  
    let large_n = (n as u32) << 15; ②  
    let f32_bits = base | large_n; ③  
    let m = f32::from_bits(f32_bits); ④  
    2.0 * ( m - 0.5 ) ⑤  
}
```

- ① Underscores mark the sign/mantissa/exponent boundaries
- ② Align the input byte n to 32 bits, then increase its value by shifting its bits 15 places to the left
- ③ Take a bitwise OR, merging the base with the input byte
- ④ Interpret f32_bits (which is type u32) as an f32
- ⑤ Normalize the output range

As a complete program, `mock_rand()` from [Listing 5.16](#) can be incorporated into a test program fairly easily, as shown in [Listing 5.18](#).

Listing 5.17. Output from [Listing 5.18](#)

```
max of input range: 11111111 -> 0.99609375  
mid of input range: 01111111 -> 0.49609375
```

```
min of input range: 00000000 -> 0
```

Listing 5.18. Generating a f32 that lies between 0 and 1 from an arbitrary input byte without division (ch5/ch5-u8-to-mock-rand.rs)

```
fn mock_rand(n: u8) -> f32 {
    let base: u32 = 0b0_01111110_000000000000000000000000;
    let large_n = (n as u32) << 15;
    let f32_bits = base | large_n;
    let m = f32::from_bits(f32_bits);
    2.0 * ( m - 0.5 )
}

fn main() {
    println!("max of input range: {:08b} -> {:?}", 0xff, mock_rand(0xff));
    println!("mid of input range: {:08b} -> {:?}", 0x7f, mock_rand(0x7f));
    println!("min of input range: {:08b} -> {:?}", 0x00, mock_rand(0x00));
}
```

5.5 IMPLEMENTING A CPU IN SOFTWARE TO ESTABLISH THAT FUNCTIONS ARE ALSO DATA

One of the fairly mundane, yet utterly intriguing, details about computing is that instructions are also just numbers. Operations and the data that is being operated on share the same encoding. This means that, as a general computing device, your computer can emulate other computers' *instruction sets* by emulating them in software. While we cannot pull apart a CPU to see how it works, we can construct one with code.

Working through this section will teach you about how a computer operates at a fundamental level. It will show you how functions operate and what the term “pointer” means. We won’t have an assembly language, we’ll actually be programming directly in hex. It will also introduce you to other terms you may have heard of in passing, such as “the stack”.

We’ll implement a subset of a system that was available to consumers in the 1970s called CHIP-8. CHIP-8 was supported by a number of manufacturers, but was fairly primitive even by the standards of the day. (It was created to write games, rather than for commercial or scientific applications.) One device was the COSMAC VIP. It had a single-color display with a resolution of 64x32 (0.0002

megapixels), 2KB RAM, 1.76 Mhz CPU and was sold for USD275. Oh, and you needed to assemble the computer yourself. It also contained games programmed by the world's first female game developer: Joyce Weisbecker.

5.5.1 CPU 1: “the Adder”

We'll build up our understanding by starting with a minimal core. Let's start by building an emulator that only supports a single instruction: addition.

To understand what's happening within [Listing 5.24](#), there are three main things to learn:

- becoming familiar with new terminology
- How to interpret opcode
- Understanding the main loop

GLOSSARY OF TERMS RELATED TO CPU EMULATION

Dealing with CPUs and emulation involves learning some terms that:

- an *operation* (often shortened to op) refers to procedures that are supported by the system natively. You may also encounter equivalent phrases such as “implemented in hardware”, “intrinsic operation” as you explore further.
- the *registers* are containers for data that CPU accesses directly. For most operations, operands must be moved to registers for the operation to function. For the CHIP-8, each register is a u8.
- an *opcode* is a number that maps to an operation. On the CHIP-8 platform, opcodes include both the operation and the operands' registers.

DEFINING THE CPU

The first operation that we want to support is addition. The operation takes two registers (x and y) as operands, adding the value stored in y to x. To implement

this, we'll use the minimal amount of code possible. Our initial CPU contains only two registers and the space for a single opcode.

Listing 5.19. Definition of the CPU used in [Listing 5.24](#)

```
struct CPU {  
    current_operation: u16, ①  
    registers: [u8; 2],    ②  
}
```

① All CHIP-8 opcodes are u16 values

② These two registers are sufficient for addition

So far, the “CPU” is inert. To perform addition, the following steps need to be taken:

- Initialize a CPU
- Load u8 values into registers
- Load the addition opcode into current_operation
- Perform the operation

There is no ability to store data in memory.

LOADING VALUES INTO REGISTERS

The process for “booting up” the CPU consists of writing to the fields of the CPU struct.

Listing 5.20. Extract from [Listing 5.24](#) demonstrating the CPU initialization process

```
fn main() {  
    let mut cpu = CPU {  
        current_operation: 0,  
        registers: [0; 2],  
    };  
  
    cpu.current_operation = 0x8014; ①  
    cpu.registers[0] = 5;           ②  
    cpu.registers[1] = 10;          ②  
}
```

- ① Operations involving two registers are grouped together with a left-hand 8. 0x8014 is the opcode for adding between register (0) and register (1). The right-hand 4 indicates addition.
- ② Registers are only able to hold u8 values.

UNDERSTANDING THE EMULATOR'S MAIN LOOP

Now that data has been loaded, the CPU is almost able to do some work.

The `run()` method performs the bulk of the emulator's work. It is emulating CPU cycles.

It uses the following pattern:

- read opcode (eventually, from memory)
- decode instruction
- match decoded instruction to known opcodes
- dispatch execution of the operation to a specific function

Listing 5.21. Extract from [Listing 5.24](#) that shows the first functionality being added to the emulator.

```
impl CPU {
  fn read_opcode(&self) -> u16 { ①
    self.current_operation      ①
  }                               ①

  fn run(&mut self) {
    // loop {                      ②
      let opcode = self.read_opcode();

      let c = ((opcode & 0xF000) >> 12) as u8; ③
      let x = ((opcode & 0x0F00) >> 8) as u8; ③
      let y = ((opcode & 0x00F0) >> 4) as u8; ③
      let d = ((opcode & 0x000F) >> 0) as u8; ③

      match (c, x, y, d) {
        (0x8, _, _, 0x4) => self.add_xy(x, y), ④
        _                 => todo!("opcode {:04x}", opcode), ⑤
      }
    } // }                          ②
  }

  fn add_xy(&mut self, x: u8, y: u8) {
    self.registers[x as usize] += self.registers[y as usize];
  }
}
```



```
}  
}
```

- ① `read_opcode()` will become more complex when reading from memory is introduced
- ② Avoid running this code in a loop for now
- ③ The opcode decoding process is explained fully in the next section
- ④ Dispatch execution to the “hardware circuit” responsible for performing it
- ⑤ A full emulator contains several dozen operations

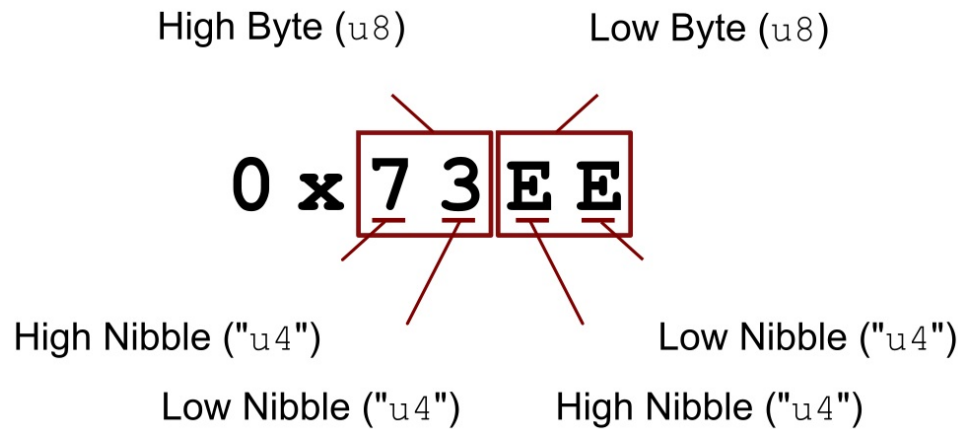
HOW TO INTERPRET CHIP-8 OPCODES

It is important for our CPU to be able to interpret its opcode (0x8014). This section provides a thorough explanation of the process used in this chapter’s code and its variable naming conventions.

CHIP-8 opcodes are `u16` values made up of 4 *nibbles*. A nibble is half of a byte. It is a 4-bit value. Because there is no 4-bit type in Rust, splitting the `u16` values into those parts is fiddly. To make matters more complicated, nibbles are often re-combined to form 8-bit and 12-bit values depending on context.

To simplify talking about the parts of each opcode, let’s introduce some standard terminology. Each opcode is made up of two bytes, the *high byte* and the *low byte*. And each byte is made up of two nibbles, the *high nibble* and the *low nibble* respectively. [Figure 5.3](#) provides an illustration.

Figure 5.3. Terms used to refer to parts of CHIP-8 opcodes



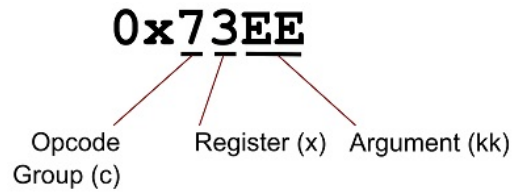
Documentation manuals for the CHIP-8 introduce several variables, including *kk*, *nnn*, *x* and *y*. [Table 5.2](#) explains their role, location and width.

Table 5.2. Variables used within CHIP-8 opcode descriptions

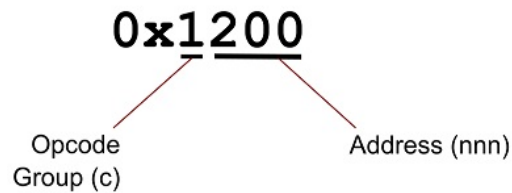
Variable	Bit Length	Location	Description
<i>n</i> *	4	low byte, low nibble	a number of bytes
<i>x</i>	4	high byte, low nibble	a CPU register
<i>y</i>	4	low byte, high nibble	a CPU register
<i>c</i> †	4	high byte, high nibble	an opcode group
<i>d</i> †*‡	4	low byte, low nibble	an opcode sub-group
<i>kk</i> ‡	8	low byte, both nibbles	an integer
<i>nnn</i> ‡	12	high byte, low nibble and low byte, both nibbles	a memory address
* <i>n</i> and <i>d</i> occupy the same location, but are used in mutually exclusive contexts. † The variable names <i>c</i> and <i>d</i> are used within <i>Rust in Action</i> , but not in other CHIP-8 documentation. ‡ Not used in Listing 5.24 , but used in Listing 5.29 and Listing 5.34 .			

There are three main form of opcodes, as illustrated by [Figure 5.4](#). The decoding process involves matching on the high nibble of the first byte and then applying one of three strategies.

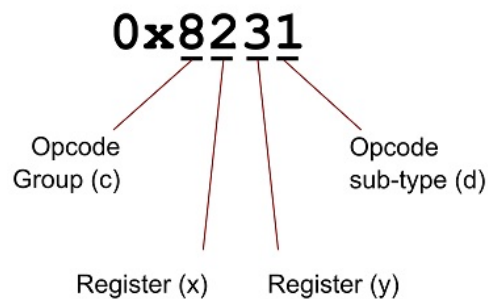
Figure 5.4. CHIP-8 opcodes are decoded in multiple ways. The approach to use depends on the value of the left-most nibble.



Interpretation
Add 238 (0xEE) to register 3.



Interpretation
Jump to memory address (0x200)



Interpretation
Perform a bitwise OR operation with registers x and y. Store result within register x.

To extract nibbles from bytes, we'll need to use the *right shift* (>>) and *logical AND* (&) *bitwise operations*. These operations were introduced during [Section 5.3.1](#). [Listing 5.22](#) demonstrates applying them to the current problem.

Listing 5.22. Extracting variables from an opcode using the right shift and logical AND bitwise operations.

```
fn main() {
    let opcode: u16 = 0x71E4;           ①

    let c = (opcode & 0xF000) >> 12;   ②
    let x = (opcode & 0x0F00) >> 8;     ②
    let y = (opcode & 0x00F0) >> 4;     ②
    let d = (opcode & 0x000F) >> 0;     ②

    assert_eq!(c, 0x7);                 ①
    assert_eq!(x, 0x1);                 ①
    assert_eq!(y, 0xE);                 ①
    assert_eq!(d, 0x4);                 ①
}
```

```

let nnn = opcode & 0x0FFF;      ③
let kk  = opcode & 0x00FF;      ③

assert_eq!(nnn, 0x1E4);
assert_eq!(kk,  0xE4);
}

```

- ① The 4 nibbles from opcode are available as individual variables after processing.
- ② Single nibbles can be selected with the AND operator (&) to filter bits that should be retained then a shift to move the bits to the lowest significant place. Hexadecimal notation is convenient for these operations, as each hexadecimal digit represents 4 bits. A 0xF value selects all bits from a nibble.
- ③ Multiple nibbles can be selected by increasing the width of the filter. For our purposes, shifting bits rightward is unnecessary.

We’re now able to decode instructions. The next step is actually executing them.

5.5.2 First working emulator

[Listing 5.24](#) is the full code of our proto-emulator: “the Adder”. When executed, it prints out the result that you have probably spent too long coding up an emulator to produce:

Listing 5.23. Output of [Listing 5.24](#)

```
5 + 10 = 15
```

Listing 5.24. Implementing the beginnings of CHIP-8 emulator (ch5/ch5-cpu1/src/main.rs)

```

struct CPU {
    current_operation: u16,
    registers: [u8; 2],
}

impl CPU {
    fn read_opcode(&self) -> u16 {
        self.current_operation
    }

    fn run(&mut self) {

```

```

    // loop {
        let opcode = self.read_opcode();

        let c = ((opcode & 0xF000) >> 12) as u8;
        let x = ((opcode & 0x0F00) >> 8) as u8;
        let y = ((opcode & 0x00F0) >> 4) as u8;
        let d = ((opcode & 0x000F) >> 0) as u8;

        match (c, x, y, d) {
            (0x8, _, _, 0x4) => self.add_xy(x, y),
            _ => todo!("opcode {:04x}", opcode), ①
        }
    // }
}

fn add_xy(&mut self, x: u8, y: u8) {
    self.registers[x as usize] += self.registers[y as usize];
}

}

fn main() {
    let mut cpu = CPU {
        current_operation: 0,
        registers: [0; 2],
    };

    cpu.current_operation = 0x8014;
    cpu.registers[0] = 5;
    cpu.registers[1] = 10;

    cpu.run();

    assert_eq!(cpu.registers[0], 15);

    println!("5 + 10 = {}", cpu.registers[0]);
}

```

5.5.3 CPU 2: “the Multiplier”

CPU 1 could execute a single instruction. CPU 2 is able to execute several instructions in sequence with the addition of a working main loop and a variable that we’ll call `position_in_memory`. `position_in_memory` holds the memory address of the CPU’s next instruction.

[Listing 5.29](#) makes the following substantive changes to [Listing 5.24](#):

- Addition of 4kb of memory (Line 8).
- Inclusion of a fully-fledged main loop and stopping condition (Lines 14-31). At each step in the loop, memory at `position_in_memory` is

accessed, decoded into an opcode, `position_in_memory` is then incremented to the next memory address and the opcode is executed. The CPU will continue to run forever until the stopping condition—an opcode of `0x0000`—is encountered.

- Removal of the `current_instruction` field of the CPU struct, which is replaced by a section of the main loop that decodes bytes from memory (Lines 15-17).
- Opcodes are written into memory (Lines 51-53).

Listing 5.25. Output of [Listing 5.29](#)

```
5 + 10 + 10 + 10 = 35
```

EXPANDING THE CPU TO SUPPORT MEMORY

The CPU definition from [Listing 5.24](#) can only process a single instruction. Some modifications are required to make it more useful. In particular, we need to add memory. Here is the CPU’s definition used in [Listing 5.29](#) and [Listing 5.34](#):

Listing 5.26. Defining a CPU struct. It contains 16 general purpose registers, 1 special purpose register. For convenience, we’ll include the system’s memory within the struct.

```
struct CPU {  
    registers: [u8; 16],           ①  
    position_in_memory: usize,    ②  
    memory: [u8; 4096],           ③  
}
```

- ① 16 registers means that a hexadecimal number (0 to F) can address them.
- ② This is a special register, typically known as the “program counter”. We’ll use a different term so that its purpose is more understandable to readers new to systems programming. The `usize` type is used to save some type conversions later on when it indexes memory.
- ③ The CHIP-8 has 4 KiB RAM (`0x1000` in hexadecimal). Within the specification, the first 512 bytes (`0x100`) are reserved for the system. Other

bytes are available for programs. This implementation will relax that restriction.

READING OPCODES FROM MEMORY

With the addition of memory within the CPU, the `read_opcode()` method requires updating.

Listing 5.27. Extract from [Listing 5.29](#) illustrating how to read an opcode from memory by combining two `u8` values into a single `u16`

```
fn read_opcode(&self) -> u16 {
    let p = self.position_in_memory;
    let op_byte1 = self.memory[p] as u16;
    let op_byte2 = self.memory[p + 1] as u16;

    op_byte1 << 8 | op_byte2          ①
}
```

① To create a `u16` opcode, we combine two values from memory with the logical OR operation. They need to be cast as `u16` to start with, otherwise the left-shift will set all of the bits to 0.

HANDLING INTEGER OVERFLOW

Within the CHIP-8, the last register is used as a *carry flag*. When set, this flag indicates that an operation has overflowed the `u8` size of a register.

Listing 5.28. Extract from [Listing 5.29](#) illustrating how to handle overflow within CHIP-8 operations

```
fn add_xy(&mut self, x: u8, y: u8) {
    let arg1 = self.registers[x as usize];
    let arg2 = self.registers[y as usize];

    let (val, overflow_detected) = arg1.overflowing_add(arg2); ①
    self.registers[x as usize] = val;

    if overflow_detected {
        self.registers[0xF] = 1;
    } else {
        self.registers[0xF] = 0;
    }
}
```

```
}
```

① The `overflowing_add()` method for `u8` returns `(u8, bool)`. The `bool` is `true` when overflow is detected.

SECOND WORKING EMULATOR

Listing 5.29. Adding the ability for the emulator to process multiple instructions (ch5/ch5-cpu2/src/main.rs)

```
struct CPU {
    registers: [u8; 16],
    position_in_memory: usize,
    memory: [u8; 0x1000],
}

impl CPU {
    fn read_opcode(&self) -> u16 {
        let p = self.position_in_memory;
        let op_byte1 = self.memory[p] as u16;
        let op_byte2 = self.memory[p + 1] as u16;

        op_byte1 << 8 | op_byte2
    }

    fn run(&mut self) {
        loop {
            let opcode = self.read_opcode();
            self.position_in_memory += 2;

            let c = ((opcode & 0xF000) >> 12) as u8;
            let x = ((opcode & 0x0F00) >> 8) as u8;
            let y = ((opcode & 0x00F0) >> 4) as u8;
            let d = ((opcode & 0x000F) >> 0) as u8;

            match (c, x, y, d) {
                (0, 0, 0, 0) => { return; },
                (0x8, _, _, 0x4) => self.add_xy(x, y),
                _ => todo!("opcode {:04x}", opcode),
            }
        }
    }

    fn add_xy(&mut self, x: u8, y: u8) {
        let arg1 = self.registers[x as usize];
        let arg2 = self.registers[y as usize];

        let (val, overflow_detected) = arg1.overflowing_add(arg2);
        self.registers[x as usize] = val;

        if overflow_detected {
            self.registers[0xF] = 1;
        } else {

```



```

        self.registers[0xF] = 0;
    }
}

fn main() {
    let mut cpu = CPU {
        registers: [0; 16],
        memory: [0; 4096],
        position_in_memory: 0,
    };

    cpu.registers[0] = 5;
    cpu.registers[1] = 10;
    cpu.registers[2] = 10; ④
    cpu.registers[3] = 10; ④

    cpu.memory[0] = 0x80; cpu.memory[1] = 0x14; ⑤
    cpu.memory[2] = 0x80; cpu.memory[3] = 0x24; ⑥
    cpu.memory[4] = 0x80; cpu.memory[5] = 0x34; ⑦

    cpu.run();

    assert_eq!(cpu.registers[0], 35);

    println!("5 + 10 + 10 + 10 = {}", cpu.registers[0]);
}

```

- ① Continue execution beyond processing a single instruction
- ② Increment `position_in_memory` to point to the next instruction
- ③ Short circuit the function to terminate execution when the opcode `0x0000` is encountered
- ④ Initialize a few registers with values
- ⑤ Load the opcode `0x8014` to memory. `0x8014` means “add the value in register 1 to register 0”
- ⑥ Load the opcode `0x8024` to memory. `0x8024` means “add the value in register 2 to register 0”
- ⑦ Load the opcode `0x8034` to memory. `0x8014` means “add the value in register 3 to register 0”

5.5.4 CPU 3: Adding functions

We have nearly built all of the emulator machinery. This section adds the ability for you to call functions. There is no programming language support however, so any programs will still need to be written in binary. As well as implementing

functions, this section attempts to validate an assertion made at the start: functions are also data.

EXPANDING THE CPU TO INCLUDE SUPPORT FOR A STACK

To build functions, we require some additional opcodes to be implemented:

- the CALL opcode ($0x2nnn$ where nnn is a memory address) sets `position_in_memory` to nnn , the address of the function
- the RETURN opcode ($0x00EE$) sets `position_in_memory` to the memory address of the previous CALL

To enable these to opcodes to work together, the CPU needs to have some specialized memory available for storing addresses. This is known as “the stack”. Each CALL adds an address to the stack, by incrementing the stack pointer and writing NNN to that position in the stack. Each RETURN removes the top address, by decrementing the stack pointer.

Listing 5.30. Detail of the emulated CPU in [Listing 5.34](#) to include a stack and stack pointer

```
struct CPU {  
    registers: [u8; 16],  
    position_in_memory: usize,  
    memory: [u8; 4096],  
    stack: [u16; 16],      ①  
    stack_pointer: usize,  ②  
}
```

① The stack’s maximum “height” is 16. After 16 nested function calls, the program will encounter a *stack overflow*.

② Giving the `stack_pointer` type `usize` makes it easier to be used to index values within stack

The Rust-like pseudo-code at [Listing 5.31](#) illustrates what is happening within the `main()` of [Listing 5.34](#). Unfortunately, the function has neither a name, arguments, nor variables. Everything appears in hexadecimal numbers. As we

have only implemented a single opcode so far, our ability to do anything interesting is limited. Adding a value to a register twice will need to suffice.

Listing 5.31. A pseudo-code illustration of the “add_twice” function, implemented at memory address 0x100 in [Listing 5.34](#).

```
fn add_twice(register_1: u8, register_2: u8) {  
    // 0x8014;  
    register_1 += register_2;  
  
    // 0x8014;  
    register_1 += register_2;  
  
    // 0x00EE;  
    return;  
}
```

IMPLEMENTING CALL AND RETURN

Calling a function is a three step process:

- store the current memory location on the stack
- increment the stack pointer
- set the current memory location (which) to the intended memory address

Returning from a function involves reversing the calling process:

- decrement the stack pointer
- retrieve the calling memory address from the stack
- set the current memory location to the intended memory address

Listing 5.32. Extract from [Listing 5.34](#) focusing on the `call()` and `ret()` methods.

```
fn call(&mut self, addr: u16) {  
    let sp = self.stack_pointer;  
    let stack = &mut self.stack;  
  
    if sp > stack.len() {  
        panic!("Stack overflow!")  
    }  
}
```

```

    stack[sp] = self.position_in_memory as u16; ①
    self.stack_pointer += 1;                    ②
    self.position_in_memory = addr as usize;    ③
}

fn ret(&mut self) {
    if self.stack_pointer == 0 {
        panic!("Stack underflow");
    }

    self.stack_pointer -= 1;
    self.position_in_memory = self.stack[self.stack_pointer] as usize; ④
}

```

- ① Add the “current” `position_in_memory` to the stack. This memory address is two bytes higher than the calling location, as it has been incremented within the body of the `run()` method.
- ② Increment `self.stack_pointer`. This will prevent store the `self.position_in_memory` from being overwritten until it needs to be accessed again in a subsequent return.
- ③ Modifying `self.position_in_memory` has the effect of jumping to that address.
- ④ Jump to the position in memory where an earlier call was made.

THIRD WORKING EMULATOR

Now that we have all of the pieces ready, let’s assemble them into a working program. [Listing 5.34](#) is able to compute a (hard-coded) mathematical expression:

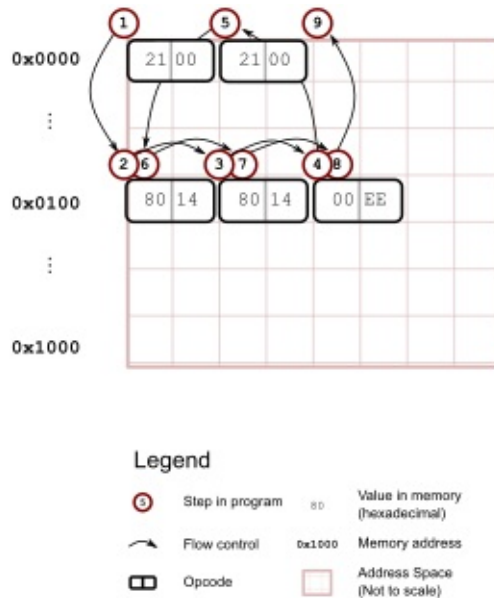
Listing 5.33. Output of [Listing 5.34](#)

```
5 + (10 * 2) + (10 * 2) = 45
```

This calculation is made without the source code that you may be used to. You will need to make do with interpreting hexadecimal numbers. To help, [Figure 5.5](#) illustrates of what is happening within the CPU during `cpu.run()`. The arrows reflect the state of the `cpu.position_in_memory` variable as it makes its way through the program.

Figure 5.5. Illustrating the control flow of the function implemented

within the emulated CPU from [Listing 5.34](#)



Listing 5.34. Emulating a CPU that incorporates calling and returning from a user-defined function (ch5/ch5-cpu3/src/main.rs)

```
struct CPU {
    registers: [u8; 16],
    position_in_memory: usize,
    memory: [u8; 4096],
    stack: [u16; 16],
    stack_pointer: usize,
}

impl CPU {
    fn read_opcode(&self) -> u16 {
        let p = self.position_in_memory;
        let op_byte1 = self.memory[p] as u16;
        let op_byte2 = self.memory[p + 1] as u16;

        op_byte1 << 8 | op_byte2
    }

    fn run(&mut self) {
        loop {
            let opcode = self.read_opcode();
            self.position_in_memory += 2;

            let c = ((opcode & 0xF000) >> 12) as u8;
            let x = ((opcode & 0x0F00) >> 8) as u8;
            let y = ((opcode & 0x00F0) >> 4) as u8;
            let d = ((opcode & 0x000F) >> 0) as u8;

            let nnn = opcode & 0xFFFF;
            // let kk = (opcode & 0x00FF) as u8;
        }
    }
}
```

```

        match (c, x, y, d) {
            ( 0, 0, 0, 0) => { return; },
            ( 0, 0, 0xE, 0xE) => self.ret(),
            (0x2, _, _, _) => self.call(nnn),
            (0x8, _, _, 0x4) => self.add_xy(x, y),
            _ => todo!("opcode {:04x}", opcode),
        }
    }
}

fn call(&mut self, addr: u16) {
    let sp = self.stack_pointer;
    let stack = &mut self.stack;

    if sp > stack.len() {
        panic!("Stack overflow!")
    }

    stack[sp] = self.position_in_memory as u16;
    self.stack_pointer += 1;
    self.position_in_memory = addr as usize;
}

fn ret(&mut self) {
    if self.stack_pointer == 0 {
        panic!("Stack underflow");
    }

    self.stack_pointer -= 1;
    self.position_in_memory = self.stack[self.stack_pointer] as usize;
}

fn add_xy(&mut self, x: u8, y: u8) {
    let arg1 = self.registers[x as usize];
    let arg2 = self.registers[y as usize];

    let (val, overflow_detected) = arg1.overflowing_add(arg2);
    self.registers[x as usize] = val;

    if overflow_detected {
        self.registers[0xF] = 1;
    } else {
        self.registers[0xF] = 0;
    }
}

}

fn main() {
    let mut cpu = CPU {
        registers: [0; 16],
        memory: [0; 4096],
        position_in_memory: 0,
        stack: [0; 16],
        stack_pointer: 0,
    };

    cpu.registers[0] = 5;
    cpu.registers[1] = 10;

```

```

cpu.memory[0x000] = 0x21; cpu.memory[0x001] = 0x00; ①
cpu.memory[0x002] = 0x21; cpu.memory[0x003] = 0x00; ②
                                                    ③
cpu.memory[0x100] = 0x80; cpu.memory[0x101] = 0x14; ④
cpu.memory[0x102] = 0x80; cpu.memory[0x103] = 0x14; ⑤
cpu.memory[0x104] = 0x00; cpu.memory[0x105] = 0xEE; ⑥

cpu.run();

assert_eq!(cpu.registers[0], 45);
println!("5 + (10 * 2) + (10 * 2) = {}", cpu.registers[0]);
}

```

- ① Opcode 0x2100: CALL the function at 0x100
- ② Opcode 0x2100: CALL the function at 0x100
- ③ Opcode 0x0000: HALT execution is implicit
- ④ Opcode 0x8014: ADD register 1's value to register 0
- ⑤ Opcode 0x8014: ADD register 1's value to register 0
- ⑥ Opcode 0x00EE: RETURN

As you delve into systems' documentation, you will find that real-life functions are more complicated than simply jumping to a pre-defined memory location. Operating systems and CPU architectures differ in calling conventions and their capabilities. Sometimes operands will need to be added to the stack, sometimes inserted into defined registers. Still, while the specific mechanics may differ, the process will be roughly similar to what you have just encountered. Congratulations on making it this far.

5.5.5 CPU 4: Adding the rest

With a few extra opcodes, it's possible to implement multiplication and many more functions within your inchoate CPU.

Check the source code that comes along with the book, specifically the ch5/ch5-cpu4 example, at <https://github.com/rust-in-action/code> for a fuller implementation of the CHIP-8 specification.

The last step in learning about CPUs and data is to understand how control flow works. Within CHIP-8, control flow works by comparing values in registers,

then modifying `position_in_memory` depending on the outcome. There are no while loops or for loops within a CPU. Creating them in programming languages is the art of the compiler writer.

5.6 SUMMARY

This has been a large, in-depth chapter. You have learned:

- How to treat a typed value as untyped bytes, then extract individual bits to produce new typed data
- How a CPU interprets instructions from memory, which are encoded as integers
- How a platform's endianness can invert the interpretation of the value that is being represented
- That the same bit pattern means different things, depending on its type
- How to navigate between base-2, base-10 and base-16 representations of integers
- How to write and run unit tests in your programs
- How to use left shift and right shift operations to single out individual bits

6 MEMORY

This chapter covers:

- What a pointer is and why some of them are “smart”
- What the terms “the stack” and “the heap” mean
- How a program views its memory and why that’s a lie

The chapter provides you with some of the tacit knowledge held by systems programmers about how the computer’s memory operates. It aims to be the most accessible guide to pointers and memory management available. You will be learning about how applications interact with an operating system. Programmers who understand these dynamics can use that knowledge to maximize their programs' performance, while minimizing their memory footprint.

Memory is a shared resource and the operating system is an arbiter. To make its life easier, it lies to your program about how much memory is available and where it’s located. Revealing the truth behind those lies requires us to work through some prior knowledge. That is the work of the first two sections of the chapter. Each of the three sections builds on the previous one.

None of them assume that you’ve encountered the topic before. There is a fairly large body of theory to cover, but all of it is explained by example. Among other examples, you create your first graphical application in the book.

The chapter introduces little new Rust syntax, as the material is quite dense. You’ll learn how to construct pointers, how to interact with an operating system via its native API and how to interact with other programs through Rust’s foreign function interface.

6.1 POINTERS

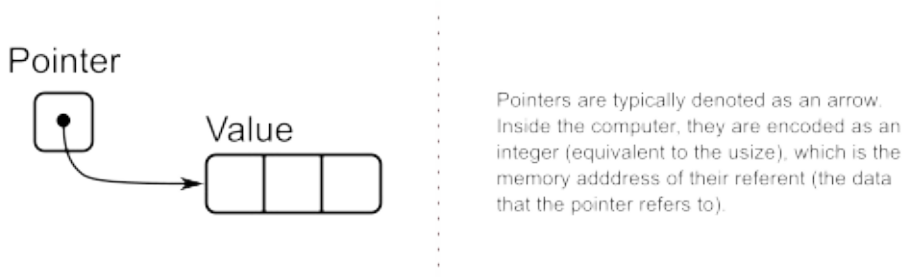
Pointers are how computers refer to data that isn't immediately accessible. The topic tends to have an aura of mystique to it. That's not necessary. If you've ever read a book's table of contents, then you've used a pointer. Pointers are just numbers that refer to somewhere else.

If you've never encountered systems programming before, there is lots of terminology to grapple with that is describing unfamiliar concepts. Thankfully though, what's sitting underneath the abstraction is not too difficult to understand. If you've used a table of contents, then you've used a pointer.

The first thing to understand is the notation. [Figure 6.1](#) introduces 3 concepts:

- The arrow refers to some unknown location in memory
- Each box represents a block of memory. In this figure, each block refers to a `usize` width. Other figures use a byte, or perhaps even a bit, as the chunk of memory they're referring to.
- The rounded box underneath "Value" represents three blocks of memory that are next to each other

Figure 6.1. Notation for illustrating a pointer. In Rust code, this is most frequently encountered as `&T` and `&mut T`, where `T` is the type of the value.



Authors use the notation in [Figure 6.1](#) to indicate that the two we don't know what the address held in the "Pointer" box will be, but they do want to demonstrate that "Pointer" and "Value" are connected.

For newcomers, pointers are scary and awe-inspiring. Their proper use requires

you to know how exactly how your program is laid out in memory. Imagine reading a table of contents that says chapter 4 starts on page 97, but it actually it starts on page 107. That will be frustrating, but at least you can cope with the mistake. A computer doesn't experience frustration. But it also lacks any intuition that it has been pointed to the wrong place. It will just keep working, correctly or incorrectly, as if it had been given the correct location. The fear of pointers is that you will introduce some impossible to debug error.

We can think of data stored within the program's memory as scattered around within physical RAM somewhere. To make use of that RAM, there needs to be some sort of retrieval system in place. An *address space* is that retrieval system. Pointers are encoded as memory *addresses*, which are represented as integers of type `usize`. An address points to somewhere within the address space. For the moment, think of the address space as all of your RAM laid out in a single line end to end.

Why are memory addresses encoded as `usize`? Surely there's no 64-bit computer with 2^{64} bytes of RAM. The range of the address space is a façade provided by the operating system and the CPU. Programs only know an orderly series of bytes, irrespective of the amount of RAM that is actually available in the system. We discuss how this works later in virtual memory section of this chapter.



Important

Rust uses the same underlying representation in memory for multiple semantics. For example, `&T` and `&mut T` both look the same once your source code has been compiled. To reiterate, Rust's ownership rules and safety checks are invisible in the compiled binary file. This is a consequence of the *zero-cost abstractions* philosophy.

6.2 EXPLORING RUST'S REFERENCE AND POINTER TYPES

This section teaches you how to work with several of Rust's pointer types. *Rust*

in Action tries to stick to the following guidelines:

- *reference* is used to signal that the Rust compiler will provide its safety guarantees
- *pointer* is used when referring to something more primitive, with an implication that we'll be responsible for maintaining safety. There is a connotation of being unsafe.
- *raw pointer* is used for types where it's important to make their unsafe nature explicit

Throughout this section, we'll be expanding on a common code fragment introduced by [Listing 6.1](#). A pictorial view of what's happening follows the code at [Figure 6.2](#).

Listing 6.1. Mimicking pointers with references. Two global variables, `b` and `c` are pointed to by references. Those references hold addresses of `b` and `c` respectively.

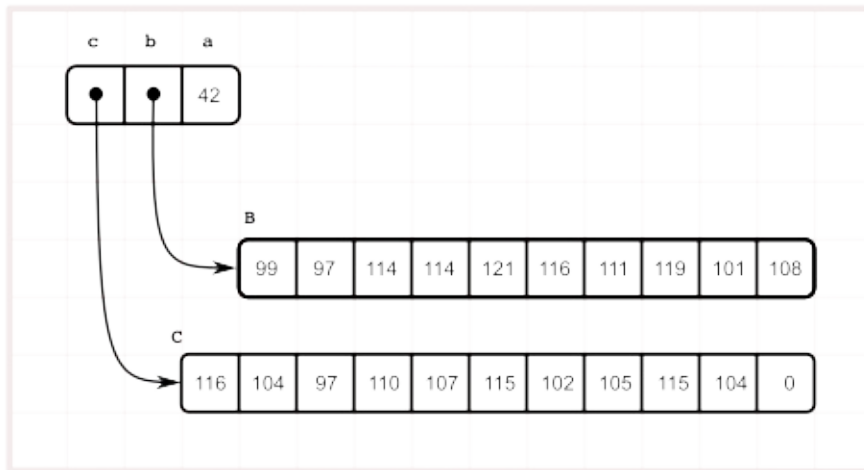
```
static B: [u8; 10] = [99, 97, 114, 114, 121, 116, 111, 119, 101, 108];
static C: [u8; 11] = [116, 104, 97, 110, 107, 115, 102, 105, 115, 104, 0];

fn main() {
    let a = 42;
    let b = &B; ①
    let c = &C; ①

    println!("a: {}, b: {:p}, c: {:p}", a, b, c); ②
}
```

- ① We use the same type of reference for this example for simplicity. Later examples distinguish *smart pointers* from *raw pointers* and require different types.
- ② The `{:p}` syntax asks Rust to format the variable as a pointer. That is, print out the memory address that the value is pointing to.

Figure 6.2. An abstract view of how two pointers operate alongside a standard integer. The important lesson is that the programmer may not know the location of the referent data beforehand.

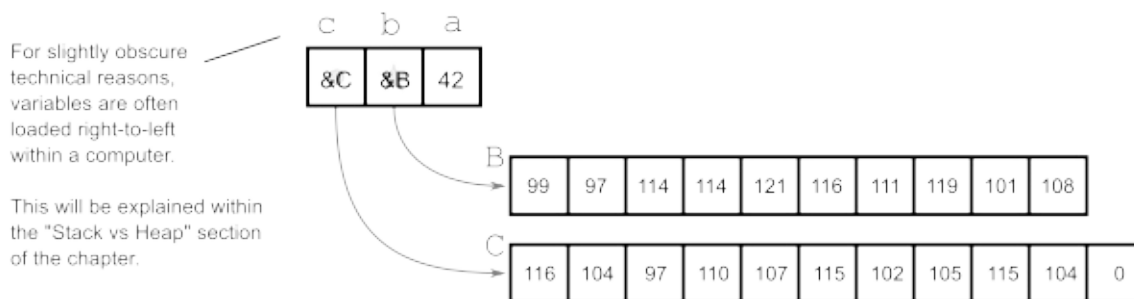


[Listing 6.1](#) has three variables within its `main()` function. `a` is rather trivial, it's just an integer. The two others are more interesting. `b` and `c` are references. They refer to two opaque arrays of data, `B` and `C`. For the moment, consider Rust references as equivalent to pointers.

Listing 6.2. Output of [Listing 6.1](#)

```
a: 42, b: 0x556fd40eb480, c: 0x556fd40eb48a ①
```

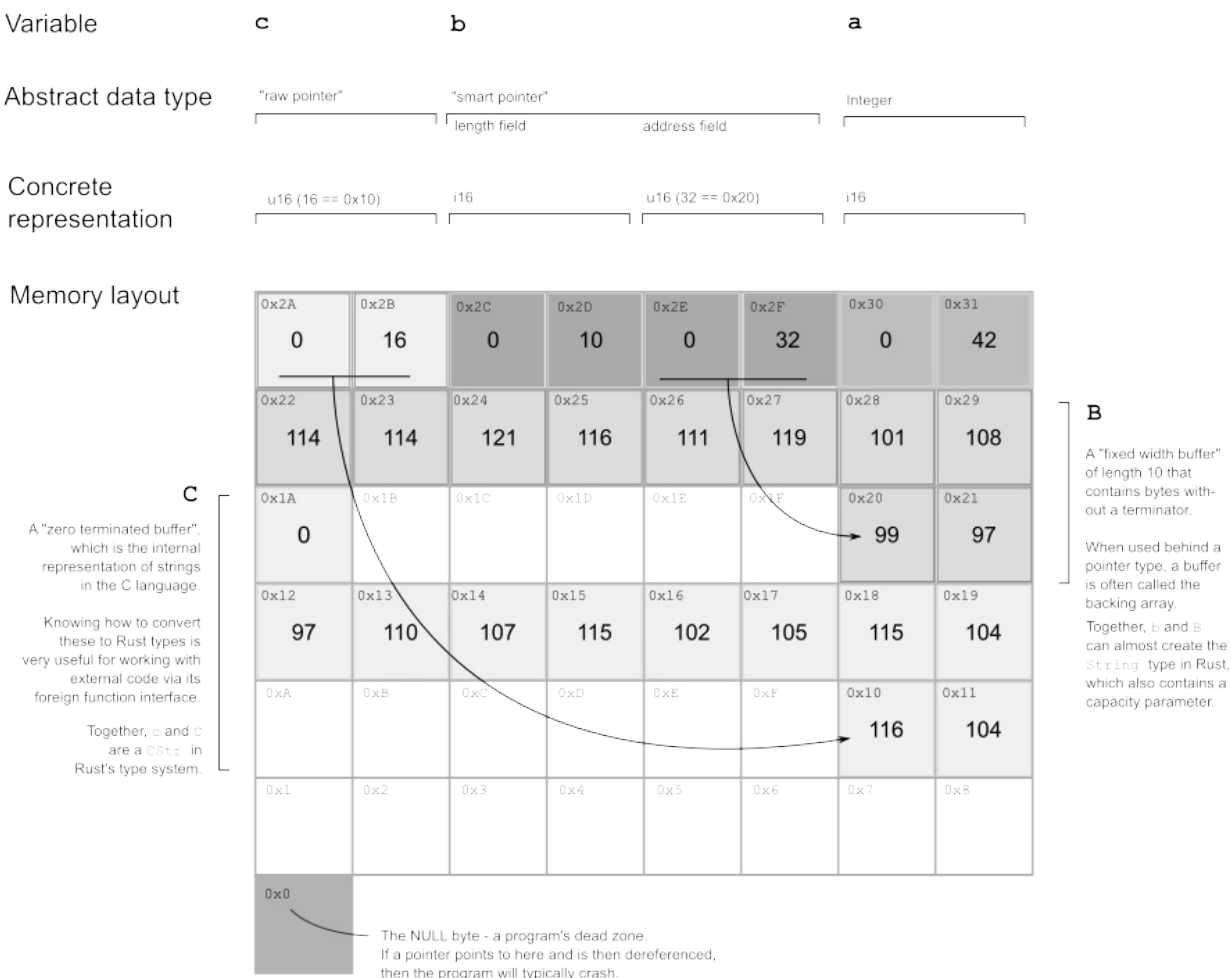
① The exact memory addresses are likely to be different on your machine if you run the code



One problem with portraying pointers as arrows to disconnected arrays is that they de-emphasize that the address space is contiguous and shared between all variables.

[Figure 6.3](#) provides a view of the same example into an imaginary address space of 49 bytes that has a pointer width of two bytes (16 bits). You'll notice that the variables `b` and `c` look different in memory, despite being the same type in [Listing 6.1](#). That's due to [Listing 6.1](#) lying to you. The gritty details—and a code example that's closer to this diagram—are coming shortly.

Figure 6.3. An illustrative address space of the program provided in [Listing 6.1](#). It provides an illustration of the relationship between addresses (typically written in hexadecimal) and integers (typically written in decimal). White cells represent unused memory.



For a more thorough examination of what is happening under the hood, [Listing 6.4](#) produces much more output. It uses more sophisticated types instead of references to correlate more accurately with what is presented in [Figure 6.3](#).

Listing 6.3. Output from [Listing 6.4](#)

```
a (an unsigned integer):
  location: 0x7ffe8f7ddfd0
  size:      8 bytes
  value:     42

b (a reference to B):
  location: 0x7ffe8f7ddfd8
  size:      8 bytes
  points to: 0x55876090c830

c (a "box" for C):
  location: 0x7ffe8f7ddfe0
  size:      16 bytes
  points to: 0x558762130a40

B (an array of 10 bytes):
  location: 0x55876090c830
  size:      10 bytes
  value:      [99, 97, 114, 114, 121, 116, 111, 119, 101, 108]

C (an array of 11 bytes):
  location: 0x55876090c83a
  size:      11 bytes
  value:      [116, 104, 97, 110, 107, 115, 102, 105, 115, 104, 0]
```

Listing 6.4. Comparing references and `Box<T>` to several types demonstrate how they differ internally

```
use std::mem::size_of;

static B: [u8; 10] = [99, 97, 114, 114, 121, 116, 111, 119, 101, 108];
static C: [u8; 11] = [116, 104, 97, 110, 107, 115, 102, 105, 115, 104, 0];

fn main() {
    let a: usize = 42; ①
    let b: &[u8; 10] = &B; ②
    let c: Box<[u8]> = Box::new(C); ③

    println!("a (an unsigned integer):");
    println!(" location: {:p}", &a);
    println!(" size:      {:?} bytes", size_of::<usize>());
    println!(" value:     {:?}", a);
    println!();

    println!("b (a reference to B):");
    println!(" location: {:p}", &b);
    println!(" size:      {:?} bytes", size_of::<&[u8; 10]>());
    println!(" points to: {:p}", b);
    println!();

    println!("c (a \"box\" for C):");
    println!(" location: {:p}", &c);
    println!(" size:      {:?} bytes", size_of::<Box<[u8]>>());
```

```

println!(" points to: {:p}", c);
println!();

println!("B (an array of 10 bytes):");
println!(" location: {:p}", &B);
println!(" size:      {:?} bytes", size_of::<[u8; 10]>());
println!(" value:     {:?}", B);
println!();

println!("C (an array of 11 bytes):");
println!(" location: {:p}", &C);
println!(" size:      {:?} bytes", size_of::<[u8; 11]>());
println!(" value:     {:?}", C);
}

```

- ① `usize` is the size of a memory address for the CPU the code is compiled for. That CPU is called the *compile target*.
- ② `&[u8; 10]` is read as “a reference to an array of 10 bytes”. The array is located in the “static” area of memory, and the reference itself (a pointer of width `usize` bytes) is placed on the stack.
- ③ The `Box<[u8]>` type is read as “a boxed byte slice”. When programmers place values “inside” a box, ownership of the value moves to the owner of the box.

For readers who are interested in decoding the text within `B` and `C`, [Listing 6.5](#) is a short program that matches [Figure 6.3](#) more closely. It contains a number of new Rust features and some relatively arcane syntax that haven’t been introduced yet. They’ll be explained shortly.

Listing 6.5. Using Rust’s tools to print from strings provided by external sources. Almost creates a memory address layout that resembles [Figure 6.3](#).

```

use std::borrow::Cow; ①
use std::ffi::CStr; ②
use std::os::raw::c_char; ③

static B: [u8; 10] = [99, 97, 114, 114, 121, 116, 111, 119, 101, 108];
static C: [u8; 11] = [116, 104, 97, 110, 107, 115, 102, 105, 115, 104, 0];

fn main() {
    let a = 42; ④
    let b: String; ⑤
    let c: Cow<str>; ⑥

    unsafe {
        let b_ptr = &B as *const u8 as *mut u8; ⑦
    }
}

```



```

        b = String::from_raw_parts(b_ptr, 10, 10); ⑧

        let c_ptr = &C as *const u8 as *const c_char; ⑨
        c = CStr::from_ptr(c_ptr).to_string_lossy(); ⑩
    }

    println!("a: {}, b: {}, c: {}", a, b, c);
}

```

- ① Cow stands for “copy on write”. It is a smart pointer type that is able to read from its pointer location without needing to copy it first. This is handy when a buffer is provided to you from an external source. Avoiding copies increases runtime performance.
- ② CStr is a C-like string type from the foreign function interface module of the Rust standard library (`std::ffi`). It allows Rust to read in 0-terminated strings.
- ③ This is not strictly needed, but does make the code’s intent clear. `c_char` is almost certainly a type alias for Rust’s `i8` type, with the possibility of a platform-specific nuances. C does not define the width of its `char` type in its standard, although it’s one byte wide in practice. Retrieving the type alias from the `std::os::raw` module allows for differences.
- ④ Each of the variables are introduced here, so that they’re accessible from `println!` later. If we created `b` and `c` within the `unsafe` block, they would be out of scope later on.
- ⑤ `String`, as it happens, is a smart pointer type. It holds a pointer to a backing array and a field to store the size.
- ⑥ Cow accepts a type parameter for the data it points to. `str` is the type returned by `CStr.to_string_lossy()` and so is appropriate here.
- ⑦ References cannot be cast directly to `*mut T`, which is the type required by `String::from_raw_parts()`. Thankfully `*const T` can be cast to `*mut T`, leading to the this double cast syntax.
- ⑧ `String::from_raw_parts()` accepts a pointer (`*mut T`) to an array of bytes, a size and a capacity parameter.
- ⑨ Here we convert a `*const u8` to a `*const i8`, aliased to `c_char`. The conversion to `i8` works successfully because we remain under 128, following the ASCII standard.
- ⑩ Conceptually, `CStr::from_ptr()` takes responsibility for reading the pointer until it reaches a 0 and then generates a `Cow<str>` from that result.

To understand the code in [Listing 6.5](#), there is quite a bit of ground to cover. We first need to work through what a “raw pointer” is, then discuss a number of feature-rich alternatives that have been built around them.

6.2.1 Raw pointers in Rust

A “raw” pointer is a memory address without Rust’s standard guarantees. They are inherently unsafe. For example, unlike references (`&T`), they can be `null`.

If you’ll forgive the syntax, raw pointers are denoted as `*const T` and `*mut T` for immutable and mutable raw pointers respectively. That is, even though each is a single type, they contain three tokens: `*`, `const` or `mut`, and their type `T`.

- A raw pointer to a `String` looks like `*const String`.
- A raw pointer to an `i32` is written as `*mut i32`.

Before we put pointers into practice, here are two other things that are useful to know:

- The difference between a `*mut T` and a `*const T` is minimal. They can be freely cast between one another and tend to be used interchangeably. They act as in-source documentation.
- Rust references (`&mut T` and `&T`) compile down to raw pointers. That means that it’s possible to access the performance of raw pointers without needing to venture into unsafe blocks.

Here is a small example that creates a raw pointer from an `i64` value. It then prints the value and its address in memory via the `{:p}` syntax.

Listing 6.6. Creating a raw pointer (`*const T`) by coercing a reference to a value (`&T`) into one

```
fn main() {  
    let a: i64 = 42;  
    let a_ptr = &a as *const i64; ①  
  
    println!("a: {} ({:p})", a, a_ptr); ②  
}
```

- ① Cast a reference to a (&a) to a constant raw pointer to an i64 (*const i64)
- ② Print the value of the variable a (42) and its address in memory (0x7ff...)

The terms pointer and memory address are sometimes used interchangeably. They are integers that represent a location in virtual memory. From the compiler's point of view though, there is one important difference. Rust's pointer types—*const T and *mut T—always point to the starting byte of T and they also know the width of type T in bytes. A memory address might refer to anywhere in memory.

An i64 is 8 bytes wide (64 bits ÷ 8 bits per byte). Therefore, if an i64 is stored at address 0x7fffd, then each of the bytes between 0x7fffd..0x8004 must be fetched from RAM to recreate the integer's value. The process of fetching data from RAM from a pointer is known as *dereferencing a pointer*.

Listing 6.7. Identifying a value's address by casting a reference to it as a raw pointer via std::mem::transmute

```
fn main() {  
    let a: i64 = 42;  
    let a_ptr = &a as *const i64;  
    let a_addr: usize = unsafe { std::mem::transmute(a_ptr) }; ①  
  
    println!("a: {} ({:p}...0x{:x})", a, a_ptr, a_addr + 7);  
}
```

- ① transmute() interprets the bytes at the location as another type. Using transmute() here is considered unidiomatic Rust, but allows us to postpone introducing more syntax.

Under the hood, references (&T and &mut T) are implemented as raw pointers. But they come with extra guarantees and should always be preferred. Accessing the value of a raw pointer is always unsafe.

Using raw pointers in Rust code is like working with pyrotechnics. Usually the results are fantastic, sometimes they're painful and very occasionally they're tragic. Raw pointers should be considered an intermediary step between

receiving something from the operating system or another programming language and into a reference.

To demonstrate their volatility, let's work through a quick example with Rust's raw pointers. Creating a pointer of arbitrary types from any integer is perfectly legal. Dereferencing that pointer must occur within an unsafe block, implying that the programmer takes full responsibility for any consequences.

```
fn main() {  
    let ptr = 42 as *const Vec<String>; ①  
  
    unsafe {  
        let new_addr = ptr.offset(4);  
        println!("{:p} -> {:p}", ptr, new_addr);  
    }  
}
```

① Pointers can be created from any integral value safely. An `i32` is not a `Vec<String>`, but Rust is quite comfortable to ignore that here.

To reiterate - raw pointers are not safe. They have a number of properties that mean that their use is strongly discouraged within day-to-day Rust code.

- They do not own their values. The Rust compiler does not check that the referent data is still valid when they're accessed
- Multiple raw pointers to the same data is allowed. Every raw pointer can have write/read/write access to that data. This means that there is no time when Rust can guarantee that the shared data is valid.

Notwithstanding those warnings, there are a small number of valid reasons to make use of raw pointers.

- It's unavoidable. Perhaps some operating system call or 3rd party code requires a raw pointer. Raw pointers are very common with C code that provides an external interface.
- Shared access to something is essential and runtime performance is paramount. Perhaps multiple components within your application require equal access to some expensive-to-compute variable. If

you're willing to take on the risk of one of those components poisoning every other component with some silly mistake, then raw pointers are an option of last resort.

6.2.2 Rust's pointer ecosystem

Given that raw pointers are unsafe, what is the safer alternative? The alternative is to use *smart pointers*. In the Rust community, a smart pointer is some pointer type that has some kind of super power, over and above the ability to deference a memory address. You will probably encounter the term *wrapper type* as well. Rust's smart pointer types tend to wrap raw pointers and bestow them with added semantics.

A narrower definition of “smart pointer” is common in the C communities. There, authors (generally) imply that the term smart pointer means the C equivalents of Rust's `core::ptr::Unique`, `core::ptr::Shared` and `std::rc::Weak` types. You will be introduced to those four types shortly.



Note

The term *fat pointer* refers to memory layout. Thin pointers, such as raw pointers, are a single `usize` wide. Fat pointers are usually two `usize` wide. Occasionally more.

Rust has an extensive set of pointer (and pointer-like) types in its standard library. Each has their own role, strengths and weaknesses. Given their unique properties, rather than writing them out as a list, let's model them as characters in a card-based role playing game.

Raw Pointer



The cousins `*mut T` and `*const T` are the free radicals of the pointer world. Lightning fast, but wildly unsafe.

Powers

- Speed
- Can interact with outside world

Weaknesses

- Unsafe

Image: Detail of *Acrobats Beneath Cherry Trees*: *Spinning Tops and Balancing* by Utagawa Yoshiharu

Box<T>



Store anything in a box. Accepts almost any type for long term storage. The workhorse of a new, safe programming era.

Powers

- Store a value in central storage, in a location called "the heap"

Weaknesses

- Size increase

Image: Detail from 15th edition of *Caesar's Column: A Story of the Twentieth Century* by Ignatius Donnelly

Rc<T>



The reference counted pointer, `Rc<T>` is Rust's competent, yet miserly bookkeeper. It knows who has borrowed what and when.

Powers

- Shared access to values

Weaknesses

- Size increase
- Runtime cost
- Not threadsafe

Image: Detail of *The Miser*, from *The Dance of Death* by Wenceslaus Hollar

Arc<T>



`Arc<T>` is Rust's abassador. It can share values across threads, guaranteeing that they will not interfere with each other.

Powers

- Shared access to values
- Threadsafe

Weaknesses

- Size increase
- Runtime cost

Image: Detail of *The Grand Vizir giving Audience to the English Ambassador* by Robert Pranker

Cell<T>



An expert in metamorphosis, Cell confers the ability to mutate immutable values.

Powers

- Interior mutability

Weaknesses

- Size increase
- Performance

Image: Detail from *On the Disposition of Iron in Variegated Strata* by George Mew

RefCell<T>



Perform mutation on immutable references with RefCell<T>. Its mind-bending powers come with some costs.

Powers

- Interior mutability
- Can be nested within Rc and Arc, which only accept immutable refs

Weaknesses

- Size increase
- Runtime speed
- Lack of compile-time guarantees

Image: Detail of figure 187 from *The Principles of Light and Color* by Edwin Babbitt

Cow<T>



Why write something down when you only need to read it? Perhaps if you want to make modifications. This is the role of Cow (copy on write).

Powers

- Avoids writes when only read access is used

Weaknesses

- Possible size increase

Image: Detail of *Lessons for the flute alone* as written by Thomas Britten

String



Acting as a guide on how to deal with the uncertainties of user input, String shows us how to build safe abstractions.

Powers

- Grows dynamically as required
- Guarantees correct encoding at runtime

Weaknesses

- Size, can over-allocate

Image: Detail from *Remarks on the Abracadabra of the Nineteenth Century* by William Leo-Wolf

Vec<T>



The bedrock of dynamically sized types within Rust, `Vec<T>` understands how to provide a home for your data as needed.

Powers

- Grows dynamically as required

Weaknesses

- Size, can over-allocate

Image: Detail from *Chronologischer Raupenkalender, oder, Naturgeschichte der europäischen Raupen* by Christian Friedrich Vogel

RawVec<T>



The bedrock of `Vec<T>` and other dynamically sized types. Understands how to provide a home for your data as needed.

Powers

- Grows dynamically as required
- Works with the memory allocator to find space

Weaknesses

- Not directly applicable from your code

Image: Detail from *Chronologischer Raupenkalender, oder, Naturgeschichte der europäischen Raupen* by Christian Friedrich Vogel

Unique<T>



Sole owner of a value, a unique pointer is guaranteed to possess full control.

Powers

- Base for types requiring exclusive possession of values, such as `String`

Weaknesses

- Not appropriate for application code directly

Image: Detail from illustrations from *From Earth to the Moon* by Emile-Antoine Bayard

Shared<T>



Sharing ownership is hard. `Shared<T>` makes life a little bit easier.

Powers

- Shared ownership
- Can align memory to `T`'s width, even when empty

Weaknesses

- Not appropriate for application code directly

Image: Detail from *New Pictures in Old Frames: being a book of verse for girls and boys, written in old French forms, etc* by Gertrude Bradley

Each of the pointer types introduced here will be used extensively throughout the book. As such, we'll give them fuller treatment when they're needed. For now, the two novel attributes that appear within the "powers" section of some of these

cards are *interior mutability* and *shared ownership*. They warrant some discussion:

- Interior mutability - You may wish to provide an argument to a method that takes immutable values, yet you need to retain mutability. If you're willing to pay the runtime performance cost, it's possible to fake immutability. If the method requires an owned value, wrap the argument in `Cell<T>`. References can be wrapped in `RefCell<T>`. It is common when using reference counted types `Rc<T>` and `Arc<T>`, which only accept immutable arguments, to also be wrapped in `Cell<T>` or `RefCell<T>`. The resulting type might look like `Rc<RefCell<T>>`. This means that you pay the runtime cost twice, but with significantly more flexibility.
- Shared ownership - Some objects, such as network connections or access to some operating system service perhaps, are difficult to mould into the pattern of having a single place with read/write access at any given time. Code might be simplified if two parts of the program can share access to that single resource. Rust will allow you to do this, but at the expense of a runtime cost.

6.2.3 Smart pointer building blocks

You may arise at a situation when you wish to build your own smart pointer type with its own semantics. Perhaps a new research paper has been released and you wish to incorporate its results into your own work. Perhaps you're conducting the research! Regardless, it might be useful to note that Rust's pointer types are extensible. That is, they're designed with extension in mind.

All of the programmer-facing pointer types, such as `Box<T>`, are built from more primitive types that live deeper within Rust, often in its core or `alloc` modules.

The C++ smart pointer types have Rust counterparts:

Here are some useful starting points for when you investigate building your own smart pointer types:

- `core::ptr::Unique` is the basis for types such as `String`, `Box<T>` and the pointer field of `Vec<T>`
- `core::ptr::Shared` is the basis for `Rc<T>` and `Arc<T>`. It can handle situations where shared access is desired.

In addition, the following tools can also be very handy in certain situations.

- Deeply inter-linked data structures may benefit from `std::rc::Weak` and `std::arc::Weak` for single and multi-threaded programs respectively. They allow access to data within an `Rc/Arc` without incrementing its reference count. This can prevent never-ending cycles of pointers.
- `alloc::raw_vec::RawVec` supports `Vec<T>` and `VecDeque<T>` (an expandable, double-ended queue that hasn't appeared in the book so far). It understands how to allocate and deallocate memory in a smart way for any given type.
- `std::cell::UnsafeCell` sits behind both `Cell<T>` and `RefCell<T>`. If you would like to provide interior mutability to your types, its implementation is worth investigating.

A full treatment of building new safe pointers touches on some of Rust's internals. These building blocks have their own building blocks. Unfortunately, explaining every detail will diverge too far from our goals for this chapter.

6.3 PROVIDING PROGRAMS WITH MEMORY FOR THEIR DATA

This section attempt to demystify the terms: “the stack” and “the heap”. They often appear in contexts that pre-suppose you already know what they mean. That isn't the case here. We'll cover the details of what they are, why they are and how to make use of that knowledge to make your programs leaner and faster.

Some people hate wading through the details though. For those readers, here is salient difference between the stack and the heap:

- the stack is fast
- the heap is slow

That difference leads to the following axiom: “when in doubt, prefer the stack”. To place data onto the stack, the compiler must know the type’s size at compile time. Translated to Rust, that means “when in doubt, use types that implement `Sized`”.

Now that you’ve got the gist, it’s time to learn when to take the slow path and how to avoid it when you want to go faster.

6.3.1 The stack

The stack is often described by analogy: think of a stack of dinner plates waiting in the cupboard of a commercial kitchen. Cooks are taking plates off the stack to serve food, lowly dishwashers are placing new plates on the top.

The unit—the plate—of a computing stack is the *stack frame*, also known as the *allocation record*. You are probably used to think of This is a group of variables and other data.

Like many descriptions in computing, “the stack” and “the heap” are analogies that only partially fit. If you are learning about them for the first time, you may need to untangle yourself from the mental picture of them created by its name.

The stack is often described by analogy to a stack of dinner plates waiting in the cupboard. Unfortunately, that mental picture is inaccurate. Here are some differences:

- the stack actually contains two levels of “objects”: *stack frames* and data
- the stack grants programmers with access to multiple elements stored within it, rather than the top item only.
- the stack can include elements of arbitrary size, where the implication of the dinner plate analogy is that all elements must be the same size.

- as an implementation detail, items within the stack are not necessarily removed from RAM when they are inaccessible from code. When dinner plates are picked up, they've really gone. Instead, some designated machinery within the CPU marks the location of the new top.

So why is the stack called the stack? Because of the usage pattern. Entries on the stack are made in a Last In, First Out (LIFO) manner.

The entries in the stack are called *stack frames*. Stack frames are created as function calls are made. As a program makes progress, a cursor within the CPU updates to reflect the current address of the current stack frame. The cursor is known as the *stack pointer*. As functions are called within functions, the stack pointer decreases in value as the stack grows. When a function returns, the stack pointer increases.

Stack frames contain a function's state during the call. When a function is called within a function, the older function's values are effectively frozen in time. They are also known as a *activation frames*, and less commonly *allocation records*.

[\[39\]](#)

Unlike dinner plates, every stack frame is a different size. The stack frame contains space for its function's arguments, a pointer to the original call site and local variables (except the data which is allocated on the heap).



Tip

If you are unfamiliar with what the term “call site” means, refer to the CPU emulation section of chapter 5.

To understand what is happening more fully, let's consider a thought experiment. Imagine a diligent, yet absurdly single-minded cook in a commercial kitchen. The cook takes each table's docket and places them in a queue. The cook has a fairly bad memory, so she writes down what her current order is in her own notebook. As new orders come in, she updates her notebook to refer to the new order. As she completes orders, her notebook is changed to the next item on the queue. Unfortunately for customers in this restaurant, the book operated in a last

in, first out manner. Hopefully, you will not be one of the early orders during tomorrow's lunch rush. In this experiment, her notebook plays the role of the stack pointer. The stack itself is comprised of variable-length dockets, representing stack frames.

Like stack frames, restaurant dockets contain some metadata. The table number can act as the return address.

The stack's primary role is to make space for local variables. Why is the stack fast? All of a function's variables are side-by-side in memory. That speeds up access.

Improving the ergonomics of functions that can only accept `String` or `&str`

As a library author, it can simplify downstream application code if your functions could accept both `&str` and `String` types. Unfortunately, these two types have very different representations in memory. One (`&str`) is allocated on the stack, the other (`String`) also allocates memory on the heap. That means that types cannot be trivially cast between one another. It's possible to work around this with Rust's generics. Consider the example of validating a password. For the purposes of the example, a strong password is one that's at least 6 characters long.

Listing 6.8. Validating a password by checking its length

```
fn is_strong(password: String) -> bool {  
    password.len() > 5  
}
```

`password_is_strong` can only accept `String`. That means that the following code won't work:

```
let pw = "justok";  
let is_strong = is_strong(pw);
```

Generic code can help.

In cases where read-only access is required, use functions with the type signature `fn x<T: AsRef<str>>(a: T)` rather than `fn x(a: String)`. The fairly unwieldy type signature reads as "function `x` takes an argument `password` of type `T`, where `T` implements `AsRef<str>`". Implementors of `AsRef<str>` behave as a reference to `str`, even when they're not.

Here is the code snippet again, now with the new signature in place:

Listing 6.9. Accepting any type `T` that implements `AsRef<str>`. `AsRef<str>` means that it can behave as a reference to `str`.

```
fn is_strong<T: AsRef<str>>(password: T) -> bool { ①
    password.as_ref().len() > 5
}
```

① Calling code can provide a `String` or a `&str` as password.

When read/write access to the argument is required, normally you can make use of `AsRef<T>`'s sibling trait `AsMut<T>`. Unfortunately for this example, `&'static str` cannot become mutable and so another strategy can be deployed: implicit conversion. It's possible to ask Rust to accept only those types which can be converted to `String`. We'll then perform that conversion within the function and apply any required business logic to that newly created `String`.

Listing 6.10. Accepting any type τ that can be converted to a `String`. This can circumvent the issue that `&str` is an immutable value.

```
fn is_strong<T: Into<String>>(password: T) -> bool {
    password.into().len() > 5
}
```

This implicit conversion strategy does have significant risks though. If a `String`-ified version of the password variable needs to be created multiple times in the pipeline, it would be much more efficient to require an explicit conversion within the calling application. That way the `String` will be created once and reused.

6.3.2 The heap

This section introduces “the heap”. The heap is an area of a program’s memory for types that do not have sizes known at compile time.

What does it mean to have no known size at compile time? In Rust, there are two senses of this word. Some types grow and shrink over time as required. Obvious cases are `String` and `Vec<T>`. Other types, even though they don’t change size at runtime, are unable to tell the Rust compiler how much memory to allocate. These are known as *dynamically sized types*. Slices (`[T]`) are the commonly cited example. Slices have no compile-time length. Internally, they’re a pointer to some part of an array. But slices actually represent some number of elements within that array. Another example is a *trait object*, which haven’t been described in this book so far. Trait objects allow Rust programmers to mimic some features of dynamic languages, by allowing multiple types to be wedged

into the same container.

WHAT IS THE HEAP?

You will gain a fuller understanding of *what* the heap is once you work through the next section on virtual memory. For now, let's concentrate on what it is *not*. Once those points are clarified, we'll then work back towards some form of truth.

The word “heap” implies disorganization. A closer analogy would be warehouse space in some medium-sized business. As deliveries arrive (as variables are created), the warehouse makes space available. As the business carries out its work, those materials are used and the warehouse space can now be made available for new deliveries. At times, there are gaps and perhaps a bit of clutter is left around over time. But overall, there is a good sense of order.

Another mistake that the “heap” has no relation to the data structure known as “a heap”. That data structure is often used to create priority queues. It's an incredibly clever tool in its own right, but right now it's a complete distraction. The heap is not a data structure. It's an area of memory.

Now that those two distinctions are made, let's inch towards an explanation. The critical difference from a usage point of view is that variables on the heap must be accessed via a pointer, whereas this is not required variables accessed on the stack.

Although it's a trivial example, let's consider two variables *a* and *b*. They're both representing integers, 40 and 60 respectively. In one of those cases though, the integer happens to live on the heap.

```
let a: i32 = 40;  
let b: Box<i32> = Box::new(60);
```

Now, let's demonstrate that critical difference. The following code won't compile:

```
let result = a + b;
```

The boxed value assigned to `b` is only accessible via a pointer. To access that value, we need to *dereference* it. The *dereference operator* is a unary `*`, which prefixes the variable name:

```
let result = a + *b;
```

This syntax can be difficult to follow at first, as the symbol is also used for multiplication, but it does become more natural over time. [Listing 6.11](#) is a complete example.

Listing 6.11. Creating variables on the heap implies creating that variable via a pointer type, such as `Box<T>`.

```
fn main() {  
    let a: i32 = 40; ①  
    let b: Box<i32> = Box::new(60); ②  
  
    println!("{}", a, b, a + *b); ③  
}
```

- ① 40 lives on the stack
- ② 60 lives on the heap
- ③ To access the 60, we need to dereference it.

To get a feel for what the heap is and what is happening within memory as a program is running, let's consider a tiny example. All we will be doing is creating some numbers “on the heap”, then adding their values together.

A further introduction to using the heap is provided by the larger [Listing 6.13](#). When run, the program produces some fairly trivial output: two 3s. Still, it's really the internals of the program's memory that are important here, not its results. A pictorial view of the program's memory as it runs follows the code at [Figure 6.4](#).

Listing 6.12. Output from [Listing 6.13](#)

Listing 6.13. Allocating and de-allocating memory on the heap via `Box<T>` (ch6/ch6-heap-via-box/src/main.rs)

```
use std::mem::drop; ①

fn main() {
    let a = Box::new(1); ②
    let b = Box::new(1); ②
    let c = Box::new(1); ②

    let result1 = *a + *b + *c; ③

    drop(a); ④
    let d = Box::new(1);
    let result2 = *b + *c + *d;

    println!("{}", result1, result2);
}
```

- ① Bring manual `drop()` into local scope
- ② Allocate values “on the heap”
- ③ The unary `*` is called the dereference operator. It returns the value within the box. `result1` holds the value 3.
- ④ Invoke `drop()`, freeing up memory for other uses

[Listing 6.13](#) places four values “on the heap” and removes one. It contains some new—or at least less familiar—syntax that might be worthwhile to cover and/or recap:

- `Box::new(T)` allocates `T` on the heap. “Box” is term that can be deceptive if you don’t share its intuition. Something that has been “boxed” lives on the heap, with a pointer to it on the stack. This is demonstrated in the first column of [Figure 6.4](#), where the number `0x100` at address `0xfff` points to the value 1, at address `0x100`. However, there is no actual “box” of bytes enclosing a value. Nor is the value hidden or concealed in some way.
- `std::mem::drop` brings the function `drop()` into local scope. `drop()` is used to delete objects before their scope ends. Types that

implement `Drop`, have a `drop()` method, but explicitly calling it is illegal within user code. `std::mem::drop` is an escape hatch from that rule.

- Asterisks next to variables (`*a`, `*b`, `*c`, `*d`) are unary operators. This is the *dereference operator*. Dereferencing a `Box::(T)` returns `T`. In our case, the variables `a`, `b`, `c` and `d` are references that refer to integers.

Each column illustrates what is happening inside memory at 6 lines of code. The stack appears as the boxes along the top, and the heap appears along the bottom. The figure omits several details, but should help you gain an intuition of the relationship between the stack and the heap.



Note

If you have experience with a debugger and wish to explore what is happening, be sure to compile your code with no optimizations. Compile your code with `cargo build` (or `cargo run`), rather than `cargo build --release`. Using the `--release` flag actually ends up optimizing all of the allocations and arithmetic away. If you are invoking `rustc` manually, the command to `rustc --codegen opt-level=0` if you are invoking `rustc` manually.

Figure 6.4. A view into a program's memory layout during the execution of [Listing 6.13](#)

How to Read This Diagram

The top box represents the stack.
The bottom box represents the heap.

Note:

The stack grows downwards, rather than upwards as its name suggests. It starts at the top of the address space, which is 4096 bytes (0xfff) here for simplicity's sake.

The heap begins at the bottom, plus an offset, which is 256 (0x100) here. The space between 0 and the offset is reserved for the program's executable instructions and variables that last the lifetime of the program.

The stack and the heap are talked about as two entities, but both occupy the same address space. Starting at opposite ends of the address space avoids conflicts.

The top box represents the **stack**.

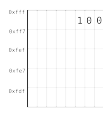
The stack grows downwards, rather than upwards as its name suggests.

For simplicity's sake, this example's address space is 4096 bytes. In a more realistic scenario, such as with a 64-bit CPU, the address space is 2^{32} bytes long.

The bottom box represents the **heap**.

The heap begins at the bottom of the address space, plus an offset, which is 256 (0x100) here.

The space between 0 and the offset is reserved for the program's executable instructions and variables that last the lifetime of the program.



Values within cells describe the state of memory at the end of the current line.

Each cell represents an individual byte (the u8 type in Rust terminology).

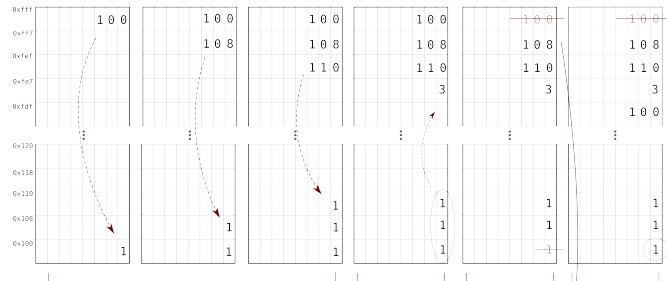
Blank cells represent uninitialized memory.

Each span of bytes is the width of usize, which is 8 bytes on a 64-bit CPU.

Program execution over time

```
let a = Box::new(1);
let b = Box::new(1);
let c = Box::new(1);
let result1 = *a + *b + *c;
drop(a);
let d = Box::new(1);
```

Memory layout over time



i32 values are allocated on the heap and a pointer to that value's address is placed on the stack (the integers are "boxed").

The three integers are added together and their sum is placed on the stack.

The boxed value probably hasn't been deleted from the heap, but the memory allocator has marked that location as free for re-use.

The space occupied by "a" is re-used by "d".

How to interpret this diagram

The top box represents the **stack**.

The stack grows downwards, rather than upwards as its name suggests.

For simplicity's sake, this example's address space is 4096 bytes. In a more realistic scenario, such as with a 64-bit CPU, the address space is 2^{32} bytes long.



The bottom box represents the **heap**.

The heap begins at the bottom of the address space, plus an offset, which is 256 (0x100) here.

The space between 0 and the offset is reserved for the program's executable instructions and variables that last the lifetime of the program.

Rust's super power

The lifetime of the variable "a" has ended at this point.

Accessing this memory address is now invalid. Its data will still be there on the stack, but it's **impossible** to access it within safe Rust.

Note: this behavior is implementation-dependent. Memory allocators are free to make other decisions.

6.3.3 What is dynamic memory allocation?

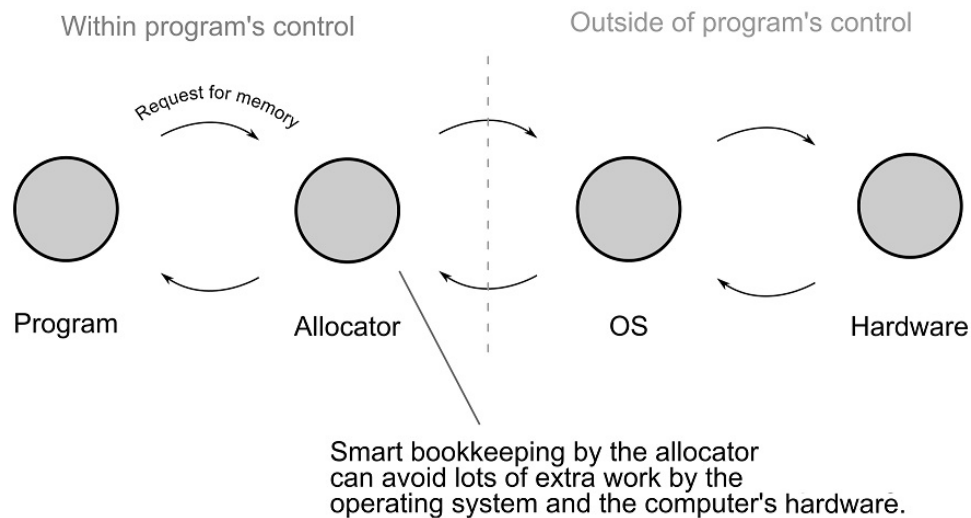
At any given time, a running program has a fixed number of bytes to get its work done in. When the program would like more memory, it needs to ask for more from the operating system. This is known as *dynamic memory allocation*.

Dynamic memory allocation is a three step process:

- Step 1: Request memory from the operating system via a *system call*. In the UNIX family of operating systems, this system call is `alloc()`. In MS Windows, the call is named `HeapAlloc()`.
- Step 2: Make use of the allocated memory in the program.
- Step 3: Release memory that isn't needed back to the operating system via `free()` (`HeapFree()` in MS Windows)

As it turns out, there is an intermediary between the program and the operating system: the allocator. The allocator—a specialist sub-program that is embedded into yours behind the scenes—can perform optimizations that avoid lots of work within the operating system and CPU.

Figure 6.5. Conceptual view of dynamic memory allocation. Requests for memory originate and terminate at the program, but involve several other components. At each stage, the components may short-circuit the process and return quickly.



Let's examine performance impact of dynamic memory allocation and strategies to reduce that impact. Before starting, let's recap why there's a performance difference between the stack and the heap. Remember that the stack and the heap are conceptual abstractions only. They do not exist as physical partitions of your computer's memory. So what accounts for their different performance characteristics?

The accessing data on the stack is very fast access because a function's local variables, which are allocated on the stack, reside next to each other in RAM. This is sometimes referred to as *contiguous layout*. Contiguous layout is very *cache-friendly*. Variables allocated on the heap are extremely unlikely to reside next to each other. Moreover, accessing data on the heap involves dereferencing the pointer. That implies a page table lookup and a trip to main memory.

Table 6.1. A simplistic, yet practical, table for comparing the stack and the heap

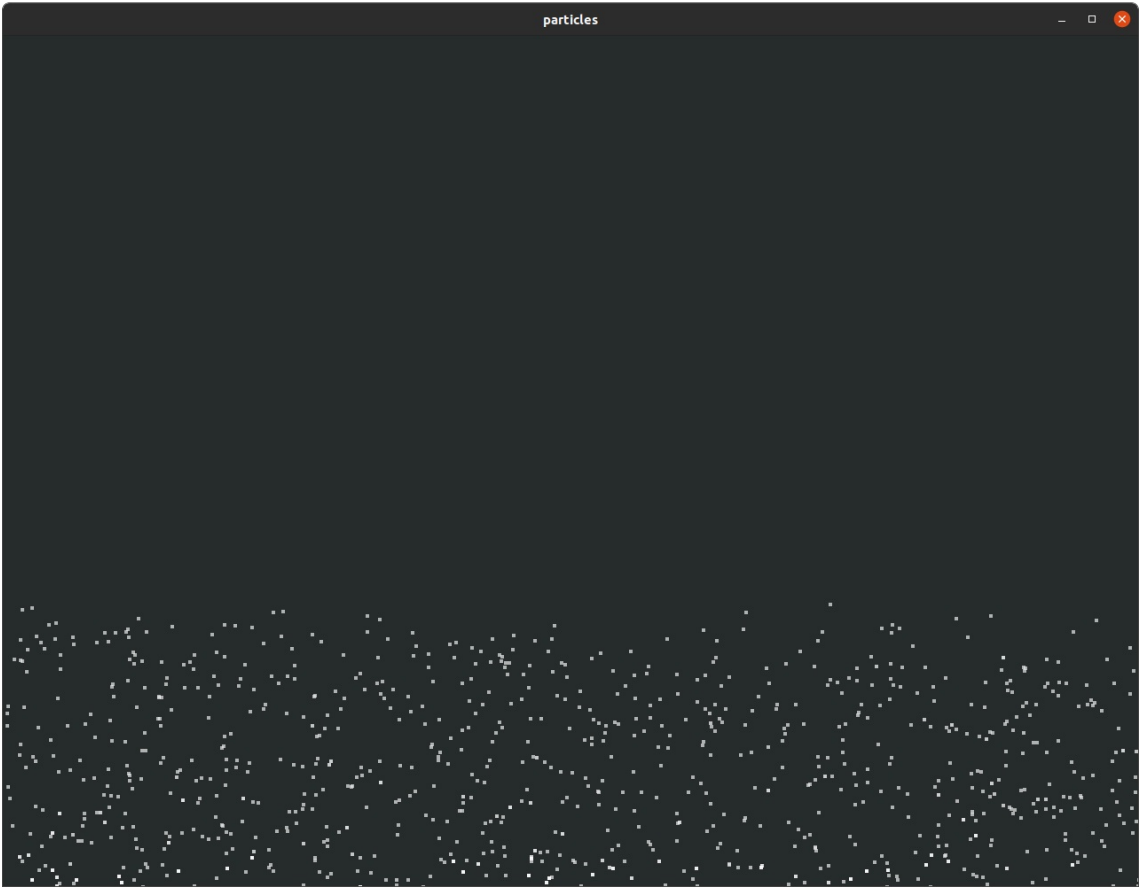
Stack	Heap
Simple	Complex
Safe	Dangerous [*]
Fast	Slow
Rigid	Flexible

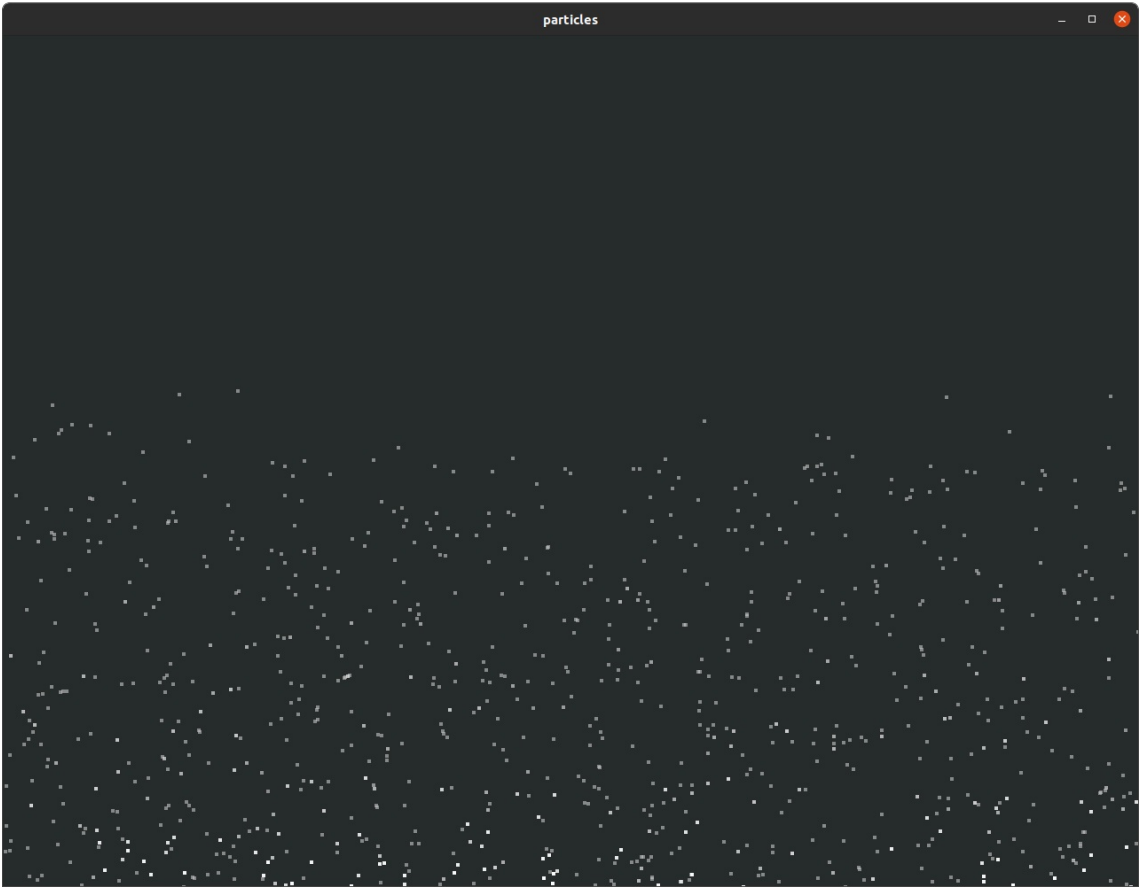
^{*} (Not in safe Rust!)

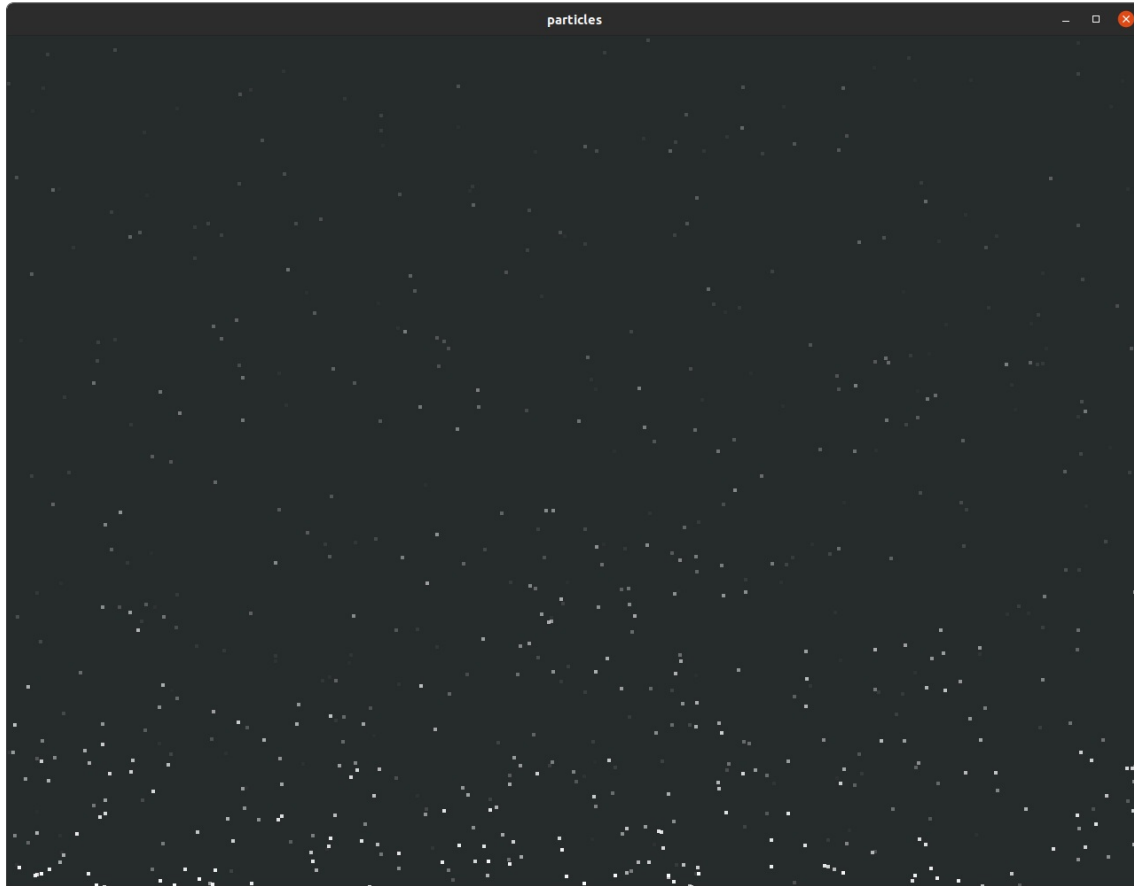
There is a trade off for the stack's increased speed. Data structures on the stack must stay the same size during the lifetime of the program. Data structures allocated on the heap are more flexible. Because they are accessed via a pointer, that pointer can be changed

To quantify this impact, we need to learn how to measure the cost. To get a large number of measurements, we need a program that creates and destroys many values. Let's create a toy program: a background element to a video game.

Figure 6.6. Screenshots from the result of running [Listing 6.15](#)







After running [Listing 6.15](#), you should see a window appear on your screen filled with a dark grey background. White snow-like dots will start to float from the bottom and fade as they approach the top. If you check the console output, streams of numbers will be appearing. Their significance will be explained once the code is discussed.

[Listing 6.15](#) contains three major sections:

- a memory allocator (the `ReportingAllocator` struct) with the responsibility of recording the time that dynamic memory allocations have taken to fulfil
- definitions of the structs `World` and `Particle` and how they behave over time
- the `main()` function, that deals with window creation and initialization

Listing 6.14. Build dependencies for [Listing 6.15](#) (ch6/ch6-particles/Cargo.toml)

```
[package]
name = "ch6-particles"
version = "0.1.0"
authors = ["TS McNamara <author@rustinaction.com>"]
edition = "2018"

[dependencies]
piston_window = "0.117"      ①
piston2d-graphics = "0.39"  ②
rand = "0.8"                 ③
```

- ① piston_window provides a wrapper around the core functionality of the piston game engine. This will allow us to easily draw things on screen, largely irrespective of host environment.
- ② piston2d-graphics provides vector mathematics, which is important to simulate movement.
- ③ rand provides random number generators and associated functionality.

Listing 6.15. A graphical application that creates and destroys many heap-allocated objects (ch6/ch6-particles/main.rs)

```
use graphics::math::{Vec2d, add, mul_scalar}; ①
use piston_window::*; ②
use rand::prelude::*; ③

use std::alloc::{GlobalAlloc, System, Layout}; ④
use std::time::Instant; ⑤

#[global_allocator] ⑥
static ALLOCATOR: ReportingAllocator = ReportingAllocator;

struct ReportingAllocator; ⑦

unsafe impl GlobalAlloc for ReportingAllocator {
    unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
        let start = Instant::now();
        let ptr = System.alloc(layout); ⑧
        let end = Instant::now();
        let time_taken = end - start;
        let bytes_requested = layout.size();

        eprintln!("{}", bytes_requested, time_taken.as_nanos());
        ptr
    }
}
```

```

    unsafe fn dealloc(&self, ptr: *mut u8, layout: Layout) {
        System.dealloc(ptr, layout);
    }
}

struct World {
    current_turn: u64,
    particles: Vec<Box<Particle>>,
    height: f64,
    width: f64,
    rng: ThreadRng,
}

struct Particle {
    height: f64,
    width: f64,
    position: Vec2d<f64>,
    velocity: Vec2d<f64>,
    acceleration: Vec2d<f64>,
    color: [f32; 4],
}

impl Particle {
    fn new(world : &World) -> Particle {
        let mut rng = thread_rng();
        let x = rng.gen_range(0.0..=world.width);
        let y = world.height;
        let x_velocity = 0.0;
        let y_velocity = rng.gen_range(-2.0..0.0);
        let x_acceleration = 0.0;
        let y_acceleration = rng.gen_range(0.0..0.15);

        Particle {
            height: 4.0,
            width: 4.0,
            position: [x, y].into(),
            velocity: [x_velocity, y_velocity].into(),
            acceleration: [x_acceleration, y_acceleration].into(),
            color: [1.0, 1.0, 1.0, 0.99],
        }
    }

    fn update(&mut self) {
        self.velocity = add(self.velocity, self.acceleration);
        self.position = add(self.position, self.velocity);
        self.acceleration = mul_scalar(self.acceleration, 0.7);
        self.color[3] *= 0.995;
    }
}

impl World {
    fn new(width: f64, height: f64) -> World {
        World {
            current_turn: 0,
            particles: Vec::<Box<Particle>>::new(),
            height: height,
            width: width,
            rng: thread_rng(),
        }
    }
}

```

```

fn add_shapes(&mut self, n: i32) {
    for _ in 0..n.abs() {
        let particle = Particle::new(&self); ②0
        let boxed_particle = Box::new(particle);
        self.particles.push(boxed_particle);
    }
}

fn remove_shapes(&mut self, n: i32) {
    for _ in 0..n.abs() {
        let mut to_delete = None;
        for (i, particle) in self.particles.iter().enumerate() {
            if particle.color[3] < 0.02 {
                to_delete = Some(i);
            }
            break;
        }
        if let Some(i) = to_delete {
            self.particles.remove(i);
        } else {
            self.particles.remove(0);
        };
    }
}

fn update(&mut self) {
    let n = self.rng.gen_range(-3..=3);

    if n > 0 {
        self.add_shapes(n);
    } else {
        self.remove_shapes(n);
    }

    self.particles.shrink_to_fit();
    for shape in &mut self.particles {
        shape.update();
    }
    self.current_turn += 1;
}

fn main() {
    let (width, height) = (1280.0, 960.0);
    let mut window: PistonWindow = WindowSettings::new("particles", [width, height])
        .exit_on_esc(true)
        .build()
        .expect("Could not create a window.");

    let mut world = World::new(width, height);
    world.add_shapes(1000);

    while let Some(event) = window.next() {
        world.update();

        window.draw_2d(&event, |ctx, renderer, _device| {
            clear([0.15, 0.17, 0.17, 0.9], renderer);

            for s in &mut world.particles {

```

```

        let size = [s.position[0], s.position[1], s.width, s.height];
        rectangle(s.color, size, ctx.transform, renderer);
    }
    });
}
}

```

- ① `graphics::math::Vec2d` provides mathematical operations and conversion functionality for 2D vectors
- ② `piston_window` provides the tools to create a GUI program and draw shapes to on that window.
- ③ `rand` provides random-number generators and related functionality.
- ④ `std::alloc` provides facilities for controlling memory allocation.
- ⑤ `std::time` provides access to the system's clock.
- ⑥ The `#[global_allocator]` attribute marks the following value (ALLOCATOR) as satisfying the `GlobalAlloc` trait.
- ⑦ `ReportingAllocator` prints the time taken for each allocation to `STDOUT` as the program runs. This provides a fairly accurate indication of the time taken for dynamic memory allocation.
- ⑧ Defer the actual memory allocation to the system's default memory allocator.
- ⑨ `world` contains data that is useful for the lifetime of the program.
- ⑩ `Particle` is an object in 2D space.
- ⑪ Start at a random position along the bottom of the window.
- ⑫ Rise vertically over time.
- ⑬ Increase the speed of the rise over time.
- ⑭ The `into()` method converts the arrays of type `[f64; 2]` into `Vec2d`
- ⑮ Use a fully saturated white that is has a tiny amount of transparency.
- ⑯ Move the particle to its next position.
- ⑰ Slow down the particle's rate of increase as it travels across the screen.
- ⑱ Slowly make the particle more transparent over time.
- ⑲ Using `Box<Particle>`, rather than simply `Particle`, incurs an extra memory allocation when every particle is created.
- ⑳ Create a `Particle` as a local variable on the stack

Take ownership of `particle`, moving its data to the heap and create a

reference to that data on the stack

Push the reference into `self.shapes`.

For `n` iterations, remove the first particle that is invisible. If there are no invisible particles, then remove the oldest.

Return a random integer between -3 and 3, inclusive.

[Listing 6.15](#) is a fairly long code example, but hopefully it does not contain any code that's too alien from what you've already seen. Towards the end at line 142, the code example introduces Rust's closure syntax. If you look at the call to `window.draw_2d()`, it has a second argument with vertical bars surrounding two variable names (`|ctx, renderer, _device| { ... }`). Those vertical bars provide space for the closure's arguments and the curly braces are its body.

A closure is a function that is defined in-line and can access variables from its surrounding scope. They're often called a *anonymous* or *lambda* functions.

Closures are a common feature within idiomatic Rust code, but this book tends to avoid them where possible to keep examples approachable to programmers from an imperative or object-oriented background. They are explained fully in chapter 11. In the interim, it's sufficient to say that they are a convenient shorthand for defining functions. Let's focus on generating some evidence that allocating variables on the heap (many millions of times) can have a performance impact on your code.

6.3.4 Analyzing the impact of dynamic memory allocation

If you run [Listing 6.15](#) from a terminal window, you'll soon see two columns of numbers filling it up. These columns represent the number of bytes allocated and the duration in nanoseconds taken to fulfil the request. That output can be sent to a file for further analysis, as shown by the steps in [Listing 6.16](#).

Listing 6.16. Creating a report of memory allocations by redirecting `STDERR` from `ch6-particles` to a file

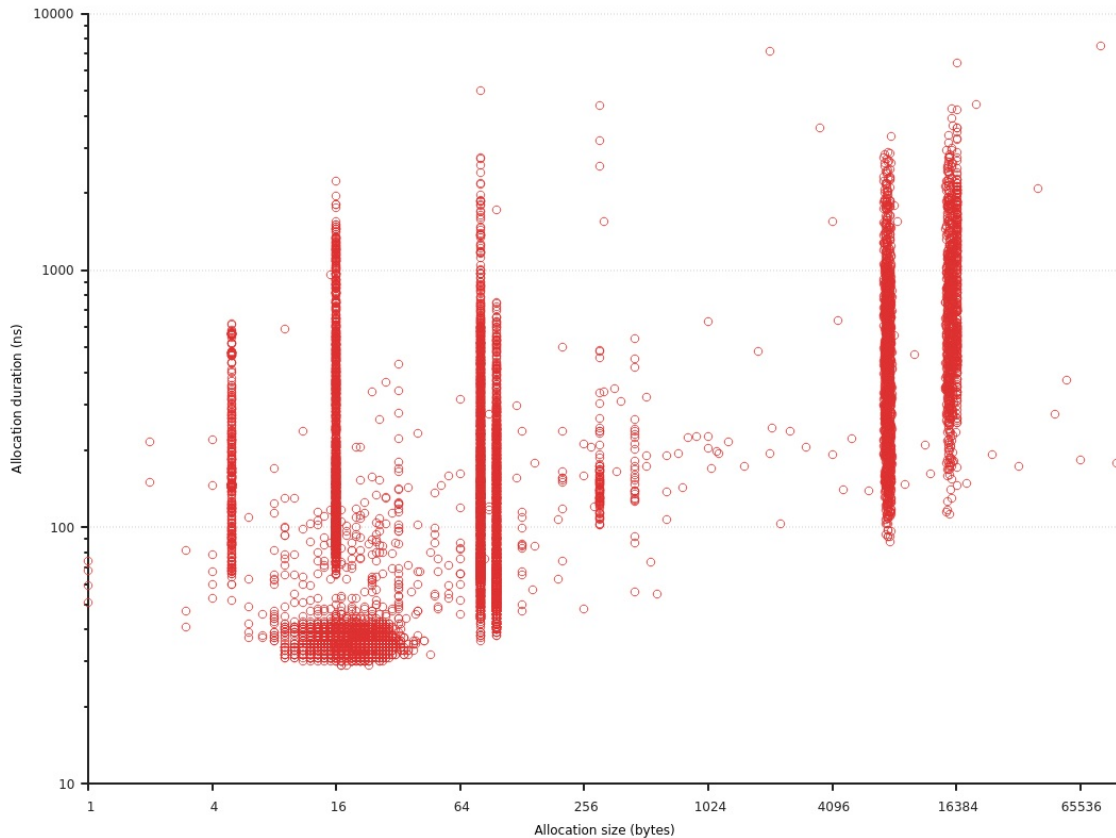
```
$ cd ch6-particles
$ cargo run -q 2> alloc.tsv ①
$ head alloc.tsv ②
4      219
5      83
48     87
9      78
9      93
19     69
15     960
16     40
14     70
16     53
```

① Run `ch6-particles` in "quiet" mode

② View the first 10 lines of output

One interesting aspect from this short extract is that memory allocation speed is not correlated with allocation size. When every heap allocation is plotted, this becomes even clearer:

Figure 6.7. Plotting heap allocation times against allocation size shows that there is no clear relationship between the two. The time taken to allocate memory is essentially unpredictable, even when requesting the same amount of memory multiple times.



To generate your own version of [Figure 6.7](#), the following gnuplot script can be tweaked as desired.

Listing 6.17. Script used to generate [Figure 6.7](#) with gnuplot (ch6/alloc.plot)

```
set key off
set rmargin 5
set grid ytics noxtics nocbtics back
set border 3 back lw 2 lc rgbcolor "#222222"

set xlabel "Allocation size (bytes)"
set logscale x 2
set xtics nomirror out
set xrange [0 to 100000]

set ylabel "Allocation duration (ns)"
set logscale y
set yrange [10 to 10000]
set ytics nomirror out

plot "alloc.tsv" with points \
    pointtype 6 \
    pointsize 1.25 \
    linecolor rgbcolor "#22dd3131"
```

Although larger memory allocations do tend to take longer than shorter ones, it's not guaranteed. The range of durations for allocating memory of the same number is over an order of magnitude. It might take 100 nanoseconds, it might take 1,000.

Does it matter? Probably not. But it might. If you have a 3 GHz CPU, then your processor is capable of performing 3 billion operation per second. If there is a 100 nanosecond delay between each of those operations, your computer can only perform 30 million operations in the same timeframe. Perhaps those hundreds of microseconds really do count for your application.

Some general strategies for minimizing heap allocations include:

- Using arrays of uninitialized objects. Instead of creating objects from scratch as required, create a bulk lot of them with zeroed values. When the time comes to activate one of those objects, set its values to non-zero. This can be a very dangerous strategy, as you're circumventing Rust's lifetime checks.
- Using an allocator that is tuned for your application's access memory profile. Memory allocators are often sensitive to the sizes where they perform best at.
- Investigate `arena::Arena` and `arena::TypedArena`. They allow objects to be created on-the-fly, but `alloc()` and `free()` are only called when the arena is created and destroyed.

[\[39\]](#) To be precise, the activation frame is called a stack frame when allocated on the stack.

6.4 VIRTUAL MEMORY

This section explains what the term *virtual memory* means and why it exists. You will be able to use this knowledge to speed up your programs, by building software that “goes with the grain”. CPUs can compute faster when they're able to access memory quickly. Understanding some of the dynamics of the computer

architecture can help to provide CPUs with memory efficiently.

6.4.1 Background

I have spent far too much of my life playing computer games. As enjoyable and challenging as I've found them, I've often wondered about whether I would have been better off spending my teenage years doing something more productive. Still, it's left me with plenty of memories. Some of those memories still leave a bitter taste. Occasionally, someone would enter the game and obliterate everyone with near perfect aim and seemingly impossibly high health ratings. Other players would decry "cheater!" but were more-or-less helpless in defeat. While waiting in in-game purgatory, I would sit wondering: "How is that possible? How are those tweaks to the game actually made?"

By working through this section's examples, you would have built the core of a tool that's capable of inspecting and modifying values of a running program.

Glossary of terms related to virtual memory

Terminology within this area is particularly arcane. It is often tied to decisions made many decades ago when the earliest computers were being designed. Here is a quick reference of some of the most important terms:

- Page - Fixed-size block of *words* of *real memory*. Typically 4kb in size for 64-bit operating systems.
- Word - Any type that is size of a pointer. This corresponds to the width of the CPU's registers. In Rust, `usize` and `isize` are word-length types.
- Page fault - An error raised by the CPU when a valid memory address is requested that is not currently in physical RAM. Signals to the operating system that at least one page must be swapped back in to memory.
- Swapping - Migrating a page of memory stored temporarily on disk from main memory upon request.
- Virtual memory - The program's view of its memory. All data accessible to a program is provided in its address space by the operating system.
- Real memory - The operating system's view of the physical memory available on the system. In many technical texts, real memory is defined independently from physical memory, which becomes much more of an electrical engineering term.
- Page table - The data structure maintained by the operating system to manage translating from virtual to real memory.

- Segment - A segment is a block within virtual memory. Virtual memory is divided into blocks to minimize the space required to translate between virtual and physical addresses.
- Segmentation fault - An error raised by the CPU when an illegal memory address is requested.

MMU: A component of the CPU that manages memory address translation. Maintains a cache of recently-translated addresses called the TLB, which stands for the translation lookaside buffer although that terminology has fallen from fashion.



Note

One term that has not been defined in any technical sense so far in this book is “process”. If you’ve encountered it before and have been wondering why it has been omitted, it will be introduced properly when concurrency is introduced. For now, consider the terms “process” and its peer “operating system process” to refer to a running program.

6.4.2 Step 1: Having a Process Scan Its Own Memory

Intuitively, a program’s memory is a series of bytes that starts at location 0 and ends at location n . If a program reports 100kB of RAM usage, it would seem that n would be somewhere near 100,000. Let’s test that intuition.

We’ll create a small command-line program that looks through memory, starting at location 0 and ending at 10,000. As it’s a small program, it shouldn’t occupy more than 10,000 bytes.

When executed, the program will not perform as intended. Sadly, it will crash. You’ll learn why the crash occurs as you follow through this section.

Listing 6.18. Attempting to scan through a running program’s memory byte by byte, starting at zero. This listing introduces the syntax for creating *raw pointers* and *dereferencing* (reading) them. (ch6/ch6-memscan-1/src/main.rs)

```
fn main() {  
    let mut n_nonzero = 0;
```

```

for i in 0..10000 {
    let ptr = i as *const u8; ①
    let byte_at_addr = unsafe { *ptr }; ②

    if byte_at_addr != 0 {
        n_nonzero += 1;
    }
}

println!("non-zero bytes in memory: {}", n_nonzero);
}

```

① Convert `i` to a `*const T`, a “raw pointer” of type `u8`. Raw pointers allow programmers to inspect raw memory addresses. We treat every address as a unit, ignoring the fact that most values span multiple bytes.

② *Dereference* the pointer. That is, read the value at address `i`. Another way of saying this is read the value being pointed to.

[Listing 6.18](#) crashes because it is attempting to dereference a NULL pointer. When `i` equals 0, `ptr` can’t really be dereferenced. Incidentally, this is why all raw pointer dereferences must occur within an `unsafe` block. How about we attempt to start from a non-zero memory address? Given that the program is executable code, there should be several thousand bytes of non-zero data to iterate through at least.

Listing 6.19. Scanning a process’s memory, starting from 1 to avoid dereferencing a NULL pointer

```

fn main() {
    let mut n_nonzero = 0;

    for i in 1..10000 { ①
        let ptr = i as *const u8;
        let byte_at_addr = unsafe { *ptr };

        if byte_at_addr != 0 {
            n_nonzero += 1;
        }
    }

    println!("non-zero bytes in memory: {}", n_nonzero);
}

```

① Start at 1 rather than 0 to avoid NULL pointer exception

This unfortunately does not solve the issue completely. [Listing 6.19](#) still crashes upon execution and the number of non-zero bytes is never printed to the console. This is due to what's known as a *segmentation fault*.

Segmentation faults are generated when the CPU and operating system detect that your program is attempting to access memory regions that it isn't entitled to. Memory regions are divided into segments. That explains the name.

Let's try a different approach. Rather than attempting to scan through bytes, let's look for the addresses of things that we know exist. We've spend lots of time learning about pointers, let's put that to use. [Listing 6.21](#) creates several values, examining their addresses.

Every run of [Listing 6.21](#) will (probably) generate unique values. Here is the output of one run:

Listing 6.20. Output from [Listing 6.21](#)

```
GLOBAL:    0x7ff6d6ec9310
local_str: 0x7ff6d6ec9314
local_int: 0x23d492f91c
boxed_int: 0x18361b78320
boxed_str: 0x18361b78070
fn_int:    0x23d492f8ec
```

As you can see, values appear to be scattered across a very wide range. So despite your program (hopefully) only needing a few kilobytes of RAM, a few variables live in giant locations. These are *virtual addresses*. As explained in the heap vs stack section, the stack starts at the top of the address space and the heap near the bottom. In this run, the highest value is 0x7ff6d6ec9314. That's approximately $2^{64} \div 2$. That number is due to the operating system reserving half of the address space for itself.

Listing 6.21. Printing out the address of several variables within a program to examine its address space (ch6/ch6-memscan-3/src/main.rs)

```
static GLOBAL: i32 = 1000; ①
```

```

fn noop() -> *const i32 {
    let noop_local = 12345; ②
    &noop_local as *const i32 ③
}

fn main() {
    let local_str = "a";      ④
    let local_int = 123;      ④
    let boxed_str = Box::new('b'); ④
    let boxed_int = Box::new(789); ④
    let fn_int = noop();      ④

    println!("GLOBAL:    {:p}", &GLOBAL as *const i32); ⑤
    println!("local_str: {:p}", local_str as *const str); ⑤
    println!("local_int: {:p}", &local_int as *const i32); ⑤
    println!("boxed_int: {:p}", Box::into_raw(boxed_int)); ⑤
    println!("boxed_str: {:p}", Box::into_raw(boxed_str)); ⑤
    println!("fn_int:    {:p}", fn_int); ⑤
}

```

- ① Create a global static, which is a global variable in Rust programs
- ② Create a local variable within `noop()` so that something is from outside of `main()` has a memory address
- ③ Return the address of `noop_local` as a raw pointer
- ④ Create various values of several types, including values on the heap.
- ⑤ Print out the values' addresses

By now, you'll be pretty good at being able to access addresses of stored values. There are actually two small lessons that you also picked up on:

- Some memory addresses are illegal. The operating system will shut your program down if it attempts to access memory that is out of bounds.
- Memory addresses are not arbitrary. Although they seem to be spread quite far apart within the address space, values are clustered together within pockets.

Before pressing on with the cheat program, let's step back and look at the system that's operating behind the scenes to translate these virtual addresses to real memory.

6.4.3 Translating virtual addresses to physical addresses

Accessing data in a program requires virtual addresses—the only addresses that the program itself has access to—gets translated into physical addresses. This process involves a dance between the program, the operating system, the CPU, the RAM hardware and occasionally hard drives and other devices.

The CPU is responsible for performing this translation, but the operating system stores the instructions. CPUs contain a *memory management unit* (MMU) that is designed for this one job. For every running program, every virtual address is mapped to a physical address. Those instructions are stored in memory too (at a pre-defined address). That means, in the worst case, every attempt at accessing memory addresses incurs two memory lookups.

The worst case is possible to avoid. The CPU maintains a cache of recently translated addresses. That is, the CPU has its own (fast) memory to speed up accessing memory. For historic reasons, this cache is known as the *translation lookaside buffer*, often abbreviated as TLB. For programmers optimizing for performance, keep your data structures lean and avoid deeply nested structures. Reaching the capacity of the TLB (typically around 100 pages for x86 processors) can be costly.

Looking into how the translation system operates reveals more, often quite complex, details. Virtual addresses are grouped into blocks called *pages*, which are typically 4kb in size. This practice avoids needing to store a translation mapping for every single variable in every single program. Having a uniform size for each page also assists to avoid a phenomenon known as *memory fragmentation*, where pockets of empty, yet unusable, space appear within available RAM.



Note

This is a general guide only. The details of how the OS and CPU cooperate to manage memory differs significantly in some

environments. In particular, constrained environments such as microcontrollers may use real addressing. For those interested in learning more, the research field is known as *computer architecture*.

The operating system and CPU can play some interesting tricks when data lives within pages of virtual memory:

- Having a virtual address space allows the operating system to overallocate. Programs that ask for more memory than the machine can physically provide are able to be accommodated.
- Inactive pages of memory can be swapped to disk in a byte-for-byte manner until its requested by the active program. Swapping is often used in periods of high contention for memory, but can be used more generally depending on an operating system's whims.
- Other size optimizations can be performed, such as compression. A program will see its memory intact. Behind the scenes, the operating system has compressed the program's wasteful data usage.
- Programs are able to share data quickly. If your program requests a large block of zeroes, say for a newly-created array, the operating system might point you towards a page filled with zeroes that is currently being used by three other programs. None of the programs are aware that the others are looking at the same physical memory and the zeroes have different positions within their virtual address space.
- Paging can speed up the loading of shared libraries. As a special case of the previous bullet point, if a shared library has already been loaded by another program, the operating system can avoid loading it into memory twice by pointing the new program to the old data.
- Paging adds security between programs. As you discovered earlier in the section, some parts of the address space are illegal to access. The operating system has other attributes that it can add. If an attempt is made to write to a read only page, the program will be terminated by the operating system.

Making effective use of the virtual memory system in day-to-day programs

requires thinking about how data is represented in RAM. Here are some guidelines:

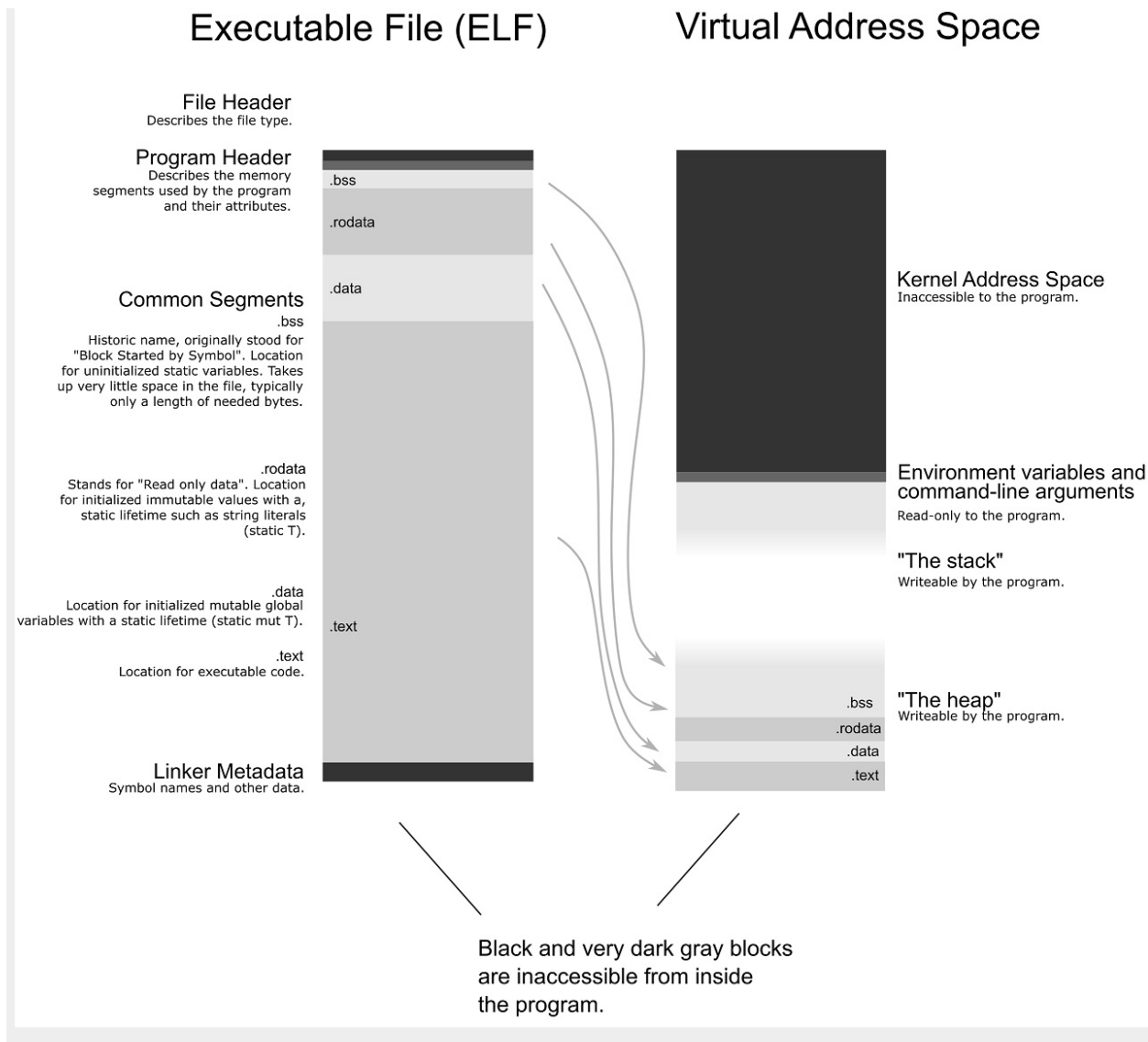
- Keep hot working portions of your program within 4kb of size. This will maintain very fast lookups.
- If 4kb is unreasonable for your application, then the next target to keep under is $4\text{kb} \times 100$. That rough guide should mean that the CPU can maintain its translation cache (the TLB) in good order to support your program.
- Avoid deeply nested data structures with pointer spaghetti. If a pointer points to another page, then performance will suffer.
- Test the ordering of your nested loops. CPUs read small blocks of bytes from the RAM hardware, known as a *cache line*. When processing an array, you can take advantage of this by investigating whether you are doing column-wise operations or row-wise operations.

One thing to note: virtualization makes this situation worse. If you're running an app inside a virtual machine, the hypervisor must also translate addresses for its guest operating systems. This is why many CPUs ship with “virtualization support”, which can reduce this extra overhead. Running containers within virtual machines adds another layer of indirection and therefore latency. For bare metal performance, run apps on bare metal.

How does an executable file turn into a program's virtual address space?

The layout of executable files (aka binaries) have many similarities to the address space diagram that we saw earlier in the heap vs stack section of the chapter.

While the exact process is dependent on the operating system and file format, here is a representative example. Each of the segments of the address space that we have been discussing are described by binary files. When the executable is started, the operating system loads the right bytes into the right places. Once the virtual address space is created, the CPU can be told to jump to the start of the .text segment and the program begins executing.



6.4.4 Step 2: Working with the operating system to scan an address space

Our task is to scan our program's memory while it's running. As we've discovered, the operating system maintains the instructions for mapping between a virtual address and a physical address. Perhaps we can ask the operating system to tell us what is happening?

Operating systems provide an interface for programs to be able to make requests, known as *system calls*. Within Windows, the `KERNEL.DLL` provides the

necessary functionality to be able to inspect and manipulate memory of a running process.



Note

Why Windows? Well, many Rust programmers use MS Windows as a platform. Also, its functions are quite well named and don't require as much prior knowledge as the POSIX API.

When you run [Listing 6.24](#), you should see lots of output with many sections similar to the following:

Listing 6.22. Output from [Listing 6.24](#)

```
MEMORY_BASIC_INFORMATION {           ①
  BaseAddress: 0x00007ffbe8d9b000,
  AllocationBase: 0x0000000000000000,
  AllocationProtect: 0,                ②
  RegionSize: 17568124928,
  State: 65536,                        ②
  Protect: 1,                          ②
  Type: 0                              ②
}
MEMORY_BASIC_INFORMATION {
  BaseAddress: 0x00007fffffff0000,
  AllocationBase: 0x00007fffffff0000,
  AllocationProtect: 2,
  RegionSize: 65536,
  State: 8192,
  Protect: 1,
  Type: 131072
```

- ① MEMORY_BASIC_INFORMATION is a struct defined within the Windows API.
- ② These fields are the integer representations of enums defined within the Windows API. It's possible to decode these to the enum variant names, but this isn't available without adding extra code to the listing.

Listing 6.23. Dependencies for [Listing 6.24](#) (ch6/ch6-meminfo-win/Cargo.toml)

```
[package]
name = "meminfo"
version = "0.1.0"
authors = ["Tim McNamara <author@rustinaction.com>"]
edition = "2018"
```

```
[dependencies]
winapi = "0.2" ①
kernel32-sys = "0.2" ②
```

① Defines some useful type aliases

② Provides interaction with KERNEL.DLL from the Windows API

Listing 6.24. Inspecting a program's memory the via the Windows API (ch6/ch6-meminfo-win/src/main.rs)

```
use kernel32;
use winapi;

use winapi::{
    DWORD, ①
    HANDLE, ②
    LPVOID, ②
    PVOID, ③
    SIZE_T, ④
    LPSYSTEM_INFO, ⑤
    SYSTEM_INFO, ⑥
    MEMORY_BASIC_INFORMATION, ⑥
};

fn main() {
    let this_pid: DWORD; ⑦
    let this_proc: HANDLE; ⑦
    let min_app_addr: LPVOID; ⑦
    let max_app_addr: LPVOID; ⑦
    let mut base_addr: PVOID; ⑦
    let mut proc_info: SYSTEM_INFO; ⑦
    let mut mem_info: MEMORY_BASIC_INFORMATION; ⑦

    const MEMINFO_SIZE: usize = std::mem::size_of::<MEMORY_BASIC_INFORMATION>();

    unsafe { ⑧
        base_addr = std::mem::zeroed();
        proc_info = std::mem::zeroed();
        mem_info = std::mem::zeroed();
    }

    unsafe { ⑨
        this_pid = kernel32::GetCurrentProcessId();
        this_proc = kernel32::GetCurrentProcess();
        kernel32::GetSystemInfo(&mut proc_info as LPSYSTEM_INFO); ⑩
    };

    min_app_addr = proc_info.lpMinimumApplicationAddress; ⑪
    max_app_addr = proc_info.lpMaximumApplicationAddress; ⑪

    println!("{:?}", this_pid, this_proc);
    println!("{:?}", proc_info);
    println!("min: {:p}, max: {:p}", min_app_addr, max_app_addr);
```

```

loop {
    let rc: SIZE_T = unsafe {
        kernel32::VirtualQueryEx(
            this_proc, base_addr,
            &mut mem_info, MEMINFO_SIZE as SIZE_T)
    };

    if rc == 0 {
        break
    }

    println!("{::#?}", mem_info);
    base_addr = ((base_addr as u64) + mem_info.RegionSize) as PVOID;
}
}

```

- ① In Rust, this would be known as a u32
- ② Pointer types for various internal APIs without an associated type. In Rust, “void pointers” are defined in `std::os::raw::c_void`. A `HANDLE` is a pointer to some opaque resource within Windows.
- ③ In Windows, data type names are often prefixed with a shorthand for their type. `P` stands for pointer. `LP` stands for long pointer, e.g. 64 bit.
- ④ `u64` (usize on this machine)
- ⑤ A pointer to a `SYSTEM_INFO` struct
- ⑥ Some structs defined by Windows internally
- ⑦ These variables will be initialized from within `unsafe` blocks. To make them accessible in the outer scope, they need to be defined here.
- ⑧ This block guarantees that all memory is initialized
- ⑨ This block of code is where system calls are made
- ⑩ Rather than use a return value, this function makes use of a C idiom to provide its result to the caller. We provide a pointer to some pre-defined struct, then read that struct’s new values once the function has returned to see the results.
- ⑪ Renaming these variables for convenience.
- ⑫ This loop does the work of scanning through the address space
- ⑬ This system call provides information about a specific segment of the running program’s memory address space, starting at `base_addr`.

Finally we have been able to explore an address space without the operating

system killing our program. Now the question remains, how do we inspect individual variables and modify them?

6.4.5 Step 3: reading and writing bytes to processes' memory

Operating systems provide tools to read and write memory, even in other programs. This is essential for Just-In-Time compilers (JITs), debuggers and programs to help people cheat at games to work.

On Windows, the general process looks something like this in Rust-like pseudo-code:

```
let pid = some_process_id;
OpenProcess(pid);

loop address space {
    call VirtualQueryEx() to access the next memory segment
    scan the segment by calling ReadProcessMemory(), looking for a selected pattern
    call WriteProcessMemory() with the desired value
}
```

Linux provides an even simpler API via `process_vm_readv()` and `process_vm_writev()`. They're analogous to `ReadProcessMemory()` and `WriteProcessMemory()` in Windows.

6.5 WRAP UP

Memory management is a complicated area with many levels of abstraction to uncover. This chapter has tried to focus on those elements that are most salient to your work as a programmer. Now when you read your next blog post on some low-level coding technique, you should be much more able to follow along with the terminology.

You have learned:

- What a pointer is and what distinguishes pointers from memory addresses

- What a smart pointer is and that the term has a wider meaning in the Rust community than it does in the C++ community
- That there is a family of smart pointer types that can be used to suit your application's requirements
- That the standard library's smart pointer types are built from building blocks that you can also use to define your own smart pointers if required
- What the heap and stack are
- Why using the stack is faster
- How the memory allocation process operates
- How an operating system, the CPU and the program cooperate to fetch and modify your program's data
- How your program can customize its performance via altering its allocation strategy
- How to trace a program's memory allocations
- What a syscall is and how to make one on Linux and Windows

7 FILES & STORAGE

In this chapter:

- Learn how data is represented on physical storage devices
- Write your own data structures to your preferred file format
- Build a tool to read from files and inspect their contents
- Create a working key-value store that's immune from becoming corrupt

Storing data permanently on digital media is trickier than it looks. This chapter takes you through some of that detail. To transfer information held by ephemeral electrical charges in RAM to (semi-)permanent storage media—and then to be able to retrieve it again later—takes several layers of software indirection.

The chapter introduces some new concepts for Rust developers, such as how to structure projects into library crates. This is needed because one of the projects is ambitious. By the end of the chapter, you would have built a working key-value store that's guaranteed to be durable to hardware failure at any stage.

During the chapter, we'll work through a small number of side quests. For example, we implement parity bit checking and explore what it means "to hash" a value. To start with however, let's see if we can create patterns from the raw byte sequence within files.

7.1 WHAT IS A FILE FORMAT?

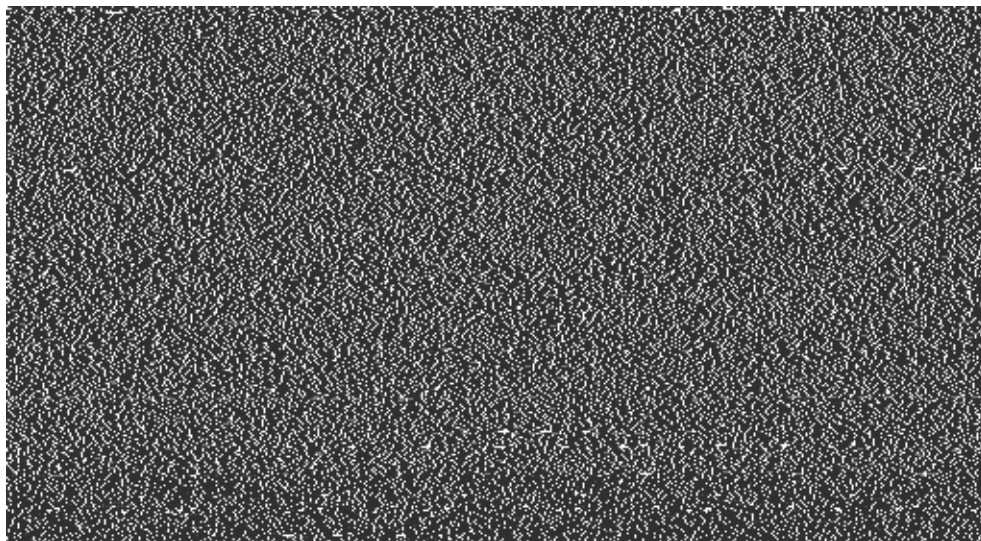
File formats are standards for working with data as an single, ordered sequence of bytes. Storage media, such as hard disk drives, work fastest when reading or writing large blocks of data in serial. This contrasts with in-memory data structures, where data layout has less of an impact.

File formats live in a large design space with trade offs in performance, human-readability and portability. Some formats are highly portable and self-describing. Others restrict themselves to being accessible within a single environment, are unable to be read by third party tools, yet are very high performance.

[Table 7.1](#) illustrates some of the design space. Each row reveals the file format's internal patterns, which are generated from the same source text. By color-coding each byte within the file, it's possible to see structural differences between each representation.

Table 7.1. Internals of four digital versions of William Shakespeare's *Much Ado About Nothing* produced by Project Gutenberg.

Figure 7.1. Plain Text



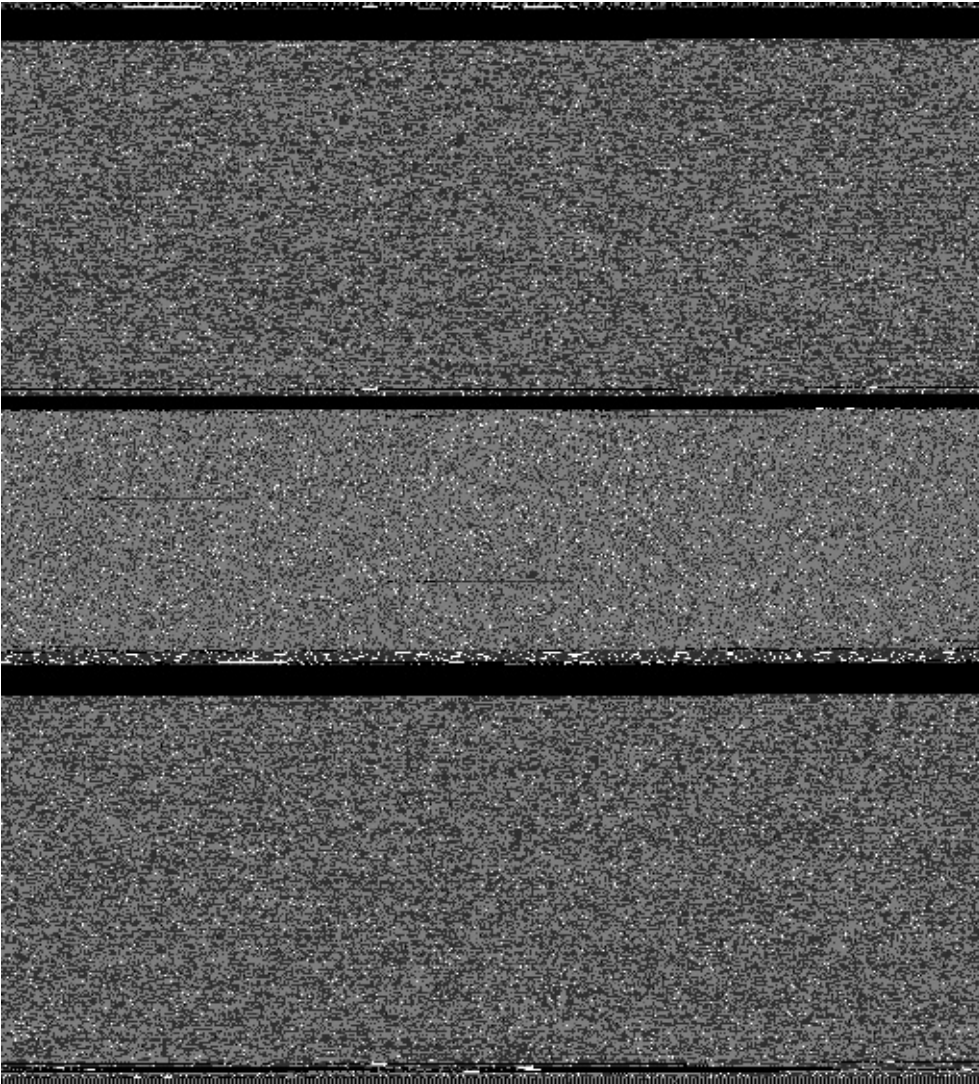
The plain text version of the play contains printable characters only. They are indicated by dark grey for letters and punctuation, and white for whitespace. Visually, the image appears to be very noisy. It lacks internal structure. That's due to the variation in length of the natural language that the file is representing. A file with regular, repeating structures, such as a file format designed to hold arrays of floating point numbers, tends to look quite different.

Figure 7.2. EPUB



The EPUB format is actually a compressed ZIP archive with a bespoke file extension. There are many bytes within the file that fall out of the range of “printable” category , as indicated by the mid-grey pixels.

Figure 7.3. MOBI



Includes four bands of NUL bytes (0x00), represented as black pixels.

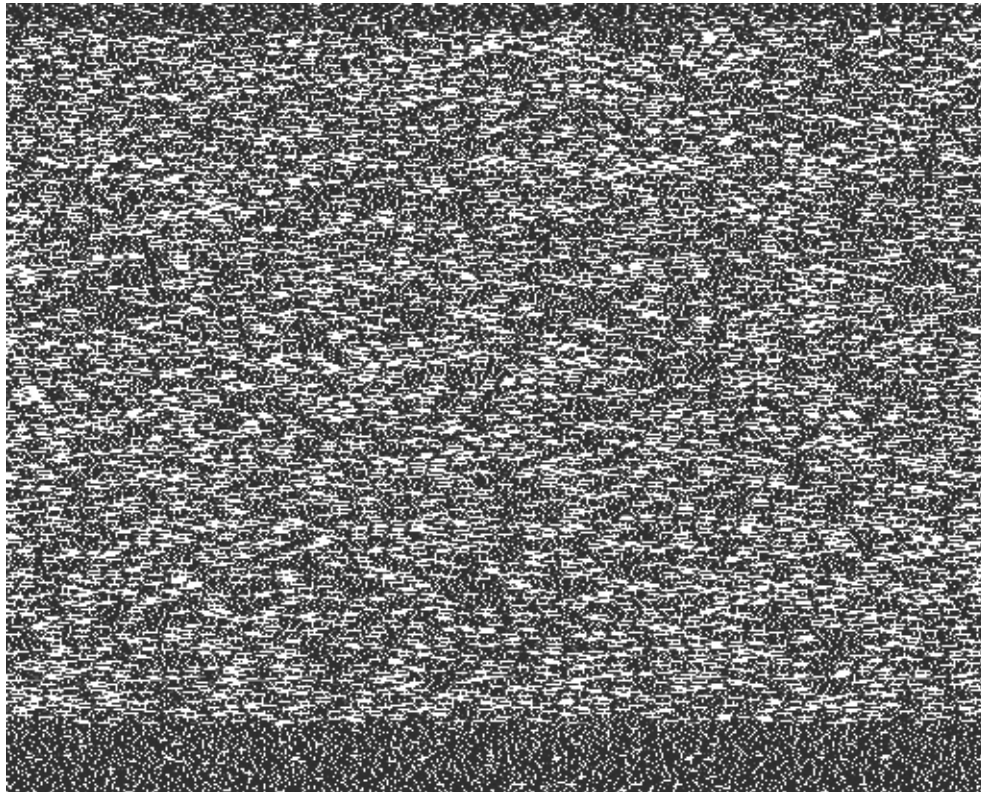
These bands probably represent the result of an engineering trade off. In some sense, these empty bytes are wasted space. But they’re probably added as padding so that the file’s sections are very easy to parse

later on.

//// None of the three major sections look like they are human readable. As the intended deployment platform is the Amazon Kindle device, that's probably intended. ////

The other feature of this file is its size. It's larger than the other versions of the play. That might imply that the file is harboring more data than just the text. Candidates include display elements like fonts through to encryption keys to enforce anti-copying restrictions within the file.

Figure 7.4. HTML



The HTML file contains a very high proportion of whitespace characters. They are indicated by white pixels. Markup languages like HTML tend to add whitespace to aid readability.

7.2 CREATING YOUR OWN FILE FORMATS FOR DATA STORAGE WITH SERDE

When working with data that needs to be stored over a long time, the proper thing to do is to use a battle-tested database. Despite this, many systems use plain text files for data storage. Configuration files, for example, are commonly designed to be both human-readable and machine-readable.

The Rust ecosystem has excellent support for converting data to many on-disk formats.

7.2.1 Writing data to disk with `serde` & the `bincode` format

`serde` is a crate that *ser*ializes and *de*serializes Rust values to and from many formats. Each format has its own strengths, many are human-readable, while others prefer to be compact so that they can be speedily sent across the network.

Using `serde` takes surprisingly little ceremony. As an example, let's use statistics about the Nigerian city of Calabar and store them in multiple output formats.

To start, let's assume that our code contains a `City` struct:

```
#[derive(Serialize, Deserialize)] ①
struct City {
    name: String,
    population: usize,
    latitude: f64,
    longitude: f64,
}
```

① The `Serialize` and `Deserialize` traits are provided by the `serde` crate. Most code implements them with this `derive` annotation. They provide the tooling to enable many external formats to interact with Rust code.

Populating that struct with data about Calabar is straightforward:

```
let calabar = City {
    name: String::from("Calabar"),
    population: 470_000,
    latitude: 4.95,
    longitude: 8.33,
};
```

Now to convert that `calabar` variable into JSON, it's one line of code:

```
let as_json = serde_json::to_string(&calabar).unwrap();
```

serde speaks much more than JSON though. The code at [Listing 7.3](#) also provides similar examples for two lesser-known formats, CBOR and bincode. CBOR and bincode are more compact than JSON, at the expense of being machine-readable only. Here is a view of that `calabar` variable in several variants:

Listing 7.1. Output of [Listing 7.3](#) demonstrating the different distributions of the bytes present in each format. JSON is readable as-is, but CBOR and bincode contain NUL bytes that require escaping.

[illegible]

To generate that output, compile the code at [Listing 7.3](#) using [Listing 7.2](#) as the crate’s TOML file.

Listing 7.2. Declaring the dependencies and setting other crate metadata for Listing 7.3 (ch7/ch7-serde-eg/Cargo.toml)

```
[package]
name = "ch7-serde-eg"
version = "0.1.0"
authors = ["Tim McNamara <author@rustinaction.com>"]
edition = "2018"

[dependencies]
bincode = "1"
serde = "1"
serde_cbor = "0.8"
serde_derive = "1"
serde_json = "1"
```

Listing 7.3. Converting a Rust struct to three different file formats

with serde (ch7/ch7-serde-eg/src/main.rs)

```
use serde_derive::{Serialize, Deserialize};

#[derive(Serialize, Deserialize)]           ①
struct City {
    name: String,
    population: usize,
    latitude: f64,
    longitude: f64,
}

fn main() {
    let calabar = City {
        name: String::from("Calabar"),
        population: 470_000,
        latitude: 4.95,
        longitude: 8.33,
    };

    let as_json = serde_json::to_string(&calabar).unwrap();    ②
    let as_cbor = serde_cbor::to_vec(&calabar).unwrap();       ②
    let as_bincode = bincode::serialize(&calabar).unwrap();    ②

    println!("json: {}", &as_json);
    println!("cbor: {:?}", &as_cbor);
    println!("cbor (as UTF-8): {:?}", String::from_utf8_lossy(&as_cbor));
    println!("bincode: {:?}", &as_bincode);
    println!("bincode (as UTF-8): {:?}", String::from_utf8_lossy(&as_bincode));
}
```

- ① The `#derive` attribute (`#[derive(Serialize, Deserialize)]`) is added to the `City` struct definition. This instructs the `serde_derive` crate to write the necessary code that will carry out the conversion between an in-memory `City` and an on-disk `City`.
- ② Unfortunately, formats can have differing APIs. Each crate is maintained independently from the `serde` core. In a sense, that's why a library like `serde` is so useful. It provides a common framework for implementing serializers and deserializers for any new format independently from one another.

7.3 IMPLEMENTING A HEXDUMP CLONE

A very handy utility for inspecting files' contents is `hexdump`. `hexdump` takes a stream of bytes, often from a file, then outputs those bytes in pairs of hexadecimal numbers. [Table 7.2](#) provides an example. As you'll know from previous chapters, two hexadecimal numbers can represent all digits from 0 to

255, which is the number of bit patterns representable within a single byte. We'll call our clone `fview`, short for file view.

Table 7.2. `fview` in operation

fview input	... fn main() { println!("Hello, world!"); } ...
fview output	... [0x00000000] 0a 66 6e 20 6d 61 69 6e 28 29 20 7b 0a 20 20 20 [0x00000010] 20 70 72 69 6e 74 6c 6e 21 28 22 48 65 6c 6c 6f [0x00000020] 2c 20 77 6f 72 6c 64 21 22 29 3b 0a 7d ...

Unless you're familiar with hexadecimal notation, the output from `fview` can be fairly opaque. If you're experienced at looking at similar output, you may notice that there are no bytes above `0x7e` (127). There are also very few bytes less than `0x21` (33), with the exception of `0x0a` (10). `0x0a` represents the newline character ('`\n`'). These byte patterns are markers that we're dealing with a plain text input source.

The source code that builds the complete `fview` is provided at [Listing 7.5](#). Because a few new features of Rust need to be introduced, we'll take a few steps to get to the full program. We'll start with [Listing 7.4](#) that uses a string literal as input and produces the output in [Table 7.2](#).

Listing 7.4. A hexdump clone with a hard-coded input that mocks file I/O. Demonstrates the use of multi-line string literals and importing the `std::io` traits via `std::io::prelude`. This enables `&[u8]` types to be read as files, via the `std::io::Read` trait. (ch7/ch7-fview-str/src/main.rs)

```
use std::io::prelude::*;                                ①

const BYTES_PER_LINE: usize = 16;
const INPUT: &'static [u8] = br#"                      ②
fn main() {
    println!("Hello, world!");
}";

fn main() -> std::io::Result<()> {
    let mut buffer: Vec<u8> = vec!();                    ③
    INPUT.read_to_end(&mut buffer)?;                     ④

    let mut position_in_input = 0;
    for line in buffer.chunks(BYTES_PER_LINE) {
        print!("[0x{:08x}] ", position_in_input);        ⑤
    }
}
```

```

        for byte in line {
            print!("{:02x} ", byte);
        }
        println!();
        position_in_input += BYTES_PER_LINE;
    }

    Ok(())
}

```

- ① The "prelude" imports traits that are heavily used in I/O operations, such as `Read` and `Write`. It's possible to include the traits manually, but they're so common that the standard library provides this convenience line to help keep your code compact.
- ② Multi-line string literals don't need their double-quotes escaped when built with *raw string literals*. Notice the `r` prefix and the `#` delimiters. The additional `b` prefix indicates that this should be treated as bytes (`&[u8]`), not as UTF-8 text (`&str`).
- ③ Make space for the program's input with an internal buffer.
- ④ "Read" our input and insert it into our internal buffer.
- ⑤ Write out the current position with up to 8 left-padded zeros.
- ⑥ Shortcut for printing a newline to stdout.

Now that we have seen the intended operation of `fview`, let's extend its capabilities to read real files.

Listing 7.5. A basic hexdump clone that demonstrates how to open a file in Rust and iterate through its contents (ch7/ch7-fview/src/main.rs)

```

use std::fs::File;
use std::io::prelude::*;
use std::env;

const BYTES_PER_LINE: usize = 16;

fn main() {
    let arg1 = env::args().nth(1);
    let fname = arg1.expect("usage: fview FILENAME");

    let mut f = File::open(&fname).expect("Unable to open file.");
    let mut pos = 0;
    let mut buffer = [0; BYTES_PER_LINE];
}

```

```

while let Ok(_) = f.read_exact(&mut buffer) {
    print!("[0x{:08x}] ", pos);
    for byte in &buffer {
        match *byte {
            0x00 => print!(". "),
            0xff => print!("# "),
            _ => print!("{:02x} ", byte),
        }
    }

    println("");
    pos += BYTES_PER_LINE;
}
}

```

- ① Import the standard library's `File` type into local scope
- ② The prelude imports lots of common functionality for convenience.
- ③ `std::env` provides fairly low-level access to the program's environment. We're not building a complicated app, so we don't need to worry about anything more fully-featured.
- ④ Changing this constant can change the effects that are generated by the program.
- ⑤ Rust iterators provide a `nth()` method that allows you to extract single values. Using `nth()` to extract an argument from `env::args()` means that the argument won't be validated, but that will suffice for now.
- ⑥ Using `nth()` does however incur some cost: it mandates that you consider an error case. It returns a `Result`. `expect()` prints the usage error message is printed to the screen when there are no command-line arguments provided or some other issue arises.
- ⑦ The calls to `expect()` should be considered a friendlier version of `unwrap()`. `unwrap()` panics abruptly on error. `expect()` panics with an error message.
- ⑧ Rather than calling `chunks()` on the input iterator, we explicitly provide a buffer to fill to `read_exact()`.

[Listing 7.5](#) introduces some new Rust:

- `while let Ok(_) { ... }` is a control flow structure. The program will continue to loop until `f.read_exact()` returns `Err`, which occurs when it has run out of bytes to read.
- `f.read_exact()` comes from the `Read` trait. The method transfers data from the source, in our case `f`, to the buffer provided as an

argument and stops when that buffer is full. It provides greater control to you as a programmer for managing memory, than the `chunks()` option used in [Listing 7.4](#), but does come with some quirks. If the buffer is longer than the number of available bytes to read, the file will return an error and the state of the buffer is undefined.`

7.4 FILE OPERATIONS IN RUST

So far in this chapter, we have invested a lot of time considering how data is translated into sequences of bytes. Let's spend some time considering another level of abstraction, the file.

Previous chapters have covered basic operations, such as opening and reading from a file. This section contains some other helpful techniques, which provide more granular control.

7.4.1 Opening a file in Rust and controlling its file mode

Files are an abstraction that's maintained by the operating system. It presents a façade of names and hierarchy above a nest of raw bytes.

Files also provide a layer of security. They have permissions attached to them that the operating system will enforce. This—in principle, at least—is what prevents a web server running under its own user account from reading files owned by others.

`std::fs::File` is the primary type for interacting with the file system. There are two methods available for creating a file, `open()` and `create()`. Use `open()` when you know the file already exists. [Table 7.3](#) explains more of their differences.

Table 7.3. Creating `File` values in Rust code and their effects on the underlying file system

Method	Return value when the file already exists	Effect on underlying file	Return value when no file exists
<code>File::open</code>	<code>Ok(File)*</code>	Opened in read-only mode, as is	<code>Err</code>
<code>File::create</code>	<code>Ok(File)*</code>	All existing bytes are truncated, and the file is opened at the beginning of the new file	<code>Ok(File)*</code>
* Assuming the user account has sufficient permission.			

When more control is required, `std::fs::OpenOptions` is available. It provides the necessary knobs to turn for any intended application. [Listing 7.18](#) provides a good example of a case where an “append” mode is requested. The application requires writeable file that is also readable and will be created if it doesn’t already exist.

Listing 7.6. Excerpt of [Listing 7.18](#) demonstrating the use of `std::fs::OpenOptions` to create a writeable file that will not be truncated when it’s opened

```
let f = OpenOptions::new()      ①
    .read(true)                ②
    .write(true)               ③
    .create(true)              ④
    .append(true)              ⑤
    .open(path)?;              ⑥
```

- ① An example of the "Builder" pattern. Each method returns a new instance of the `OpenOptions` struct with the relevant option set.
- ② Open the file for reading
- ③ Enable writing. This line is not strictly necessary, as it’s implied by `append`.
- ④ Create a file at `path` if it doesn’t already exist
- ⑤ Don’t delete any content that’s already been written to disk.
- ⑥ Open the file at `path` after unwrapping the intermediate `Result`

7.4.2 Interacting with the file system in a type-safe manner with `std::fs::Path`

Rust provides type-safe variant of `str` and `String` in its standard library: `std::path::Path` and `std::path::PathBuf`. They can be used to unambiguously work with path separators in a cross-platform way. `Path` can address files, directories and related abstractions, such as symbolic links.

Creating one is relatively straight-forward:

```
let hello = PathBuf::from("/tmp/hello.txt")
```

From there, interacting with them reveals methods that are specific to paths:

```
hello.extension() ①
```

[^①](#) Returns `Some("txt")`

The full API is straight-forward for anyone who has used code to manipulate paths before, so it won't be fleshed out here. Still, it may be worth discussing why it's included within the language as many languages omit this.



Tip

As an implementation detail, `std::fs::Path` and `std::fs::PathBuf` are implemented on top of `std::ffi::OsStr` and `std::ffi::OsString` respectively. This means that `Path` and `PathBuf` are *not* guaranteed to be UTF-8.

Why use `Path`, rather than manipulating strings directly?

- Clearer intent - `Path` provides useful methods, such as `set_extension()` that describe the intended outcome. This can assist programmers who later read the code. Manipulating strings in that manner doesn't provide that level of self-documentation.
- Portability - Some operating systems treat file system paths as case insensitive. Others don't. Using one operating system's conventions may result in issues later on when users end up expecting their host system's conventions will be followed. Additionally, path separators

are specific to operating systems and thus can differ. This means that using raw strings can lead to portability issues. Comparisons require exact matches.

- Easier debugging - If you're attempting to extract /tmp from the path /tmp/hello.txt, doing it manually can introduce subtle bugs that may only appear at runtime. Mis-counting the correct number index values after splitting the string on / introduces a bug that can't be caught at compile time.

To illustrate the subtle errors, consider the case of separators. Slashes are common in today's operating systems, but those conventions took some time to become established.

- \ is commonly used on MS Windows
- / is the convention for UNIX-like operating systems
- : was the path separator of the classic Mac OS
- > is used in the Stratus VOS operating system

Table 7.4. Comparing std::String and std::path::Path to extract a file's parent directory

<pre>fn main() { let hello = String::from("/tmp/hello.txt"); let tmp_dir = hello.split("/").nth(0); ① println!("{:?}", tmp_dir); ② }</pre> ① Split hello at its backslashes, then take the zeroith element of the resulting Vec<String> ② Mistake! Prints Some("").	<pre>use std::path::PathBuf; fn main() { let mut hello = PathBuf::from("/tmp/hello.txt"); hello.pop(); ① println!("{:?}", hello.display()); ② }</pre> ① Truncates hello in place ② Success! Prints "/tmp".
The plain String code enables you to use familiar methods, but can introduce subtle bugs that are difficult to detect at compile time. In this instance, we've used the wrong index number to access the	Using path::Path does not make your code immune to subtle errors, but can certainly help to minimize their likelihood. Path provides dedicated methods for common operations, such as setting a

parent directory (/tmp).

file's extension.

7.5 IMPLEMENTING A KEY-VALUE STORE WITH A LOG-STRUCTURED, APPEND-ONLY STORAGE ARCHITECTURE

It's time to tackle something larger. Let's begin to lift the lid on database technology.

Along the way, we'll learn the internal architecture of a family of database systems using the "log-structured, append-only" model. They're significant as case studies because they are both designed for extremely high resilience and high performance. Despite storing data on fickle media such as flash storage or a spinning hard disk drive, databases using this model are able to guarantee that a) data will never be lost and b) that their backing data files will never be corrupted.

7.5.1 The key-value model

`actionkv` is a *key-value store*. That store and retrieve sequences of bytes (`[u8]`) of arbitrary length. Each sequence has two parts. The first is a key, and the second is a value. As the `&str` type is represented as `[u8]` internally, [Table 7.5](#) uses the plain text notation, rather than the binary equivalent that's used prominently earlier in [Section 7.2.1](#).

Table 7.5. Illustrating keys and values by matching countries with their capital cities

Key	Value
"Cook Islands"	"Avarua"
"Fiji"	"Suva"
"Kiribati"	"South Tarawa"
"Niue"	"Alofi"

The key-value model enables simple queries such as "what is the capital city of Fiji?" but doesn't support asking broader queries, such as "what is the list of

capital cities for all Pacific Island states?".

7.5.2 Introducing `actionkv` v0.1: an in-memory key-value store with a command line interface

The first version of our key-value store, `actionkv` exposes us to the API that we'll use and introduces the main library code. The library code will not change as the subsequent two systems are built on top of it. Before we get to that code though, there are some pre-requisites that need to be covered.

Unlike other projects in this book, this one uses the “library” template to start with (`cargo new --lib actionkv`) It has the following structure:

```
actionkv
├── src
│   ├── akv_mem.rs
│   └── lib.rs
└── Cargo.toml
```

Using a library crate allows programmers to build re-usable abstractions within their projects. For our purposes, we'll be using the same `lib.rs` for multiple executables.

To avoid future ambiguity, we need to describe which executable binaries are produced by the project. To do so, provide a `bin` section within two square bracket pairs (`[[bin]]`) project's `Cargo.toml` file. See lines 14-16 of [Listing 7.7](#). Two square brackets indicate that the section can be repeated.

Listing 7.7. Defining dependencies and other metadata (ch7/ch7-actionkv1/Cargo.toml)

```
[package]
name = "actionkv"
version = "1.0.0"
authors = ["Tim McNamara <author@rustinaction.com>"]
edition = "2018"

[dependencies]
byteorder = "1.2"      ①
crc = "1.7"            ②
```

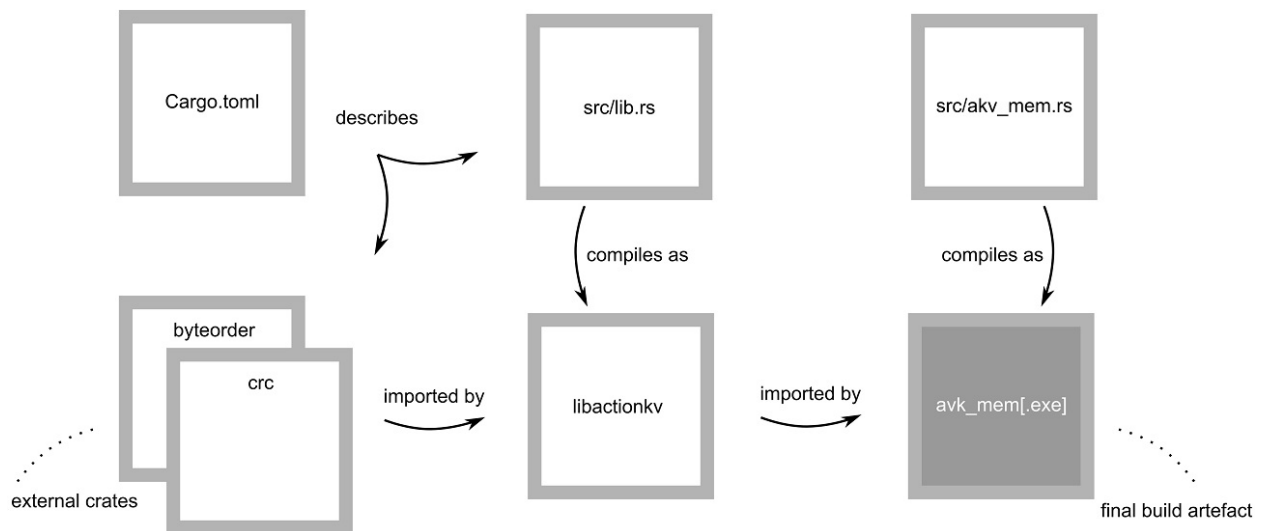
```
[lib]                                ③
name = "libactionkv"                ③
path = "src/lib.rs"                 ③

[[bin]]                              ④
name = "akv_mem"
path = "src/akv_mem.rs"
```

- ① The `byteorder` crate extends many Rust types with extra traits that enable them to be written to disk and read back into a program in a repeatable, easy-to-use way
- ② The `crc` crate provides the checksum functionality that we want to include
- ③ A `[lib]` section of `Cargo.toml` you to define your own name for the library that you're building. A crate may only have one library.
- ④ A `[[bin]]` section—of which there may be many—of `Cargo.toml` defines an executable file that will be built from this crate. The double-square bracket syntax (`[[ ]]`) is required because it unambiguously describes `bin` as being part of 1 or more elements.

The project will end up with several files. [Figure 7.5](#) illustrates the relationships and how they work together to build the `akv_mem` executable that's referred to within the section of the project's `Cargo.toml` file.

Figure 7.5. An outline of how the different files within the ActionKV project and their dependencies work together. The project's `cargo.toml` coordinates lots of activity that will ultimately result in an executable being built.



7.6 ACTIONKV V0.1 FRONT-END CODE

The public API of actionkv is comprised of four operations: “get”, “delete”, “insert” and “update”.

Table 7.6. Operations supported by actionkv

Command	Description
get <key>	retrieves the value at key from the store
insert <key> <value>	adds a key/value pair to the store
delete <key>	removes a key/value pair from the store
update <key> <value>	replace an old value with a new one

Naming is difficult

To access stored key-value pairs, should the API provide “get”, “retrieve” or perhaps “fetch”? Should setting values be “insert”, “store” or “set”? actionkv attempts to stay neutral to these decisions by deferring to the API provided by `std::collections::HashMap`.

We use Rust’s matching facilities to efficiently work with the command-line arguments and dispatch to the correct internal function:

Listing 7.8. Excerpt from [Listing 7.9](#) demonstrating the public API


```

match action {
    "get" => {
        match store.get(key).unwrap() {
            None => eprintln!("{:?} not found", key),
            Some(value) => println!("{:?}", value),
        }
    },
    "delete" => store.delete(key).unwrap(),
    "insert" => {
        let value = maybe_value.expect(&USAGE).as_ref();
        store.insert(key, value).unwrap()
    },
    "update" => {
        let value = maybe_value.expect(&USAGE).as_ref();
        store.update(key, value).unwrap()
    },
    _ => eprintln!("{}", &USAGE),
}

```

- ① action is a command-line argument, and has the type `&str`
- ② `println!` needs to use Debug syntax (`{:?}`) as `[u8]` is arbitrary bytes and therefore does not implement `Display`.
- ③ A future update that could be added for compatibility with Rust's `HashMap` would be for `insert` to return the old value, if it exists

In full, the code for `actionkv v0.1` is presented as [Listing 7.9](#). Notice that the heavy lifting of interacting with the file system is delegated to an instance of `ActionKV` called `store`. How `ActionKV` operates is explained in [Section 7.7](#).

Listing 7.9. In-memory key-value store command line application (ch7/ch7-actionkv1/src/akv_mem.rs)

```

use libactionkv::ActionKV;
#[cfg(target_os = "windows")]
const USAGE: &str = "
Usage:
    akv_mem.exe FILE get KEY
    akv_mem.exe FILE delete KEY
    akv_mem.exe FILE insert KEY VALUE
    akv_mem.exe FILE update KEY VALUE
";
#[cfg(not(target_os = "windows"))]
const USAGE: &str = "
Usage:
    akv_mem FILE get KEY
    akv_mem FILE delete KEY
    akv_mem FILE insert KEY VALUE
";

```

```

    akv_mem FILE update KEY VALUE
";

fn main() {
    let args: Vec<String> = std::env::args().collect();
    let fname = args.get(1).expect(&USAGE);
    let action = args.get(2).expect(&USAGE).as_ref();
    let key = args.get(3).expect(&USAGE).as_ref();
    let maybe_value = args.get(4);

    let path = std::path::Path::new(&fname);
    let mut store = ActionKV::open(path).expect("unable to open file");
    store.load().expect("unable to load data");

    match action {
        "get" => {
            match store.get(key).unwrap() {
                None => eprintln!("{:?} not found", key),
                Some(value) => println!("{:?}", value),
            }
        },
        "delete" => store.delete(key).unwrap(),
        "insert" => {
            let value = maybe_value.expect(&USAGE).as_ref();
            store.insert(key, value).unwrap()
        },
        "update" => {
            let value = maybe_value.expect(&USAGE).as_ref();
            store.update(key, value).unwrap()
        },
        _ => eprintln!("{}", &USAGE),
    }
}

```

- ① Although `src/lib.rs` exists within our project, it's treated the same as any other crate within our project's `src/bin.rs` file
- ② The `cfg` attribute allows MS Windows users will see the correct file extension in their help documentation. This attribute is explained at [Section 7.6.1](#).

7.6.1 Tailoring what is compiled with conditional compilation

Rust provides excellent facilities for altering what is compiled depending on the *compiler target architecture*. Generally, this is the target's operating system, but may be facilities provided by its CPU. Changing what is compiled depending on some compile-time condition is known as *conditional compilation*.

To add conditional compilation to your project, annotate your source code with `cfg` attributes. `cfg` works in conjunction with the `target` parameter provided to `rustc` during compilation.

[Listing 7.9](#) provides a usage string—common as quick documentation for command-line utilities—for multiple operating systems. It’s replicated here. When the project is built for Windows, the usage string contains a `.exe` file extension. The resulting binary files only include the data that is relevant for their target.

Listing 7.10. Excerpt from [Listing 7.9](#) demonstrating the use of conditional compilation to provide two definitions of `const USAGE` in the source code

```
#[cfg(target_os = "windows")]
const USAGE: &str = "
Usage:
    akv_mem.exe FILE get KEY
    akv_mem.exe FILE delete KEY
    akv_mem.exe FILE insert KEY VALUE
    akv_mem.exe FILE update KEY VALUE
";

#[cfg(not(target_os = "windows"))]
const USAGE: &str = "
Usage:
    akv_mem FILE get KEY
    akv_mem FILE delete KEY
    akv_mem FILE insert KEY VALUE
    akv_mem FILE update KEY VALUE
";
```

There is no negation operator for these matches. That is, `#[cfg(target_os != "windows")]` does not work. Instead, there is a function-like syntax for specifying matches. Use `#[cfg(not(...))]` for negation. `#[cfg(all(...))]` and `#[cfg(any(...))]` are also available to match on elements of a list.

The list of conditions that may trigger compilation changes is extensive:

Table 7.7. Options available to match against in `cfg` attributes

Attribute	Valid options	Notes
<code>target_arch</code>	"aarch64" "arm" "mips"	Not an exclusive list of options.

	"powerpc" "powerpc64" "x86" "x86_64"	
target_os	"android" "bitrig" "dragonfly" "freebsd" "haiku" "ios" "linux" "macos" "netbsd" "redox" "openbsd" "windows"	Not an exclusive list of options.
target_family	"unix" "windows"	
target_env	"" "gnu" "msvc" "musl"	This will often be an empty string ("").
target_endian	"big" "little"	
target_pointer_width	"32" "64"	The size (in bits) of the target architecture's pointer size. Used for <code>isize</code> , <code>usize * const</code> and <code>* mut</code> types.
target_has_atomic	"8" "16" "32" "64" "ptr"	Integer sizes that have support for <i>atomic operations</i> . During an atomic operations, the CPU takes responsibility for preventing race conditions with shared data, at the expense of performance. The word “atomic” is used in the sense of indivisible.
target_vendor	"apple" "pc" "unknown"	
test		No options, just uses a simple Boolean check
debug_assertions		No options, just uses a simple Boolean check. This attribute is present for non-optimized builds and supports the <code>debug_assert!</code> macro.

Lastly, it's possible to tweak `cfg` attributes when invoking `cargo/rustc` via the `-cfg ATTRIBUTE` command-line argument.

7.7 UNDERSTANDING THE CORE OF ACTIONKV: THE LIBACTIONKV CRATE

The command line application built in [Section 7.6](#) dispatches its work to

`libactionkv::ActionKV`. The responsibilities of the `ActionKV` struct are a) to manage interactions with the file system and b) to encode and decode data from the on-disk format.

Figure 7.6. Relationship between `libactionkv` and other components of the project



7.7.1 Initializing the `ActionKV` struct

To create an instance of `libactionkv::ActionKV`, there are two steps: 1) pointing to the file where the data is stored and 2) loading an in-memory index from the data within that file.

Listing 7.11. Excerpt of [Listing 7.9](#) demonstrating the initialization process of `libactionkv::ActionKV`

```
let mut store = ActionKV::open(path).expect("unable to open file"); ①
store.load().expect("unable to load data"); ②
```

① Step 1: open the file at path

② Step 2: create an in-memory index by loading the data from path

Both steps return `Result`, which is why the calls to `.expect()` are also present.

Let's look inside the code of `ActionKV::open()` and `ActionKV::load()`. `open()` opens the file from disk, `load()` loads the offsets of any pre-existing data into an in-memory index.

The code uses two type aliases, `ByteStr` and `ByteString`:

```
type ByteStr = [u8];
```

We'll use the `ByteStr` type alias for data that tends to be used as a string, but happens to be in a binary (raw bytes) form. Its text-based peer is the built-in `str`. Unlike `str`, `ByteStr` is not guaranteed to contain valid UTF-8 text.

Both `str` and `[u8]` (or its alias `ByteStr`) are seen in the wild as `&str` and `&[u8]` (or `&ByteStr`). They are both called “slices”.

```
type ByteString = Vec<u8>;
```

The alias `ByteString` will be the workhorse used when we want to use a type that behaves like a `String`, but can contain arbitrary binary data.

Listing 7.12. Excerpt from [Listing 7.18](#) demonstrating the use of `ActionKV::open()`

```
type ByteString = Vec<u8>; ①
type ByteStr = [u8]; ②

#[derive(Debug, Serialize, Deserialize)] ③
pub struct KeyValuePair {
    pub key: ByteString,
    pub value: ByteString,
}
```

```

#[derive(Debug)]
pub struct ActionKV {
    f: File,
    pub index: HashMap<ByteString, u64>,
}
                                     ④

impl ActionKV {
    pub fn open(path: &Path) -> io::Result<Self> {
        let f = OpenOptions::new()
            .read(true)
            .write(true)
            .create(true)
            .append(true)
            .open(path)?;
        let index = HashMap::new();
        Ok(ActionKV { f, index })
    }

    pub fn load(&mut self) -> io::Result<()> {
        let mut f = BufReader::new(&mut self.f);
                                     ⑤

        loop {
            let current_position = f.seek(SeekFrom::Current(0))?;
                                     ⑥

            let maybe_kv = ActionKV::process_record(&mut f);
                                     ⑦
            let kv = match maybe_kv {
                Ok(kv) => kv,
                Err(err) => {
                    match err.kind() {
                        io::ErrorKind::UnexpectedEof => {
                                     ⑧
                            break;
                        },
                        _ => return Err(err),
                    }
                },
            };

            self.index.insert(kv.key, current_position);
        }

        Ok(())
    }
}

```

- ① This code will be processing lots of `Vec<u8>` data. Because they'll be used in the same way as `String` tends to be used, `ByteString` is a useful alias.
- ② `ByteStr` is to `&str` what `ByteString` is to `Vec<u8>`
- ③ `Serialize` and `Deserialize` are explained later in the chapter at [Section 7.2.1](#). They instruct the compiler to generate *serialization* code to enable writing `KeyValuePair` data to disk via.
- ④ Maintains a mapping between keys and file locations.
- ⑤ The role of the `ActionKV::load()` method is to populate the index of the `ActionKV` struct, mapping keys to file positions

- ⑥ We use the `File::seek()` method to return the number of bytes from the start of the file. This becomes the value of the index.
- ⑦ `ActionKV::process_record()` reads a record in the file at its current position
- ⑧ “Unexpected” is relative. The application may not have expected to encounter the end of the file, but we expect files to be finite and so we deal with that eventuality.

What is EOF?

File operations in Rust might return an error of type `std::io::ErrorKind::UnexpectedEof`. What is "Eof"?

End of File (EOF) is a *convention* that operating systems provide to applications. There is no special marker or delimiter at the end of a file, within the file itself.

EOF is a zero byte (0x0). When reading from a file, the operating system tells the application how many bytes were successfully read from storage. If no bytes were successfully read from disk, yet no error condition was detected, then the operating system and therefore the application assume that EOF has been reached.

This works because the operating system has the responsibility for interacting with physical devices. When a file is read by an application, the application notifies the operating system that it would like to access the disk.

7.7.2 Processing an individual record

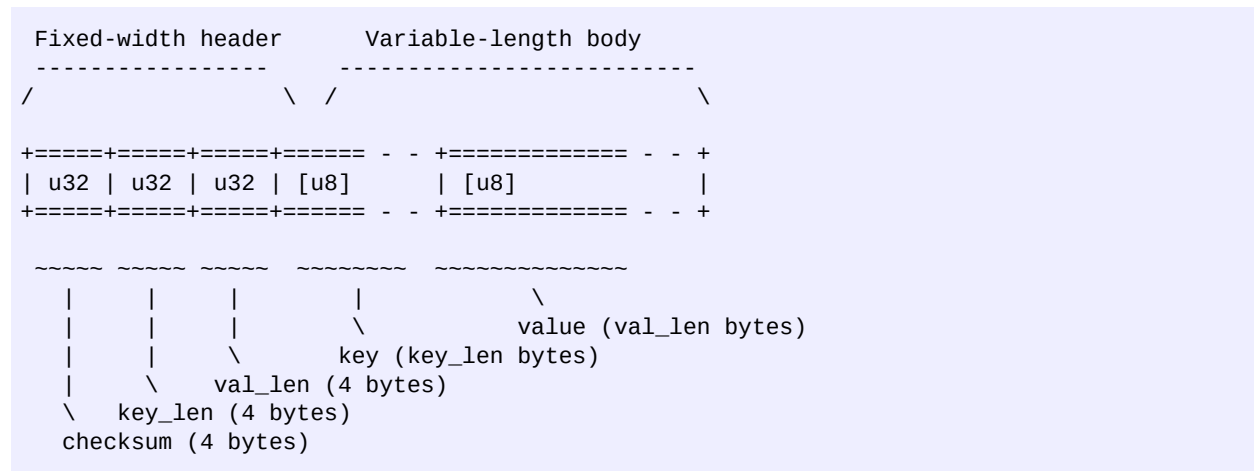
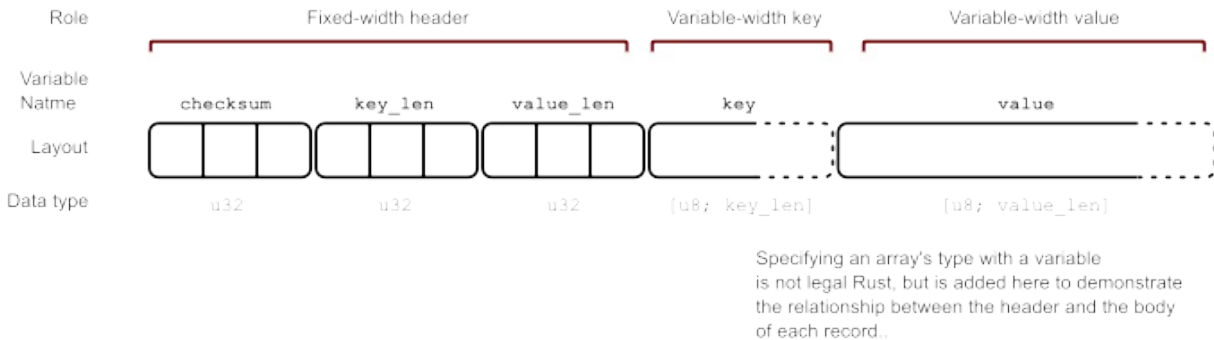
`actionkv` uses a published standard for its on-disk representation. It is an implementation of the "Bitcask" storage backend that was developed for the original implementation of the Riak database. "Bitcask" belongs to a family of file formats known in the literature as "Log-Structured Hash Tables".

What is "Riak"?

Riak is a database. It was developed during the height of the NoSQL movement and competed against similar systems MongoDB, Apache CouchDB and Tokyo Tyrant. It distinguished itself with its emphasis on resilience to failure. Although it was slower than its peers, it made a guarantee that it will never lose data. That guarantee was enabled in part because of its smart choice of data format.

Bitcask lays every record in a prescribed manner:

Figure 7.7. Illustration of a single record in the Bitcask file format. To parse a record, read the header information, then use that information to read the body. Lastly verify the body's contents with the checksum provided in the header.



Every key-value pair is prefixed by 12 bytes. Those bytes describe its length ($\text{key_len} + \text{val_len}$) and its content (checksum)

`process_record()` is the function that does the processing for this within `actionkv`. It begins by reading twelve bytes that represent three integers: a checksum, the length of the key and the length of the value. Those values are then used to read the rest of the data from disk and verify that is what's intended.

Listing 7.13. Extract from [Listing 7.18](#) focusing on the `ActionKV::process_record()` method

```

fn process_record<R: Read>(f: &mut R) -> io::Result<KeyValuePair> { ①
    let saved_checksum = f.read_u32::<LittleEndian>()?; ②
    let key_len = f.read_u32::<LittleEndian>()?; ②
    let val_len = f.read_u32::<LittleEndian>()?; ②
    let data_len = key_len + val_len;

    let mut data = ByteString::with_capacity(data_len as usize);

    {
        f.by_ref() ③
        .take(data_len as u64)
        .read_to_end(&mut data)?;
    }
    debug_assert_eq!(data.len(), data_len as usize); ④

    let checksum = crc32::checksum_ieee(&data); ⑤
    if checksum != saved_checksum {
        panic!("data corruption encountered ({:08x} != {:08x})", checksum,
saved_checksum);
    }

    let value = data.split_off(key_len as usize); ⑥
    let key = data;

    Ok( KeyValuePair { key, value } )
}

```

- ① The `f` argument accepts any type that implements `Read`. This is generally expected to be files, but `&[u8]` is another type that's acceptable here.
- ② The `byteorder` crate allows on-disk integers to be read in a deterministic manner, as discussed at [Section 7.7.3](#).
- ③ `f.by_ref()` is required because `take(n)` creates a new `Read` value. Using a reference within this short-lived block allows us to sidestep ownership issues.
- ④ `debug_assert!` tests are disabled in optimized builds, enabling debug builds to have more runtime checks.
- ⑤ A “checksum” is a number that can be used to verify that the bytes read from disk are the same as what was intended. This process is discussed at [Section 7.7.4](#).
- ⑥ The `split_off(n)` method splits a `Vec<T>` in two at `n`.

7.7.3 Writing multi-byte binary data to disk in a guaranteed byte order

One challenge that our code faces is that it needs to be able to store multi-byte data to disk in a deterministic way. This sounds easy, but computing platforms differ as to how they read numbers. Some read the four bytes of an `i32` from left-to-right, others read right-to-left. That could potentially be a problem if the program is designed to be written by one computer and loaded by another.

The Rust ecosystem provides some support here. The `byteorder` crate can extend types that implement the standard library's `std::io::Read` and `std::io::Write` traits. `std::io::Read` and `std::io::Write` are commonly associated with `std::io::File`, but are also implemented by other types, such as `[u8]` and `TcpStream`. The extensions can guarantee how multi-byte sequences are interpreted, either as little endian or big endian.

To follow what's going on with our key-value store, it will help to have an understanding how `byteorder` works. [Listing 7.15](#) is a toy application that demonstrates the core functionality. Lines 11-23 show how to write to a file and lines 28-35 show how to read from one.

The two key lines are:

```
use byteorder::{LittleEndian};  
use byteorder::{ReadBytesExt, WriteBytesExt};
```

`byteorder::LittleEndian`—and its peers `BigEndian` and `NativeEndian` that are not used in [Listing 7.15](#)—are types that declare how multi-byte data is written to and read to disk. `byteorder::ReadBytesExt` and `byteorder::WriteBytesExt` are traits. In some sense, they're invisible within the code. They extend methods to primitive types, such as `f32` and `i16` without further ceremony. Bringing them into scope with a `use` statement immediately adds powers to the types that are implemented within the source of `byteorder`. In practice, that means primitive types. Rust, as a statically-typed language, makes this transformation at compile-time. From the running program's point of view, integers always had the ability to write themselves to disk in a pre-defined order.

When executed, [Listing 7.15](#) produces a visualisation of the byte patterns that

are created by writing `1_u32`, `2_i8` and `3.0_f32` out in little-endian order:

```
[1, 0, 0, 0]
[1, 0, 0, 0, 2]
[1, 0, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 8, 64]
```

Listing 7.14. Metadata for the project at [Listing 7.15](#) (ch7/ch7-write123/Cargo.toml)

```
[package]
name = "write123"
version = "0.1.0"
authors = ["Tim McNamara <author@rustinaction.com>"]
edition = "2018"

[dependencies]
byteorder = "1.2"
```

Listing 7.15. Writing integers to disk (ch7/ch7-write123/src/main.rs)

```
use std::io::Cursor;           ①
use byteorder::{LittleEndian};  ②
use byteorder::{ReadBytesExt, WriteBytesExt};  ③

fn write_numbers_to_file() -> (u32, i8, f64) {
    let mut w = vec![];         ④

    let one: u32 = 1;
    let two: i8 = 2;
    let three: f64 = 3.0;

    w.write_u32::<LittleEndian>(one).unwrap();  ⑤
    println!("{:?}", &w);

    w.write_i8(two).unwrap();  ⑥
    println!("{:?}", &w);

    w.write_f64::<LittleEndian>(three).unwrap();  ⑤
    println!("{:?}", &w);

    (one, two, three)
}

fn read_numbers_from_file() -> (u32, i8, f64) {
    let mut r = Cursor::new(vec![1, 0, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 8, 64]);
    let one_ = r.read_u32::<LittleEndian>().unwrap();
    let two_ = r.read_i8().unwrap();
    let three_ = r.read_f64::<LittleEndian>().unwrap();

    (one_, two_, three_)
```

```

}

fn main() {
    let (one, two, three) = write_numbers_to_file();
    let (one_, two_, three_) = read_numbers_from_file();

    assert_eq!(one, one_);
    assert_eq!(two, two_);
    assert_eq!(three, three_);
}

```

- ① As files support the ability to `seek()`, that is to move backwards and forwards to different positions, something is necessary to enable a `Vec<T>` to mock being a file. `io::Cursor` plays that role, enabling an in-memory `Vec<T>` to be file-like.
- ② Used as a type argument for various `read_*`() and `write_*`() methods that are used in the program.
- ③ Traits that provide `read_*`() and `write_*`()
- ④ The variable name `w` stands for “writer”
- ⑤ Write values “to disk”. These methods return `io::Result`, which we swallow here as they won’t fail unless something is seriously wrong with the computer that is running the program.
- ⑥ Single byte types `i8` and `u8` don’t take an endianness parameter

7.7.4 Validating I/O errors with checksums

`actionkv` has no method of validating what it has read from disk is what was written to disk. Perhaps something was interrupted during the original write? While we may not be able to recover the original data if this were the case, if we could recognize the issue then we would be in a position to alert the user.

A well-worn path to overcome this problem is to use a technique called a *checksum*. Here’s how it works:

Saving to disk

- Before data is written to disk, a checking function (there are many options as to which function) is applied to those bytes
- The result of the checking function (the checksum) is written

alongside the original data.

No checksum is calculated for the bytes of the checksum. If something breaks while writing the checksum's own bytes to disk, this will also be noticed later on as an error.

Reading from disk

- Read the data and the saved checksum.
- Apply the checking function to the data.
- Compare the results of the two checking functions. If the two results do not match, an error must have occurred and the data should be considered corrupted.

Which checking function should you use? Like many things in computer science, it depends. An ideal checksum function would a) return the same result for the same input, b) always return a different result for different inputs, c) be fast and d) be easy to implement.

Table 7.8. A simplistic evaluation of different checksum functions

Checksum Technique	Size of Result	Simplicity	Speed	Reliability
Parity bit	1 bit	★★★★★	★★★★★	★★☆☆☆
CRC32	32 bits	★★★☆☆	★★★★☆	★★★★☆
cryptographic hash function	128-512 bits (or more)	★☆☆☆☆	★★☆☆☆	★★★★★

Functions that you might see in the wild will depend on your application domain. More traditional areas might see the use of simpler systems, such as “parity bit” or “CRC32”.

- The parity bit is easy and fast, but is somewhat prone to error.
- CRC32 (cyclic redundancy check, returning 32 bits) is much more complex, but its results are more trustworthy.
- Cryptographic hash functions are much more complex still while

also being significantly slower, yet provide very high levels of assurance.

IMPLEMENTING PARITY BIT CHECKING

This section here implements one of the simpler checksum schemes: parity checking. Parity checks count the number of 1s within a bitstream. They store a bit that indicates whether the count was even or odd.

Parity bits have traditionally been used for error detection within noisy communication systems, such as transmitting data over analog systems such as radio waves. For example, the ASCII encoding of text has a particular property that makes it quite convenient for this scheme. Its 128 characters only require 7 bits of storage ($128 = 2^7$). That leaves one bit spare in every byte.

Systems can also include parity bits in larger streams of bytes. [Listing 7.17](#) presents (an overly chatty) implementation. The `parity_bit()` function in lines 1-10 that takes an arbitrary stream of bytes and returns a `u8` indicating whether the count of the input's bits was even or odd.

When executed, [Listing 7.17](#) produces the following output:

Listing 7.16. Output from [Listing 7.17](#)

```
input: [97, 98, 99]                ①
97 (0b01100001) has 3 one bits
98 (0b01100010) has 3 one bits
99 (0b01100011) has 4 one bits
output: 00000001

input: [97, 98, 99, 100]           ②
97 (0b01100001) has 3 one bits
98 (0b01100010) has 3 one bits
99 (0b01100011) has 4 one bits
100 (0b01100100) has 3 one bits
result: 00000000
```

① [97, 98, 99] represents `b"abc"` as seen by the internals of the Rust compiler.

② [97, 98, 99, 100] represents `b"abcd"`

Listing 7.17. Implementing parity bit checking (ch7/ch7-paritybit/src/main.rs)

```
fn parity_bit(bytes: &[u8]) -> u8 {  
    let mut n_ones: u32 = 0; ①  
  
    for byte in bytes {  
        let ones = byte.count_ones();  
        n_ones += ones; ②  
        println!("{}", (0b{:08b}) has {} one bits", byte, byte, ones);  
    }  
    (n_ones % 2 == 0) as u8 ③  
}  
  
fn main() {  
    let abc = b"abc";  
    println!("input: {:?}", abc);  
    println!("output: {:08x}", parity_bit(abc));  
    println!();  
    let abcd = b"abcd";  
    println!("input: {:?}", abcd);  
    println!("result: {:08x}", parity_bit(abcd))  
}
```

- ① Take a byte slice as argument `bytes` and return a single byte as output. This function could have easily returned a `bool` value, but returning `u8` allows the result to be shifted into some future desired position later on.
- ② All of Rust's integer types come equipped with `count_ones()` and `count_zeros()` methods.
- ③ There are plenty of methods to optimize this function. One fairly simple approach would be to hard code a `const [u8; 256]` array of zeros and ones corresponding to the intended result, then index that array with each byte.

7.7.5 Inserting a new key-value pair into an existing database

As discussed during [Section 7.6](#), there are four operations that our code needs to support: insert, get, update and delete. We're using an append-only design, which means that the last two can be implemented as variants of insert. You may have noticed that during `load()`, the inner loop continues until the end of the file. This allows more-recent updates to overwrite stale data, including deletions.

Inserting a new record is almost the inverse of `process_record()` described at

[Section 7.7.2.](#)

```
pub fn insert(&mut self, key: &ByteStr, value: &ByteStr) -> io::Result<()> {
    let position = self.insert_but_ignore_index(key, value)?;

    self.index.insert(key.to_vec(), position);
    Ok(())
}

pub fn insert_but_ignore_index(&mut self, key: &ByteStr, value: &ByteStr) ->
io::Result<u64> {
    let mut f = BufWriter::new(&mut self.f);

    let key_len = key.len();
    let val_len = value.len();
    let mut tmp = ByteString::with_capacity(key_len + val_len);

    for byte in key {
        tmp.push(*byte);
    }

    for byte in value {
        tmp.push(*byte);
    }

    let checksum = crc32::checksum_ieee(&tmp);

    let next_byte = SeekFrom::End(0);
    let current_position = f.seek(SeekFrom::Current(0))?;
    f.seek(next_byte)?;
    f.write_u32::<LittleEndian>(checksum)?;
    f.write_u32::<LittleEndian>(key_len as u32)?;
    f.write_u32::<LittleEndian>(val_len as u32)?;
    f.write_all(&mut tmp)?;

    Ok(current_position)
}
```

- ① `key.to_vec()` converts the `&ByteStr` to a `ByteString`.
- ② `std::io::BufWriter` is a type that batches multiple `short write()` calls into fewer actual disk operations single one. This increases throughput while keeping the application code neater.
- ③ Iterating through one collection to populate another is slightly awkward, but the job gets done.

7.7.6 libactionkv full code listing

`libactionkv` performs the heavy lifting in our key-value stores. You have already explored much of it during [Section 7.7.1](#), [Section 7.7.2](#), and [Section](#)

7.7.5.

Here is the code in full:

Listing 7.18. libactionkv (ch7/ch7-actionkv1/src/lib.rs)

```
use std::collections::HashMap;
use std::fs::{File, OpenOptions};
use std::io;
use std::io::{SeekFrom, BufReader, BufWriter};
use std::io::prelude::*;
use std::path::{Path};

use byteorder::{LittleEndian, ReadBytesExt, WriteBytesExt};
use crc32::crc32;
use serde_derive::{Deserialize, Serialize};

type ByteString = Vec;
type ByteStr = [u8];

#[derive(Debug, Serialize, Deserialize)]
pub struct KeyValuePair {
    pub key: ByteString,
    pub value: ByteString,
}

#[derive(Debug)]
pub struct ActionKV {
    f: File,
    pub index: HashMap<ByteString, u64>,
}

impl ActionKV {
    pub fn open(path: &Path) -> io::Result<Self> {
        let f = OpenOptions::new()
            .read(true)
            .write(true)
            .create(true)
            .append(true)
            .open(path)?;
        let index = HashMap::new();
        Ok(ActionKV { f, index })
    }

    pub fn load(&mut self) -> io::Result<()> {
        let mut f = BufReader::new(&mut self.f);

        loop {
            let current_position = f.seek(SeekFrom::Current(0))?;

            let maybe_kv = ActionKV::process_record(&mut f);
            let kv = match maybe_kv {
                Ok(kv) => kv,
                Err(err) => {
                    match err.kind() {
                        io::ErrorKind::UnexpectedEof => {
```

```

        break;
    },
    _ => return Err(err),
}
    },
};

    self.index.insert(kv.key, current_position);
}

Ok(())
}

fn process_record<R: Read>(f: &mut R) -> io::Result<KeyValuePair> { ①
    let saved_checksum = f.read_u32::<LittleEndian>()?;
    let key_len = f.read_u32::<LittleEndian>()?;
    let val_len = f.read_u32::<LittleEndian>()?;
    let data_len = key_len + val_len;

    let mut data = ByteString::with_capacity(data_len as usize);

    {
        f.by_ref()
            .take(data_len as u64)
            .read_to_end(&mut data)?;
    }
    debug_assert_eq!(data.len(), data_len as usize);

    let checksum = crc32::checksum_ieee(&data);
    if checksum != saved_checksum {
        panic!("data corruption encountered ({:08x} != {:08x})",
            checksum, saved_checksum);
    }

    let value = data.split_off(key_len as usize);
    let key = data;

    Ok( KeyValuePair { key, value } )
}

pub fn seek_to_end(&mut self) -> io::Result<u64> {
    self.f.seek(SeekFrom::End(0))
}

pub fn get(&mut self, key: &ByteStr) -> io::Result<Option<ByteString>> { ②
    let position = match self.index.get(key) {
        None => return Ok(None),
        Some(position) => *position,
    };

    let kv = self.get_at(position)?;

    Ok(Some(kv.value))
}

pub fn get_at(&mut self, position: u64) -> io::Result<KeyValuePair> {
    let mut f = BufReader::new(&mut self.f);
    f.seek(SeekFrom::Start(position))?;
    let kv = ActionKV::process_record(&mut f)?;

```

```

        Ok(kv)
    }

    pub fn find(&mut self, target: &ByteStr) -> io::Result<Option<(u64, ByteString)>>
    {
        let mut f = BufReader::new(&mut self.f);

        let mut found : Option<(u64, ByteString)> = None;

        loop {
            let position = f.seek(SeekFrom::Current(0))?;

            let maybe_kv = ActionKV::process_record(&mut f);
            let kv = match maybe_kv {
                Ok(kv) => kv,
                Err(err) => {
                    match err.kind() {
                        io::ErrorKind::UnexpectedEof => {
                            break;
                        },
                        _ => return Err(err),
                    }
                }
            };

            if kv.key == target {
                found = Some((position, kv.value));
            }

            // important to keep looping until the end of the file,
            // in case the key has been overwritten
        }

        Ok(found)
    }

    pub fn insert(&mut self, key: &ByteStr, value: &ByteStr) -> io::Result<()> {
        let position = self.insert_but_ignore_index(key, value)?;

        self.index.insert(key.to_vec(), position);
        Ok(())
    }

    pub fn insert_but_ignore_index(&mut self, key: &ByteStr, value: &ByteStr) ->
    io::Result<u64> {
        let mut f = BufWriter::new(&mut self.f);

        let key_len = key.len();
        let val_len = value.len();
        let mut tmp = ByteString::with_capacity(key_len + val_len);

        for byte in key {
            tmp.push(*byte);
        }

        for byte in value {
            tmp.push(*byte);
        }
    }

```

```

        let checksum = crc32::checksum_ieee(&tmp);

        let next_byte = SeekFrom::End(0);
        let current_position = f.seek(SeekFrom::Current(0))?;
        f.seek(next_byte)?;
        f.write_u32::(checksum)?;
        f.write_u32::(key_len as u32)?;
        f.write_u32::(val_len as u32)?;
        f.write_all(&tmp)?;

        Ok(current_position)
    }

    #[inline]
    pub fn update(&mut self, key: &ByteStr, value: &ByteStr) -> io::Result<()> {
        self.insert(key, value)
    }

    #[inline]
    pub fn delete(&mut self, key: &ByteStr) -> io::Result<()> {
        self.insert(key, b"")
    }
}

```

- ① `process_record()` assumes that `f` is already at the right place in the file
- ② We need to wrap `Option` within `Result` to allow for the possibilities of I/O errors as well as missing values occurring
- ③ "Unexpected" is relative. The application may not have expected it, but we expect files to be finite.

If you've made it this far, you should congratulate yourself. You've implemented a key value store that will happily store and retrieve whatever you have to throw at it.

7.7.7 Working with keys and values with `HashMap` and `BTreeMap`

Working with key-value pairs happens in almost every programming language. For the tremendous benefit of learners everywhere, this task, and the data structures that support it, have many names.

You might encounter someone with a computer science background preferring to use the term *hash table*. Perl and Ruby strip that off and call them *hashes*. Lua does the opposite and uses the term *table*. Many communities name the structure

after a metaphor, such as a *dictionary* (one term is being associated with a “definition”) or a *map* (programmers, following mathematicians, are mapping from one value to another). Other communities prefer naming based on the role that the structure plays. PHP describes them as *associative arrays*. JavaScript’s objects tend to be implemented as a key/value pair collection and so generic term *object* suffices. Static languages tend to name them according to how they are implemented. C++ and Java distinguish between a *hash map* and a *tree map*.

Rust uses the terms `HashMap` and `BTreeMap` to define two implementations of the same abstract data type. Rust is closest to C++ and Java in this regard.

In this book, the terms “collection of key/value pairs” and “associative array” refer to the abstract data type. “Hash table” refers to associative arrays implemented with a hash table. `HashMap` refers to Rust’s implementation of hash tables.

What is a hash? What is hashing?

If you’ve ever been confused by the term “hash”, it may help to understand that this relates to an implementation decision made to enable non-integer keys to map to values.

`HashMap` is implemented with a *hash function*. Computer scientists will understand that this implies a certain behavior pattern in common cases. They have a “constant time lookup” in general. (Although their performance can suffer in some pathological cases, as we’ll see shortly).

A *hash function* maps between values of variable-length to fixed-length values. In practice, the return value of a hash function is an integer. That fixed-width value can then be used to build a very efficient lookup table. This internal lookup table is known as a *hash table*.

[Listing 7.19](#) is a very basic hash function for `&str` that simply interprets the first character of a string as an unsigned integer:

Listing 7.19. Implementing a basic hash function for `&str` that uses the first character of the string as an hash value

```
fn basic_hash(key: &str) -> u32 {  
    let first = key.chars()           ①  
                    .next()           ②  
                    .unwrap_or('\0'); ③  
  
    unsafe {                           ④  
        std::mem::transmute::<char, u32>(first) ④  
    }                                   ④  
}
```

```
}
```

- ① The `.chars()` iterator converts the string into a series of `char` values, which are each 4 bytes long
- ② `.next()` returns an `Option`, which will be either `Some(char)` or `None` for empty strings
- ③ In the case of an empty string, provide `NULL` as the default. `unwrap_or()` behaves as `unwrap()`, but will provide a value rather than panicking when `None` is encountered.
- ④ Interpret the memory of `first` as a `u32`, even though its type is `char`.

`basic_hash` can take any string as input—an infinite set of possible inputs—and return a fixed-width result for all of them in a deterministic manner. Great!

`basic_hash` is very fast, but has some large faults. If multiple inputs start with the same character (perhaps “Tonga” and “Tuvalu”), they will result in the same output. This happens in every instance when a infinite input space is mapped into a finite space, but it’s particularly bad here. Natural language text is not uniformly distributed.

Hash tables, including Rust’s `HashMap` deal with this phenomenon, called a *hash collision*, by providing a backup location for keys with the same hash value. That secondary storage is typically a `Vec<T>` that we’ll call the collision store. When collisions occur, the collision store is scanned from front to back whenever it is accessed. That linear scan takes longer and longer to run as the store’s size increases. Attackers can make use of this characteristic to overload the computer that is performing the hash function.

In very general terms, faster hash functions do less work to avoid being attacked. They will also perform best when their inputs will be within a defined range.

Fully understanding the internals of how hash tables are implemented is too much detail for here. It’s a fascinating topic for programmers who wish to extract optimum performance and memory usage from their programs.

7.7.8 Creating a `HashMap` and populating it with values

[Listing 7.20](#) provides a collection of key-value pairs encoded as JSON.

Listing 7.20. Using some polynesian island nations and their capital cities to demonstrate the use of an associative array in JSON notation

```
{  
  "Cook Islands": "Avarua",  
  "Fiji": "Suva",  
}
```

```
"Kiribati": "South Tarawa",  
"Niue": "Alofi",  
"Tonga": "Nuku'alofa",  
"Tuvalu": "Funafuti"  
}
```

Rust does not provide a literal syntax for `HashMap` within the standard library. To insert items into and get them out again, follow the example provided at [Listing 7.21](#).

When executed, [Listing 7.21](#) produces the following line in the console:

```
Capital of Tonga is: Nuku'alofa
```

Listing 7.21. An example of the basic operations of `HashMap` (ch7/ch7-pacific-basic/src/main.rs)

```
use std::collections::HashMap;  
  
fn main() {  
    let mut capitals = HashMap::new();  
  
    capitals.insert("Cook Islands", "Avarua");  
    capitals.insert("Fiji", "Suva");  
    capitals.insert("Kiribati", "South Tarawa");  
    capitals.insert("Niue", "Alofi");  
    capitals.insert("Tonga", "Nuku'alofa");  
    capitals.insert("Tuvalu", "Funafuti");  
  
    println!("Capital of Tonga is: {}", capitals["Tonga"]);  
}
```

- ① Type declarations of the keys and values are not required here as they are inferred by the Rust compiler
- ② `HashMap` implements `Index`, which allows for values to be retrieved via the square bracket indexing style

Writing everything out as method calls can feel needlessly verbose at times. With some support from the wider Rust ecosystem, it's possible to inject JSON string literals into Rust code. What's best is that the conversion is done at compile time, meaning no loss of runtime performance.

The output of [Listing 7.22](#) is also a single line:


```
Capital of Tonga is: "Nuku'alofa" ①
```

① The double quotes are also included as the `json!` macro returns a wrapper around `String` and this is its default representation.

Listing 7.22. Using `serde-json` crate to include JSON literals within your Rust source code (ch7/ch7-pacific-json/src/main.rs)

```
#[macro_use]                ①
extern crate serde_json;    ①

fn main() {
    let capitals = json!({    ②
        "Cook Islands": "Avarua",
        "Fiji": "Suva",
        "Kiribati": "South Tarawa",
        "Niue": "Alofi",
        "Tonga": "Nuku'alofa",
        "Tuvalu": "Funafuti"
    });

    println!("Capital of Tonga is: {}", capitals["Tonga"])
}
```

① Incorporate the `serde_json` crate into this one and make use of its macros. This brings the `json!` macro into scope.

② `json!` takes a JSON literal—and also Rust expressions that implement `Borrowed`—and converts it into a Rust value of type `serde_json::Value`. `serde_json::Value` is an enum that can represent every type within the JSON specification.

7.7.9 Retrieving values from `HashMap` and `BTreeMap`

The main operation that a key-value store provides is the ability to access its values. There are two ways to achieve this. To demonstrate them, let's assume that we have initialized `capitals` from [Listing 7.22](#).

The approach already demonstrated is to access values via square brackets:

```
capitals["Tonga"] ①
```

① Returns "Nuku'alofa"

This approach will return a read-only reference to the value (which is deceptive when dealing with examples containing string literals, as their status as references is somewhat disguised). In the syntax used by Rust's documentation, this is described as `&V`, where `&` denotes a read-only reference and `V` is the type of the value. If the key is not present, the program will panic.

Note

Index notation is supported by all types that implement the `Index` trait.

Accessing `capitals["Tonga"]` is syntactic sugar for `capitals.index("Tonga")`.

Secondly, it's possible to use the `.get()` method on `HashMap`. This returns an `Option<&V>`, providing the opportunity to recover from cases where values are missing.

```
capitals.get("Tonga") ①
```

① Returns `Some("Nuku'alofa")`

Other important operations supported by `HashMap` are:

- Deleting key-value pairs with the `.remove()` method
- Iterating over keys, values and key-value pairs with the `.keys()`, `.values()` and `.iter()` methods respectively as well as their read/write variants (`.keys_mut()`, `.values_mut()` and `.iter_mut()`).

There is no method for iterating through a subset of the data. For that, `BTreeMap` is needed.

7.7.10 How to decide between `HashMap` and

BTreeMap

If you're wondering about which backing data structure to choose, here is a simple guideline: Use `HashMap` unless you have a good reason to use `BTreeMap`.

`BTreeMap` is faster when there is a natural ordering between the keys, and your application makes use of that ordering. Specifically:

- you wish to process values in order
- you wish to select for a range of keys

Let's demonstrate these two use cases with a small example from Europe. The Dutch East India Company, known as VOC after the initials of its Dutch name *Vereenigde Oostindische Compagnie* was an extremely powerful economic and political force at its peak.

For two centuries, VOC was a dominant trader between Asia and Europe. It had its own navy, currency and established its own colonies (called trading posts). It was also the first company to issue bonds. In the beginning, investors from six business chambers (kamers) provided capital for the business.

Let's use these investments as key-value pairs. Code is available at [Listing 7.24](#). When compiled, it produces an executable that generates the following output:

Listing 7.23. Output from [Listing 7.24](#)

```
chamber Rotterdam invested 173000
chamber Hoorn invested 266868
chamber Delft invested 469400
chamber Enkhuizen invested 540000
chamber Middelburg invested 1300405
chamber Amsterdam invested 3697915
smaller chambers: Rotterdam Hoorn Delft
```

Listing 7.24. Demonstrating range queries and ordered iteration of the `BTreeMap`

```
use std::collections::BTreeMap;

fn main() {
```

```

let mut voc = BTreeMap::new();

voc.insert(3_697_915, "Amsterdam");
voc.insert(1_300_405, "Middelburg");
voc.insert( 540_000, "Enkhuizen");
voc.insert( 469_400, "Delft");
voc.insert( 266_868, "Hoorn");
voc.insert( 173_000, "Rotterdam");

for (guilders,kamer) in &voc {
    println!("chamber {} invested {}", kamer, guilders); ①
}

print!("smaller chambers: ");
for (_guilders,kamer) in voc.range(0..500_000) {           ②
    print!("{}", kamer);
}
println!("");
}

```

① Prints in sorted order. With integers, this means starting at the lowest value.

② BTreeMap allows you to select a portion of the keys that will be iterated through with the range syntax

Table 7.9. Considerations for deciding on which implementation to use to map keys to values

std::collections::HashMap with default hash function (known as SipHash in the literature)	cryptographically secure and resistant to denial of service attacks, but slower than alternative hash functions
std::collections::BTreeMap	useful for keys with an inherent ordering where cache-coherence may provide a speed boost

7.7.11 Adding database index to action_kv v2

Databases and file systems are much larger pieces of software than single files. There is a very large design space involved with storage and retrieval systems, which is why new ones are always being developed. Common to all of those systems however will be a component that is The real smarts behind the database

actionkv v0.1, built in [Section 7.5.2](#), contains a major issue that prevents it from having a decent start-up time. Every time it's run, it needs to re-build its index of where keys are stored every time it runs.

Let's add the ability for actionkv to store its own data that index *within* the same file that is being used to store application data.

It will be easier than it sounds. No changes to libactionkv are necessary. And the front-end code only requires minor additions.

The project folder now has a new structure with an extra file:

Listing 7.25. ActionKV v2: updated project structure

```
actionkv
├── src
│   ├── akv_disk.rs ①
│   ├── akv_mem.rs
│   └── lib.rs
└── Cargo.toml      ②
```

① New file included in the project

② Two updates are required here to add a new binary and dependencies

The project's Cargo.toml adds some new dependencies along with a second "[[bin]]" entry:

Listing 7.26. ActionKV v2: updated cargo.toml file (ch7/ch7-actionkv2/Cargo.toml)

```
[package]
name = "actionkv"
version = "2.0.0"
authors = ["Tim McNamara <author@rustinaction.com>"]
edition = "2018"

[dependencies]
bincode = "1"           ①
byteorder = "1"
crc = "1"
serde = "1"             ①
serde_derive = "1"      ①

[lib]
name = "libactionkv"
path = "src/lib.rs"

[[bin]]
name = "akv_mem"
path = "src/akv_mem.rs"
```

```
[[bin]]
name = "akv_disk"           ②
path = "src/akv_disk.rs"    ②
```

① New dependencies to assist with writing the index to disk

② New executable definition

When a key is accessed, we convert load the index on-disk representation to its in-memory form.

Listing 7.27. Excerpt from [Listing 7.28](#) highlighting the main change from [Listing 7.9](#). The on-disk implementation of ActionKV includes a hidden `INDEX_KEY` value that allows ActionKV to quickly access other records in the file.

```
match action {
  "get" => { ①
    let index_as_bytes = a.get(INDEX_KEY).unwrap().unwrap(); ②
    let index: HashMap<ByteString, u64> =
      bincode::deserialize(&index_as_bytes).unwrap(); ③

    match index.get(key) { ④
      None => eprintln!("{:?} not found", key), ④
      Some(idx) => { ⑤
        let kv = a.get_at(idx).unwrap(); ⑤
        println!("{:?}", kv.value), ⑤
      }
    }
  },
}
```

① Nothing changes for the end-user

② `INDEX_KEY` is the “name” of the index within the database. Two `unwrap()` calls are required because `a.index` is a `HashMap` that returns `Option` and values themselves are stored within an `Option` to facilitate possible future deletes.

③ Convert the on-disk representation to an in-memory representation.

④ Has the key that’s been requested actually been stored?

⑤ Yes! Great, now fetch that `KeyValuePair` from disk

Listing 7.28. A key-value store that persists its index data

between runs (ch7/ch7-actionkv2/src/akv_disk.rs)

```
use std::collections::HashMap;
use libactionkv::ActionKV;

#[cfg(target_os = "windows")]
const USAGE: &str = "
Usage:
    akv_disk.exe FILE get KEY
    akv_disk.exe FILE delete KEY
    akv_disk.exe FILE insert KEY VALUE
    akv_disk.exe FILE update KEY VALUE
";

#[cfg(not(target_os = "windows"))]
const USAGE: &str = "
Usage:
    akv_disk FILE get KEY
    akv_disk FILE delete KEY
    akv_disk FILE insert KEY VALUE
    akv_disk FILE update KEY VALUE
";

type ByteStr = [u8];
type ByteString = Vec<u8>;

fn store_index_on_disk(a: &mut ActionKV, index_key: &ByteStr) {
    a.index.remove(index_key);
    let index_as_bytes = bincode::serialize(&a.index).unwrap();
    a.index = std::collections::HashMap::new();
    a.insert(index_key, &index_as_bytes).unwrap();
}

fn main() {
    const INDEX_KEY: &ByteStr = b"+index";

    let args: Vec<String> = std::env::args().collect();
    let fname = args.get(1).expect(&USAGE);
    let action = args.get(2).expect(&USAGE).as_ref();
    let key = args.get(3).expect(&USAGE).as_ref();
    let maybe_value = args.get(4);

    let path = std::path::Path::new(&fname);
    let mut a = ActionKV::open(path).expect("unable to open file");

    a.load().expect("unable to load data");
    store_index_on_disk(&mut a, INDEX_KEY);

    match action {
        "get" => {
            let index_as_bytes = a.get(&INDEX_KEY).unwrap().unwrap();
            let index: HashMap<ByteString, u64> =
                bincode::deserialize(&index_as_bytes).unwrap();

            match index.get(key) {
                None => eprintln!("{:?} not found", key),
                Some(value) => println!("{:?}", value), ①
            }
        }
    }
}
```

```

    },
    "delete" => a.delete(key).unwrap(),
    "insert" => {
        let value = maybe_value.expect(&USAGE).as_ref();
        a.insert(key, value).unwrap()
    },
    "update" => {
        let value = maybe_value.expect(&USAGE).as_ref();
        a.update(key, value).unwrap()
    },
    _ => eprintln!("{}", &USAGE),
}
}

```

① To print values, we need to use `Debug` as a `[u8]` value can contain arbitrary bytes

7.8 SUMMARY

This chapter has covered lots of ground. Hopefully you have a newfound appreciation for some of the difficulties that arise from trying to do something as easy as storing information.

You have learned:

- presenting binary data as text with hexadecimal notation by implementing a hexdump clone
- translating between Rust data structures and on-disk file formats, such as JSON, with `serde` and its related crates
- reading and writing multi-byte binary data with a defined endianness
- verifying data that's sent across fallible channels via checksums and parity bits
- managing complex software projects by writing library crates
- adding multiple binaries to a project with `cargo`
- using a `lib.rs` file to build a crate and import it with a project's `main()` function, within `src.rs`
- working with key value pairs
- the internals of parity bit checking and other forms of checksum
- printing errors to `stderr` with the `eprintln!` macro

8 NETWORKING

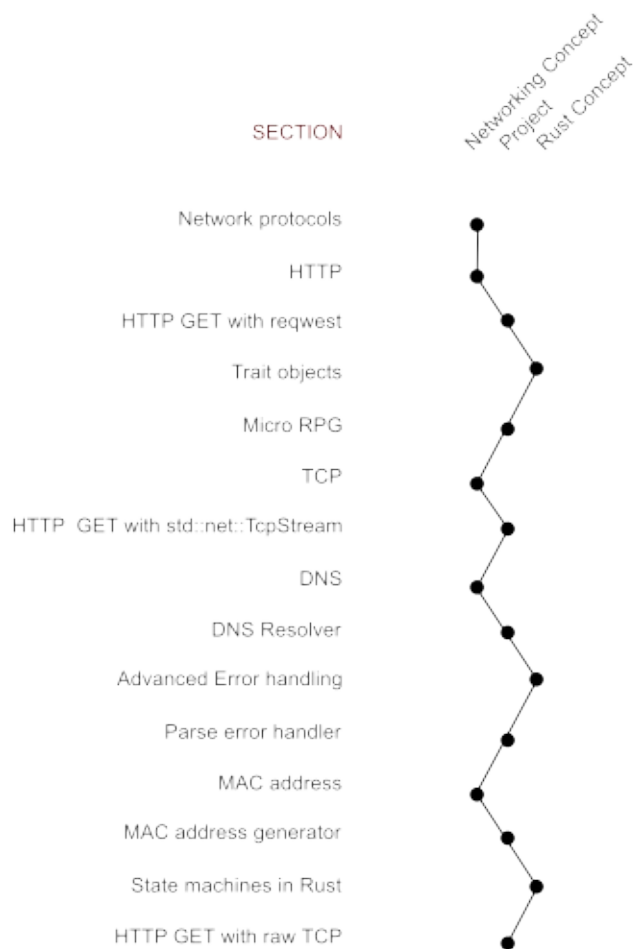
This chapter implements making HTTP requests multiple times, stripping away a layer of abstraction each time. We start by using a user-friendly library, then boil that away until we're left with manipulating raw TCP packets. Impress your friends with your ability to distinguish an IP address from a MAC address. And learn why we went straight from IPv4 to IPv6.

You'll also be learning lots of Rust in this chapter, most of it related to advanced error handling techniques which become essential for incorporating upstream crates. Several pages are devoted to error handling. This includes a thorough introduction to *trait objects*.

Networking is a difficult subject to cover in a single chapter. Each layer is a fractal of complexity. Networking experts will hopefully be kind of my lack of depth in my treatment of their topic of interest!

[Figure 8.1](#) provides an overview of the topics that the chapter covers. Some of the projects that we cover include implementing DNS resolution, generating standards-compliant MAC addresses and multiple examples of generating HTTP requests.. A hint of a role-playing game is added for light relief.

Figure 8.1. Networking chapter map. The chapter incorporates a healthy mix of theory and practical exercises.



8.1 JUST ENOUGH HTTP

Rather than trying to learn the whole networking stack, let's focus on something that's of practical use. Most readers of this book will have encountered web programming. Most web programming involves interacting with some sort of framework. Let's look there.

HTTP is the protocol that web frameworks understand. Learning more about HTTP enables us to extract the most performance out of our web frameworks. It can also help is to more easily diagnose any problems that are occurring.

Figure 8.2. Several layers of networking protocols involved with delivering content over the Internet. Includes comparisons with 2

other common models, the 7 layer OSI model and the 4 layer TCP/IP model.

How computers talk to each other

ABOUT

A view of the networking stack. Each layer relies upon the layers below it.

Occasionally layers bleed together. For example, HTML files can include directives that overwrite those provided by HTTP.

For a message to be received, each layer must be traversed from bottom to top. To send messages, the steps are reversed.

HOW TO READ

Vertical positioning typically indicates that two levels interact at that location.

Exceptions include encryption provided by TLS, Network addressing can be provided by either IPv4 or IPv6 and virtual layers are largely ignorant of physical links. (Shadows from physics do appear on upper layers in the form of latency and reliability.)

Gaps indicate that a higher level can pass directly through to the level below. A domain name or TLS security is not necessary for HTTP to function, for example.

LEGEND

-  Protocol discussed in this chapter
-  Protocol in use at this level
-  Represents hundreds of other protocols that exist at this level.
-  Represents that this protocol is available, but may not be deployed.



Networking is comprised of layers. If you're new to the field, don't be

intimidated by the flood of acronyms. The most important thing to remember is that lower levels are unaware of what's happening above them and higher levels are agnostic to what's happening below them. Lower levels receive a stream of bytes and pass it on. Higher levels don't care how messages are sent, they just want them to be sent.

HTTP is known as an application-level protocol. It relies on *TLS* (Transport Layer Security), which has replaced the better known *SSL* (Secure Socket Layer). TLS sits on top of TCP. On top of HTTP sit HTML, CSS, JavaScript, WebAssembly modules, images, video and other files. These layers do bleed together slightly though. HTTP will often redundantly include information provided at the TCP layer. HTML includes a mechanism to supplement or overwrite directives omitted or specified within HTTP.

[Figure 8.2](#) provides one view of the networking stack. Even that complicated picture is highly simplified. [Listing 8.3](#) and its output at [Listing 8.4](#)

Listing 8.1. Generating a HTTP message to `www.manning.com`

```
curl \                                ①
  --include \                          ②
  --head \                             ③
  https://www.manning.com/books/rust-in-action
```

- ① `curl` is a command-line utility for making requests over the Internet, especially HTTP
- ② Include the HTTP response codes
- ③ Use the HEAD method, which omits the message body. This shortens the length of the response, making it easier to show in [Listing 8.2](#).

Listing 8.2. Contents of an HTTP message

```
HTTP/1.1 200                                ①
Server: nginx/1.16.0
Date: Sun, 26 May 2019 04:26:26 GMT
Content-Type: text/html; charset=UTF-8      ②
Connection: keep-alive                     ③
X-Application-Context: application:production ④
Content-Language: en-US
X-Frame-Options: Deny
```

- ① This server supports version 1.1 of the HTTP protocol and this message has the 200 response code, which means everything went smoothly.
- ② The Content-Type header describes the format—and in this case the encoding—of the message body.
- ③ The Connection header relates to the underlying TCP/TLS connection. It's telling the client (curl) that it should hold the connection open if there are more requests to be made.
- ④ The host system has defined a custom header that has been passed along from whatever has generated the page.

In [Listing 8.1](#), we're only seeing what's happening at the HTTP layer. It's possible to ask for more output. That'll help identify what was actually sent.

Listing 8.3. Making an HTTP request to `www.manning.com` with the `curl` command-line utility, requesting verbose output.

```
curl -viI https://www.manning.com/books/rust-in-action
```

Listing 8.4. Example output from [Listing 8.3](#). Actual output will differ between `curl` versions, depending on how your `curl` was compiled and the details of Manning's hosting environment.

```
* Trying 35.166.24.88... ①
* TCP_NODELAY set ②
* Connected to www.manning.com (35.166.24.88) port 443 (#0) ③
* ALPN, offering h2 ④
* ALPN, offering http/1.1 ④
* successfully set certificate verify locations:
* CAfile: none
  Capath: /etc/ssl/certs ⑤
* TLSv1.3 (OUT), TLS handshake, Client hello (1): ⑥
* TLSv1.3 (IN), TLS handshake, Server hello (2): ⑥
* TLSv1.2 (IN), TLS handshake, Certificate (11): ⑥
* TLSv1.2 (IN), TLS handshake, Server key exchange (12): ⑥
* TLSv1.2 (IN), TLS handshake, Server finished (14): ⑥
* TLSv1.2 (OUT), TLS handshake, Client key exchange (16): ⑥
* TLSv1.2 (OUT), TLS change cipher, Client hello (1): ⑥
* TLSv1.2 (OUT), TLS handshake, Finished (20): ⑥
* TLSv1.2 (IN), TLS handshake, Finished (20): ⑥
* SSL connection using TLSv1.2 / ECDHE-RSA-AES256-GCM-SHA384 ⑥
* ALPN, server accepted to use http/1.1 ⑦
* Server certificate: ⑧
```

```

* subject: OU=Domain Control Validated; CN=*.manning.com
* start date: May 25 00:46:15 2019 GMT
* expire date: May 25 00:46:15 2020 GMT
* subjectAltName: host "www.manning.com" matched cert's "*.manning.com"
* issuer: C=US; ST=Arizona; L=Scottsdale; O=GoDaddy.com, Inc.;
OU=http://certs.godaddy.com/repository/; CN=Go Daddy Secure Certificate Authority - G2
* SSL certificate verify ok. ⑨
> HEAD /books/rust-in-action HTTP/1.1 ⑩
> Host: www.manning.com
> User-Agent: curl/7.61.0
> Accept: */*
>
< HTTP/1.1 200 ⑪
HTTP/1.1 200
< Server: nginx/1.16.0
Server: nginx/1.16.0
< Date: Sun, 26 May 2019 04:41:58 GMT
Date: Sun, 26 May 2019 04:41:58 GMT
< Content-Type: text/html; charset=UTF-8
Content-Type: text/html; charset=UTF-8
< Connection: keep-alive
Connection: keep-alive
< X-Application-Context: application:production
X-Application-Context: application:production
< Content-Language: en-US
Content-Language: en-US
< X-Frame-Options: Deny
X-Frame-Options: Deny
< Content-Security-Policy: frame-ancestors 'none'
Content-Security-Policy: frame-ancestors 'none'

<
* Connection #0 to host www.manning.com left intact

```

- ① DNS has resolved the `www.manning.com` host to the IP address `35.166.24.88` (DNS)
- ② TCP connection parameters being used by `curl` (Internal)
- ③ TCP connection is established, using the conventional port 443 for HTTPS (TCP)
- ④ ALPN parameters being used by `curl`. ALPN is an extension to TLS (Internal)
- ⑤ The location of SSL certificates that my computer, and by extension `curl`, trusts (Internal)
- ⑥ TLS negotiation process, where the two sides are attempting to decide which version of TLS and the encryption cyphers to use (TLS)
- ⑦ ALPN negotiation results (Internal)
- ⑧ Details from the SSL certificate (Internal)
- ⑨ Verification of the server's SSL certificate against the certificates that my

computer has stored (Internal)

⑩ Data sent by `curl` to the server (HTTP)

⑪ Response back to `curl` (HTTP)

HTTPS sits on top of TLS, which sits on top of TCP. One thing to note is that the web server itself is somewhat ignorant that content is being delivered over TLS. On lines 32 and 33 of [Listing 8.4](#), the server responds with the string “HTTP/1.1 200”. The encryption is wrapped around this content. As discussed earlier, the lower level (TLS) is ignorant of what it’s dealing with and the higher level (HTTP) doesn’t care about how it is being transported to the recipient.

Protocols don’t necessarily carry all of the semantics of their underlying layers with them, although shadows do tend to linger. For example, the HTTP header “Connection: keep-alive” on line 42 of [Listing 8.4](#) is a message that relates to the TCP layer.

TCP is used by HTTP because it also provides reliable, in-order delivery of messages. TCP also includes built-in mechanisms to handle overloading and service degradation. HTTP can rely on those guarantees and doesn’t need to implement them itself.

8.2 GENERATING AN HTTP GET REQUEST WITH REQUEST

Our first implementation will be with a high-level library that is focused on HTTP. We’ll be using `request`, as its focus is primarily on making it easy for Rust programmers to create an HTTP request.

Although it’s the shortest, the `request` implementation is the most feature complete. As well as being able to correctly interpret HTTP headers, it will also handle cases such as content redirects. Most importantly though, it understands how to handle TLS properly. In addition to expanded networking capabilities, `request` also validates the content’s encoding and ensures that it is sent to your application as a valid `String`. None of our lower-level implementations do any

of that.

Listing 8.5. Project structure for [Listing 8.7](#)

```
ch8-simple/  
├── src  
│   └── main.rs  
└── Cargo.toml
```

Listing 8.6. Crate metadata for [Listing 8.7](#) (ch8/ch8-simple/Cargo.toml)

```
[package]  
name = "ch8-simple"  
version = "0.1.0"  
authors = ["Tim McNamara <author@rustinaction.com>"]  
edition = "2018"  
  
[dependencies]  
request = "0.9"
```

Listing 8.7. Making an HTTP request with the `request` library (ch8/ch8-simple/src/main.rs)

```
use request;  
  
fn main() -> Result<(), Box<dyn std::error::Error>> { ①  
    let url = "http://www.rustinaction.com/";  
    let mut response = request::get(url)?;  
  
    let content = response.text()?;  
    print!("{}", content);  
  
    Ok::<(), Box<dyn std::error::Error>>()  
}
```

① The syntax for `Box<dyn std::error::Error>` is new. It represents a *trait object*.

If you've ever done any web programming, [Listing 8.7](#) should be straightforward. `request::get()` issues an HTTP GET request to the URL represented by `url`. The `response` variable holds a struct representing the server's response. The `response.text()` method returns a `Result` that provides access to the HTTP body, after validating that the contents are a legal `String`.

One question though - what on earth is the error side of the `Result` return type `Box<dyn std::error::Error>`? This is an example of a *trait object*, which enables Rust to support *polymorphism* at runtime. Trait objects are proxies for concrete types. The syntax `Box<dyn std::error::Error>` means a `Box`, a pointer, to any type that implements `std::error::Error`'s

Using a library that knows about HTTP allows our programs to omit many details:

- Knowing when to close the connection. HTTP has rules for telling each of the parties when the connection has ended. This isn't available to us when making requests by hand. Instead, we keep the connection open for as long as possible, and hope that the server will close
- Converting the byte stream to content. Rules for translating the message body from `[u8]` to `String` (or perhaps an image, video or some other content) are handled as part of the protocol. This can be tedious to handle by hand, as HTTP allows content to be compressed in several methods and encoded in several plain text formats.
- TCP port numbers. HTTP defaults to port 80. A library that is tailored for HTTP, such as `reqwest` can omit it. When we're building requests by hand with generic TCP crates, we need to be explicit.
- IP addresses. The TCP protocol doesn't actually know about domain names like `www.rustinaction.com`. The library is resolving the IP address for `www.rustinaction.com` on our behalf.

8.3 TRAIT OBJECTS

This section describes *trait objects* in detail. You will be developing the world's next best-selling fantasy role-playing game. If you would like to focus on networking, feel free to skip ahead to [Section 8.4](#).

There is a reasonable amount of jargon in the next several paragraphs. Brace yourself. You'll do fine.

Let's start by introducing trait objects by what they achieve and what they do, rather than focusing on what they are.

8.3.1 What do trait objects enable?

While they have several uses, they are immediately useful by allowing you to create containers of multiple types. Although players of your game may choose different races, and each race is defined in its own `struct`, you want them to be able to treat them as a single type. A `Vec<T>` won't work here, because we can't easily have types `T`, `U` and `V` wedged into `Vec<T>` without introducing some type of wrapper object.

8.3.2 What is a trait object?

Trait objects add a form of *polymorphism*, the ability to share an interface between types, to Rust via *dynamic dispatch*. Trait objects are similar to generics. Generics offer polymorphism via *static dispatch*. Choosing between generics and type objects typically involves a trade off between disk space and time. Generics involves more using disk space, with faster runtime. Trait objects use less space on disk, but incur a small runtime overhead, caused by pointer indirection.

Trait objects are *dynamically-sized types*, which means that they're always seen in the wild behind a pointer.

Trait objects have three forms of syntax: `&dyn Trait`, `&Trait`, and `Box<Trait>`. The `dyn` keyword is strongly encouraged and other forms are officially deprecated.

- `&dyn Trait`
- `&Trait`
- `Box<Trait>`
- `Box<dyn Trait>`

8.3.3 Creating a tiny role-playing game

[Listing 8.9](#) is the start of your game. Characters in the game can be one of three races: humans, elves and dwarves. They're represented by the `Human`, `Elf` and `Dwarf` structs respectively. Characters interact with things. Things are represented by the `Thing` type [\[40\]](#). `Thing` is an enum that currently represents swords and trinkets. There's only one form of interaction right now: enchantment.

Enchanting a thing involves calling the `enchant()` method:

```
character.enchant(&mut thing)
```

When enchantment is successful, `thing` glows brightly. When a mistake occurs, `thing` is transformed into a trinket.

Within [Listing 8.9](#), we create a party of characters with the following syntax:

```
let d = Dwarf {};  
let e = Elf {};  
let h = Human {};  
  
let party: Vec<&dyn Enchanter> = vec! [&d, &h, &e]; ①
```

① Although `d`, `e` and `h` are different types, using the type hint `&dyn Enchanter` tells the compiler to treat each value as a trait object. They now all have the same type.

Actually casting the spell involves choosing a spellcaster. We make use of the `rand` crate for that.

```
let spellcaster = party.choose(&mut rand::thread_rng()).unwrap();  
spellcaster.enchant(&mut it)
```

The `choose` method originates from the `rand::seq::SliceRandom` trait that is brought into scope in line 2. One of the party is chosen at random. She then attempts to enchant the object `it`.

Compiling and running [Listing 8.9](#) results in a variation of this:

Elf mutters incoherently. The Sword fizzes, then turns into a worthless trinket.

Listing 8.8. Crate metadata for the rpg project (ch8/ch8-rpg/Cargo.toml)

```
[package]
name = "rpg"
version = "0.1.0"
authors = ["Tim McNamara <author@rustinaction.com>"]
edition = "2018"

[dependencies]
rand = "0.7"
```

Listing 8.9. Example of a using a trait object, &Enchanter to enable a container to hold several types (ch8/ch8-rpg/src/main.rs)

```
use rand;
use rand::seq::SliceRandom;
use rand::Rng;

#[derive(Debug)]
struct Dwarf {}

#[derive(Debug)]
struct Elf {}

#[derive(Debug)]
struct Human {}

#[derive(Debug)]
enum Thing {
    Sword,
    Trinket,
}

trait Enchanter: std::fmt::Debug {
    fn competency(&self) -> f64;

    fn enchant(&self, thing: &mut Thing) {
        let probability_of_success = self.competency();
        let spell_is_successful = rand::thread_rng().gen_bool(probability_of_success); ①

        println!("{:?} mutters incoherently. ", self);
        if spell_is_successful {
            println!("The {:?} glows brightly.", thing);
        } else {
            println!("The {:?} fizzes, then turns into a worthless trinket.", thing);
            *thing = Thing::Trinket {};
        }
    }
}
```

```

impl Enchanter for Dwarf {
    fn competency(&self) -> f64 {
        0.5
    }
}
impl Enchanter for Elf {
    fn competency(&self) -> f64 {
        0.95
    }
}
impl Enchanter for Human {
    fn competency(&self) -> f64 {
        0.8
    }
}

fn main() {
    let mut it = Thing::Sword;

    let d = Dwarf {};
    let e = Elf {};
    let h = Human {};

    let party: Vec<&dyn Enchanter> = vec! [&d, &h, &e]; ⑤
    let spellcaster = party.choose(&mut rand::thread_rng()).unwrap();

    spellcaster.enchant(&mut it);
}

```

- ① `gen_bool()` generates a Boolean value, where `true` occurs in proportion to its argument. A value of 0.5 returns `true` 50% of the time.
- ② Dwarves are poor spell casters and their spells regularly fail.
- ③ Spells cast by elves rarely fail.
- ④ Humans are proficient at enchanting things. Mistakes are uncommon.
- ⑤ We can hold members of different types within the same `Vec` as they all implement the `Enchanter` trait.

Trait objects are a powerful construct in the language. In a sense, they provide a way to navigate Rust's rigid type system. As you learn about this feature in more detail, you will encounter some jargon. Trait objects are a form of *type erasure*. The compiler does not have access to the original type during the call to `enchant()`.

One of the frustrating things about Rust’s syntax for beginners is that traits objects and type parameters look very similar.

For example, consider these two lines:

```
use rand::Rng;
use rand::rngs::ThreadRng;
```

Although they both have something to do with random number generators, they’re quite different.

`rand::Rng` is a trait, `rand::rngs::ThreadRng` is a struct.

Trait objects make this distinction harder. When used as a function argument and similar places, the form `&Rng` is “a reference to something that implements `Rng`”, whereas `&ThreadRng` reads as “a reference to a `ThreadRng`”.

With time, the distinction between traits and types becomes easier to grasp. Types and traits are used in different places.

Common use cases for trait objects:

- creating collections of heterogeneous objects
- as a return value, they can enable functions to return multiple concrete types
- supporting *dynamic dispatch*, whereby the function that is called is determined at run-time, rather than compile time

Trait objects are not objects in the sense that an object-oriented programmer would understand. They’re perhaps closer to a mixin class. Trait objects don’t exist on their own. They’re agents of some other type.

An alternative analogy would be a singleton object that is delegated with some authority by another concrete type. In [Listing 8.9](#), the `&Enchanter` is delegated to act on behalf of three concrete types.

[\[40\]](#) Naming is hard.

8.4 TCP

Dropping down from HTTP, we encounter TCP. Rust’s standard library provides us with cross-platform tools for making TCP requests. Let’s use them.

Listing 8.10. Project structure for [Listing 8.12](#)

```
ch8-stdlib
├── src
│   └── main.rs
└── Cargo.toml
```

Listing 8.11. Project metadata for [Listing 8.12](#) (ch8/ch8-stdlib/Cargo.toml)

```
[package]
name = "ch8-stdlib"
version = "0.1.0"
authors = ["Tim McNamara <author@rustinaction.com>"]
edition = "2018"

[dependencies]
```

Listing 8.12. Using the Rust standard library to construct an HTTP GET request using `std::net::TcpStream` (ch8/ch8-stdlib/src/main.rs)

```
use std::io::prelude::*;
use std::net::TcpStream;

fn main() -> std::io::Result<()> {
    let mut connection = TcpStream::connect("www.rustinaction.com:80")?; ①

    connection.write_all(b"GET / HTTP/1.0")?; ②
    connection.write_all(b"\r\n")?; ③
    connection.write_all(b"Host: www.rustinaction.com")?; ④
    connection.write_all(b"\r\n\r\n")?; ⑤

    std::io::copy(&mut connection, &mut std::io::stdout())?; ⑥

    Ok(())
}
```

- ① We need to specify the port (80) explicitly, `TcpStream` does not know that this will become a HTTP request.
- ② GET is the HTTP method, / is the resource we're attempting to access and HTTP/1.0 is the protocol version we're requesting. Why 1.0? It does not support "keep alive" requests, which will allow our stream to close without difficulty.
- ③ In many networking protocols, `\r\n` is how a new line is signified.
- ④ The hostname provided on line 5 is actually discarded once it is converted to

an IP address. The Host HTTP header allows the server to know which host we're connecting to.

⑤ Two blank lines signifies that we've finished the request.

⑥ `std::io::copy()` streams bytes from a Reader to a Writer.

8.4.1 What is a “port number”?

Port numbers are purely virtual. They are simply `u16` values. Port numbers allow a single IP address to host multiple services.

8.4.2 Converting a hostname to an IP address

So far, we've been providing the hostname `www.rustinaction.com` to Rust. But, to send messages over the Internet, the Internet Protocol requires IP addresses. TCP knows nothing about domain names. To convert a domain name to an IP address, we rely on the Domain Name System (DNS) and its process called *domain name resolution*.

We're able to resolve names by asking a server. They're able to recursively ask other servers. DNS requests can be made over TCP, including encrypted with TLS, but are also sent over UDP. We'll be using DNS here, because it's more useful for learning purposes.

To explain how the translating from a domain name to an IP address works, we'll create a small application that does the translation. It's called `resolve`, for which [Listing 8.16](#) lists its source code. `resolve` makes use of public DNS services, but you are able to add your own easily with the `-s` argument.

Public DNS providers

At the time of writing, several companies provide DNS servers for public use. Any of the IP addresses listed here should offer roughly equivalent service.

- 1.1.1.1 and 1.0.0.1 by Cloudflare
- 8.8.8.8 and 8.8.4.4. by Google

- 9.9.9.9 by Quad9, founded by IBM
- 64.6.64.6 and 64.6.65.6 by VeriSign

`resolve` only understands a small portion of DNS protocol, but that portion is sufficient for our purposes. The project makes use of an external crate, `trust-dns`, to perform the hard work. `trust-dns` implements the RFC 1035—which defines DNS—and several later RFCs quite faithfully and uses terminology derived from it.

Some of the terms that are useful to understand are outlined in [Table 8.1](#).

Table 8.1. Terms that are used in RFC 1035, the `trust_dns` crate and Listing 8.16 and how they interlink

Term	Definition	Representation in Code
Domain Name	A domain name is almost what you probably think of when you use the term “domain name” in everyday language. The technical definition includes some special cases, such as the “root” domain which is encoded as a single dot and domain names needing to be case-insensitive.	Defined in <code>trust_dns::domain::Name</code> <pre>pub struct Name { is_fqdn: bool, ① labels: Vec<Label>, }</pre> ① “fqdn” stands for “fully-qualified domain name”
Message	A Message is a container format for both requests to DNS servers (called <i>queries</i>) and responses back to clients (called <i>answers</i>). Messages must contain a header, but other fields are not required. Message represents this with several <code>Vec<T></code> fields. They do not need to be wrapped in <code>Option</code> to represent missing	Defined in <code>trust_dns::domain::Name</code> <pre>struct Message { header: Header, queries: Vec<Query>, answers: Vec<Record>, name_servers: Vec<Record>, additional: Vec<Record>, sig0: Vec<Record>, ① edns: Option<Edns>, ② }</pre> ① “sig0” is a cryptographically signed record used for

	values as their length can be 0.	verifying the message's integrity. It is defined in RFC 2535. ② “edns” indicates whether the message includes “Extended DNS”
Message Type	A message type identifies the message as a query or an answer. Queries can also be updates, which is functionality that our code ignores.	Defined in <code>trust_dns::op::MessageType</code> <pre>pub enum MessageType { Query, Response, }</pre>
Message ID	A number that is used for senders to link queries and answers.	<pre>u16</pre>
Resource Record Type	The resource record type refers to the DNS codes that you've probably encountered if you've ever configured a domain name. Of note is how <code>trust_dns</code> handles invalid codes. The <code>RecordType</code> enum contains an <code>Unknown(u16)</code> variant that can be used for codes that it doesn't understand.	Defined in <code>trust_dns::rr::record_type::RecordType</code> <pre>pub enum RecordType { A, AAAA, ANAME, ANY, // ... Unknown(u16), ZERO, }</pre>
Query	A Query holds the domain name and the record type that we're seeking the DNS details for. They also describe the DNS class, allows queries to distinguish between messages sent over the Internet from other transport protocols.	Defined in <code>trust_dns::op::Query</code> <pre>pub struct Query { name: Name, query_type: RecordType, query_class: DNSClass, }</pre>
Op Code	An <code>OpCode</code> is, in some	Defined in <code>trust_dns::op::OpCode</code>

	sense, a subtype of <code>MessageType</code>. This is an extensibility mechanism that allows future functionality. For example, RFC 1035 defines the "Query" and "Status" op codes but others were defined later. "Notify" and "Update" are defined by RFC 1996 and RFC 2136, respectively.	<pre>pub enum OpCode { Query, Status, Notify, Update, }</pre>
--	--	---

An unfortunate consequence of the protocol—which is a consequence of reality, I suppose—is that there are many options, types and sub-types involved. The process of constructing a message that asks, “Dear DNS server, what is the IPv4 address for `domain_name`?”, is shown in [Listing 8.13](#).

Listing 8.13. Constructing a DNS message in Rust with `trust-dns` to request an IPv4 address for `domain_name`. (Excerpt from [Listing 8.16](#))

```
let mut msg = Message::new();           ①
msg
    .set_id(rand::random:::<u16>())      ②
    .set_message_type(MessageType::Query)
    .set_op_code(OpCode::Query)
    .add_query(Query::query(domain_name, RecordType::A) ③
    .set_recursion_desired(true);        ④
```

- ① A Message has little content of its own, its main role is to be a container for the query that we’re adding a few lines down.
- ② We’ll just use a random number as our message ID. The expression `rand::random:::<u16>()` can be read as “call `random()` from the `rand` crate, returning a `u16` value”.
- ③ To ask for multiple record types, such as AAAA for IPv6 addresses, add extra queries.
- ④ Request that the DNS server asks DNS servers for us on our behalf if it doesn’t know the answer.

We’re now in a position where we can meaningfully inspect the code. It has the

following structure: parse command line arguments, build a DNS message using `trust_dns` types, convert the structured data into a stream of bytes and then send those bytes across the wire. Then we need to accept the response from the server, decode the incoming bytes and print out the result.

The error handling is relatively ugly, with many calls to `unwrap()` and `expect()`. We'll address that problem shortly during [Section 8.5](#).

The end process is a command-line application that's quite simple. Running `resolve` involves very little ceremony. Given a domain name, `resolve` provides an IP address:

```
$ resolve www.rustinaction.com
35.185.44.232
```

[Listing 8.14](#) outlines the layout of the files required to build `resolve`. The listings [Listing 8.15](#) and [Listing 8.16](#) are the project's source code.

While you are experimenting with the project, you may wish to use some features of `cargo run` to speed up your process:

```
$ cargo run -q -- www.rustinaction.com ①
35.185.44.232
```

① `cargo` sends any arguments to the right of `--` to the executable that it is compiling. The `-q` option mutes any intermediate output.

Listing 8.14. Crate structure for `resolve`

```
ch8/ch8-resolve
├── Cargo.toml
└── src
    └── main.rs
```

Listing 8.15. Crate metadata for `resolve` (ch8/ch8-resolve/Cargo.toml)

```
[package]
```

```

name = "resolve"
version = "0.1.0"
authors = ["Tim McNamara <author@rustinaction.com>"]
edition = "2018"

[dependencies]
rand = "0.6"
clap = "2.33"

[dependencies.trust-dns]
version = "0.16"
default_features = false

```

Listing 8.16. resolve: a simple DNS command-line utility that can resolve the IP address for domain names (ch8/ch8-resolve/src/main.rs)

```

use std::net::{SocketAddr, UdpSocket};
use std::time::Duration;

use clap::{App, Arg};
use rand;
use trust_dns::op::{Message, MessageType, OpCode, Query};
use trust_dns::rr::domain::Name;
use trust_dns::rr::record_type::RecordType;
use trust_dns::serialize::binary::*;

fn main() {
    let app = App::new("resolve")
        .about("A simple to use DNS resolver")
        .arg(Arg::with_name("dns-server").short("s").default_value("1.1.1.1"))
        .arg(Arg::with_name("domain-name").required(true))
        .get_matches();

    let domain_name_raw = app.value_of("domain-name").unwrap();
    let dns_server_raw = app.value_of("dns-server").unwrap();

    let mut request_as_bytes: Vec<u8> = Vec::with_capacity(512); ①
    let mut response_as_bytes: [u8; 512] = [0; 512]; ②

    let domain_name = Name::from_ascii(&domain_name_raw).unwrap(); ③
    let mut msg = Message::new(); ④
    msg
        .set_id(rand::random::<u16>())
        .set_message_type(MessageType::Query) ⑤
        .add_query(Query::query(domain_name, RecordType::A))
        .set_op_code(OpCode::Query)
        .set_recursion_desired(true); ⑥

    let mut encoder = BinEncoder::new(&mut request_as_bytes); ⑦
    msg.emit(&mut encoder).unwrap(); ⑦

    let localhost = UdpSocket::bind("0.0.0.0:0").expect("cannot bind to local
socket"); ⑧
    let timeout = Duration::from_secs(3);

```

```

localhost.set_read_timeout(Some(timeout)).unwrap();
localhost.set_nonblocking(false).unwrap();

let dns_server: SocketAddr = format!("{:53", dns_server_raw).parse()
    .expect("invalid address");
let _amt = localhost.send_to(&request_as_bytes, dns_server)
    .expect("socket misconfigured");

let (_amt, _remote) = localhost.recv_from(&mut response_as_bytes)
    .expect("timeout reached");

let dns_message = Message::from_vec(&response_as_bytes)
    .expect("unable to parse response");

for answer in dns_message.answers() {
    if answer.record_type() == RecordType::A {
        let resource = answer.rdata();
        let ip = resource.to_ip_addr().expect("invalid IP address received");
        println!("{}", ip.to_string());
    }
}
}

```

- ① We allocate 512 bytes, even though this isn't strictly necessary. 512 is the number of bytes specified by DNS when messages are sent over UDP.
- ② An implementation detail of `recv_from()` (called later on) is that it relies on the length of the buffer provided to indicate how many bytes to read from the network. `Vec<T>` created with `with_capacity()` has a length of 0, leading to an early return.
- ③ Convert the string to a data type that represents domain names
- ④ A `Message` represents a DNS message, which is a container for queries (and other information, such as answers)
- ⑤ Specify that this is a DNS query, not a DNS answer (they both have the same representation over the wire)
- ⑥ Ask the DNS server to query any DNS servers that it knows about if it doesn't know the answer itself.
- ⑦ Convert the `Message` type into raw bytes with the `BinEncoder`.
- ⑧ `0.0.0.0:0` means “listen to all addresses on a random port”, and the actual port will be selected by the operating system.

UDP “connections”

UDP does not have a notion of a long-lived connection. Put another way, UDP does not have duplex

communication. That means to ask a question of a DNS server, both parties must act as clients and servers from the point of view of UDP.

Time to recap. Our overall task is to make HTTP requests. HTTP is built on TCP. But we only have a domain name (`www.rustinction.com`) when we're making the request, so we need to use DNS. But DNS is primarily delivered over UDP. So we've needed to take a diversion and learn about UDP. Now it's time to return to TCP. Before we're able to though, we need to learn how to combine error types that emerge from multiple dependencies.

8.5 ERGONOMIC ERROR HANDLING FOR LIBRARIES

Rust's error handling is safe and sophisticated. However, it offers a few challenges. When a function incorporates `Result` types from two upstream crates, the `?` operator no longer works because it can only understand a single type. This will prove to be important when we refactor our domain resolution code to work alongside our TCP code. This section discusses some of those challenges and some strategies for managing them.

8.5.1 Issue: unable to return multiple error types

Returning a `Result<T, E>` works great when there is a single error type `E`. But things become more complicated when we want to work with multiple error types.



Tip

For single-file examples, compile the code with `rustc <filename>`, rather than using `cargo build`.

If an example is named `io-error.rs`, then the shell command is: `rustc io-error.rs && ./io-error[.exe]`

To start with, let's look at a small example that covers the easy case of a single error type. We'll try to open a file that does not exist.

When run, [Listing 8.17](#) prints out a short message in Rust syntax:

```
Error: Os { code: 2, kind: NotFound, message: "No such file or directory" }
```

We won't win any awards for user experience here, but we get a chance to learn a new language feature.

Listing 8.17. A Rust program that always produces an I/O error (ch8/misc/io-error.rs)

```
use std::fs::File;

fn main() -> Result<(), std::io::Error> {
    let _f = File::open("invisible.txt"?);

    Ok(())
}
```

Now let's introduce an other error type into `main()`. This will produce a compiler error. We'll then work through some options to get the code to compile.

Listing 8.18. A function that attempts to return multiple Result types (ch8/misc/multierror.rs)

```
use std::fs::File;
use std::net::Ipv6Addr;

fn main() -> Result<(), std::io::Error> {
    let _f = File::open("invisible.txt"?);           ①
    let _localhost = "::1".parse::<Ipv6Addr>()?;    ②

    Ok(())
}
```

① `File::open()` returns `Result<(), std::io::Error>`

② `".parse::<Ipv6Addr>()` returns `Result<Ipv6Addr, std::net::AddrParseError>`

Compiling [Listing 8.18](#) (use `rustc multierror.rs`) produces quite a stern error message:


```

error[E0277]: the trait bound `std::io::Error: std::convert::From<std::net:
:AddrParseError>` is not satisfied
--> multierror.rs:6:22
|
6 |     let _localhost = "::1".parse::<Ipv6Addr>()?; ①
|                                     ^^^^^^^^^^^^^ the trait `std::convert:
:From<std::net::AddrParseError>` is not implemented for `std::io::Error`
|
= help: the following implementations were found:
      <std::io::Error as std::convert::From<std::ffi::NulError>>
      <std::io::Error as std::convert::From<std::io::ErrorKind>>
      <std::io::Error as std::convert::From<std::io::IntoInnerError<W>>>
= note: required by `std::convert::From::from`

error: aborting due to previous error

For more information about this error, try `rustc --explain E0277`.

```

The error message can be difficult to interpret if you don't know what the question mark operator (?) is doing. What are there multiple messages about `std::convert::From`? The ? operator is syntactic sugar for the `try!` macro. `try!` performs two functions. When it detects `Ok(value)`, the expression evaluates to `value`. When `Err(err)` occurs, `try!/?` returns early after attempting to convert `err` to the error type defined in the calling function.

In Rust-like pseudo-code, the `try!` macro could be defined as:

```

macro try {
  match expression {
    Result::Ok(val) => val,                                ①
    Result::Err(err) => {
      let converted = convert::From::from(err);            ②
      return Result::Err(converted);                       ③
    }
  });
}

```

- ① When an expression matches `Result::Ok(val)`, use `val`
- ② When it matches `Result::Err(err)`, convert it to the outer function's error type and then return early.
- ③ This return returns from the calling function, not the `try!` macro itself.

Looking at [Listing 8.18](#) again, we can see the macro in action.

```

fn main() -> Result<(), std::io::Error> {

```

```

let _f = File::open("invisible.txt"?; ①
let _localhost = "::1".parse::<Ipv6Addr>()?; ②

    Ok(())
}

```

- ① `File::open()` returns `std::io::Error` itself, so no conversion is necessary.
- ② `".parse()` presents `?` with a `std::net::AddrParseError`. We don't define how to convert `std::net::AddrParseError` to `std::io::Error`, so program fails to compile.

As well as saving you from needing to use explicit pattern matching to extract the value or return an error, the `?` operator also attempts to convert its argument into an error type if required. Because the signature of `main()` is `main() → Result<(), std::io::Error>`, Rust is attempting to convert the `std::net::AddrParseError` produced by `parse::<Ipv6Addr>()` into a `std::io::Error`. Don't worry though, we can fix this!

Earlier in the chapter at [Section 8.3](#), we introduced trait objects. Now we'll be able to put them to good use. Using `Box<dyn Error>` as the error variant in the `main()` function allows us to progress. The `dyn` keyword is short for "dynamic", implying that there is a runtime cost for this flexibility.

Listing 8.19. Output from [Listing 8.20](#)

```
Error: Os { code: 2, kind: NotFound, message: "No such file or directory" }
```

I suppose it's a limited form of progress. We've circled around to the error that we started with. But we've passed through the compiler error, which is what we wanted. Onwards.

Listing 8.20. Using a trait object in a return value to simplify error handling when errors originate from multiple upstream crates. (ch8/misc/traitemerror.rs)

```

use std::fs::File;
use std::error::Error;
use std::net::Ipv6Addr;

```

```
fn main() -> Result<(), Box<dyn Error>> {           ①
    let _f = File::open("invisible.txt"?;           ②
    let _localhost: Ipv6Addr = "::1".parse()?;      ③

    Ok(())
}
```

① `Box<dyn Error>` is a trait object, which represents any type that implements `Error`.

Wrapping trait objects in `Box` is necessary because their size (in bytes on the stack) is unknown at compile-time. In the case of [Listing 8.20](#), the trait object might originate from either `File::open()` or `:::1".parse()`. What actually happens depends on the circumstances encountered at runtime. A `Box` has a known size on the stack. Its *raison d'être* is to point to things that don't, such as trait objects.

8.5.2 Wrapping downstream errors by defining our own error type

The problem that we are attempting to solve is that each of our dependencies defines its own error type. Multiple error types in one function prevent returning `Result`. The first strategy we looked at was to use trait objects. But trait objects have a potentially significant downside.

Using trait objects is also known as *type erasure*. Rust is no longer able to know that an error has originated upstream. Using `Box<dyn Error>` as the error variant of a `Result` means that the upstream error types are, in a sense, lost. The original errors are now converted to exactly the same type.

It is possible to retain the upstream errors, but requires some more work on our behalf. We need to bundle upstream errors in our own type. When the upstream errors are needed later, say for reporting errors to the user, it's possible to extract them with pattern matching.

Here is the outline:

- Define an enum that includes the upstream errors as variants
- Annotate the enum with `#[derive(Debug)]`
- Implement `Display`
- Implement `Error`, which almost comes for free because we have implemented `Debug` and `Display`
- Use `map_err()` in your code to convert the upstream error to your omnibus error type. `map_err()` isn't a function that you have encountered before. We'll explain what it does when we get there.

It's possible to stop there, but there's an optional extra step that improves the ergonomics:

- Implement `std::convert::From` to remove the need to call `map_err()`

To begin, let's start back with the code that we know fails from [Listing 8.18](#).

```
use std::fs::File;
use std::net::Ipv6Addr;

fn main() -> Result<(), std::io::Error> {
    let _f = File::open("invisible.txt")?;
    let _localhost = "::1".parse::<Ipv6Addr>()?;

    Ok(())
}
```

This fails because `"".parse::<Ipv6Addr>()` does not return a `std::io::Error`. What we want to end up with is code that looks a little more like [Listing 8.21](#):

Listing 8.21. Hypothetical example of the kind of code that we would like to write

```
use std::fs::File;
use std::io::Error;           ①
use std::net::AddrParseError; ①
use std::net::Ipv6Addr;

enum UpstreamError{
    IO(std::io::Error),
    Parsing(AddrParseError),
}
```

```
fn main() -> Result<(), UpstreamError> {
    let _f = File::open("invisible.txt")?.maybe_convert_to(UpstreamError);
    let _localhost = "::1".parse::<Ipv6Addr>()?.maybe_convert_to(UpstreamError);

    Ok(())
}
```

① Bring upstream errors into local scope.

DEFINE AN ENUM THAT INCLUDES THE UPSTREAM ERRORS AS VARIANTS

The first thing to do is to return a type that can hold the upstream error types. In Rust, an enum works well. [Listing 8.21](#) does not compile, but does do this step. We'll tidy up the imports slightly.

```
use std::io;
use std::net;

enum UpstreamError{
    IO(io::Error),
    Parsing(net::AddrParseError),
}
```

Let's proceed.

ANNOTATE THE ENUM WITH `#[DERIVE(DEBUG)]`

Easy. That's a single line change. We add `#[derive(Debug)]` to line 4. The best kind of change.

```
use std::io;
use std::net;

#[derive(Debug)]
enum UpstreamError{
    IO(io::Error),
    Parsing(net::AddrParseError),
}
```

IMPLEMENT `STD::FMT::DISPLAY`

We'll cheat slightly and implement `Display` by simply using `Debug`. We know that this is available to us, because errors must define `Debug`.

```
use std::fmt;
use std::io;
use std::net;

#[derive(Debug)]
enum UpstreamError{
    IO(io::Error),
    Parsing(net::AddrParseError),
}

impl fmt::Display for UpstreamError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{:?}", self)
    }
}
```

① Implement `Display` in terms of `Debug` via the `"{:?}"` syntax.

IMPLEMENT `STD::ERROR::ERROR`

Another easy change. Excellent.

```
use std::error; ①
use std::fmt;
use std::io;
use std::net;

#[derive(Debug)]
enum UpstreamError{
    IO(io::Error),
    Parsing(net::AddrParseError),
}

impl fmt::Display for UpstreamError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{:?}", self)
    }
}

impl error::Error for UpstreamError { } ②
```

① Bring the `std::error::Error` trait into local scope

② Defer to default method implementations. The compiler will fill in the blanks.

The `impl` block is—well, we can rely on default implementations provided by the compiler—especially terse. There are default implementations of every method defined by `std::error::Error`, so we’re asking the compiler to do all of the work for us.

USE `MAP_ERR()` IN YOUR CODE TO CONVERT THE UPSTREAM ERROR TO YOUR OMNIBUS ERROR TYPE

Back at [Listing 8.21](#), we wanted to have a `main()` that looks like this:

```
fn main() -> Result<(), UpstreamError> {
    let _f = File::open("invisible.txt")?.maybe_convert_to(UpstreamError);
    let _localhost = "::1".parse::<Ipv6Addr>()?.maybe_convert_to(UpstreamError);

    Ok(())
}
```

I can’t offer you that, but I can give you this:

```
fn main() -> Result<(), UpstreamError> {
    let _f = File::open("invisible.txt").map_err(UpstreamError::IO)?;
    let _localhost = "::1".parse::<Ipv6Addr>().map_err(UpstreamError::Parsing)?;

    Ok(())
}
```

This new code works! Here’s how. The `map_err()` function maps an error to a function. Variants of our `UpstreamError` enum can be used as functions here. The `?` operator needs to be at the end. Otherwise the function may have returned before the code has a chance to convert the error.

[Listing 8.23](#) provides the new code. When run, it produces this message to the console:

Listing 8.22. Output from [Listing 8.23](#)

```
Error: IO(0s { code: 2, kind: NotFound, message: "No such file or directory" })
```

Listing 8.23. Wrapping upstream errors in our own type to retain

type safety (ch8/misc/wraperror.rs)

```
use std::io;
use std::fmt;
use std::net;
use std::fs::File;
use std::net::Ipv6Addr;

#[derive(Debug)]
enum UpstreamError {
    IO(io::Error),
    Parsing(net::AddrParseError),
}

impl fmt::Display for UpstreamError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{:?}", self)
    }
}

impl error::Error for UpstreamError { }

fn main() -> Result<(), UpstreamError> {
    let _f = File::open("invisible.txt").map_err(UpstreamError::IO)?;
    let _localhost = "::1".parse::<Ipv6Addr>().map_err(UpstreamError::Parsing)?;

    Ok(())
}
```

It's also possible to remove the calls to `map_err()`. To enable that, we need to implement `From`.

IMPLEMENT `std::convert::From` TO REMOVE THE NEED TO CALL `MAP_ERR()`

The `std::convert::From` trait has a single required method, `from()`. We need two `impl` blocks to enable our two upstream error types to be convertible.

```
impl From<io::Error> for UpstreamError {
    fn from(error: io::Error) -> Self {
        UpstreamError::IO(error)
    }
}

impl From<net::AddrParseError> for UpstreamError {
    fn from(error: net::AddrParseError) -> Self {
        UpstreamError::Parsing(error)
    }
}
```


Now the `main()` returns to a simple form of itself:

```
fn main() -> Result<(), UpstreamError> {
    let _f = File::open("invisible.txt"?);
    let _localhost = "::1".parse::<Ipv6Addr>()?;

    ok(())
}
```

The full code listing is provided by [Listing 8.24](#). Implementing `From` places the burden of extra syntax on the library writer. It results in a much easier experience for people using your crate.

Listing 8.24. Implementing `std::convert::From` for our wrapper error type to simplify its use by downstream programmers (ch8/misc/wraperror2.rs)

```
use std::io;
use std::fmt;
use std::net;
use std::fs::File;
use std::net::Ipv6Addr;

#[derive(Debug)]
enum UpstreamError {
    IO(io::Error),
    Parsing(net::AddrParseError),
}

impl fmt::Display for UpstreamError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{:?}", self) // <1> Implement Display in terms of Debug
    }
}

impl error::Error for UpstreamError { }

impl From<io::Error> for UpstreamError {
    fn from(error: io::Error) -> Self {
        UpstreamError::IO(error)
    }
}

impl From<net::AddrParseError> for UpstreamError {
    fn from(error: net::AddrParseError) -> Self {
        UpstreamError::Parsing(error)
    }
}

fn main() -> Result<(), UpstreamError> {
    let _f = File::open("invisible.txt").map_err(UpstreamError::IO)?;
```

```
let _localhost = "::1".parse::<Ipv6Addr>().map_err(UpstreamError::Parsing)?;  
    Ok(())  
}
```

8.5.3 Cheat with `unwrap()` and `expect()`

The final approach for dealing with multiple error types is to use `unwrap()` and `expect()`. This is a reasonable approach when writing a `main()` function, but isn't recommended for library authors. Your users don't want their programs to crash because of things outside of their control.

Now that we have the tools to handle multiple error types in a function, we can continue our journey.

8.6 MAC ADDRESSES

Several pages ago now in [Listing 8.16](#), you implemented a DNS resolver. That enables conversions from a host name, such as `www.rustinaction.com`, to an IP address. Now that we have an IP address to connect to, we want to connect to it.

The Internet Protocol enables devices to contact each other via their IP addresses. But that's not sufficient. Networking using the Ethernet standards—which, almost everything developed since 1998 or so uses-- a unique identifier for every piece of hardware that's connected.

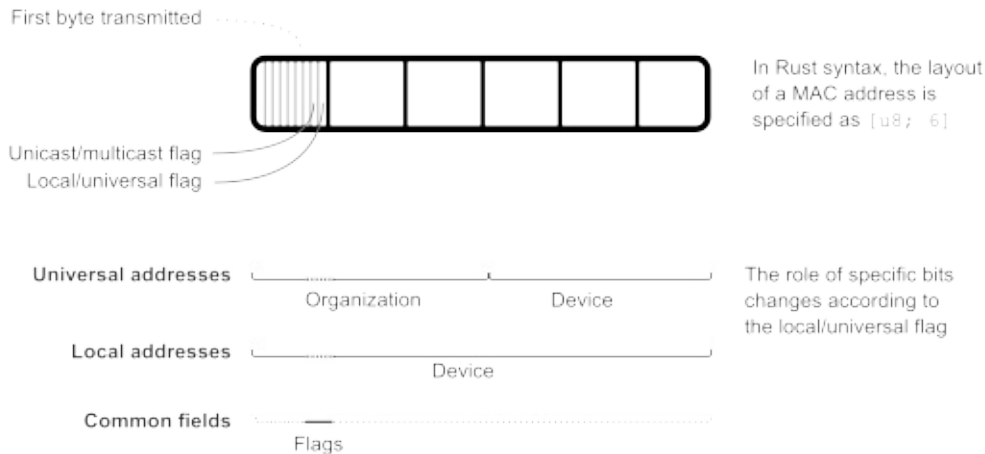
Why a second number? The answer is partially technical and partially historic. Ethernet networking and the Internet started life independently. Ethernet's focus was local area networks. The Internet was developed to enable communication between networks. Ethernet is the addressing system understood by devices that share a physical link. Or a radio link, in the case of WiFi, Bluetooth and other wireless technologies. Perhaps a better way to express this is that MAC addresses are used by devices that share electrons.

IP addresses are hierarchical, but MAC addresses are not. Addresses appearing close together numerically are not close together physically or organizationally.

MAC addresses are 48 bits (6 bytes) wide. IP addresses are 32 bits (4 bytes) for IPv4 and 128 bits (16 bytes) for IPv6.

Figure 8.3. MAC address in-memory layout

MAC Address Layout



There are 2 forms of MAC address:

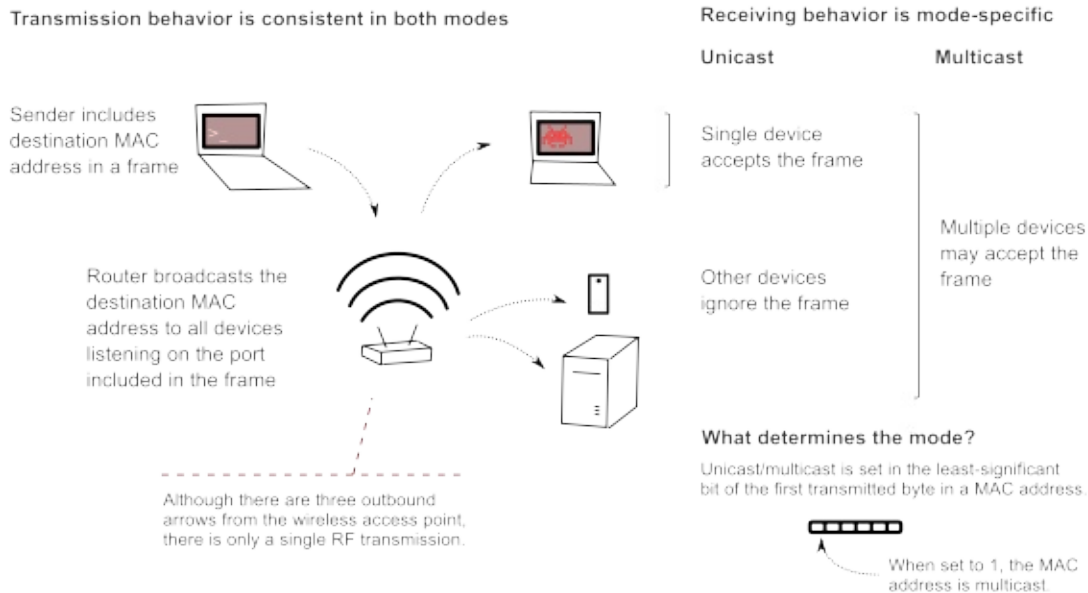
- *universally administered addresses* or *universal* are set when devices are manufactured. Manufacturers use a prefix that they are assigned by the IEEE Registration Authority and a scheme of their choosing for the remaining bits.
- *locally administered addresses* or *local* allow devices to create their own MAC address without registration. When setting a device's MAC address yourself in software, you should make sure that your address is set to the local form.

MAC addresses have two modes: *unicast* and *multicast*. Their transmission behavior is identical. The distinction is made when a device makes a decision whether to accept a *frame*. A frame is a term used by the Ethernet protocol for a byte slice at this level. Analogies to frame include the terms packet, wrapper and envelope.

Figure 8.4. The differences between multicast and unicast MAC

addresses

Unicast vs Multicast MAC Addresses



Unicast addresses are intended to transport information between two points that are in direct contact, say between a laptop and a router. Wireless access points complicate matters somewhat, but don't change the fundamentals. A multicast address can be accepted

The term *unicast* is somewhat misleading. There is a broadcast step. Unicast addresses differ in which devices take action on them, not which data is transmitted over the wire (or through the radio waves).

8.6.1 Generating MAC addresses

When we begin talking raw TCP later at [Listing 8.36](#), we'll be creating a virtual hardware device. To convince anything to talk to us, we need to learn how to assign our virtual device a MAC address. The `macgen` project at [Listing 8.26](#) generates MAC addresses.

Listing 8.25. Crate metadata for `macgen` (`ch8/ch8-mac/Cargo.toml`)

```
[package]
```

```

name = "ch8-macgen"
version = "0.1.0"
authors = ["Tim McNamara <author@rustinaction.com>"]
edition = "2018"

[dependencies]
rand = "0.7"

```

Listing 8.26. macgen, a MAC address generator (ch8/ch8-mac/src/main.rs)

```

extern crate rand;

use rand::RngCore;
use std::fmt;
use std::fmt::Display;

#[derive(Debug)]
struct MacAddress([u8; 6]); ①

impl Display for MacAddress {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        let octet = &self.0;
        write!(
            f,
            "{:02x}::{:02x}::{:02x}::{:02x}::{:02x}::{:02x}", ②
            octet[0], octet[1], octet[2], octet[3], octet[4], octet[5] ②
        )
    }
}

impl MacAddress {
    fn new() -> MacAddress {
        let mut octets: [u8; 6] = [0; 6];
        rand::thread_rng().fill_bytes(&mut octets);
        octets[0] |= 0b_0000_0011; ③
        MacAddress { 0: octets }
    }

    fn is_local(&self) -> bool {
        (self.0[0] & 0b_0000_0010) == 0b_0000_0010
    }

    fn is_unicast(&self) -> bool {
        (self.0[0] & 0b_0000_0001) == 0b_0000_0001
    }
}

fn main() {
    let mac = MacAddress::new();
    assert!(mac.is_local());
    assert!(mac.is_unicast());
    println!("mac: {}", mac);
}

```

- ① Use the newtype pattern to wrap a bare array without any extra overhead
- ② Convert each byte to hexadecimal notation
- ③ Force MAC to be locally assigned and unicast by setting the two least-significant bits to 1 with the *bitwise OR assignment operator*

The code from [Listing 8.26](#) should feel legible. Line 25 contains some relatively obscure syntax though. `octets[0] |= 0b_0000_0011` coerces the two flag bits described at [Figure 8.3](#) to the 1 state. That designates every MAC address we generate as locally assigned and unicast.

8.7 IMPLEMENTING STATE MACHINES WITH RUST'S ENUMS

Another pre-requisite for handling network messages is being able to define a state machine. Our code needs to adapt to changes in connectivity.

[Listing 8.36](#) contains a state machine, implemented with a loop, a match and a Rust enum. Because of Rust's expression-based nature, control flow operators also return values. Every time around the loop, the state is mutated in place.

Listing 8.27. Pseudo-code state machine implementation with a repeated match on a enum work together

```
enum HttpState {
    Connect,
    Request,
    Response,
}

loop {
    state = match state {
        HttpState::Connect if !socket.is_active() => {
            socket.connect();
            HttpState::Request
        }
        HttpState::Request if socket.may_send() => {
            socket.send(data);
            HttpState::Response
        }
        HttpState::Response if socket.can_recv() => {
            received = socket.recv();
            HttpState::Response
        }
    }
}
```

```

    }
    HttpState::Response if !socket.may_recv() => {
        break;
    }
    _ => state,
}
}

```

More advanced methods to implement finite state machines do exist. This is the simplest, however. We'll be making use of it in [Listing 8.36](#). The best enable the transitions to become part of the type system itself.

8.8 RAW TCP

But we're still far too high level! To dig deeper though, we're going to need to get some assistance from the operating system.

Integrating with the raw TCP packets typically requires root/superuser access. The operating system starts to get quite grumpy when an unauthorized user asks to make raw network requests. We can get around this (on Linux) by creating a proxy device that non-super users are allowed to communicate with directly.

Don't have Linux?

If you're running another operating system, there are many virtualization options available.

- The Multipass project provides very fast Ubuntu virtual machines on macOS and Windows hosts
- On Windows, another option to look into is "WSL", the "Windows Subsystem for Linux"
- Oracle VirtualBox is an open source project with excellent support for many host operating systems

8.9 CREATING A VIRTUAL NETWORKING DEVICE

The `ip` utility can create TUN/TAP devices. The higher level `tunctl` utility may also be useful here. To create a device called `tap-rust`, execute the command in

[Listing 8.28](#) in your Linux console:

Listing 8.28. Command to create a TAP device on Linux 2.2 and above

```
sudo \           ①  
ip tuntap \      ②  
  add \          ③  
  mode tap \     ④  
  name tap-rust \ ⑤  
  user $USER     ⑥
```

- ① Execute as the root user
- ② Tell ip that we're managing TUN/TAP devices
- ③ Use the "add" sub-command
- ④ Use the "TUN" tunnelling mode
- ⑤ Give your device a unique name
- ⑥ Grant access to your non-root user account

When successful, ip prints no output. To confirm that tap-rust has been added, we can use the `ip tuntap list` sub-command.

Listing 8.29. Command to list TUN/TAP devices

```
ip tuntap list
```

When executed, you should see the tap-rust device in the list of devices in the output.

```
$ ip tuntap list  
tap-rust: tap persist user 1000
```

Now that we have created a networking device, we also need to allocate it an IP address and tell our system to forward packets to it.

Listing 8.30. Commands to enable tap-rust to access the Internet


```
sudo ip link set tap-rust up ①
sudo ip addr add 192.168.42.100/24 dev tap-rust ②

sudo iptables -t nat -A POSTROUTING -s 192.168.42.0/24 -j MASQUERADE ③
sudo sysctl net.ipv4.ip_forward=1 ④
```

- ① Establish a network device called “tap-rust” and activate it.
- ② Assign the IP address 192.168.42.100 to the “tap-rust” device.
- ③ Enable Internet packets to reach the source IP address mask (-s 192.168.42.100/24) by appending a rule (-A POSTROUTING) that dynamically maps IP addresses to a device (-j MASQUERADE).
- ④ Instruct the kernel to enable IPv4 packet forwarding.

To remove the device once you have completed this chapter, use "del" rather than "add".

Listing 8.31. Removing the rust TAP device created earlier (ch8/ch8-mget/teardown.sh)

```
sudo ip tuntap del mode tap name tap-rust
```

8.10 “RAW” HTTP

We should now have all the knowledge we need to take on the challenge of using HTTP at the TCP level. The mget project (mget is short for “manually get”) spans [Listing 8.34](#), [Listing 8.35](#), [Listing 8.36](#) and [Listing 8.37](#). It is a large project, but is immensely satisfying to understand and build.

Each file provides a different role:

- `main.rs` ([Listing 8.34](#)) handles command-line parsing and weaves together the functionality provided by its peer files. This is where error types are combined using the process outlined at [Section 8.5.2](#).
- `ethernet.rs` ([Listing 8.35](#)) generates MAC address using the logic from [Listing 8.26](#) and converts between MAC address types defined

by `smoltcp` and our own.

- `http.rs` ([Listing 8.36](#)) does most of the work. It carries out the work of interacting with the server to make the HTTP request.
- `dns.rs` ([Listing 8.37](#)) performs DNS resolution using the structure provided earlier



Tip

The source code for every code listing is available from <https://github.com/rust-in-action/code>.



Note

Important acknowledgement: [Listing 8.36](#) was derived from the HTTP client example within the `smoltcp` crate itself. `whitequark` has built an absolutely fantastic networking library.

Listing 8.32. File structure for the `mget` project

```
ch8-mget
├── src
│   ├── main.rs
│   ├── http.rs
│   ├── ethernet.rs
│   └── dns.rs
└── Cargo.toml
```

Listing 8.33. Crate metadata for `mget` (`ch8/ch8-mget/Cargo.toml`)

```
[package]
name = "mget"
version = "0.1.0"
authors = ["Tim McNamara <author@rustinaction.com>"]
edition = "2018"

[dependencies]
clap = "2"           ①
rand = "0.7"         ②
url = "2"            ③

[dependencies.trust-dns] ④
version = "0.16"
default_features = false

[dependencies.smoltcp] ⑤
version = "0.6"
default_features = true
```

```
features = ["proto-igmp", "proto-ipv4", "verbose"]
```

- ① Provide command-line argument parsing
- ② Used to select a random port number
- ③ Used for URL parsing and validation
- ④ `trust-dns` enables connecting to a DNS server
- ⑤ `smoltcp` provides a TCP implementation

Listing 8.34. `mget` command line parsing and overall coordination (ch8/ch8-mget/src/main.rs)

```
use clap::{App, Arg};
use smoltcp::phy::TapInterface;
use url::Url;

mod dns;
mod http;
mod ethernet;

fn main() {
    let app = App::new("mget")
        .about("GET a webpage, manually")
        .arg(Arg::with_name("url").required(true)) ①
        .arg(Arg::with_name("tap-device").required(true)) ②
        .arg(Arg::with_name("dns-server").default_value("1.1.1.1")) ③
        .get_matches(); ④

    let url_text = app.value_of("url").unwrap();
    let dns_server_text = app.value_of("dns-server").unwrap();
    let tap_text = app.value_of("tap-device").unwrap();

    let url = Url::parse(url_text) ⑤
        .expect("error: unable to parse <url> as a URL"); ⑤

    if url.scheme() != "http" { ⑤
        eprintln!("error: only HTTP protocol supported"); ⑤
        return; ⑤
    } ⑤

    let tap = TapInterface::new(&tap_text) ⑤
        .expect("error: unable to use <tap-device> as a network interface"); ⑤

    let domain_name = url.host_str().expect("domain name required"); ⑤
    let _dns_server: std::net::Ipv4Addr = dns_server_text ⑤
        .parse() ⑤
        .expect("error: unable to parse <dns-server> as an IPv4 address"); ⑤

    let addr = dns::resolve(dns_server_text, domain_name).unwrap().unwrap(); ⑥

    let mac = ethernet::MacAddress::new().into(); ⑦
```

```
http::get(tap, mac, addr, url).unwrap(); ⑧  
}
```

- ① Require a URL to download data from
- ② Require a TAP networking device to use to connect with
- ③ Make it possible for the user to select which DNS server to use
- ④ Parse the command-line arguments
- ⑤ Validate the command-line arguments
- ⑥ Convert the URL's domain name into an IP address that can be connected to
- ⑦ Generate a random unicode MAC address
- ⑧ Make the HTTP GET request

Listing 8.35. Ethernet type conversion and MAC address generation (ch8/ch8-mget/src/ethernet.rs)

```
use rand;  
use std::fmt;  
use std::fmt::Display;  
  
use rand::RngCore;  
use smoltcp::wire;  
  
#[derive(Debug)]  
pub struct MacAddress([u8; 6]);  
  
impl Display for MacAddress {  
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {  
        let octet = self.0;  
        write!(f, "{:02x}:{:02x}:{:02x}:{:02x}:{:02x}:{:02x}",  
            octet[0], octet[1], octet[2], octet[3], octet[4], octet[5]  
        )  
    }  
}  
  
impl MacAddress {  
    pub fn new() -> MacAddress {  
        let mut octets: [u8; 6] = [0; 6];  
        rand::thread_rng().fill_bytes(&mut octets); ①  
        octets[0] |= 0b_0000_0010; ②  
        octets[0] &= 0b_1111_1110; ③  
        MacAddress { 0: octets }  
    }  
}  
  
impl Into<wire::EthernetAddress> for MacAddress {  
    fn into(self) -> wire::EthernetAddress {  
        wire::EthernetAddress { 0: self.0 }  
    }  
}
```

```
}
```

- ① Generate a random number
- ② Ensure that the local address bit is set to 1
- ③ Ensure unicast bit is set to 0

Listing 8.36. Manually creating an HTTP request, using TCP primitives (ch8/ch8-mget/src/http.rs)

```
use std::collections::BTreeMap;
use std::fmt;
use std::net::IpAddr;
use std::os::unix::io::AsRawFd;

use smoltcp::iface::{EthernetInterfaceBuilder, NeighborCache, Routes};
use smoltcp::phy::TapInterface;
use smoltcp::phy::wait as phy_wait;
use smoltcp::socket::{SocketSet, TcpSocket, TcpSocketBuffer};
use smoltcp::time::Instant;
use smoltcp::wire::{EthernetAddress, IpAddress, IpCidr, Ipv4Address};
use url::Url;

#[derive(Debug)]
enum HttpState {
    Connect,
    Request,
    Response,
}

#[derive(Debug)]
pub enum UpstreamError {
    Network(smoltcp::Error),
    InvalidUrl,
    Content(std::str::Utf8Error),
}

impl fmt::Display for UpstreamError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{:?}", self)
    }
}

impl From<smoltcp::Error> for UpstreamError {
    fn from(error: smoltcp::Error) -> Self {
        UpstreamError::Network(error)
    }
}

impl From<std::str::Utf8Error> for UpstreamError {
    fn from(error: std::str::Utf8Error) -> Self {
        UpstreamError::Content(error)
    }
}
```

```

fn random_port() -> u16 {
    49152 + rand::random::<u16>() % 16384
}

pub fn get(tap: TapInterface, mac:EthernetAddress, addr: IpAddr, url: Url) ->
Result<(), UpstreamError> {
    let domain_name = url.host_str().ok_or(UpstreamError::InvalidUrl)?;

    let neighbor_cache = NeighborCache::new(BTreeMap::new());

    let tcp_rx_buffer = TcpSocketBuffer::new(vec![0; 1024]);
    let tcp_tx_buffer = TcpSocketBuffer::new(vec![0; 1024]);
    let tcp_socket = TcpSocket::new(tcp_rx_buffer, tcp_tx_buffer);

    let ip_addrs = [
        IpCidr::new(IpAddress::v4(192, 168, 42, 1), 24),
    ];

    let fd = tap.as_raw_fd();
    let mut routes = Routes::new(BTreeMap::new());
    let default_gateway = Ipv4Address::new(192, 168, 100, 1);
    routes.add_default_ipv4_route(default_gateway).unwrap();
    let mut iface = EthernetInterfaceBuilder::new(tap)
        .ethernet_addr(mac)
        .neighbor_cache(neighbor_cache)
        .ip_addrs(ip_addrs)
        .routes(routes)
        .finalize();

    let mut sockets = SocketSet::new(vec![]);
    let tcp_handle = sockets.add(tcp_socket);

    let http_header = format!(
        "GET {} HTTP/1.0\r\nHost: {}\r\nConnection: close\r\n\r\n",
        url.path(),
        domain_name,
    );

    let mut state = HttpState::Connect;
    'http: loop {
        let timestamp = Instant::now();
        match iface.poll(&mut sockets, timestamp) {
            Ok(_) => {}
            Err(e) => { //
                eprintln!("error: {:?}", e);
            }
        }

        {
            let mut socket = sockets.get::<TcpSocket>(tcp_handle);

            state = match state {
                HttpState::Connect if !socket.is_active() => {
                    eprintln!("connecting");
                    socket.connect((addr, 80), random_port())?;
                    HttpState::Request
                }
                HttpState::Request if socket.may_send() => {
                    eprintln!("sending request");

```

```

        socket.send_slice(http_header.as_ref()?);
        HttpState::Response
    }
    HttpState::Response if socket.can_recv() => {
        socket.recv(|raw_data| {
            let output = String::from_utf8_lossy(raw_data);
            println!("{}", output);
            (raw_data.len(), ())
        })?;
        HttpState::Response
    }
    HttpState::Response if !socket.may_recv() => {
        eprintln!("received complete response");
        break 'http;
    }
    _ => state,
}
}

    phy_wait(fd, iface.poll_delay(&sockets, timestamp)).expect("wait error");
}

Ok(())
}

```

Listing 8.37. Creating DNS queries to translate domain names to IP addresses (ch8/ch8-mget/src/dns.rs)

```

use std::error::Error;
use std::net::{SocketAddr, UdpSocket};
use std::time::Duration;

use trust_dns::op::{Message, MessageType, OpCode, Query};
use trust_dns::proto::error::ProtoError;
use trust_dns::rr::domain::Name;
use trust_dns::rr::record_type::RecordType;
use trust_dns::serialize::binary::*;

fn message_id() -> u16 {
    let candidate = rand::random();
    if candidate == 0 {
        return message_id();
    }
    candidate
}

#[derive(Debug)]
pub enum DnsError {
    ParseDomainName(ProtoError),
    ParseDnsServerAddress(std::net::AddrParseError),
    Encoding(ProtoError),
    Decoding(ProtoError),
    Network(std::io::Error),
    Sending(std::io::Error),
    Receiving(std::io::Error),
}

```

```

impl std::fmt::Display for DnsError {
    fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
        write!(f, "{:#?}", self)
    }
}

impl std::error::Error for DnsError {
    // use default methods
}

pub fn resolve(dns_server_address: &str, domain_name: &str) ->
Result<Option<std::net::IpAddr>, Box<dyn Error>> {
    let domain_name = Name::from_ascii(domain_name)
    . map_err(DnsError::ParseDomainName)?; ①
    let dns_server_address = format!("{}", dns_server_address); ①
    let dns_server: SocketAddr = dns_server_address.parse()
    .map_err(DnsError::ParseDnsServerAddress)?; ①

    let mut request_buffer: Vec<u8> = Vec::with_capacity(64); ②
    let mut response_buffer: [u8; 512] = [0; 512]; ③

    let mut request = Message::new();
    request.add_query(Query::query(domain_name, RecordType::A)); ④
    request
        .set_id(message_id())
        .set_message_type(MessageType::Query)
        .set_op_code(OpCode::Query)
        .set_recursion_desired(true); ⑤

    let timeout = Duration::from_secs(5);
    let localhost = UdpSocket::bind("0.0.0.0").map_err(DnsError::Network)?; ⑥
    localhost.set_read_timeout(Some(timeout)).map_err(DnsError::Network)?;
    localhost.set_nonblocking(false).map_err(DnsError::Network)?;

    let mut encoder = BinEncoder::new(&mut request_buffer);
    request.emit(&mut encoder).map_err(DnsError::Encoding)?;

    let _n_bytes_sent = localhost
        .send_to(&request_buffer, dns_server)
        .map_err(DnsError::Sending)?;

    loop { ⑦
        let (_b_bytes_rcv, remote_port) = localhost.recv_from(&mut response_buffer)
        .map_err(DnsError::Receiving)?;
        if remote_port == dns_server {
            break;
        }
    }

    let response = Message::from_vec(&response_buffer).map_err(DnsError::Decoding)?;
    for answer in response.answers() {
        if answer.record_type() == RecordType::A {
            let resource = answer.rdata();
            let server_ip = resource.to_ip_addr().expect("invalid IP address received");
            return Ok(Some(server_ip));
        }
    }
}

Ok(None)

```



```
}
```

- ① Attempt to build the internal data structures using the raw text input
- ② We know that our DNS request will be very small, so we only need a small amount of space to hold it
- ③ DNS over UDP uses a maximum packet size of 512 bytes
- ④ DNS messages can hold multiple queries, but here we're only using a single one
- ⑤ Ask the DNS server that we're connecting to make requests on our behalf if it doesn't know the answer
- ⑥ Binding to port 0 asks the operating system to allocate a port on our behalf
- ⑦ There is a minuscule chance another UDP message will be received on our port from some unknown sender. To avoid that, we ignore packets from IP addresses that we don't expect.

`mget` is an ambitious project. It brings together all the threads from the chapter, is dozens of lines long and yet is less capable than `request::get(url)` call we made at [Listing 8.7](#). Hopefully it's revealed several interesting avenues for you to explore. Perhaps surprisingly, there are several more networking layers to unwrap.

8.11 WRAPPING UP

Well done for making your way through a lengthy and challenging chapter.

You have learned:

- implementing `std::fmt::Display` with little fuss in terms of `std::fmt::Debug`
- resolving domain names to IP addresses
- parsing input to your Rust programs safely, such as creating type-safe IP addresses from raw strings with `"127.0.0.1".parse::<Ipv4Addr>()`.
- generating valid MAC addresses
- what the question mark operator (the `try!` macro) is doing under the

hood and why compile errors mention conversion traits

- using a loop and match with an enum to implement a simple finite state machine
- creating wrapper error types with an enum, then implementing `std::convert::Format` on the upstream errors to enable your enum to work with the `?` operator
- implementing the `std::error::Error` trait by implementing `std::fmt::Display` and `std::fmt::Debug`, then using default implementations of the required methods
- using trait objects to create a collection of heterogeneous objects

9 TIME AND TIME KEEPING

In this chapter:

- how a computer keeps time
- how operating systems represent timestamps
- synchronize with the world's atomic clocks with the Network Time Protocol (NTP)

During this chapter, you'll be producing an NTP client that can request the current time from the world's network of public time servers. It's a fully-functioning client that can be included into your own computer's boot process to keep it in sync with the world.

Understanding of how time works within computers supports your efforts to build resilient applications. The system's clock jumps both backwards and forwards in time. Knowing why this happens will allow you to anticipate and prepare for that eventuality.

Your computer also contains multiple physical and virtual clocks. It takes some knowledge to understand the limitations of each and when they're appropriate. Understanding the limitations of each should foster a healthy skepticism of micro benchmarks and other time-sensitive code.

Some of the hardest software engineering involves distributed systems that need to agree on what the time is. If you have the resources of Google, then you're able to maintain a network atomic clocks that provide a worldwide time synchronization of 7ms. The closest open source alternative is CockroachDB. It relies on the Network Time Protocol (NTP), which may have a (worldwide) latency of approximately dozens of milliseconds. But that doesn't make it useless. When deployed within a local network, NTP allows computers to agree on the time to within a few milliseconds or less.

On the Rust side of the equation, this chapter invests lots of time interacting with the operating system internals. You'll become more confident with `unsafe` blocks and using raw pointers. Readers will become familiar with `chrono`, the *de facto* standard crate for high-level time and clock operations.

9.1 BACKGROUND

It's easy to think that a day has 86,400 seconds. $60 \text{ seconds} \times 60 \text{ minutes} \times 24 \text{ hours} = 86,400 \text{ seconds}$. But the Earth's rotation isn't quite that perfect. The length of each day fluctuates due to tidal friction with the moon and other effects such as torque at the boundary of the Earth's core and its mantle.

Software does not tolerate these imperfections. Most systems assume that most seconds have an equal duration. The mismatch presents several problems. In 2012, a large number of services—including high profile sites such as Reddit and Mozilla's Hadoop infrastructure—stopped functioning after a leap second was added to their clocks. And at times, clocks can go back in time too. Few software systems are prepared for the same timestamp to appear twice. That makes it very difficult to debug the logs.

There are two options for resolving this impasse:

- keep the length of each second fixed—which is good for computers, while being irritating for humans. Over time, “midday” will drift towards sunset or sunrise.
- adjust the length of each year to keep the sun's position relative to noon in the same place from year to year—good for humans, but sometimes highly irritating for computers

In practice, both options have been chosen. The world's atomic clocks use their own time zone with fixed-length seconds called TAI. Everything else uses time zones that are periodically adjusted called UTC.

TAI is spoken by the world's atomic clocks and they maintain a fixed-length year. UTC adds leap seconds to TAI about once every 18 months. In 1972, TAI

and UTC were 10 seconds apart. By 2016, they had drifted to 36 seconds apart.

As well as the issues with Earth's fickle rotational speed, the physics of your own computer make it very challenging to keep accurate time. There are also (at least) two clocks running on your system. One is a battery-powered device called the real-time clock. The other one is known as *system time*. System time increments itself based on *hardware interrupts* provided by the computer's motherboard. Somewhere in your system, a quartz crystal is oscillating rapidly.

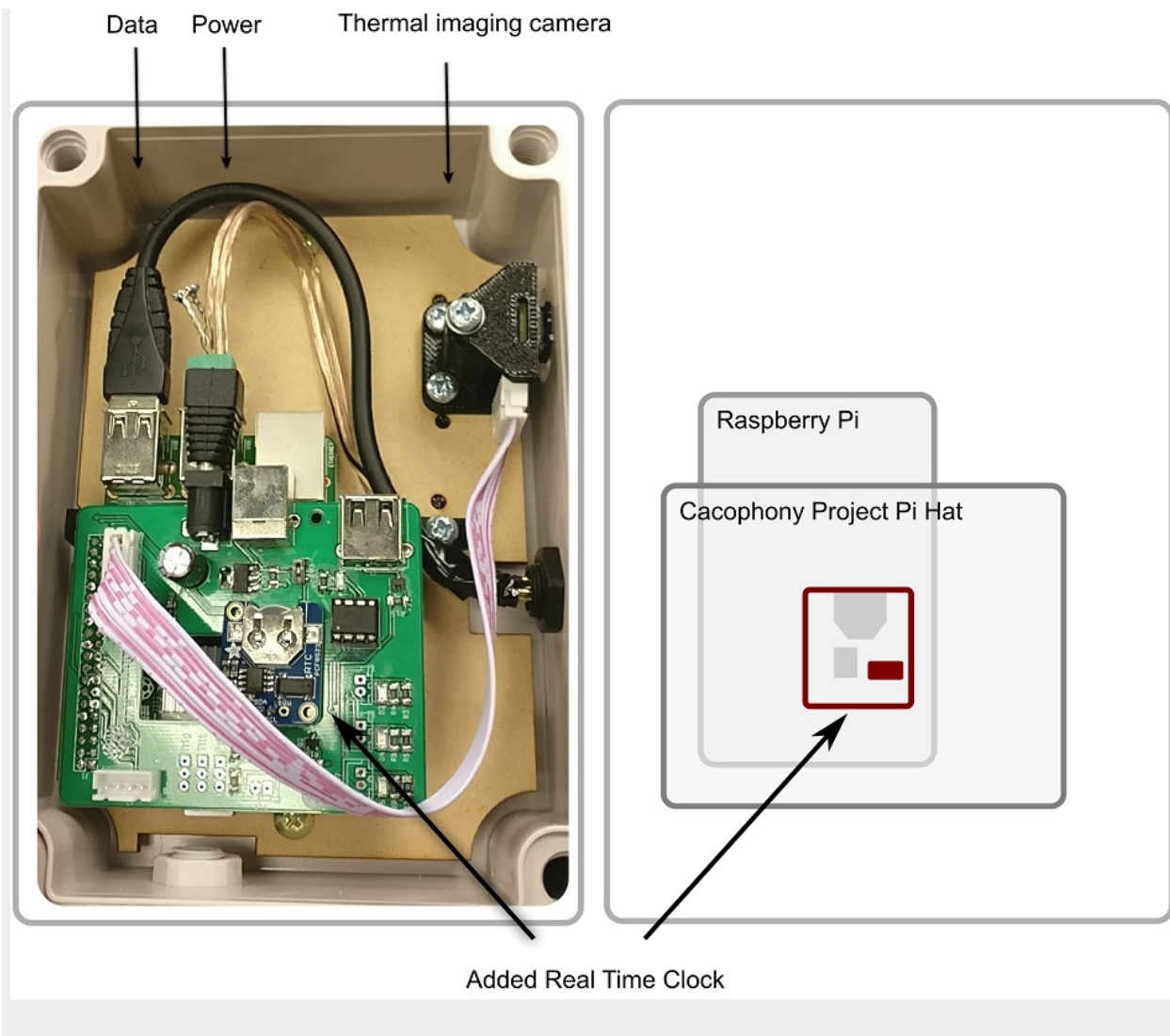
Dealing with hardware platforms without a real-time clock

The Raspberry Pi does not include a battery-supported real-time clock. When the computer turns on, the system clock is set to the epoch time. During boot, it uses the network time protocol (NTP) to identify the current time.

What about situations where there is no network connection? This is the situation faced by the Cacophony Project, which develops devices to support New Zealand's native bird species by applying computer vision to accurately identify pest species.

The main sensor of the device is a thermal imaging camera. Footage needs to be annotated with accurate timestamps. To enable this, the Cacophony Project team decided to add an additional real-time clock to their custom board, known as a "Raspberry Pi Hat"

Figure 9.1. Viewing the internals of a prototype for the Cacophony Project's automated pest detection system



9.2 SOURCES OF TIME

Computers can't look at the clock on the wall to determine what time it is. They need to figure it out by themselves. To explain how this happens, let's consider how digital clocks operate generally, then consider how computer systems operate given some difficult constraints, such as operating without power.

Digital clocks consist of two main parts. The first part is some component that ticks at regular intervals. The second part is a pair of counters. One counter increments as ticks occur. The other increments as seconds occur.

Determining "now" within digital clocks means comparing up the number of

seconds against some pre-determined starting point. The starting point is known as the *epoch*.

Embedded hardware aside, when your computer is turned off, a small battery-powered clock is running. Its electric charge causes a quartz crystal to oscillate rapidly. The clock measures those oscillations and updates its internal counters.

In a running computer, the CPU clock frequency becomes the source of regular ticks. A CPU core operates at a fixed frequency. ^[41] Inside the hardware, a counter can be accessed via CPU instructions ^[42] and/or by accessing pre-defined CPU registers.

Relying on a CPU's clock can actually cause problems in niche scientific and other high-accuracy domains, such as profiling an application's behaviour. When computers use multiple CPUs, which is especially common in high performance computing, each CPU has a slightly different clock rate. Moreover, CPUs perform out-of-order execution. This means that it's impossible for someone creating a benchmarking/profiling software suite to deterministically know how long a function has taken between two timestamps. The CPU instructions requesting the current timestamp may have been shifted around.

^[41] Dynamic adjustments to a CPU's clock speed do occur in many processors to conserve power, but they happen infrequently enough from the point of view of the clock that they're insignificant.

^[42] For example, Intel-based processors support the RDTSC instruction, which stands for "Read Time Stamp Counter".

9.3 DEFINITIONS

Unfortunately, this chapter needs to introduce some jargon that will be important to be able to refer to for anyone looking deeply into the subject matter.

- absolute time - A term to describe the time that you would tell someone if they were to ask for the time. Also referred to as *wall*

clock time and *calendar* time.

- realtime clock - The *realtime clock* is a physical clock that's embedded into the computer's motherboard that keeps time when power is off. Also known as the *CMOS clock*.
- system clock - The operating system's view of the time is known as the *system clock*. Upon boot, the operating system takes over timekeeping duties from the realtime clock. All applications derive their idea of the time from the system time. The *system clock* experiences jumps, as it can be manually set to a different position. This jumpiness can confuse some applications.
- monotonically increasing - A *monotonically increasing* clock never provides the same time twice. This is a useful property for a computer application, as log messages will never have a repeated timestamp among other advantages. Unfortunately, preventing time adjustments means being permanently bound to the local clock's skew. The *system clock* is not monotonically increasing.
- steady clock - A *steady clock* provides two guarantees: 1) its seconds are all equal length, and 2) it is *monotonically increasing*. Values from steady clocks are unlikely to align to the *system clock* time or absolute time. They typically start at 0 when computers boot up, then count upwards as an internal counter progresses. Although potentially useless for knowing the absolute time, they are very handy for calculating the duration between two points in time.
- high accuracy - A clock is highly accurate if the length of its seconds are regular. The difference between two clocks is known as *skew*. Accurate clocks have very little skew against the atomic clocks that are humanity's best engineering effort at keeping accurate time.
- high resolution - A *high resolution* clock is able to give accuracy down to 10 nanoseconds, or perhaps below. High resolution clocks are typically implemented within CPU chips, as there are few devices that can maintain time at such high frequency. CPUs are able to do this, because their units of work are measured in cycles and cycles have the same duration. A 1Ghz CPU core takes 1 nanosecond to compute 1 cycle.

- fast clock - A *fast clock* takes very little time to read the time. Fast clocks sacrifice accuracy and precision for speed.

9.4 ENCODING TIME

There are many ways to represent time within a computer. The typical approach is to use a pair of 32-bit integers. The first counts the number of seconds that have elapsed. The second represents a fraction of a second. The precision of the fractional part depends on the device in question.

The starting point is arbitrary. The most common epoch in UNIX-based systems is 1 Jan 1970 UTC. Alternatives include 1 Jan 1900 (which happens to be used by NTP), 1 Jan 2000 for more recent applications and 1 Jan 1601, which is the beginning of the Gregorian calendar.

Using fixed-width integers presents two main advantages and two main challenges:

- Advantages
 - simplicity: it's very easy to understand the format
 - efficiency: integer arithmetic is the CPU's favorite activity

Disadvantages

- fixed-range: all fixed-integer types are finite, implying that time will eventually wrap around to 0 again
- imprecise: integers are discrete, while time is continuous. Different systems make different trade offs relating to sub-second accuracy, leading to rounding errors.

It's also important to note that the general approach is inconsistently implemented. Here are some things seen in the wild to represent the seconds component:

- UNIX timestamps - 32 bit integer - milliseconds since "epoch", e.g.

1 Jan 1970.

- MS Windows FILETIME structures (since Windows 2000) - 64bit unsigned integer - 100 nanosecond increments since 1 Jan 1601 (UTC)
- Rust community's `chrono` crate - 32 bit signed integer - used to implement the `NaiveTime`, alongside an enum to represent timezones where appropriate. Has relatively few quirks. One of which is sneaking leap seconds into the nanoseconds field.
- `time_t` (meaning "time type", but also called simple time or calendar time) within the C standard library (`libc`)
 - Dinkumware's `libc` - unsigned long int, e.g. a 32-bit unsigned integer
 - GNU `libc` - long int, e.g. a 32-bit signed integer
 - AVR `libc` - uses a 32-bit unsigned integer, and its epoch begins on Midnight, 1 January 2000 UTC (https://www.nongnu.org/avr-libc/user-manual/time_8h_source.html)

Fractional parts tend to use the same type as their whole-second counterparts, but this isn't guaranteed.

9.4.1 Representing time zones

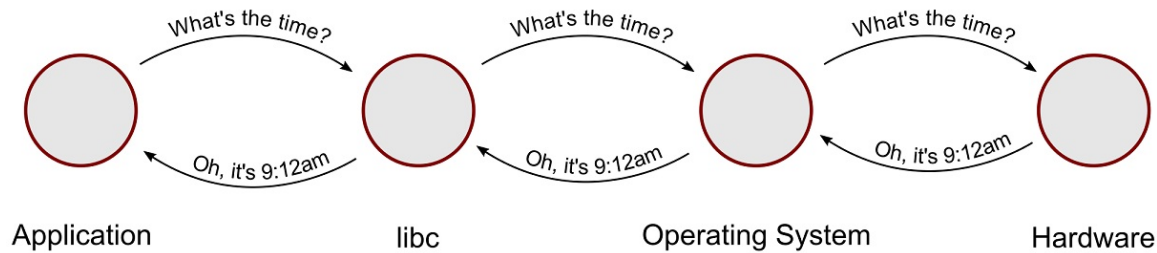
Time zones are political divisions, rather than technical ones. A soft consensus appears to have been formed around storing another integer that represents the number of seconds offset from UTC.

9.5 cLock V0.1.0: TEACHING AN APPLICATION HOW TO TELL THE TIME

To begin coding our NTP client, let's start by learning how to read the time.

Figure 9.2. How an application tells the time. It gets this information from the operating system, usually via functionality provided by the

system's libc implementation.



[Listing 9.2](#) might almost feel too small to be a fully-fledged example. Running the code will result in the current timestamp formatted according to the ISO 8601 standard.

Listing 9.1. Crate configuration for [Listing 9.2](#) (ch9/ch9-clock0/Cargo.toml)

```
[package]
name = "clock"
version = "0.1.0"
authors = ["Tim McNamara <author@rustinaction.com>"]
edition = "2018"

[dependencies]
chrono = "0.4"
```

Listing 9.2. Reading the system time in the local time zone and print it to the screen (ch9/ch9-clock0/src/main.rs)

```
use chrono::Local;

fn main() {
    let now = Local::now(); ①
    println!("{}", now);
}
```

① Ask for the time in the system's local time zone.

There is lots of complexity being hidden by these 8 lines of code. Much of it will be peeled away during the course of the chapter.

The magic is provided by `chrono::Local`. It returns a typed value that contains a

time zone. Attempts to interact with timestamps that don't include time zones, or to perform other forms of illegal time arithmetic, result in the program refusing to compile.

9.6 cLock V0.1.1: FORMATTING TIMESTAMPS TO COMPLY WITH ISO 8601 AND EMAIL STANDARDS

The application that we'll be creating is called `cLock`. `cLock` reports the current time. During the chapter, it will be incrementally enhanced to support setting the time manually and via NTP.

For the moment, compiling the code from [Listing 9.9](#) and sending the `--use-standard` timestamp flag should produce something like the following output:

Listing 9.3. Output from compiling and running the code of `cLock` v0.1.1, provided at [Listing 9.9](#)

```
$ cd path/to/ch9/ch9-clock1
$ cargo run -- --use-standard rfc2822 ①
warning: associated function is never used: `set` ②
--> src/main.rs:12:8 ②
| ②
| ②
12 |     fn set() -> ! { ②
|     ^^^ ②
| ②
= note: `#[warn(dead_code)]` on by default ②
warning: 1 warning emitted ②
    Finished dev [unoptimized + debuginfo] target(s) in 0.01s ②
    Running `target/debug/clock --use-standard rfc2822` ②
Sat, 20 Feb 2021 15:36:12 +1300 ③
```

- ① Two dashes after a cargo command send the remaining arguments to the application that's being built
- ② Output from the compiler. It is complaining about the `set()` function, which is marked as `unimplemented!()` in the source code and never called.
- ③ Output from the program itself, telling the current time.

9.6.1 Refactoring the clock v0.1.0 code to support wider architecture

It makes sense to spend a short period of time creating a scaffold for the larger application that clock will become.

Within clock, we'll make a small cosmetic change. Rather than using functions to read the time and adjust it, we'll use static methods of a struct Clock.

Listing 9.4. Excerpt from [Listing 9.9](#) showing the change from [Listing 9.2](#) relating to how the time is read from the local system clock

```
use chrono::{DateTime};
use chrono::{Local};

struct Clock;

impl Clock {
    fn get() -> DateTime<Local> { ①
        Local::now()
    }

    fn set() -> ! { ②
        unimplemented!() ③
    }
}
```

- ① `DateTime<Local>` is a `Datetime` with the `Local` timezone information
- ② The exclamation mark (!) indicates to the compiler that the function will never return. That is, a return value is impossible. It is referred to as the "Never type".
- ③ If the `unimplemented!()` macro (or its shorter cousin `todo!()`) is reached at runtime, then the program panics.

Clock is purely acting as a namespace at this stage. Adding a struct in now provides some extensibility later on. As the application grows, it might become useful for Clock to contain some state between calls or implement some trait to support some new functionality.



Tip

A struct with no fields is known as a "zero sized" type. It will not occupy any memory in the resulting application and is purely a compile-time construct.

9.6.2 Formatting the time as a UNIX timestamp or a formatted string according to ISO 8601, RFC 2822, and RFC 3339

Timestamps that will be sent to stdout are produced using functionality provided by chrono:

Listing 9.5. Excerpt from [Listing 9.9](#) showing the methods used to format dates

```
let now = Clock::get();
match std {
    "timestamp" => println!("{}", now.timestamp()),
    "rfc2822"    => println!("{}", now.to_rfc2822()),
    "rfc3339"    => println!("{}", now.to_rfc3339()),
    _           => unreachable!(),
}
```

clock (thanks to chrono) supports three time formats. ISO 8601 support is provided by the RFC 3339, which is a slightly stricter standard. Every RFC 3339-compliant timestamp is an ISO 8601-compliant timestamp, but the inverse is not true.

9.6.3 Providing a full command-line interface

Command line arguments are part of the environment provided to an application from its operating system when it's established. They're raw strings.

Rust provides some support for accessing the raw `Vec<String>` via `std::env::args`, but it can be tedious to develop lots of parsing logic for moderately sized applications. Our code wants to be able to validate certain input—for example that the desired output format is one that clock actually

supports—and validating input tends to be irritatingly complex. To avoid this frustration, `clock` makes use of the `clap` crate.

`clap` provides facilities for constructing **command line applications**. In particular, it makes the task of interpreting command-line arguments and providing help documentation. There are two main types that are useful for getting started: `clap::App` and `clap::Arg`. Each `clap::Arg` represents a command-line argument and the options that it can represent. `clap::App` collects them into a single application.

To support the public API at [Table 9.1](#), the code at [Listing 9.6](#) uses three `Arg` structs that are wrapped together within a single `App`:

Table 9.1. Usage examples for executing `clock` application from the command-line, each of which need to be supported by our parser

Usage	Description	Example output
<code>clock</code>	Default usage. Print the current time.	2018-06-17T11:25:19.467585800+12:00
<code>clock get</code>	Provide "get" action explicitly with default formatting.	2018-06-17T11:25:19.467585800+12:00
<code>clock get --use-standard timestamp</code>	Provide "get" action and a formatting standard.	1529191458
<code>clock get -s timestamp</code>	Provide "get" action and a formatting standard using shorter notation.	1529191458
<code>clock set <datetime></code>	Provide "set" action explicitly with default parsing rules.	
<code>clock set --use-standard timestamp <datetime></code>	Provide "set" action explicitly and indicate that the input will be a UNIX timestamp.	

Listing 9.6. Excerpt from [Listing 9.9](#) demonstrating the use of `clap` to parse command-line arguments

```
let app = App::new("clock")
    .version("0.1.1")
    .about("Gets and (aspirationally) sets the time.")
    .arg(Arg::with_name("action")
        .takes_value(true)
        .possible_values(&["get", "set"]))
```

```

        .default_value("get"))
    .arg(Arg::with_name("std")
        .short("s")
        .long("use-standard")
        .takes_value(true)
        .possible_values(&["rfc2822", "rfc3339", "timestamp"])
        .default_value("rfc3339"))
    .arg(Arg::with_name("datetime")
        .help("When <action> is 'set', apply <datetime>. Otherwise, ignore."));

let args = app.get_matches();

```

clap creates some usage documentation on your behalf. This can be read from the project's root directory:

Listing 9.7. Reviewing the automatically generated usage documentation for clock

```

$ cd /path/to/ch9/ch9-clock1
$ cargo run -q -- --help
clock 0.1
Gets and sets (aspirationally) the time.

USAGE:
    clock.exe [OPTIONS] [ARGS]

FLAGS:
    -h, --help            Prints help information
    -V, --version          Prints version information

OPTIONS:
    -s, --use-standard <std>    [default: rfc3339] [possible values: rfc2822, rfc3339,
timestamp]

ARGS:
    <action>        [default: get] [possible values: get, set]
    <datetime>      When <action> is 'set', apply <datetime>. Otherwise, ignore.

```

① Execute `cargo run "quietly"` (with `-q`) and send `--help` to the resulting binary

Listing 9.8. Crate configuration for Listing 9.9 (ch9/ch9-clock1/Cargo.toml)

```

[package]
name = "clock"
version = "0.1.1"
authors = ["Tim McNamara <author@rustinaction.com>"]
edition = "2018"

```



```
[dependencies]
chrono = "0.4"
clap = "2"
```

②

- ① The version number has been incremented by 0.0.1.
- ② A new dependency has been added. We'll pin to the major version of the clap crate.

9.6.4 The full c1ock v0.1.1 code listing

With the pre-requisite knowledge covered, let's move to seeing everything in a single place.

Listing 9.9. Producing formatted dates from the command line (ch9/ch9-clock1/src/main.rs)

```
use clap::{App, Arg};
use chrono::{DateTime};
use chrono::{Local};

struct Clock;

impl Clock {
    fn get() -> DateTime<Local> {
        Local::now()
    }

    fn set() -> ! {
        unimplemented!()
    }
}

fn main() {
    let app = App::new("clock")
        .version("0.1")
        .about("Gets and (aspirationally) sets the time.")
        .arg(Arg::with_name("action")
            .takes_value(true)
            .possible_values(&["get", "set"])
            .default_value("get"))
        .arg(Arg::with_name("std")
            .short("s")
            .long("use-standard")
            .takes_value(true)
            .possible_values(&["rfc2822", "rfc3339", "timestamp"])
            .default_value("rfc3339"))
        .arg(Arg::with_name("datetime")
            .help("When <action> is 'set', apply <datetime>. Otherwise, ignore."));
```

```

let args = app.get_matches();

let action = args.value_of("action").unwrap(); ①
let std = args.value_of("std").unwrap();         ①

if action == "set" {
    unimplemented!() ②
}

let now = Clock::get();
match std {
    "timestamp" => println!("{}", now.timestamp()),
    "rfc2822"   => println!("{}", now.to_rfc2822()),
    "rfc3339"   => println!("{}", now.to_rfc3339()),
    _           => unreachable!(),
}
}

```

① A default value (via `default_value("get")` and `default_value("rfc3339")`) has been supplied to each of these arguments, so it's safe to `unwrap()` on these two lines.

② Abort early as we're not ready to set the time yet.

9.7 cLock V0.1.2: SETTING THE TIME

Setting the time is complicated by the fact that each operating system has its own mechanism for doing so. This requires that we use operating system-specific conditional compilation to create a cross-portable tool.

9.7.1 Common behavior

[Listing 9.13](#) provides two implementations of setting the time. They both follow a common pattern:

1. Parse a command-line argument to create a `DateTime<FixedOffset>` value. The `FixedOffset` timezone is provided by `chrono` as a proxy for “whichever timezone is provided by the user”. `chrono` can't know at compile-time which timezone will be selected.
2. Convert the `DateTime<FixedOffset>` to a `DateTime<Local>` to enable timezone comparisons.
3. Instantiate an operating system-specific struct that's used as an argument

for the necessary *system call* (system calls are function calls provided by the operating system)

4. Set the system's time within an unsafe block. Unsafety is required because responsibility is delegated to the operating system.
5. Print the updated time.



Warning

This code uses functions to teleport the system's clock to a different time. This jumpiness can cause system instability.

Some applications that expect monotonically increasing time. A smarter (and more complex) approach is to adjust the length of a second for n seconds until the desired time is reached.

Functionality is implemented within the `Clock` struct that was introduced at [Section 9.6.1](#).

9.7.2 Setting the time in operating systems that use `libc`

POSIX-compliant operating systems can have their time set via a call to `settimeofday()`, which is provided by `libc`. `libc` is the “C Standard Library” and has lots of historic connections with UNIX operating systems. The C language was developed to write UNIX. Even today, interacting with a UNIX derivative involves using the tools provided by the C language.

There are two mental hurdles required for Rust programmers to understanding this code:

1. the arcane types provided by `libc`
2. the unfamiliarity of providing arguments as pointers

Let's address both.

LIBC IS TYPE-HEAVY AS IT USES MANY ALIASES

Dealing with the time APIs within `libc` means dealing with many types and type aliases. The code is intended to be easy to implement, which means providing local implementors (hardware designers) the opportunity to change aspects as their platforms require. The way this is done is to use type aliases everywhere, rather than sticking to a defined integer type.

Near the start of [Listing 9.10](#), you will encounter the line

```
libc::time::{timeval, time_t, suseconds_t};
```

which represents two type aliases and a struct definition. In Rust syntax, they're defined like this:

```
type time_t = i64;
type suseconds_t = i64;

pub struct timeval {
    pub tv_sec: time_t,
    pub tv_usec: suseconds_t,
}
```

`time_t` represents the seconds that have elapsed since “the Epoch”. `suseconds_t` represents the fractional component of the current second.

NON-WINDOWS `cLOCK` CODE

`libc` provides a handy function, `settimeofday`. The project's `Cargo.toml` file requires two extra lines to bring `libc` bindings into the crate:

```
[target.'cfg(not(windows))'.dependencies]    ①
libc = "0.2"
```

① These two lines can be added to the end of the file

Listing 9.10. Setting the time in a `libc` environment, such as Linux and BSD operating systems

```
#[cfg(not(windows))]
fn set<Tz: TimeZone>(t: DateTime<Tz>) -> () {    ①
```

```

    use libc::settimeofday;                                ②
    use libc::{suseconds_t, time_t, timeval, timezone};    ②

    let t = t.with_timezone(&Local); // variable names indicate the data's
progression
    let mut u: timeval = unsafe { zeroed() }; // through the function

    u.tv_sec = t.timestamp() as time_t;
    u.tv_usec = t.timestamp_subsec_micros() as suseconds_t;

    unsafe {
        let mock_tz: *const timezone = std::ptr::null();    ③
        settimeofday(&u as *const timeval, mock_tz);
    }
}

```

- ① `t` is sourced from the command line and has already been parsed.
- ② Operating system-specific imports are made within the function, to avoid polluting the global scope. `libc::settimeofday` is a function that modifies the system clock. `suseconds_t`, `time_t`, `timeval`, and `timezone` are all types that are used interact with it.
- ③ The `timezone` parameter of `settimeofday()` appears to be some historic accident. Non-null values generate an error.

This code cheekily, and probably perilously, avoids checking whether the `settimeofday` function was successful. It's very possible that it didn't. That will be remedied in the next iteration of `clock`.

9.7.3 Setting the time on MS Windows

The code for MS Windows is similar to its `libc` peers. It is somewhat wordier, as the struct that sets the time has more fields than the second and subsecond part.

The rough equivalent of `libc` is called `kernel32.dll`, which is accessible after also including the `winapi` crate.

WINDOWS API INTEGER TYPES

Windows provides its own take on what to call integral types. This code only makes use of the `WORD` type, but it can be useful to remember two other common

types that have emerged since computers used 16-bit CPUs.

Windows Type	Rust Type	Remarks
WORD	u16	Refers to the width of a CPU “word”, as it was when Windows was initially created.
DWORD	u32	“Double word”
QWORD	u64	“Quadruple word”
LARGE_INTEGER	i64	A type defined as a crutch to enable 32-bit and 64-bit platforms to share code.
ULARGE_INTEGER	u64	An unsigned version of LARGE_INTEGER

REPRESENTING THE TIME IN WINDOWS

Windows provides multiple time types. Within `clock` however, we’re mostly interested in `SYSTEMTIME`. Another type that is provided is `FILETIME`, described here to avoid confusion.

Windows Type	Rust Type	Remarks
SYSTEMTIME	<code>winapi::SYSTEMTIME</code>	Contains fields for the year, month, day of the week, day of the month, hour, minute, second and millisecond.
FILETIME	<code>winapi::FILETIME</code>	Analogous to <code>libc::timeval</code> Contains second and millisecond fields. Microsoft’s documentation warns that on 64-bit platforms, its use can cause irritating overflow bugs without finicky type casting, which is why it’s not employed here.

WINDOWS `clock` CODE

As the `SYSTEMTIME` struct contains many fields, generating one takes a little bit longer.

Listing 9.11. Setting the time using the Windows `kernel.dll` API

```
#[cfg(windows)]
fn set(<Tz: TimeZone>(t: DateTime<Tz>) -> () {
    use winapi::{SYSTEMTIME, WORD};
    use kernel32::{GetSystemTime, SetSystemTime};

    let t = t.with_timezone(&Local);
```

```

let mut systime: SYSTEMTIME = unsafe { zeroed() } ;

let dow = match t.weekday() {
    Weekday::Mon => 1,
    Weekday::Tue => 2,
    Weekday::Wed => 3,
    Weekday::Thu => 4,
    Weekday::Fri => 5,
    Weekday::Sat => 6,
    Weekday::Sun => 0,
};

let mut ns = t.nanosecond();
let mut leap_second = 0;
let is_leap = ns > 1_000_000_000;

if is_leap {
    ns -= 1_000_000_000;
    leap_second += 1;
}

systime.wYear = t.year() as WORD;
systime.wMonth = t.month() as WORD;
systime.wDayOfWeek = dow as WORD;
systime.wDay = t.day() as WORD;
systime.wHour = t.hour() as WORD;
systime.wMinute = t.minute() as WORD;
systime.wSecond = (leap_second + t.second()) as WORD;
systime.wMilliseconds = (ns / 1_000_000) as WORD;

let systime_ptr = &systime as *const SYSTEMTIME;

unsafe {
    SetSystemTime(systime_ptr);
}

```

- ① The `chrono::Datelike` trait provides the `weekday()` method. Microsoft's developer documentation provides the conversion table.
- ② As an implementation detail, `chrono` represents leap seconds by adding an extra second within the nanoseconds field. To convert the nanoseconds to milliseconds, as required by Windows, we need to account for this.
- ③ See the warning below.

9.7.4 c1ock v0.1.2 Full code listing

[Listing 9.12](#) and [Listing 9.13](#) provide the full source listing for `c1ock v0.1.2`.

Listing 9.12. Crate configuration for [Listing 9.13](#) (ch9/ch9-

clock0/Cargo.toml)

```
[package]
name = "clock"
version = "0.1.2"
authors = ["Tim McNamara <author@rustinaction.com>"]
edition = "2018"

[dependencies]
chrono = "0.4"
clap = "2"

[target.'cfg(windows)'.dependencies]
winapi = "0.2"
kernel32-sys = "0.2"

[target.'cfg(not(windows))'.dependencies]
libc = "0.2"
```

Listing 9.13. Cross-portable code for setting of the system time

```
#[cfg(windows)] use kernel32;
#[cfg(windows)] use winapi;
#[cfg(not(windows))] use libc;

use chrono::{DateTime, Local, TimeZone};
use clap::{App, Arg};
use std::mem::zeroed;

struct Clock;

impl Clock {
    fn get() -> DateTime<Local> {
        Local::now()
    }

    #[cfg(windows)]
    fn set<Tz: TimeZone>(t: DateTime<Tz>) -> () {
        use chrono::Weekday;
        use kernel32::SetSystemTime;
        use winapi::{SYSTEMTIME, WORD};

        let t = t.with_timezone(&Local);

        let mut systime: SYSTEMTIME = unsafe { zeroed() };

        let dow = match t.weekday() {
            Weekday::Mon => 1,
            Weekday::Tue => 2,
            Weekday::Wed => 3,
            Weekday::Thu => 4,
            Weekday::Fri => 5,
            Weekday::Sat => 6,
            Weekday::Sun => 0,
        };
    }
};
```



```

let mut ns = t.nanosecond();
let is_leap_second = ns > 1_000_000_000;

if is_leap_second {
    ns -= 1_000_000_000;
}

systime.wYear = t.year() as WORD;
systime.wMonth = t.month() as WORD;
systime.wDayOfWeek = dow as WORD;
systime.wDay = t.day() as WORD;
systime.wHour = t.hour() as WORD;
systime.wMinute = t.minute() as WORD;
systime.wSecond = t.second() as WORD;
systime.wMilliseconds = (ns / 1_000_000) as WORD;

let systime_ptr = &systime as *const SYSTEMTIME;

unsafe { ①
    SetSystemTime(systime_ptr);
}
}

#[cfg(not(windows))]
fn set<Tz: TimeZone>(t: DateTime<Tz>) -> () {
    use libc::settimeofday;
    use libc::{suseconds_t, time_t, timeval, timezone};

    let t = t.with_timezone(&Local);
    let mut u: timeval = unsafe { zeroed() };

    u.tv_sec = t.timestamp() as time_t;
    u.tv_usec = t.timestamp_subsec_micros() as suseconds_t;

    unsafe {
        let mock_tz: *const timezone = std::ptr::null();
        settimeofday(&u as *const timeval, mock_tz);
    }
}

fn main() {
    let app = App::new("clock")
        .version("0.1.2")
        .about("Gets and (aspirationally) sets the time.")
        .after_help("Note: UNIX timestamps are parsed as whole seconds since 1st
January 1970 0:00:00 UTC. For more accuracy, use another format.")
        .arg(Arg::with_name("action")
            .takes_value(true)
            .possible_values(&["get", "set"])
            .default_value("get"))
        .arg(Arg::with_name("std")
            .short("s")
            .long("use-standard")
            .takes_value(true)
            .possible_values(&["rfc2822", "rfc3339", "timestamp"])
            .default_value("rfc3339"))
        .arg(Arg::with_name("datetime")
            .help("When <action> is 'set', apply <datetime>. Otherwise, ignore."));
}

```

```

let args = app.get_matches();

let action = args.value_of("action").unwrap();
let std = args.value_of("std").unwrap();

if action == "set" {
    let t_ = args.value_of("datetime").unwrap();

    let parser = match std {
        "rfc2822" => DateTime::parse_from_rfc2822,
        "rfc3339" => DateTime::parse_from_rfc3339,
        _ => unimplemented!(),
    };

    let err_msg = format!("Unable to parse {} according to {}", t_, std);
    let t = parser(t_).expect(&err_msg);

    Clock::set(t)
}

let now = Clock::get();

match std {
    "timestamp" => println!("{}", now.timestamp()),
    "rfc2822" => println!("{}", now.to_rfc2822()),
    "rfc3339" => println!("{}", now.to_rfc3339()),
    _ => unreachable!(),
}
}

```

① From the perspective of the Rust compiler, giving something else direct access to memory is unsafe. Rust cannot guarantee that the Windows kernel will be well behaved.

9.8 IMPROVING ERROR HANDLING

Those readers who have dealt with operating systems before will probably be dismayed at some of the code within [Section 9.7](#). It doesn't check to see whether the calls to `settimeofday()` and `SetSystemTime()` were actually successful.

There are multiple reasons why setting the time might fail. The most obvious one is that the user who is attempting to set the time lacks permission to do so.

The robust approach is to have `Clock::set(t)` return `Result`. As that will require modifying two functions that we have already spent some time explaining in-depth, let's introduce a workaround that makes use of the operating system's error reporting instead.

```
fn main() {
    // ...
    if action == "set" {
        // ...

        Clock::set(t);

        let maybe_error = std::io::Error::last_os_error();           ①
        let os_error_code = &maybe_error.raw_os_error();           ①
        match os_error_code {
            Some(0) => (),                                           ②
            None => (),                                             ②
            _ => eprintln!("Unable to set the time: {:?}", maybe_error),
        }
    }
}
```

- ① `maybe_error` is a Rust type that we can deconstruct to convert it into a raw `i32` value that's easy to match on.
- ② Matching on a raw integer saves importing an enum, but sacrifices type safety. Production-ready code shouldn't cheat in this way.

After calls to `Clock::set(t)`, Rust will happily talk to the operating system via `std::io::Error::last_os_error()` to see if an error code has been generated.

9.9 CLOCK V0.1.3 RESOLVING DIFFERENCES BETWEEN CLOCKS WITH THE NETWORK TIME PROTOCOL (NTP)

Coming to a consensus about the correct time is known formally as *clock synchronisation*. There are multiple international standards for synchronizing clocks. This section focuses on the most prominent one: the Network Time Protocol (NTP). NTP has existed since the mid-1980s and has proven to be very stable. Its on-wire format has not changed in the first four revisions of the protocol, with backwards compatibility retained the entire time.

NTP operates in two modes that can loosely be described as “always on” and “request/response”.

The “always on” mode allows multiple computers to work in a peer to peer

fashion to converge on an agreed definition of “now”. It requires a software daemon/service to be running constantly on each device but can achieve very tight synchronization within local networks.

The “request/response” mode is much simpler. Local clients request the time via a single message and then parse the response. Keeping track of the elapsed time on the client and comparing timestamps

NTP allows clients to request the time from computers who are closer to atomic clocks. But that only gets us part of the way. Let’s say that your computer asks 10 computers what they think the time is. Now we have 10 assertions about the time, and the network lag will differ for each source.

9.9.1 Sending NTP requests and interpreting responses

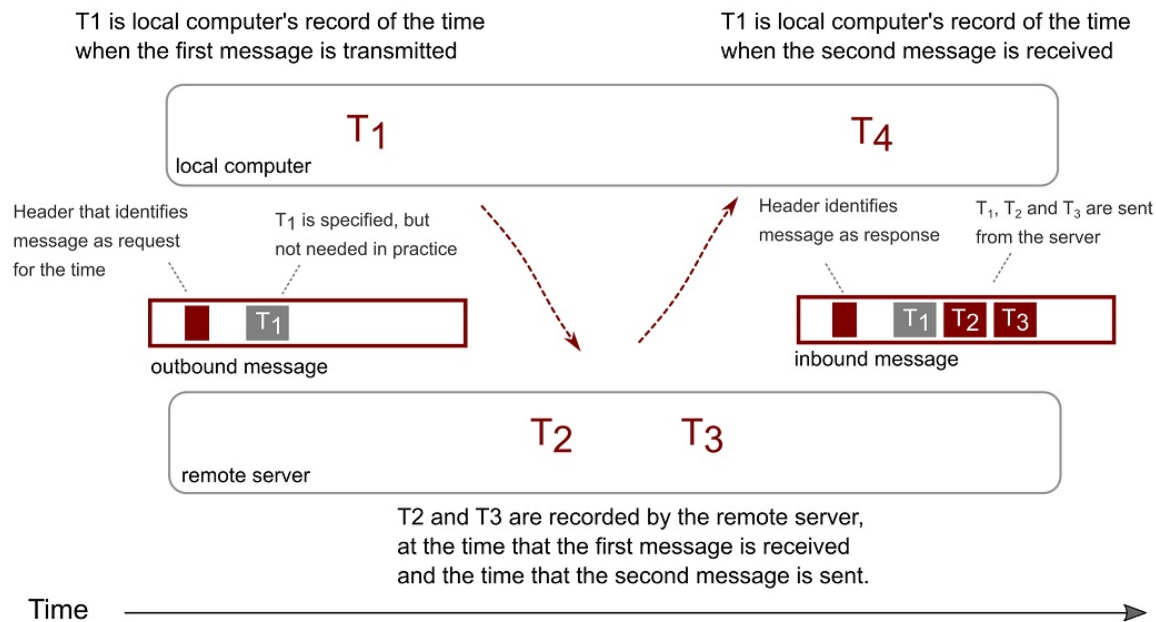
Let’s consider the client-server situation where your computer wants to correct its own time. For every computer that you check with — let’s call them *time servers* — there are two messages:

- The message from your computer to each time server is the *request*
- The reply is known as the *response*

These two messages generate four time points. They occur in serial, and their names are designated by the specification (RFC 2030):

- T_1 : the client’s timestamp of when the request was sent. Referred to as t_1 in code.
- T_2 : the time server’s timestamp when the request was received. Referred to as t_2 in code.
- T_3 : the time server’s timestamp of when it sends its response. Referred to as t_3 in code.
- T_4 : the client’s timestamp of when the response was received. Referred to as t_4 in code.

Figure 9.3. Timestamps that are defined within the Network Time Protocol



To see what this means in code, spend a few moments looking through [Listing 9.14](#). Lines 2-12 deal with establishing a connection. Lines 14-21 produce T₁ to T₄. Lines 2-12 deal with establishing a connection. Lines 14-21 produce T₁ to T₄.

Listing 9.14. Defining a function that sends NTP messages

```
fn ntp_roundtrip(host: &str, port: u16) -> Result<NTPResult, std::io::Error> {

    let destination = format!("{}", host, port);
    let timeout = Duration::from_secs(1);

    let request = NTPMessage::client();
    let mut response = NTPMessage::new();

    let message = request.data;

    let mut udp = UdpSocket::bind(LOCAL_ADDR)?;
    udp.connect(&destination).expect("unable to connect");

    let t1 = Utc::now();
    udp.send(&message)?;
    udp.set_read_timeout(Some(timeout));
    udp.recv_from(&mut response.data)?;
    let t4 = Utc::now();
```

①
②
③

```

let t2: DateTime<Utc> = response.rx_time().unwrap().into();    ④
let t3: DateTime<Utc> = response.tx_time().unwrap().into();    ⑤

Ok(NTPResult {
    t1: t1,
    t2: t2,
    t3: t3,
    t4: t4,
})
}

```

- ① This code cheats slightly by not even bothering to encode t_1 in the outbound message. In practice, this works perfectly well and requires fractionally less work.
- ② Send a request payload (which is defined elsewhere) to the server.
- ③ This call will *block* the application until data is ready to be received.
- ④ `rx_time()` stands for received timestamp and is the time that the server received the client’s message.
- ⑤ `tx_time()` stands for transmitted timestamp and is the time that the server sent the reply

T_1 to T_4 , encapsulated in [Listing 9.14](#) as `NTPResult`, are all that’s required to judge whether the local time matches the servers. The protocol contains some more sophistication relating to error handling, but that’s avoided here for simplicity. Otherwise, it’s a perfectly capable NTP client.

9.9.2 Adjusting the local time as a result of the server’s response

Given that our client has received at least one (and hopefully a few more) NTP responses, all that’s left to do is to calculate the “right” time. But wait, which time is “right”? All we have are relative timestamps. There is still no universal truth that we’ve been given access to.

The NTP documentation provides two equations to help resolve the situation. For those readers who don’t enjoy Greek letters, feel free to skim or even skip the next few paragraphs.

Our aim is to calculate two values:

1. the *time offset* is what we're ultimately interested in. It is denoted as θ (theta) by the official documentation. When θ is a positive number, our clock is fast. When it is negative, our clock is slow.
2. the *delay* caused by network congestion, latency and other noise. This is denoted as δ (delta). A large δ implies that the reading is less reliable. Our code will use this value to prefer to follow servers that respond quickly.

Table 9.2. How to calculate δ and θ in NTP

$\delta = (T_4 - T_1) - (T_3 - T_2)$	$(T_4 - T_1)$ calculates the total time spent on the client's side. $(T_3 - T_2)$ calculates the total time spent on the server's side. The difference between the two differences, e.g. δ, is an estimate of the difference between the clocks, plus a delay caused because of network traffic and processing.
$\theta = ((T_2 - T_1) + (T_4 - T_3)) / 2$	We take the average of the two pairs of timestamps.

The mathematics can be deceptively confusing because there is always an innate desire to know what the time *actually is*. That's impossible to know. All we have are assertions.

NTP is designed to operate multiple times per day, with participants nudging their clocks incrementally over time. Given sufficient adjustments, θ tends to 0 while δ remains relatively stable.

The standard is quite prescriptive about the formula to carry out the adjustments. For example, the reference implementation of NTP includes some useful filtering to limit the effect of bad actors and other spurious results.

We're going to cheat. We'll just take a mean of the differences, weighted by $1/\theta^2$. This aggressively penalizes slow servers.

To minimize the likelihood of any negative outcomes:

- We’re checking the time with “known good” actors. In particular, we’ll use time servers hosted by major operating system vendors and other reliable sources to minimize the chances of someone sending us a questionable result.
- No single result will affect the result too much. We’ll cap any adjustments we make to the local time to 200ms.

Listing 9.15. Adjusting the time according to the responses from of multiple time servers

```
fn check_time() -> Result<f64, std::io::Error> {
    const NTP_PORT: u16 = 123;

    let servers = [
        "time.nist.gov",
        "time.apple.com",
        "time.euro.apple.com",
        "time.google.com",           ①
        "time2.google.com",         ①
        //"time.windows.com",        ②
    ];

    let mut times = Vec::with_capacity(servers.len());

    for &server in servers.iter() {
        print!("{}", =>, server );

        let calc = ntp_roundtrip(&server, NTP_PORT);

        match calc {
            Ok(time) => {
                println!(" {}ms away from local system time", time.offset());
                times.push(time);
            },
            Err(_) => println!(" ? [response took too long]"),
        };
    }

    let mut offsets = Vec::with_capacity(servers.len());
    let mut offset_weights = Vec::with_capacity(servers.len());

    for time in &times {
        let offset = time.offset() as f64;
        let delay = time.delay() as f64;

        let weight = 1_000_000.0 / (delay*delay);           ③
        if weight.is_finite() {
            offsets.push(offset);
            offset_weights.push(weight);
        }
    }
}
```



```

    let avg_offset = weighted_mean(&offsets, &offset_weights);

    Ok(avg_offset)
}

```

- ① Google's time servers implement leap seconds by expanding the length of a second, rather than adding an extra second. Thus, for one day approximately every 18 months, this server may report a different time than the others.
- ② At the time of writing, Microsoft's time server appeared to be providing a time that was 15 seconds ahead of its peers.
- ③ Penalize slow servers by substantially decreasing their relative weights

9.9.3 Converting between time representations that use different precisions and epochs

`chrono` represents the fractional part of a second down to nanosecond precision, whereas NTP can represent times that differ between approximately 250 picoseconds. That's roughly 4 times more precise. The different internal representations used imply that some accuracy is likely to be lost during conversions.

The mechanism for telling Rust that two types can be converted is the `From` trait. `From` provides the `from()` method, which is encountered early on in one's Rust career in example such as `String::from("Hello, world!")`.

[Listing 9.16](#) provides implementations of `From` and `Into` that allow explicit type conversion. This code allows the line `response.rx_time().unwrap().into();` within [Listing 9.14](#) to work.

Listing 9.16. Converting between `chrono::DateTime` and NTP timestamps

```

#[derive(Default, Debug, Copy, Clone)]
struct NTPTimestamp {
    seconds: u32,
    fraction: u32,
}

impl From<NTPTimestamp> for DateTime<Utc> {
    fn from(ntp: NTPTimestamp) -> Self {

```

①
①
①
①

```

        let secs = ntp.seconds as i64 - NTP_TO_UNIX_SECONDS; ②
        let mut nanos = ntp.fraction as f64;
        nanos *= 1e9;
        nanos /= 2_f64.powi(32); ③

        Utc.timestamp(secs, nanos as u32)
    }
}

impl From<DateTime<Utc>> for NTPTimestamp {
    fn from(utc: DateTime<Utc>) -> Self {
        let secs = utc.timestamp() + NTP_TO_UNIX_SECONDS;
        let mut fraction = utc.nanosecond() as f64;
        fraction *= 2_f64.powi(32);
        fraction /= 1e9;

        NTPTimestamp {
            seconds: secs as u32,
            fraction: fraction as u32,
        }
    }
}

```

- ① Our internal type to represent a NTP timestamp
- ③ Defined as 2_208_988_800, which is the number of seconds between 1 Jan 1900 (the NTP epoch) and 1 Jan 1970 (the UNIX epoch).
- ② These conversions can be implemented using bit shift operations, at the expense of even less readability.

From has a reciprocal peer, Into. Implementing From will allow Rust to automatically generate an Into implementation on its own, except in advanced cases. In those advanced cases, it's likely that developers already possess the knowledge required to implement Into manually and so probably don't need assistance here.

9.9.4 c1ock v0.1.3 full code listing

The whole code listing for c1ock is presented in [Listing 9.17](#).

Taken in its full glory, the whole of c1ock can look quite large and imposing. Hopefully there is no new Rust syntax within [Listing 9.17](#) to digest.

Listing 9.17. Full listing for the command-line NTP client implemented in this chapter. (ch9/ch9-clock3/src/main.rs)

```

#[cfg(windows)] use kernel32;
#[cfg(windows)] use winapi;
#[cfg(not(windows))] use libc;

use byteorder::{BigEndian, ReadBytesExt};
use chrono::{DateTime, Duration as ChronoDuration, TimeZone, Timelike};
use chrono::{Local, Utc};
use clap::{App, Arg};
use std::mem::zeroed;
use std::net::UdpSocket;
use std::time::Duration;

const NTP_MESSAGE_LENGTH: usize = 48; ①
const NTP_TO_UNIX_SECONDS: i64 = 2_208_988_800;
const LOCAL_ADDR: &'static str = "0.0.0.0:12300"; ②

#[derive(Default, Debug, Copy, Clone)]
struct NTPTimestamp {
    seconds: u32,
    fraction: u32,
}

struct NTPMessage {
    data: [u8; NTP_MESSAGE_LENGTH],
}

#[derive(Debug)]
struct NTPResult {
    t1: DateTime<Utc>,
    t2: DateTime<Utc>,
    t3: DateTime<Utc>,
    t4: DateTime<Utc>,
}

impl NTPResult {
    fn offset(&self) -> i64 {
        let duration = (self.t2 - self.t1) + (self.t4 - self.t3);
        duration.num_milliseconds() / 2
    }

    fn delay(&self) -> i64 {
        let duration = (self.t4 - self.t1) - (self.t3 - self.t2);
        duration.num_milliseconds()
    }
}

impl From<NTPTimestamp> for DateTime<Utc> {
    fn from(ntp: NTPTimestamp) -> Self {
        let secs = ntp.seconds as i64 - NTP_TO_UNIX_SECONDS;
        let mut nanos = ntp.fraction as f64;
        nanos *= 1e9;
        nanos /= 2_f64.powi(32);

        Utc.timestamp(secs, nanos as u32)
    }
}

impl From<DateTime<Utc>> for NTPTimestamp {
    fn from(utc: DateTime<Utc>) -> Self {

```

```

        let secs = utc.timestamp() + NTP_TO_UNIX_SECONDS;
        let mut fraction = utc.nanosecond() as f64;
        fraction *= 2_f64.powi(32);
        fraction /= 1e9;

        NTPTimestamp {
            seconds: secs as u32,
            fraction: fraction as u32,
        }
    }
}

impl NTPMessage {
    fn new() -> Self {
        NTPMessage {
            data: [0; NTP_MESSAGE_LENGTH],
        }
    }

    fn client() -> Self {
        const VERSION: u8 = 3; // 0000_0011
        const MODE: u8 = 3;

        let mut msg = NTPMessage::new();

        msg.data[0] |= VERSION << 3; ③
        msg.data[0] |= MODE;           ③
        msg                                     ④
    }

    fn parse_timestamp(&self, i: usize) -> Result<NTPTimestamp, std::io::Error> {
        let mut reader = &self.data[i..i + 8]; ⑤
        let seconds = reader.read_u32::<BigEndian>()?;
        let fraction = reader.read_u32::<BigEndian>()?;

        Ok(NTPTimestamp {
            seconds: seconds,
            fraction: fraction,
        })
    }

    fn rx_time(&self) -> Result<NTPTimestamp, std::io::Error> { ⑥
        self.parse_timestamp(32)
    }

    fn tx_time(&self) -> Result<NTPTimestamp, std::io::Error> { ⑦
        self.parse_timestamp(40)
    }
}

fn weighted_mean(values: &[f64], weights: &[f64]) -> f64 {
    let mut sum = 0.0;
    let mut sum_of_weights = 0.0;

    for (v, w) in values.iter().zip(weights) {
        sum += v * w;
        sum_of_weights += w;
    }

    sum / sum_of_weights
}

```

```

}

fn ntp_roundtrip(host: &str, port: u16) -> Result<NTPResult, std::io::Error> {
    let destination = format!("{:}:{:}", host, port);
    let timeout = Duration::from_secs(1);

    let request = NTPMessage::client();
    let mut response = NTPMessage::new();

    let message = request.data;

    let udp = UdpSocket::bind(LOCAL_ADDR)?;
    udp.connect(&destination).expect("unable to connect");

    let t1 = Utc::now();
    udp.send(&message)?;
    udp.set_read_timeout(Some(timeout))?;
    udp.recv_from(&mut response.data)?;
    let t4 = Utc::now();

    let t2: DateTime<Utc> = response.rx_time().unwrap().into();
    let t3: DateTime<Utc> = response.tx_time().unwrap().into();

    Ok(NTPResult {
        t1: t1,
        t2: t2,
        t3: t3,
        t4: t4,
    })
}

fn check_time() -> Result<f64, std::io::Error> {
    const NTP_PORT: u16 = 123;

    let servers = [
        "time.nist.gov",
        "time.apple.com",
        "time.euro.apple.com",
        "time.google.com",
        "time2.google.com",
        //"time.windows.com",
    ];

    let mut times = Vec::with_capacity(servers.len());

    for &server in servers.iter() {
        print!("{:} =>", server);

        let calc = ntp_roundtrip(&server, NTP_PORT);

        match calc {
            Ok(time) => {
                println!(" {:}ms away from local system time", time.offset());
                times.push(time);
            }
            Err(_) => println!(" ? [response took too long]"),
        };
    }

    let mut offsets = Vec::with_capacity(servers.len());

```

```

let mut offset_weights = Vec::with_capacity(servers.len());

for time in &times {
    let offset = time.offset() as f64;
    let delay = time.delay() as f64;

    let weight = 1_000_000.0 / (delay * delay); ③
    if weight.is_finite() {
        offsets.push(offset);
        offset_weights.push(weight);
    }
}

let avg_offset = weighted_mean(&offsets, &offset_weights);

Ok(avg_offset)
}

struct Clock;

impl Clock {
    fn get() -> DateTime<Local> {
        Local::now()
    }

    #[cfg(windows)]
    fn set<Tz: TimeZone>(t: DateTime<Tz>) -> () {
        use chrono::Weekday;
        use kernel32::SetSystemTime;
        use winapi::{SYSTEMTIME, WORD};

        let t = t.with_timezone(&Local);

        let mut systime: SYSTEMTIME = unsafe { zeroed() };

        let dow = match t.weekday() {
            Weekday::Mon => 1,
            Weekday::Tue => 2,
            Weekday::Wed => 3,
            Weekday::Thu => 4,
            Weekday::Fri => 5,
            Weekday::Sat => 6,
            Weekday::Sun => 0,
        };

        let mut ns = t.nanosecond();
        let is_leap_second = ns > 1_000_000_000;

        if is_leap_second {
            ns -= 1_000_000_000;
        }

        systime.wYear = t.year() as WORD;
        systime.wMonth = t.month() as WORD;
        systime.wDayOfWeek = dow as WORD;
        systime.wDay = t.day() as WORD;
        systime.wHour = t.hour() as WORD;
        systime.wMinute = t.minute() as WORD;
        systime.wSecond = t.second() as WORD;
        systime.wMilliseconds = (ns / 1_000_000) as WORD;
    }
}

```

```

    let systime_ptr = &systime as *const SYSTEMTIME;

    unsafe {
        SetSystemTime(systime_ptr);
    }
}

#[cfg(not(windows))]
fn set<Tz: TimeZone>(t: DateTime<Tz>) -> () {
    use libc::settimeofday;
    use libc::{suseconds_t, time_t, timeval, timezone};

    let t = t.with_timezone(&Local); // variable names indicate the data's
    progression
    let mut u: timeval = unsafe { zeroed() }; // through the function

    u.tv_sec = t.timestamp() as time_t;
    u.tv_usec = t.timestamp_subsec_micros() as suseconds_t;

    unsafe {
        let mock_tz: *const timezone = std::ptr::null();
        settimeofday(&u as *const timeval, mock_tz);
    }
}

fn main() {
    let app = App::new("clock")
        .version("0.1.3")
        .about("Gets and sets the time.")
        .after_help("Note: UNIX timestamps are parsed as whole seconds since
1st January 1970 0:00:00 UTC. For more accuracy, use another format.")
        .arg(Arg::with_name("action")
            .takes_value(true)
            .possible_values(&["get", "set", "check-ntp"])
            .default_value("get"))
        .arg(Arg::with_name("std")
            .short("s")
            .long("use-standard")
            .takes_value(true)
            .possible_values(&["rfc2822", "rfc3339", "timestamp"])
            .default_value("rfc3339"))
        .arg(Arg::with_name("datetime")
            .help("When <action> is 'set', apply <datetime>.
Otherwise, ignore."));

    let args = app.get_matches();

    let action = args.value_of("action").unwrap(); // default_value() has been
    supplied,
    let std = args.value_of("std").unwrap(); // so it's safe to use .unwrap()

    if action == "set" {
        let t_ = args.value_of("datetime").unwrap();

        let parser = match std {
            "rfc2822" => DateTime::parse_from_rfc2822,
            "rfc3339" => DateTime::parse_from_rfc3339,
            _ => unimplemented!(),
        }
    }
}

```

```

};

let err_msg = format!("Unable to parse {} according to {}", t_, std);
let t = parser(t_).expect(&err_msg);

Clock::set(t);
} else if action == "check-ntp" {
    let offset = check_time().unwrap() as isize;
    let adjust_ms = offset.signum() * offset.abs().min(200) / 5;
    let adjust_ms_ = ChronoDuration::milliseconds(adjust_ms as i64);
    let now: DateTime<Utc> = Utc::now() + adjust_ms_;
    Clock::set(now);
}

let maybe_error = std::io::Error::last_os_error();
let os_error_code = &maybe_error.raw_os_error();
match os_error_code {
    Some(0) => (),
    None => (),
    _ => eprintln!("Unable to set the time: {:?}", maybe_error),
}

let now = Clock::get();

match std {
    "timestamp" => println!("{}", now.timestamp()),
    "rfc2822" => println!("{}", now.to_rfc2822()),
    "rfc3339" => println!("{}", now.to_rfc3339()),
    _ => unreachable!(),
}
}

```

- ① 12*4 bytes, e.g. the width of 12 32-bit integers
- ② 12300 is the default port for NTP
- ③ The first byte of every NTP message contains three fields, but we only need to set two of them.
- ④ msg.data[0] now equal to 0001_1011 (27 in decimal)
- ⑤ This takes a slice to the first byte
- ⑥ "RX" stands for receive.
- ⑦ "TX" stands for transmit.

9.10 SUMMARY

In this chapter, you have learned

- Different strategies for handling the imperfect nature of the Earth's

rotation around the Sun

- Parsing timestamps from strings into Rust objects via the chrono crate
- Interacting with libc, including accessing type aliases and calling functions via unsafe blocks
- Using system calls to set the system clock
- Accessing error codes provided by the operating system
- Dealing with precision differences between internal representations
- Converting between types by implementing From
- Silencing intermediate output from cargo via the -q option

10 PROCESSES, THREADS AND CONTAINERS

So far this book has almost completely avoided two fundamental terms of systems programming: threads and processes. Instead, the book has used the single term: program. This chapter expands our vocabulary.

Processes, threads and containers are abstractions created to enable multiple tasks to be carried out at the same time. They enable *concurrency*. Its peer term, *parallelism*, means to make use of multiple physical CPU cores at the same time.

Counter-intuitively, it is possible to have a concurrent system on a single CPU core. Because accessing data from memory and I/O take a very long time, threads requesting data can be set to a “blocked” state. Blocked threads are re-scheduled when their data is available.

Concurrency, or doing multiple things at the same time, is difficult to introduce into a computer program. Employing concurrency effectively involves both new concepts and new syntax.

The aim of this chapter is to give you the confidence to explore more advanced material. You will have a solid understanding of the different tools that you have available to you as an applications programmer. It exposes you to standard library and the well engineered crates `crossbeam` and `rayon`. It won't give you sufficient background to be able to implement your own concurrency crates, but will enable you to use them.

The chapter follows the following structure:

- An introduction into Rust's closure syntax at [Section 10.1](#). Closures are also known as anonymous functions and lambda functions. The syntax is important because the standard library and many (perhaps all) external crates rely on that syntax to provide support for their

concurrency model.

- A quick lesson at [Section 10.2](#) on what a thread is and how to create (“spawn”) them.
- [Section 10.2.3](#) discusses why programmers are warned against spawning tens of thousands of threads.
- [Section 10.4](#) introduces the chapter’s large example. It implements multi-threaded parser and code generator using multiple strategies. As a nice aside, the example creates procedural art from its input.
- The chapter concludes with an overview of other forms of concurrency, such as processes and containers.

10.1 ANONYMOUS FUNCTIONS

This chapter is fairly dense. Let’s get some points on the board quickly with some basic syntax and some practical examples. We’ll circle back to fill in a lot of the conceptual and theoretical material.

Threads—and other forms of code that can run out of band—use a form of function definition that we’ve avoided for the bulk of the book.

Defining a function looks like this:

```
fn add(a: i32, b: i32) -> i32 {  
    a + b  
}
```

The (loosely) equivalent lambda function is:

```
let add = |a,b| { a + b };
```

Lambda functions are denoted by the pair of vertical bars followed by curly brackets. The pair of vertical bars allows arguments to be defined.

Lambda functions in Rust can read variables from within their scope. They are closures.

Unlike regular functions, lambda functions cannot be defined in global scope. [Listing 10.1](#) gets around this by defining one within its `main()`.

Listing 10.1. Defining two functions, one of which is a lambda function, and checking that they produce the same result

```
fn add(a: i32, b: i32) -> i32 {
    a + b
}

fn main() {
    let lambda_add = |a,b| { a + b };

    assert_eq!(add(4,5), lambda_add(4,5));
}
```

When [Listing 10.1](#) is run, it executes happily (and silently). Let's now see how to put this functionality to work.

10.2 SPAWNING THREADS

To spawn a thread in Rust, we pass an anonymous function to `std::thread::spawn()`.

As described in [Section 10.1](#), anonymous functions are defined with two vertical bars to provide arguments and then curly brackets for the function's body: `|| { ... }`. Arguments to `spawn` don't take any arguments though, so you will typically encounter this syntax:

```
thread::spawn(|| { ①
    // ...         ②
});
```

① Anonymous function is defined in-line

② Variables come into the function's scope as

When the spawned thread wishes to access variables that are defined in the parent's scope, Rust will usually complain that they must be "moved" there. To indicate that, anonymous functions take a `move` keyword.

```
thread::spawn(move || { ①
    // ...
});
```

① The `move` keyword allows the anonymous function to access variables from their wider scope.

There are some necessary considerations for using variables defined in the outer scope. To reduce friction at compile time, implement copy. Values originating in outer scopes may need to have the 'static lifetime. Spawned sub-threads can outlive their parents. From the point of view of the compiler, values may need to be able to be accessible forever.

A simple task waits: sleeping the CPU for 300 milliseconds. If you have a 3Ghz CPU, you're giving it a rest of nearly 1 billion cycles. Those electrons will be very relieved.

When executed, [Listing 10.3](#) will print the total duration (in "wall clock time") of both threads executing.

Listing 10.2. Example output for [Listing 10.3](#)

```
300.218594ms...
```

Listing 10.3. Sleeping a subthread for 300ms

```
use std::{thread, time};

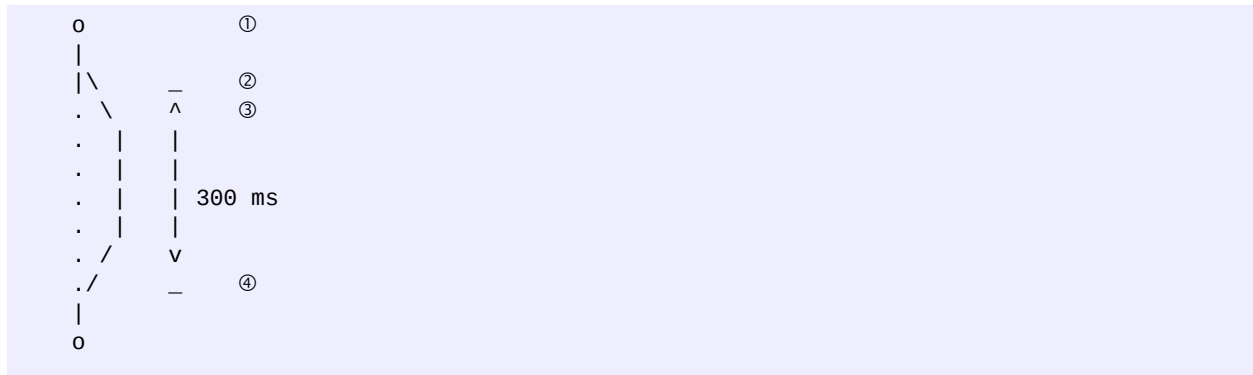
fn main() {
    let start = time::Instant::now();

    let handler = thread::spawn(move || {
        let pause = time::Duration::from_millis(300);
        thread::sleep(pause.clone());
    });

    handler.join().unwrap();

    let finish = time::Instant::now();

    println!("{:02?}", finish.duration_since(start));
}
```



- ① `main()` begins executing in the main thread
- ② `thread::spawn()` is called, creating a child thread. The handler is created in the main thread.
- ③ `handler.join()` called here. Execution is suspended by the operating system.
- ④ Child thread as completed its work. It is "joined" back into the main thread with the `handler.join()` function.

10.2.1 What does it mean to "join" threads?

The term "join()" is fairly ubiquitous, but what does it mean? It means "wait for the other thread to finish". Both threads are running, but the thread that is currently waiting is "blocked". The `join()` function instructs the operating system to defer scheduling it again until the condition has reached: the condition in this case is the other thread to finish.

10.2.2 Creating more threads takes almost no time at all

In ideal settings, adding a second thread doubles the work we can do in the same amount of time. That's what happens with [Listing 10.5](#). As with [Listing 10.3](#), [Listing 10.5](#) prints out almost the same output:

Listing 10.4. Example output of [Listing 10.5](#)

300.242328ms

Listing 10.5. Creating two sub-threads to perform work on our behalf

```
use std::{thread, time};

fn main() {
    let start = time::Instant::now();

    let handler_1 = thread::spawn(move || {
        let pause = time::Duration::from_millis(300);
        thread::sleep(pause.clone());
    });

    let handler_2 = thread::spawn(move || {
        let pause = time::Duration::from_millis(300);
        thread::sleep(pause.clone());
    });

    handler_1.join().unwrap();
    handler_2.join().unwrap();

    let finish = time::Instant::now();

    println!("{:?}", finish.duration_since(start));
}
```

As you'll notice, there is a negligible impact from creating a thread or two. But, if you've had any exposure to the field before, you may have heard that threads don't scale. What does that mean?

Well, the marginal impact of the next thread isn't constant. Every thread requires its own memory, and by implication, we'll eventually exhaust our system's memory. Before that terminal point though, thread creation will begin to trigger slowdowns in other areas. As the number of threads to schedule increases, the scheduler's work increases. When there are many threads to schedule, deciding which thread to schedule next takes more time.

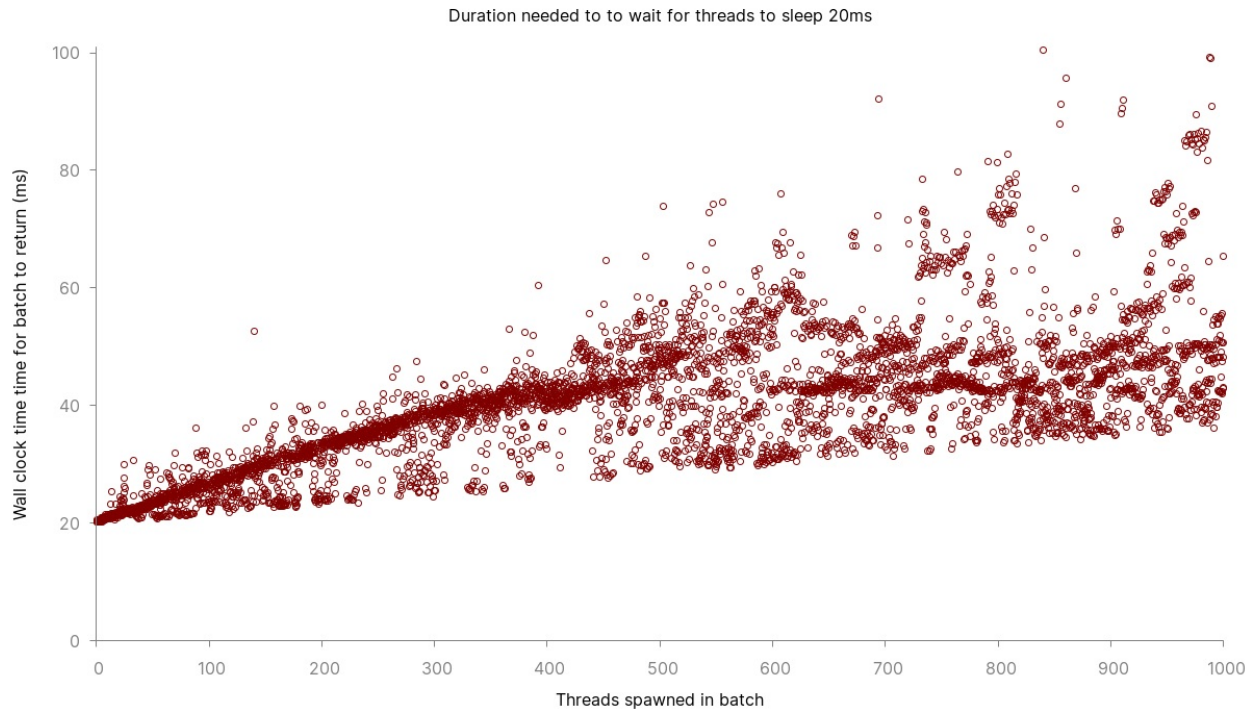
10.2.3 Effect of spawning many threads

Spawning threads is not free. It demands memory and CPU time. Switches between threads also invalidates caches.

[Figure 10.1](#) shows the data generated by successive runs of [Listing 10.6](#). The variance stays quite tight until about 400 threads per batch. After that, there's

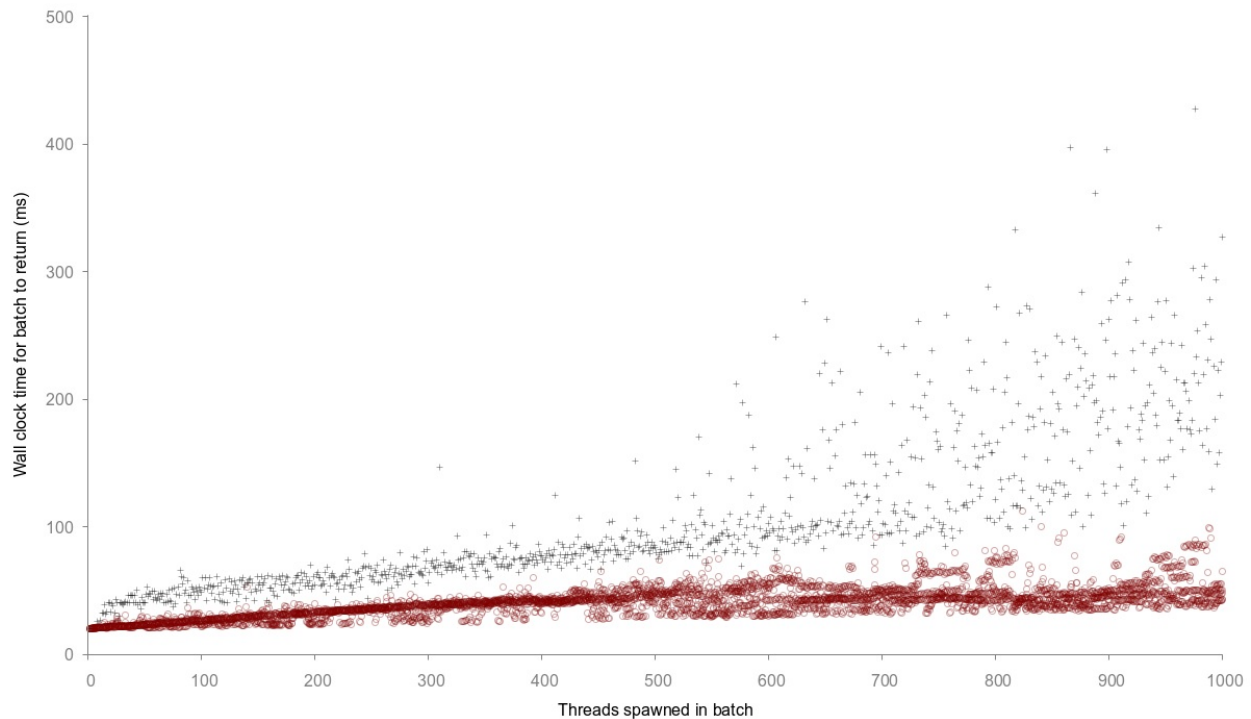
almost no knowing how long a 20ms sleep will take.

Figure 10.1. Duration needed to wait for threads to sleep 20ms



And if you're thinking that sleeping is not a representative workload, the next plot is even more telling. It asks each thread to enter a "spin loop".

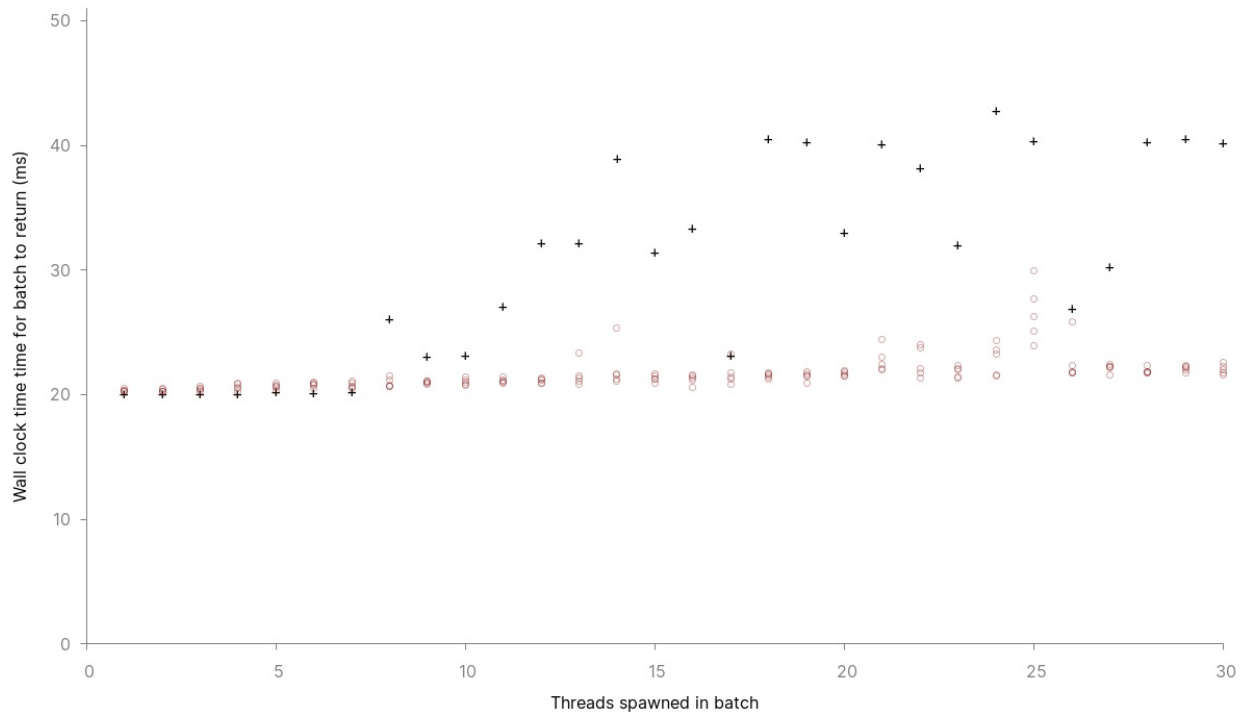
Figure 10.2. Comparing the time taken to Duration needed to wait for threads to consume 100% CPU in a spin loop for 20ms (plus symbols), overlaid against the time taken to sleep (circles)



There are features of [Figure 10.2](#) that is worth focusing in on very briefly. First, for the first 7 or so batches, the spin loop version returned closer to 20ms. The operating system’s sleep functionality isn’t perfectly accurate. If you want to “sleep” pause a thread for very short amounts of time, or if your application is very sensitive to timing, use a spin loop [\[43\]](#).

Secondly, CPU-intensive multi-threading doesn’t scale well past the number of physical cores. The benchmarking was performed on a 6 core CPU with hyper-threading disabled (the Intel i7-8750H). As soon as the thread count exceeds the core count, performance degrades quickly.

Figure 10.3. Comparing the time taken to wait for 20ms using sleep and spin loop strategies, focusing on a small number of threads per batch.



Now that we've seen the effects of threading, let's look at the code that generated the input data to the plots [Figure 10.1](#) and [Figure 10.2](#). You are welcome to reproduce the results. To do so, write output of the [Listing 10.6](#) and [Listing 10.7](#) to files and analyze the resulting data.

Listing 10.6. Using `thread::sleep` to measure the impact of increasing numbers of threads on performance

```
use std::{thread, time};

fn main() {
    for n in 1..1001 {
        let mut handlers: Vec<thread::JoinHandle<()>> = Vec::with_capacity(n);

        let start = time::Instant::now();
        for _m in 0..n {
            let handle = thread::spawn(|| {
                let pause = time::Duration::from_millis(20);
                thread::sleep(pause);
            });
            handlers.push(handle);
        }

        while let Some(handle) = handlers.pop() {
            handle.join();
        }
    }
}
```

```

    let finish = time::Instant::now();
    println!("{}", n, finish.duration_since(start));
}
}

```

Listing 10.7. Using a spin loop waiting strategy. (c10/ch10-busythreads/src/main.rs)

```

use std::{thread, time};

fn main() {
    for n in 1..1001 {
        let mut handlers: Vec<thread::JoinHandle<>> = Vec::with_capacity(n);

        let start = time::Instant::now();
        for _m in 0..n {
            let handle = thread::spawn(|| {
                let start = time::Instant::now();
                let pause = time::Duration::from_millis(20);
                while start.elapsed() < pause {
                    thread::yield_now();
                }
            });
            handlers.push(handle);
        }

        while let Some(handle) = handlers.pop() {
            handle.join();
        }

        let finish = time::Instant::now();
        println!("{}", n, finish.duration_since(start));
    }
}

```

The control flow we've chosen for lines 12-14 is slightly odd. Rather than iterating through the handlers vector, we're calling `pop()` and draining it.

Listing 10.8. What we would expect to see in [Listing 10.7](#)

```

for handle in &handlers {
    handle.join();
}

```

Listing 10.9. Code that's used in [Listing 10.7](#)

```

while let Some(handle) = handlers.pop() {
    handle.join();
}

```

Why use the more complex control flow mechanism? It might help to remember that once we `join()` a thread back to the main thread, it won't exist. Rust won't allow us to retain a reference to something that doesn't exist. Therefore, to call `join()` on any threads within handlers, they must be removed.

Listing 10.10. Using a spin loop waiting strategy, . (c10/ch10-busythreads-broken/src/main.rs)

```
use std::{thread, time};

fn main() {
    for n in 1..1001 {
        let mut handlers: Vec<thread::JoinHandle<>> = Vec::with_capacity(n);

        let start = time::Instant::now();
        for _m in 0..n {
            let handle = thread::spawn(|| {
                let start = time::Instant::now();
                let pause = time::Duration::from_millis(20);
                while start.elapsed() < pause {
                    thread::yield_now();
                }
            });
            handlers.push(handle);
        }

        for handle in &handlers {
            handle.join();
        }

        let finish = time::Instant::now();
        println!("{}", n, finish.duration_since(start));
    }
}
```

Listing 10.11. Output generated when attempting to compile [Listing 10.10](#)

```
$ cargo run -q
error[E0507]: cannot move out of `*handle` which is behind a shared reference
--> src/main.rs:20:13
   |
20 |         handle.join();
   |         ^^^^^^^ move occurs because `*handle` has type
   |         `std::thread::JoinHandle<>`, which does not implement the `Copy` trait

error: aborting due to previous error

For more information about this error, try `rustc --explain E0507`.
```

```
error: Could not compile `ch10-busythreads-broken`.
To learn more, run the command again with --verbose.
```

This error is saying that taking a reference isn't valid here. That's because multiple threads might also be taking their own references to the underlying threads. And those references need to be valid.

Astute readers know that there is actually a simpler way to get around this problem than what was used in [Listing 10.7](#). Simply remove the ampersand:

Listing 10.12. What we could have used in [Listing 10.7](#)

```
for handle in handlers {
    handle.join();
}
```

What we've encountered is one of those rare cases where taking a reference to an object causes more issues than using the object directly. Iterating over handlers directly retains ownership. That pushes any concerns about shared access to the side, and we proceed as intended.

YIELDING CONTROL WITH `THREAD::YIELD_NOW()`

As a reminder, the busy loop within [Listing 10.7](#) is the following code:

Listing 10.13. Excerpt from [Listing 10.7](#) showing the current thread yielding execution

```
while start.elapsed() < pause {
    thread::yield_now();
}
```

`std::thread::yield_now()` is a signal to the operating system that the current thread should be de-scheduled. This allows other threads to proceed while the current thread is still waiting for the 20ms time period to arrive.

A downside to yielding is that we don't know if we'll be able to resume exactly

at 20ms.

An alternative to yielding is to use the function

`std::sync::atomic::spin_loop_hint()`. `spin_loop_hint()` avoids the operating system, instead it is a signal directly to the CPU. A CPU might use that hint to turn off functionality, saving power use. The instruction is not present for every CPU. On platforms that don't support it, it does nothing.

10.2.4 Shared variable

In our threading benchmarks, we created pause variables in each thread. If you're not sure what I'm referring to, here is an excerpt:

Listing 10.14. Extract from [Listing 10.7](#) emphasizing the needless creation of `time::Duration` instances in each thread

```
let handle = thread::spawn(|| {
    let start = time::Instant::now();
    let pause = time::Duration::from_millis(20); ①
    while start.elapsed() < pause {
        thread::yield_now();
    }
});
```

① This variable doesn't need to be created in each thread.

What we want to be able to do is write something like this:

Listing 10.15. Attempting to share a variable in multiple sub-threads ([ch10/ch10-sharedpause-broken/src/main.rs](#))

```
use std::{thread, time};

fn main() {
    let pause = time::Duration::from_millis(20);
    let handle1 = thread::spawn(|| {
        thread::sleep(pause);
    });
    let handle2 = thread::spawn(|| {
        thread::sleep(pause);
    });

    handle1.join();
```

```
    handle2.join();
}
```

When attempting to run [Listing 10.15](#), we receive a very verbose—and surprisingly helpful—error message:

```
$ cargo run -q
error[E0373]: closure may outlive the current function, but it borrows `pause`,
which is owned by the current function
--> src/main.rs:5:33
5 |     let handle1 = thread::spawn(|| {
6 |         thread::sleep(pause);
7 |     });
8 |
note: function requires argument type to outlive `'static'
--> src/main.rs:5:19
5 |     let handle1 = thread::spawn(|| {
6 |         thread::sleep(pause);
7 |     });
8 |
help: to force the closure to take ownership of `pause` (and any other referenced
variables), use the `move` keyword
5 |     let handle1 = thread::spawn(move || {
6 |         thread::sleep(pause);
7 |     });
8 |
error[E0373]: closure may outlive the current function, but it borrows `pause`,
which is owned by the current function
--> src/main.rs:8:33
8 |     let handle2 = thread::spawn(|| {
9 |         thread::sleep(pause);
10 |     });
11 |
note: function requires argument type to outlive `'static'
--> src/main.rs:8:19
8 |     let handle2 = thread::spawn(|| {
9 |         thread::sleep(pause);
10 |     });
11 |
help: to force the closure to take ownership of `pause` (and any other referenced
variables), use the `move` keyword
8 |     let handle2 = thread::spawn(move || {
9 |         thread::sleep(pause);
10 |     });
11 |

error: aborting due to 2 previous errors

For more information about this error, try `rustc --explain E0373`.
```

```
error: Could not compile `ch10-sharedpause-broken`.
```

To learn more, run the command again with `--verbose`.

The fix is to add the `move` keyword to where the closures are created.

Listing 10.16. Using a variable defined in a parent scope in multiple closures by adding the `move` keyword. This switches the closures to use move semantics, which in turn relies on `copy`.

```
use std::{thread,time};

fn main() {
    let pause = time::Duration::from_millis(20);
    let handle1 = thread::spawn(move || {
        thread::sleep(pause);
    });
    let handle2 = thread::spawn(move || {
        thread::sleep(pause);
    });

    handle1.join();
    handle2.join();
}
```

The details of why this works are interesting. Read [Section 10.3](#) to look explore the details.

^[43] It's also possible to use both. Sleep for the bulk of the time and a spin loop towards the end.

10.3 DIFFERENCES BETWEEN CLOSURES (`|| {}`) AND FUNCTIONS (`fn`)

There are some differences between closures and functions. The differences prevent closures and functions from being used inter-changeably, which can cause problems for learners.

Closures and functions have different internal representations. Closures are anonymous structs that implement the `std::ops::FnOnce` trait, and potentially `std::ops::Fn` and `std::ops::FnMut`. Those structs are invisible in source code,

but contain any variables from the closure's environment that are used inside it.

Functions are implemented as *function pointers*. A function pointer is a pointer that points to code, not data. Code, when used in this sense, is computer memory that has been marked as executable.

To complicate matters, closures that do not enclose any variables from their environment are also function pointers.

Forcing the compiler to reveal the type of closures

The concrete type of a Rust closure is inaccessible as source code. The compiler creates it. To retrieve it, force a compiler error.

```
fn main() {  
    let a = 20;  
    let add_to_a = |b| { a + b }; ①  
    add_to_a == (); ②  
}
```

① Closures are values and can be assigned to a variable.

② A quick method to inspect a value's type is to attempt to perform an illegal operation on it. The compiler will quickly report it as an error message.

Among other errors, the compiler produces this one when attempting to compile the snippet:

```
Compiling playground v0.0.1 (/playground)  
error[E0369]: binary operation `==` cannot be applied to type  
`[closure@src/main.rs:3:20: 3:33]`  
  --> src/main.rs:4:14  
   |  
4 |     add_to_a == ();  
   |     ^^^^^^^  ^^  -- ()  
   |  
   | [closure@src/main.rs:3:20: 3:33]
```

10.4 PROCEDURALLY GENERATED AVATARS FROM A MULTI-THREADED PARSER AND CODE

GENERATOR

This section applies the syntax that we learned in [Section 10.2](#) to an application.

Let's say that we want the users of our app to have unique pictorial avatars by default. One approach for doing this is to take their username, take the digest of a hash function, then use those digits as parameter inputs to some procedural generation logic. Using this approach, everyone will have a visually similar, yet completely distinctive default avatars.

Our application creates parallax lines. It does this by using the characters within the base16 alphabet as opcodes for a LOGO-like language.

10.4.1 How to run `render-hex` and its intended output

In this section, we'll be producing three variations. They'll all be invoked in the same way.

Listing 10.17. A shell session demonstrating how to invoke `render-hex` and show its output

```
$ cd ch10-render-hex ①

$ cargo run -- $(echo 'Rust in Action' | sha256sum | cut -f1 -d' ') ②

$ ls ③
4705db24888d030540ada6d306f65ae770fdb8ab15d6a6899e755f1c1b93d973.svg  Cargo.lock
Cargo.toml  src  target

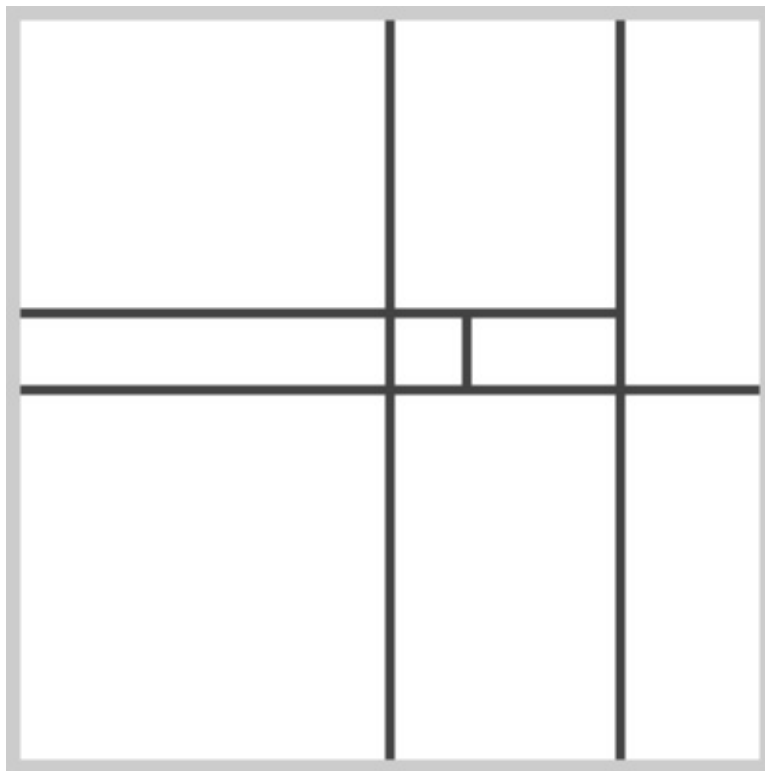
$ cat 4705db24888d030540ada6d306f65ae770fdb8ab15d6a6899e755f1c1b93d973.svg ④
<svg height="400" style='style="outline: 5px solid #800000;''
viewBox="0 0 400 400" width="400" xmlns="http://www.w3.org/2000/svg">
<rect fill="#ffffff" height="400" width="400" x="0" y="0"/>
<path d="M200,200 L200,360 L200,640 L200,200 L200,0 L200,0 L200,0 L200,-80 L200,
360 L200,680 L200,-120 L200,520 L200,200 L200,200 L320,200 L200,200 L400,200 L560,
200 L200,200 L200,200 L200,200 L200,200 L200,440 L200,200 L320,200 L200,200 L440,
200 L200,200 L200,-40 L200,400 L200,400 L200,400 L200,680 L200,-80 L200,200 L200,
200 L200,200 L200,200 L-120,200 L200,200 L200,200 L160,200 L-40,200 L200,200 L200,
440 L200,200 L-40,200 L520,200 L-160,200 L560,200 L200,200 L200,-80 L200,400 L200,
600 L200,200 L240,200 L240,200 L240,160 L240,160 L-120,160 L320,160 L320,160 L320,
520 L320,-80 L320,320" fill="none" stroke="#2f2f2f" stroke-opacity="0.9"
stroke-width="5"/>
<rect fill="#ffffff" fill-opacity="0.0" height="400" stroke="#cccccc"
```

```
stroke-width="15" width="400" x="0" y="0"/>
```

- ① Enter the project directory
- ② Create a base16 input and feed that to the resulting binary produced by cargo run (which will be the file target/debug/render-hex[.exe]). In this case, we'll use shell commands to produce one from the string "Rust in Action".
- ③ Verify that render-hex produces an output file
- ④ Inspect the file produced by invoking render-hex.

The resulting file renders as [Figure 10.4](#). To render SVG files, open them in a web browser or a vector image program, such as Inkscape.

Figure 10.4. The SHA256 of “Rust in Action” displayed as a diagram



10.4.2 Single-threaded render-hex overview

render-hex converts its input to an SVG file. The SVG file format succinctly describes drawings using mathematical operations. It's able to be viewed in a web

browser. Very little of the program relates to multi-threading at this stage, so I'll skip much of the details.

The program has a simple pipeline comprised of four steps:

- receive input from STDIN
- parse the input into operations that describe the movement of a pen across a sheet of paper
- convert the movement operations into their SVG equivalent
- generate an SVG file

Why can't we directly create path data from input? Splitting this process in to two steps allows more transformations.

This pipeline is managed directly within `main()`:

Listing 10.18. The `main()` function of `render-hex` (`ch10/ch10-render-hex/src/main.rs`). It parses command-line arguments and manages the SVG generation pipeline.

```
fn main() {  
    let args = env::args().collect::<Vec<String>>();  
    let input = args.get(1).unwrap();  
    let default_filename = format!("{}", input);  
    let save_to = args.get(2).unwrap_or(&default_filename);  
  
    let operations = parse(input);  
    let path_data = convert(&operations);  
    let document = generate_svg(path_data);  
    svg::save(save_to, &document).unwrap();  
}
```

① Command-line argument parsing.

② SVG generation pipeline.

INPUT PARSING

Our job here is to convert hexadecimal digits to instructions for a virtual pen that is travelling across a canvas. The `Operation` enum represents instructions. The term “operation” is used rather than instructions to avoid colliding with the

terminology used within the SVG specification for path drawing.

```
#[derive(Debug, Clone, Copy)]
enum Operation {
    Forward(usize),
    TurnLeft,
    TurnRight,
    Home,
    Noop(usize),
}
```

To parse, we treat every byte as an independent instruction. Numerals are converted to distances. Letters change the orientation of the drawing.

```
fn parse(input: &str) -> Vec<Operation> {
    let mut steps = Vec::<Operation>::new();
    for byte in input.bytes() {
        let step = match byte {
            b'0' => Home,
            b'1'..=b'9' => {
                let distance = (byte - 0x30) as isize; ①
                Forward(distance * (HEIGHT/10))
            },
            b'a' | b'b' | b'c' => TurnLeft,           ②
            b'd' | b'e' | b'f' => TurnRight,          ②
            _ => Noop(byte),                          ③
        };
        steps.push(step);
    }
    steps
}
```

- ① In ASCII, numerals start at 0x30 (48 in base 10). So this converts the u8 value of b'2' to 2. Performing this operation on the whole range of u8 could cause a panic. We are safe here, thanks to guarantee provided by our pattern matching.
- ② There is lots of opportunity to add more instructions to produce more elaborate diagrams, without increasing the parsing complexity.
- ③ Although we don't expect any illegal characters, there may be some in the input stream. Using a Noop operation allows us to decouple parsing from producing output.

INTERPRET INSTRUCTIONS

The Artist struct maintains the state of the diagram. Conceptually, the Artist is

holding a pen at the coordinate (x, y) and is moving it in the direction of heading.

```
#[derive(Debug)]
struct Artist {
    x: isize,
    y: isize,
    heading: Orientation,
}
```

To “move”, Artist implements several methods, two of which are highlighted in [Listing 10.19](#). Rust’s match expressions are used to succinctly refer to and modify internal state.

Listing 10.19. Two of the methods defined within the `impl Artist` block of `render-hex`. (code/ch10-render-hex/src/main.rs)

```
fn forward(&mut self, distance: isize) {
    match self.heading {
        North => self.y += distance,
        South => self.y -= distance,
        West  => self.x += distance,
        East  => self.x -= distance,
    }
}

fn turn_right(&mut self) {
    self.heading = match self.heading {
        North => East,
        South => West,
        West  => North,
        East  => South,
    }
}
```

The `convert()` function makes use of the Artist struct. Its role is to convert the `Vec<Operation>` from `parse()` to a `Vec<Command>`. That output that will be used later to generate an SVG. As a nod to the LOGO language, the Artist is given the local variable name `turtle`.

Listing 10.20. Extract from `render-hex` focusing on the `convert()` function. (code/ch10-render-hex/src/main.rs)

```
fn convert(operations: &Vec<Operation>) -> Vec<Command> {
    let mut turtle = Artist::new();
```

```

let mut path_data = Vec::<Command>::with_capacity(1+operations.len());
path_data.push(Command::Move(Position::Absolute, (HOME_X, HOME_Y).into())); ①

for op in operations {
    match *op {
        Forward(distance) => turtle.forward(distance), ②
        TurnLeft          => turtle.turn_left(),         ②
        TurnRight         => turtle.turn_right(),        ②
        Home              => turtle.home(),              ②
        Noop(byte)        => eprintln!("warning: illegal byte encountered: {:?}",
byte),
    };
    path_data.push(Command::Line(Position::Absolute, (turtle.x, turtle.y).into()));
③
    turtle.wrap();                                     ④
}
path_data
}

```

- ① To start, the turtle is positioned in the center of the drawing area.
- ② We don't generate a Command immediately. Instead we modify the internal state of turtle.
- ③ Create a Command::Line, e.g. a straight line, towards the turtle's current position.
- ④ If the turtle is out of bounds, return it to the center.

GENERATING AN SVG

The process of generating the SVG file is rather mechanical. `generate_svg()` (lines 134-164) does the work.

SVG documents look a lot like HTML documents, although the tags and attributes are different. The `<path>` tag is most important for our purposes. It has a `d` attribute (`d` is short for data) that describes how the path should be drawn. `convert()` produces a `Vec<Command>` that maps directly to the path data.

SOURCE CODE FOR THE SINGLE-THREADED VERSION OF RENDER-HEX

`render-hex` has an orthodox project structure. The whole project sits within a (fairly large) `main.rs` file, managed by cargo:

Listing 10.21. Project structure for render-hex

```
ch10-render-hex/  
├── src  
│   └── main.rs  
└── Cargo.toml
```

Listing 10.22. Project metadata for render-hex (code/ch10-render-hex/Cargo.toml)

```
[package]  
name = "render-hex"  
version = "0.1.0"  
authors = ["Tim McNamara <author@rustinaction.com>"]  
edition = "2018"  
  
[dependencies]  
svg = "0.6.0"
```

Listing 10.23. Source code for render-hex (code/ch10-render-hex/src/main.rs)

```
use std::env;  
  
use svg::Document;  
use svg::node::element::{Path, Rectangle};  
use svg::node::element::path::{Command, Position, Data};  
  
use crate::Orientation::{North, East, West, South};  
use crate::Operation::{Forward, TurnLeft, TurnRight, Home, Noop};  
  
const WIDTH: isize = 400;  
const HEIGHT: isize = WIDTH;  
const HOME_Y: isize = HEIGHT/2;  
const HOME_X: isize = WIDTH/2;  
const STROKE_WIDTH: usize = 5;  
  
#[derive(Debug, Clone, Copy)]  
enum Orientation {  
    North, East, West, South,  
}  
  
#[derive(Debug, Clone, Copy)]  
enum Operation {  
    Forward(isize),  
    TurnLeft,  
    TurnRight,  
    Home,  
    Noop(u8),  
}
```



```

#[derive(Debug)]
struct Artist { ⑩
    x: isize,
    y: isize,
    heading: Orientation,
}

impl Artist {
    fn new() -> Artist {
        Artist {
            heading: North,
            x: HOME_X,
            y: HOME_Y,
        }
    }

    fn home(&mut self) {
        self.x = HOME_X;
        self.y = HOME_Y;
    }

    fn forward(&mut self, distance: isize) { ⑩
        match self.heading { ⑩
            North => self.y += distance, ⑩
            South => self.y -= distance, ⑩
            West  => self.x += distance, ⑩
            East  => self.x -= distance, ⑩
        } ⑩
    }

    fn turn_right(&mut self) { ⑪
        self.heading = match self.heading { ⑪
            North => East, ⑪
            South => West, ⑪
            West  => North, ⑪
            East  => South, ⑪
        } ⑪
    }

    fn turn_left(&mut self) { ⑪
        self.heading = match self.heading { ⑪
            North => West, ⑪
            South => East, ⑪
            West  => South, ⑪
            East  => North, ⑪
        } ⑪
    }

    fn wrap(&mut self) { ⑫
        if self.x < 0 {
            self.x = HOME_X;
            self.heading = West;
        } else if self.x > WIDTH {
            self.x = HOME_X;
            self.heading = East;
        }

        if self.y < 0 {
            self.y = HOME_Y;
            self.heading = North;
        } else if self.y > HEIGHT {

```

```

        self.y = HOME_Y;
        self.heading = South;
    }
}

fn parse(input: &str) -> Vec<Operation> {
    let mut steps = Vec::<Operation>::new();
    for byte in input.bytes() {
        let step = match byte {
            b'0' => Home,
            b'1'..=b'9' => {
                let distance = (byte - 0x30) as isize; ⑬
                Forward(distance * (HEIGHT/10))
            },
            b'a' | b'b' | b'c' => TurnLeft,
            b'd' | b'e' | b'f' => TurnRight,
            _ => Noop(byte), ⑭
        };
        steps.push(step);
    }
    steps
}

fn convert(operations: &Vec<Operation>) -> Vec<Command> {
    let mut turtle = Artist::new();

    let mut path_data = Vec::<Command>::with_capacity(1+operations.len());
    path_data.push(Command::Move(Position::Absolute, (HOME_X, HOME_Y).into()));

    for op in operations {
        match *op {
            Forward(distance) => turtle.forward(distance),
            TurnLeft          => turtle.turn_left(),
            TurnRight         => turtle.turn_right(),
            Noop(byte)        => eprintln!("warning: illegal byte encountered: {:?}" , byte),
            Home               => turtle.home(),
        };
        path_data.push(Command::Line(Position::Absolute,
            (turtle.x, turtle.y).into()));
        turtle.wrap();
    }
    path_data
}

fn generate_svg(path_data: Vec<Command>) -> Document {
    let background = Rectangle::new()
        .set("x", 0)
        .set("y", 0)
        .set("width", WIDTH)
        .set("height", HEIGHT)
        .set("fill", "#ffffff");

    let border = background.clone()
        .set("fill-opacity", "0.0")
        .set("stroke", "#cccccc")
        .set("stroke-width", 3*STROKE_WIDTH);

    let sketch = Path::new()

```

```

        .set("fill", "none")
        .set("stroke", "#2f2f2f")
        .set("stroke-width", STROKE_WIDTH)
        .set("stroke-opacity", "0.9")
        .set("d", Data::from(path_data));

    let document = Document::new()
        .set("viewBox", (0, 0, HEIGHT, WIDTH))
        .set("height", HEIGHT)
        .set("width", WIDTH)
        .set("style", "style=\"outline: 5px solid #800000;\"" )
        .add(background)
        .add(sketch)
        .add(border);

    document
}

fn main() {
    let args = env::args().collect::<Vec<String>>();
    let input = args.get(1).unwrap();
    let default_filename = format!("{}", ".svg", input);
    let save_to = args.get(2).unwrap_or(&default_filename);

    let operations = parse(input);
    let path_data = convert(&operations);
    let document = generate_svg(path_data);
    svg::save(save_to, &document).unwrap();
}

```

- ① operation and Orientation enum types are defined later. Including them with the use keyword here removes a lot of noise from the source code later on.
- ② HEIGHT and WIDTH provide the bounds of the drawing.
- ③ HOME_Y and HOME_X constants allow us to easily reset where we are drawing from to the middle. y is the vertical coordinate, x is the horizontal.
- ④ STROKE_WIDTH is a parameter for the output SVG. This defines how each line drawn looks.
- ⑤ Using descriptions, rather than numerical values, avoids mathematics.
- ⑥ To produce richer output, feel free to extend the operations available to your programs.
- ⑦ Using an isize allows you to extend this example to implement a "Reverse" operation without adding a new variant.
- ⑧ Noop is used when we encounter illegal input. To write error messages, we retain the illegal byte.
- ⑨ The Artist struct maintains the current state.
- ⑩ forward() mutates self within the match expression.

- ⑪ `turn_left()` and `turn_right()` mutate `self` outside of the match expression.
- ⑫ `wrap()` ensures that the drawing stays within bounds.
- ⑬ In ASCII, numerals start at 0x30 (48). So this converts the `u8` value of `b'2'` to 2. Performing this operation on the whole range of `u8` could cause a panic. We are safe here, thanks to guarantee provided by our pattern matching.
- ⑭ Although we don't expect any illegal characters, there may be some in the input stream. Using a `Noop` operation allows us to decouple parsing from producing output.

10.4.3 Spawning a thread per logical task

`render-hex` presents several opportunities for parallelism. We'll focus on one of them, the `parse()` function. To add parallelism, it's a 2 step process:

- Refactor code to use a functional style
- Use the `rayon` crate and its `par_iter()` method

USING A FUNCTIONAL PROGRAMMING STYLE

The first step is to convert our `for` loop into a functional programming style. Rather than `for`, the toolkit includes the `map()`, `collect()` and a closure. Compare [Listing 10.24](#) and [Listing 10.25](#) to see how the refactor looks:

Listing 10.24. The `parse()` function used in [Listing 10.23](#)

```
fn parse(input: &str) -> Vec<Operation> {
    let mut steps = Vec::<Operation>::new();
    for byte in input.bytes() {
        let step = match byte {
            b'0' => Home,
            b'1'..=b'9' => {
                let distance = (byte - 0x30) as isize;
                Forward(distance * (HEIGHT/10))
            },
            b'a' | b'b' | b'c' => TurnLeft,
            b'd' | b'e' | b'f' => TurnRight,
            _ => Noop(byte),
        };
        steps.push(step);
    }
}
```

```

    }
    steps
}

```

Listing 10.25. New `parse()` function using functional programming constructs, such as `map()` and closures (ch10-render-hex-functional/src/main.rs)

```

fn parse(input: &str) -> Vec<Operation> {
    input.bytes().map(|byte|{
        match byte {
            b'0' => Home,
            b'1'..=b'9' => {
                let distance = (byte - 0x30) as isize;
                Forward(distance * (HEIGHT/10))
            },
            b'a' | b'b' | b'c' => TurnLeft,
            b'd' | b'e' | b'f' => TurnRight,
            _ => Noop(byte),
        })}.collect()
}

```

[Listing 10.25](#) is both shorter, more declarative and closer to idiomatic Rust style. The primary change is that there is no longer a need to create a temporary variable, `steps`. The partnership of `map()` and `collect()` removes the need for it.

`map()` is a method on iterators. It takes a closure and applies it to every element of the iterator. The trick is that `map()` is also an iterator, allowing many translations to be chained together. `collect()` stores the output of an iterator into a `Vec<T>`. In this case, the type parameter `T` is the type `Operation`.

USING A PARALLEL ITERATOR

We’re going to cheat here and make use of a crate from the Rust community: `rayon`. `rayon` is explicitly designed to add “data parallelism”. Data parallelism applies the same function (or closure!) on different data, such as a `Vec<T>`.

To use `rayon`, start by including it as a crate dependency:

Listing 10.26. New dependency (`rayon`) added to `Cargo.toml` (ch10-

render-hex-parallel-iterator/Cargo.toml)

```
[dependencies]
svg = "0.6.0"
rayon = "1"
```

At the head of the `main.rs`, add `rayon` and its “prelude”. The prelude brings several traits into the crate’s scope. This has the effect of providing a `par_bytes()` method on string slices and `par_iter()` method on byte slices. Those methods enable multiple threads to cooperatively process data.

Listing 10.27. Adding `rayon` into our project (ch10-render-hex-parallel-iterator/Cargo.toml)

```
use rayon::prelude::*;
```

To include the new functionality, convert the input string slice to a byte slice with `as_bytes()`, then add a single method, `par_iter()`, before the call to `map()`:

```
fn parse(input: &str) -> Vec<Operation> {
    input.as_bytes().par_iter().map(|byte|{    ①
        match byte {
            b'0' => Home,
            b'1'..=b'9' => {
                let distance = (byte - 0x30) as isize;
                Forward(distance * (HEIGHT/10))
            },
            b'a' | b'b' | b'c' => TurnLeft,
            b'd' | b'e' | b'f' => TurnRight,
            _ => Noop(*byte),    ②
        })}.collect()
}
```

- ① Adding `par_iter()` here is a “cheat mode” available to all Rust programmers. It is guaranteed to never introduce race conditions.
- ② The byte variable has the type `&u8`, whereas the `Operation::Noop(u8)` variant requires a dereferenced value.

10.4.4 Using a thread pool and task queue

Sometimes, we don't have a tidy iterator that we want to apply a function to. Another pattern to consider is the task queue. Tasks can originate anywhere and be added to a queue. A fleet of worker threads can pick tasks once they've finished their current one.

There are many approaches to modelling a task queue. We could create a `Vec<Task>` and `Vec<Result>` and share references to them across threads. To prevent each thread from overwriting each other, we would need a data protection strategy.

The most common tool to protect data shared between threads is `Arc<Mutex<T>>`. Fully expanded, that's your value `T`, e.g. `Vec<Task>` or `Vec<Result>` here, protected by a `std::sync::Mutex`, which itself wrapped within `std::sync::Arc`. A `Mutex` is a “mutually-exclusive” lock. Mutually-exclusive, in this context, means that no one has special rights. A lock held by any thread prevents all others. Awkwardly, a `Mutex` must itself be protected between threads. So we call in extra support. The `Arc` provides safe multi-threaded access to the `Mutex`.

`Mutex` and `Arc` are not unified into a single type to provide programmers with added flexibility. Consider a struct with several fields. You may only need a `Mutex` on a single field, but you could put the `Arc` around the whole struct. This approach provides faster read access to the fields that are not protected by the `Mutex`. A single `Mutex` retains maximum protection for the field that has read/write access.

The lock approach, while workable, is cumbersome. Channels offer a simpler alternative.

Channels have two “ends”, a sending end and a receiving end. Programmers don't get access to what is happening inside the channel. But, placing data at the sending end, they'll appear at the receiving end at some stage in the future. They can be used as a task queue because multiple items can be sent, even if a receiver is not ready to receive any messages.

Channels are fairly abstract. They hide their internal structure, preferring to delegate access to two helper objects. One can `send()`, the other can `recv()`

(receive). Importantly, we don't get access to *how* channels transmit any information sent through the channel.



Tip

By convention from radio and telegraph operators, Sender is called tx as shorthand for "transmission" and the Receiver is called rx.

This section of the chapter uses the channels implementation from the `crossbeam` crate, rather than channels from the Rust standard library. The API is consistent, but `crossbeam` provides greater functionality and flexibility. We spend a little bit of time here explaining how to use them. If you would prefer to see them used as a task queue, feel free to skip ahead.

ONE-WAY COMMUNICATION

Channels offer one-way, or simplex, communication between two points. [Listing 10.29](#) provides a quick example.

The output from [Listing 10.29](#) is simple:

```
Ok(42)
```

Listing 10.28. Cargo.toml metadata

```
[package]
name = "ch10-channels-intro"
version = "0.1.0"
authors = ["Tim McNamara <author@rustinaction.com>"]
edition = "2018"

[dependencies]
crossbeam = "0.7"
```

Listing 10.29. Creating a channel that receives i32 messages from a worker thread.

```
#[macro_use] ①
extern crate crossbeam; ②

use std::thread;
```



```

use crossbeam::channel::{unbounded}; ③

fn main() {
    let (tx, rx) = unbounded(); ④
    thread::spawn(move || {
        tx.send(42);
    });
    select!{ ⑤
        recv(rx) -> msg => println!("{:?}", msg), ⑥
    }
}

```

- ① The `select!` macro used on line 12 originates here.
- ② The `crossbeam` crate provides many concurrency primitive. Notably for us, it includes excellent implementations of channels for us to make use of.
- ③ `crossbeam::channels` has two types of channels, bounded and unbounded. Bounded channels have a fixed capacity.
- ④ Creating a channel involves calling a function that returns `Sender<T>` and `Receiver<T>`. In this example, the compiler detects that we are using a number and creates `Sender<i32>` and `Receiver<i32>`.
- ⑤ The `select!` macro takes its name from other messaging systems, such as the POSIX sockets API. It allows the main thread to block and wait for a message.
- ⑥ `recv(rx)` is syntax defined by the macro.

WHAT CAN BE SENT THROUGH A CHANNEL?

Mentally, you might be thinking of a channel like you would be thinking of a network protocol. Over the wire, you only have the type `[u8]` available to you. That byte stream needs to be parsed and validated before its contents can be interpreted. Channels are richer than `[u8]` because there is no network overhead. You have the full power of Rust's type system.

I recommend using an enum for messages, as it works perfectly with Rust's type system.

TWO-WAY COMMUNICATION

For bi-directional full duplex communication is awkward to model with a single channel. An approach that's simpler to work with is to create two sets of senders

and receivers, one for each direction. [Listing 10.31](#) provides an example of this.

When executed, [Listing 10.31](#) produces three lines of output.

Listing 10.30. Output produced by [Listing 10.31](#)

```
Ok(Pong)
Ok(Pong)
Ok(Pong)
```

Listing 10.31. Sending messages to and from a spawned thread (ch10/ch10-channels-complex/src/main.rs)

```
#[macro_use]
extern crate crossbeam;

use crossbeam::channel::unbounded;
use std::thread;

use crate::ConnectivityCheck::*;

#[derive(Debug)]
enum ConnectivityCheck { ①
    Ping,                ①
    Pong,                ①
    Pang,                ①
}                        ①

fn main() {
    let n_messages = 3;
    let (requests_tx, requests_rx) = unbounded();
    let (responses_tx, responses_rx) = unbounded();

    thread::spawn(move || loop { ②
        match requests_rx.recv().unwrap() {
            Pong => eprintln!("unexpected pong response"),
            Ping => responses_tx.send(Pong).unwrap(),
            Pang => return, ③
        }
    });

    for _ in 0..n_messages {
        requests_tx.send(Ping).unwrap();
    }
    requests_tx.send(Pang).unwrap();

    for _ in 0..n_messages {
        select! {
            recv(responses_rx) -> msg => println!("{:?}", msg),
        }
    }
}
```

- ① Defining a bespoke message type makes interpreting messages later very simple.
- ② Because all control flow is an expression, Rust allows the `loop` keyword to be here.
- ③ The `Pang` message indicates the thread should shut down.

IMPLEMENTING A TASK QUEUE

After spending some time discussing channels, it's time to apply them to the problem first introduced in [Listing 10.23](#).

You'll notice that the code that follows shortly in [Listing 10.33](#) it's quite a bit more complex than the parallel iterator approach seen earlier at [Listing 10.25](#).

Listing 10.32. Project metadata for the channel-based task queue implementation of `render-hex` (ch10/ch10-render-hex-threadpool/Cargo.toml)

```
[package]
name = "render-hex"
version = "0.1.0"
authors = ["Tim McNamara <author@rustinaction.com>"]
edition = "2018"

[dependencies]
svg = "0.6"
crossbeam = "0.7" ①
```

- ① The `crossbeam` crate is a new dependency to the project.

Listing 10.33. Partial source code for the channel-based task queue implementation of `render-hex`. This listing focuses on the `parse()` function. The rest of the code is the same as [Listing 10.23](#). (ch10/ch10-render-hex-threadpool/src/main.rs)

```
extern crate crossbeam;
use std::thread;
use std::env;

use crossbeam::channel::{unbounded};
```

```

enum Work { ①
    Task((usize, u8)), ②
    Finished, ③
}

fn parse_byte(byte: u8) -> Operation { ④
    match byte {
        b'0' => Home,
        b'1'..b'9' => {
            let distance = (byte - 0x30) as isize;
            Forward(distance * (HEIGHT/10))
        },
        b'a' | b'b' | b'c' => TurnLeft,
        b'd' | b'e' | b'f' => TurnRight,
        _ => Noop(byte),
    }
}

fn parse(input: &str) -> Vec<Operation> {
    let n_threads = 2;
    let (todo_tx, todo_rx) = unbounded(); ⑤
    let (results_tx, results_rx) = unbounded(); ⑥
    let mut n_bytes = 0;
    for (i, byte) in input.bytes().enumerate() {
        todo_tx.send(Work::Task((i, byte))).unwrap(); ⑦
        n_bytes += 1; ⑧
    }

    for _ in 0..n_threads { ⑨
        todo_tx.send(Work::Finished).unwrap(); ⑨
    } ⑨

    for _ in 0..n_threads {
        let todo = todo_rx.clone(); ⑩
        let results = results_tx.clone(); ⑩
        thread::spawn(move || {
            loop {
                let task = todo.recv();
                let result = match task {
                    Err(_) => break,
                    Ok(Work::Finished) => break,
                    Ok(Work::Task((i, byte))) => (i, parse_byte(byte)),
                };
                results.send(result).unwrap();
            }
        });
    }
    let mut ops = vec![Noop(0); n_bytes]; ⑪
    for _ in 0..n_bytes {
        let (i, op) = results_rx.recv().unwrap();
        ops[i] = op;
    }
    ops
}

```

① Create a type for the messages that we will send through the channels.

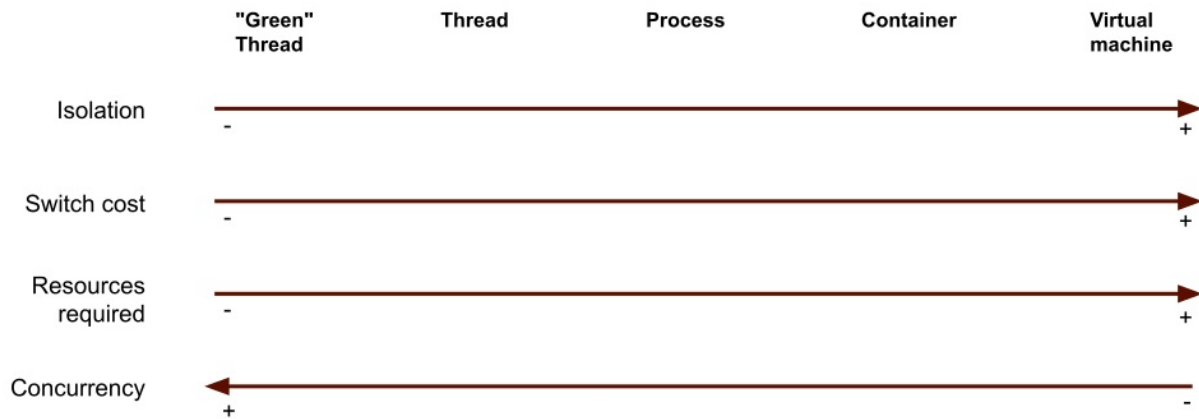
- ② The `usize` field of this tuple will indicate the position that the byte has that's being processed. This is necessary because they can be returned out of order.
- ③ We'll give worker threads a marker message to indicate that it's time for them to shut themselves down.`
- ④ Extract the functionality that workers will need to carry out to simplify the logic.
- ⑤ Create one channel for tasks to be completed.
- ⑥ Create one channel for the decoded instructions to be returned to.
- ⑦ Fill the task queue with work.
- ⑧ Keep track of how many tasks there are to do.
- ⑨ Send each thread a signal that it's time to shut down.
- ⑩ When cloned, channels can be shared between threads.
- ⑪ Because results can be returned in arbitrary order, we'll initialize a complete `Vec<Command>` that will be overwritten by our incoming results. A vector is used, rather than an array because that is what is used by the type signature and we don't want to refactor the whole program to suit this new implementation.

When independent threads are introduced, the order in which tasks are completed becomes non-deterministic. [Listing 10.33](#) includes some additional complexity to handle this. Previously, we have created an empty `Vec<Command>` for the commands that we interpret from our input. Once parsed, `main()` repeatedly added elements via the vector's `push()` method. Now, at line 147, fully initialize the vector. Its contents don't matter. They'll all be overwritten. Even so, I've chosen to use `Command::Noop` to ensure that a mistake won't result in a corrupt SVG file.

10.5 CONCURRENCY AND TASK VIRTUALIZATION

This section attempts to explain the difference between differing models of concurrency.

Figure 10.5. Trade offs relating to different forms of task isolation in computing. In general terms, increasing the isolation level increases the overhead.



The primary benefit of more costly forms of task virtualization is isolation. What is meant by the term "isolation"?

Isolated tasks cannot interfere with each other. Interference comes in many forms. Examples include corrupting memory, saturating the network and congestion saving to disk. If a thread is “blocked” waiting for the console to print output to the screen, none of the coroutines acting in that thread are able to progress.

Isolated tasks cannot access each other’s data without permission. Independent threads in the same process share a memory address space and all threads have equal access to data within that space. Processes, however, are prohibited from inspecting each other’s memory.

Isolated tasks cannot cause another task to crash. A failure in one task should not cascade into other systems. If a process induces a kernel panic, all processes are shut down. By conducting work in virtual machines, tasks can proceed even when other tasks are unstable.

Isolation is a continuum. Complete isolation is impractical. It implies that input and output is impossible. Moreover, isolation is often implemented in software.

Running extra software implies taking on extra runtime overhead.

A small glossary of relating to concurrency

This sub-field is filled with jargon. Here is a very brief introduction to some important terms and how we'll be using them in this chapter.

- Program - A program, or application, is a brand name. It's a name that we use refer to a software package. When we execute a program, the operating system creates a process.
- Executable - A file that is able to be loaded into memory, then run. Running an executable means creating a process and a thread for it, then changing the CPU's instruction pointer to the first instruction of the executable.
- Task - This chapter uses the term "task" in an abstract sense. Its meaning changes as the level of abstraction changes. When discussing processes, a task is one of the process's threads. When referring to a thread, a task might be a function call. When referring to an operating system, a task might be a running program, which might be comprised of multiple processes.
- Process - Running programs execute as processes. A process has its own virtual address space, at least one thread, and lots of bookkeeping managed by the operating system. File descriptors, environment variables, scheduling priority are managed per process. A process has a virtual address space, executable code, open handles to system objects, a security context, a unique process identifier, environment variables, a priority class, minimum and maximum working set sizes, and at least one thread of execution. Each process is started with a single thread, often called the primary thread, but can create additional threads from any of its threads. Running programs begin their life as a single process, but isn't uncommon to spawn sub-processes to do work.
- Thread - The thread metaphor is used to hint that multiple threads can work together as a whole. Thread of execution. A sequence of CPU instructions that appear in serial. Multiple threads can run concurrently, but instructions within the sequence are intended to be executed one after another.
- Co-routine (also known as "fibre", "green thread", and "lightweight thread") - Tasks that switch within a thread. The switching between tasks becomes the responsibility of the program itself, rather than the operating system.

Two theoretical concepts are important to distinguish:

- Concurrency Multiple tasks—of any level of abstraction—running at the same time
- Parallelism Multiple threads executing on multiple CPUs, at the same time

Outside of the fundamental terminology, there are also inter-related two terms that appear frequently: *asynchronous programming* and *non-blocking I/O*.

Many operating systems provide non-blocking I/O facilities, where data from multiple sockets is batched into queues and periodically polled as a group.

- Non-blocking I/O - Normally a thread is de-scheduled when it asks for data from I/O

devices such as the network. The thread is marked as *blocked*, while it awaits data to arrive. When programming with non-blocking I/O, the thread can continue executing even while it waits for data. But, there is a contradiction. How can a thread continue to execute if it doesn't have any input data to process? The answer lies in asynchronous programming.

- Asynchronous programming - Asynchronous programming describes programming for cases where the control-flow is not pre-determined. Instead, events outside the control of the program itself impact the sequence of what is executed. Those events are typically I/O-related, such as a device driver signalling that it is ready, or are related to functions returning in another thread.

The asynchronous programming model is typically more complicated for the developer, but results in a faster runtime for I/O-heavy workloads. Speed increases because there are fewer syscalls. This implies fewer *context switches* between *user space* and *kernel space*.

10.5.1 Threads

A thread is the lowest level of isolation that an operating system understands. The operating system can schedule threads.

Smaller forms of concurrency are invisible to the operating system. You may have encountered terms such as coroutines, fibers and green threads. Switching between tasks here is managed by the process itself. The operating system is ignorant of the fact that a program is processing multiple tasks.

For threads and other forms of concurrency, context switching is required.

10.5.2 What is a context switch?

Intermediate tasks required to switch between tasks at the same level of virtualization. For threads to switch, CPU registers need to be cleared, CPU caches might need to be flushed, variables within the operating system need to be reset. As isolation increases, so does the cost of the context switch. More needs to be switched.

CPUs can only execute instructions in serial. To do more than one task, a computer needs to be able to hit the "save game" button, switch to a new task and resume at that task's save spot. The CPU is save scum. Why is the CPU

constantly switching tasks? Because it has so much time available. Programs often need to access data from memory, disk, or the network. Because waiting for data is incredibly slow, there's often sufficient time to do something else in the meantime.

10.5.3 Processes

Threads exist within a process. The distinguishing characteristic of a process is that its memory is independent from other processes. The operating system, in conjunction with the CPU, protects a process's memory from all others.

To share data between processes, Rust channels and data protected by `Arc<Mutex<_>>` won't suffice. You need some support from the operating system. Re-using network sockets is common.

Most operating systems provide specialized forms of inter-process communication (IPC). They're faster, while being less portable.

10.5.4 Web Assembly

Web Assembly (wasm) is interesting because it is an attempt at isolating tasks within the process boundary itself. It's impossible for tasks running inside a wasm module to access memory available to other tasks. Originating in web browsers, wasm treats all code as potentially hostile. If you use third-party dependencies, it's likely that you haven't verified the behavior of all of the code that your process executes.

In some sense, Web Assembly modules are given access to address spaces within your process's address space. Wasm address spaces are called *linear memory*. That runtime interprets any request for data within linear memory and makes its own request to the actual virtual memory. Code within the wasm module is unaware of any memory addresses that the process has access to.

10.5.5 Containers

Containers are extensions to processes. The operating system provides additional. In addition to independent memory address spaces, a container has independent access to several other shared resources, such as the filesystem. Rather than address space, the term used for protections covering these other resources is namespaces.

10.5.6 Why use an operating system at all?

It's possible to run an application as its own operating system. We can compile a "unikernel". A unikernel is a minimal operating system paired with a single application. The compilation process strips out everything from the operating system that isn't used by the application that's being deployed.

10.6 WHAT YOU HAVE LEARNED

- The `while let Some(item) = collection.pop()` pattern to safely drain a list
- Converting `for` loops into a functional programming style with `map()` and closures. Using a functional programming style is considered idiomatic Rust.
- How to spawn threads
- Spawning thousands of threads can cause problems
- How to create a spin loop
- How to share variables between threads
- How to define a closure and how they're different from functions
- Generating vector graphics
- Multi-tasking with parallel iterators
- Using a task queue and thread pool
- Message passing with Rust's channels

11 KERNEL

Let's build an operating system. By the end of the chapter, you'll be running your own OS (or at least, a minimal subset of an OS). Not only that, but you would have compiled your own bootloader, kernel and the Rust language itself for your new operating system.

This chapter covers many features of Rust that are important for programming without an operating system. Accordingly, the chapter is important for programmers who work with Rust for embedded devices.

11.1 A FLEDGLING OPERATING SYSTEM (FLEDGEOS)

This section implements an operating system kernel. The kernel of an operating system performs several important roles, such as interacting with hardware, memory management and coordinating work. Typically work is coordinated through processes and threads. We won't be able to cover much of that in this chapter. But we will get off the ground. We'll fledge. So let's call the system we're building *FledgeOS*.

11.1.1 Setting up a development environment for developing an operating system kernel

Creating an executable for an operating system that doesn't exist yet is a complicated process. For instance, we need to compile the core Rust language for the OS from your current one. But, your current environment only understands your current environment. Let's extend it.

Several tools are needed to help us out.

There are several components that you need to install and/or configure.

- QEMU is a virtualization technology. It runs operating systems for any machine, on any of its supported hosted architectures. Visit <https://www.qemu.org/> for installation instructions.
- The bootimage crate and its dependencies. See [Listing 11.1](#) for installation instructions and [Listing 11.2](#) for validating that the installation has been successful.

Listing 11.1. Instructions for installing the pre-requisites for this chapter's examples

```
cargo install cargo-binutils      ①
cargo install bootimage          ②
rustup toolchain install nightly ③
rustup default nightly           ③
rustup component add rust-src     ④
rustup component add llvm-tools-preview ⑤
```

- ① These utilities are "pure Rust" versions of tools that do low-level operations on executable files.
- ② The bootimage crate drives the other utilities. It also compiles the bootloader.
- ③ The “nightly” compiler is required for developers to access internal compiler features essential for creating a new kernel.
- ④ We will be compiling Rust for our new OS from source. That will enable you to write code from your current operating system that targets another.
- ⑤ LLVM is a compiler that is bundled within the Rust compiler. These tools allow you to make use of LLVM directly.

To validate that each of the tools has been correctly installed, you can look for them within the "sysroot" use these commands.

Listing 11.2. Validating that the chapter's pre-requisites are correctly installed.

```
$ rustc --print sysroot
/home/tsm/.rustup/toolchains/nightly-x86_64-unknown-linux-gnu ①

$ find $(rustc --print sysroot) -type f -name 'llvm-*' -printf '%f\n' ②
llvm-size
llvm-readobj
```

```
llvm-objcopy
llvm-objdump
llvm-strip
llvm-profdata
llvm-ar
llvm-nm
```

```
$ cargo xbuild --help
Wrapper for `cargo build` that cross-compiles core, compiler_builtins and alloc
# ...
```

- ① The toolchain's name should start with “nightly”. If your toolchain is running stable or beta different, try running `rustup default nightly` again to resolve.
- ② Execute the `find` command looking for files that start with the `llvm-` prefix and then print their full names. On MS Windows, a similar result would be produced with the `dir` command. First, change to the directory indicated by `rustc --print sysroot`, then execute `dir llvm-* /b/s`.

As mentioned earlier, creating an operating system kernel for an operating system that doesn't exist is complicated. Several steps are required:

- Creating a machine-readable definition of the conventions that the operating system uses, such as the intended CPU architecture. This is the *target platform*, also known as a *compiler target* or simply *target*. You have seen targets before. Try executing `rustup target list` for a list that you can compile Rust to.
- Rust needs to be compiled for the target definition. We'll suffice with a subset of Rust called *core* that excludes the standard library (crates under `std`). This creates the new target.
- Compile the OS kernel for the new target, using the new Rust.
- Compile a bootloader that can load the new kernel.
- Execute the bootloader in a virtual environment, which will, in turn run the kernel.

Thankfully, for us the `bootimage` does all of this for us.

11.1.2 fledgeos project structure



Source code is available in the `ch11` directory at <https://github.com/rust-in-action/code/>

The source code files for the FledgeOS projects (`code/ch11/ch11-fledgeos-*`) use a slightly different structure than most cargo projects. The projects include two extra files:

- the project root directory contains a `fledge.json` file. This is the definition of the compiler target that `bootimage` and friends will be building.
- extra configuration parameters to cargo are provided in `.cargo/config.toml`. They tell cargo that it needs to compile the `std::core` module itself for this project, rather than relying on it being pre-installed.

Listing 11.3. Project layout for FledgeOS projects, with `fledgeos-0` as an example

```
ch11-fledgeos-0
├── src
│   └── main.rs
├── .cargo
│   └── config.toml ①
├── Cargo.toml
└── fledge.json      ②
```

① Extra configuration options for cargo

② A machine-readable definition of the FledgeOS kernel

11.1.3 A minimal operating system kernel

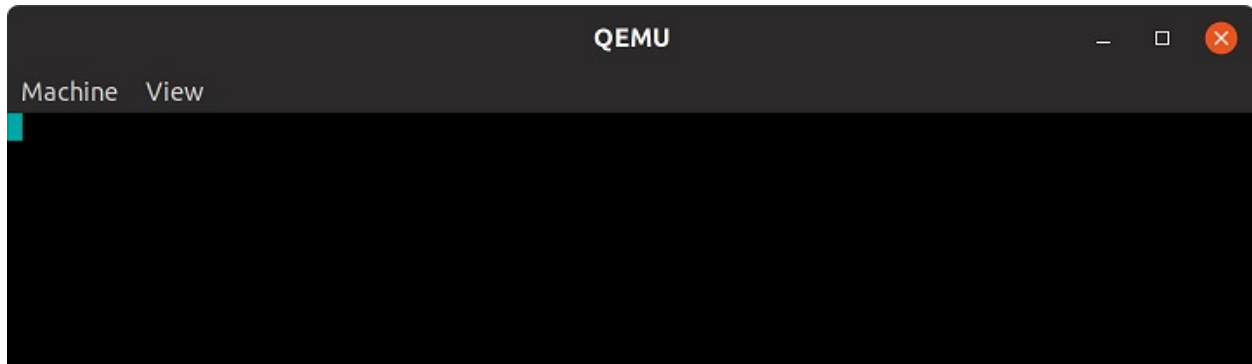
The source code for the smallest kernel that I can think of that does *something* is provided at [Listing 11.8](#). It doesn't do much, at all. It creates a pale blue box in the top-left corner.

To create your own pale blue box, use the standard `cargo run` command in a console window from within the `code/ch11/ch11-fledgeos-0` project directory:

Listing 11.4. How to compile and run the first version of FledgeOS

```
$ git clone https://github.com/rust-in-action/code rust-in-action
$ cd rust-in-action/ch11/ch11-fledgeos-0
$ cargo run
```

Figure 11.1. Expected output from running [Listing 11.8](#) with `bootimage`



Don't worry about *how* the colored block became cyan. We'll discuss the retro-computing details for that shortly. For now, success is being able to compile your own version of Rust, an OS kernel using that Rust, a bootloader that puts your kernel into the right place and having them work together.

Listing 11.5. Compiler target definition for FledgeOS (ch11/ch11-fledgeos-0/fledge.json)

```
{
  "llvm-target": "x86_64-unknown-none",
  "data-layout": "e-m:e-i64:64-f80:128-n8:16:32:64-S128",
  "arch": "x86_64",
  "target-endian": "little",
  "target-pointer-width": "64",
  "target-c-int-width": "32",
  "os": "none",
  "linker": "rust-lld",
  "linker-flavor": "ld.lld",
  "executables": true,
  "features": "-mmx,-sse,+soft-float",
  "disable-redzone": true,
  "panic-strategy": "abort"
}
```

Listing 11.6. Project metadata for [Listing 11.8](#) (ch11/ch11-

fledgeos-0/Cargo.toml)

```
[package]
name = "fledgeos"
version = "0.1.0"
authors = ["Tim McNamara <author@rustinaction.com>"]
edition = "2018"

[dependencies]
bootloader = "0.9"

[package.metadata.bootimage]
run-command = ["qemu-system-x86_64", "-drive", "format=raw,file={}" ] ①
```

① Update cargo run to invoke a QEMU session. The curly braces ({}) will be replaced by path to the operating system image created during the build step.

Listing 11.7. Extra build-time configuration for cargo (ch11/ch11-fledgeos-0/.cargo/config.toml)

```
[build]
target = "fledge.json"

[unstable]
build-std = ["core", "compiler_builtins"]
build-std-features = ["compiler-builtins-mem"]

[target.'cfg(target_os = "none")']
runner = "bootimage runner"
```

Listing 11.8. Creating an operating system kernel that paints a block of color on the screen. (ch11/ch11-fledgeos-0/src/main.rs)

```
#![feature(core_intrinsics)] ①
#![no_std] ②
#![no_main] ③

use core::intrinsics;
use core::panic::PanicInfo; ④

#[panic_handler] ⑤
#[no_mangle]
fn panic(_info: &PanicInfo) -> ! { ⑥
    unsafe {
        intrinsics::abort(); ⑦
    }
}

#[no_mangle] ⑧
pub extern "C" fn _start() -> ! { ⑨
```



```

let framebuffer = 0xb8000 as *mut u8; ⑩
unsafe {
    framebuffer.offset(1).write_volatile(0x30); ⑪
}
loop {}
}

```

- ① Enable "intrinsics", which are functions provided by the LLVM compiler itself. This attribute unlocks the `core::intrinsics` module. It requires the nightly compiler.
- ② Exclude the standard library (`std::*`) from the crate. This greatly simplifies compilation to new targets, but restricts the crate to functionality provided by `core::*`.
- ③ Instruct `rustc` to avoid setting up the program to start at `main()`.
- ④ The `PanicInfo` type allows the panic handler to inspect where the panic occurred.
- ⑤ The `panic_handler` attribute instructs the compiler to associate this function with any panics that may occur.
- ⑥ The `!` return type is known as the "never type". Functions that use `!` as their return value may never actually return.
- ⑦ To return `!`, our function must never return. In `_start()`, we loop forever, but that's inappropriate here. The `core::intrinsics::abort()` function also has the `!` return type, so we'll use it here.
- ⑧ The `no_mangle` attribute prevents Rust from renaming the function in the resulting binary. That's important here to enable the linker to interface with our compiled binary at a known point.
- ⑨ `extern "C"` means that Rust should use C's conventions for compiling this function. This allows `_start()` to be accessed by other programs, in a well-known manner.
- ⑩ The bootloader sets up address `0xb8000` as the location of the VGA buffer. A "VGA buffer" is a memory region known about by hardware that controls the screen output.
- ⑪ This line sets the byte of memory at `0xb8001` to `0x30`. This has the effect of setting the background to cyan.

Where to go to learn more about operating system development

The `cargo bootimage` command takes care of lots of nuisance and irritation. It provides a very simple interface—a single command—to a very complicated process. But, if you're a tinkerer, you might like to know what's happening beneath the surface. In that case, you should search out Philipp Oppermann's "blog_os" project and the small ecosystem of tools that has emerged from it.

Now that we have got our first kernel is live, let's learn a little bit about how it works.

11.1.4 Panic handling

Rust won't allow you a program to compile that doesn't have a mechanism to deal with panics. Normally, it inserts panic handling itself. This is one of the parts of the "Rust runtime". But, we started our code with `#[no_std]`.

Avoiding the standard library is useful. It greatly simplifies compilation. But manual panic handling is one of its costs.

Listing 11.9. Excerpt from [Listing 11.8](#) focusing on the panic handling functionality.

```
#![feature(core_intrinsics)] ①
#![no_std] ②
#![no_main] ③

use core::intrinsics;
use core::panic::PanicInfo; ④

#[panic_handler]
#[no_mangle]
pub fn panic(_info: &PanicInfo) -> ! {
    unsafe {
        intrinsics::abort(); ⑤
    }
}
```

There is an alternative to `intrinsics::abort()`. We could use an infinite loop as the panic handler, as shown at [Listing 11.10](#). The disadvantage of that approach is that any errors in the program will trigger a CPU core to run at 100% until it is shut down manually.

Listing 11.10. Using an infinite loop as a panic handler

```
#[panic_handler]
#[no_mangle]
pub fn panic(_info: &PanicInfo) -> ! {
    loop { }
}
```

The `PanicInfo` info struct provides information about where the panic originated from. This information includes the file name and line number of the source code. It'll become in handy when we implement proper panic handling.

11.1.5 Writing to the screen with VGA-compatible text mode

The bootloader sets some magic bytes with raw assembly code at boot mode. At startup, the bytes are interpreted by hardware. The hardware switches its display to a 80x25 grid. It also sets up some fixed memory buffer that is interpreted by the hardware for printing to the screen.

VGA-compatible text mode in 20 seconds

The display is split into a 80x25 grid cells. Each cell is represented in memory by two bytes. In Rust-like syntax, those two bytes include several fields:

```
struct VGACell {
    is_blinking: u1,      ①
    background_color: u3, ①
    is_bright: u1,        ①
    character_color: u3,   ①
    character: u8,         ②
}
```

① These 4 fields occupy a single byte in memory.

② Characters available are drawn from the "Code page 437" encoding, which is (approximately) an extension of ASCII.

VGA text mode has a 16 color palette. Three bits make up the main 8 colors. Foreground colors also have an additional bright variant.

```
#[repr(u8)]
enum Color {
```

```

Black = 0,    White = 8,
Blue = 1,     BrightBlue = 9,
Green = 2,    BrightGreen = 10,
Cyan = 3,     BrightCyan = 11,
Red = 4,      BrightRed = 12,
Magenta = 5,  BrightMagenta = 13,
Brown = 6,    Yellow = 14,
Gray = 7,     DarkGray = 15,
}

```

Mutating the frame buffer instructs the hardware to change what's presented on screen. The snippet used in [Listing 11.8](#) and repeated here is how one writes to memory:

```

let mut framebuffer = 0xb8000 as *mut u8;
unsafe {
    framebuffer.offset(1).write_volatile(0x30); ①
}

```

① If the intent of `0x30` is unclear, `0x3 << 4` is functionally identical. This alternative syntax that might indicate that we're altering the three bits that represent the background color.

This initialization at boot time makes it very easy to display things on screen. Each of the points in the 80x25 grid are mapped to locations in memory. This area of memory is called the "framebuffer". Our bootloader designates `0xb8000` as the start of a 4,000 byte framebuffer.

To actually set the value, our code uses some new methods that you haven't encountered before:

Listing 11.11. Excerpt from [Listing 11.8](#) focusing on modifying the VGA framebuffer

```

let mut framebuffer = 0xb8000 as *mut u8;
unsafe {
    framebuffer.offset(1).write_volatile(0x30);
}

```

Another way to write `framebuffer.offset(1).write_volatile(0x30)` is to use the dereference operator (*) and *pointer arithmetic* to set the memory location to `0x30`:

Listing 11.12. Excerpt from

```
let mut framebuffer = 0xb8000 as *mut u8;
unsafe {
    *(framebuffer + 1) = 0x30; ①
}
```

① Set the memory location 0xb8001 to 0x30.

The coding style from [Listing 11.12](#) may be more familiar to programmers who have worked heavily with pointers before. The code used in [Listing 11.11](#) prevents any typos from interfering with our intent.

In [Listing 11.11](#), `offset()` method moves through the address space in increments that align to the size of the pointer. A `*mut u8` moves one byte at a time. `*mut u32` moves 4 bytes at a time. `write_volatile()` issues a "volatile" write. Volatile prevents the compiler's optimizer from optimizing away the write instruction. A smart compiler might simply notice that we are using lots of constants everywhere and initialize the program such that the memory is simply set to the value that we want it to be. That could be useful, and is probably fine in this instance. More complex programs will benefit from preventing the optimization, however.

11.1.6 `_start()`, the "main()" function for FledgeOS

An operating system kernel does not include the concept of a `main()` function in the sense that you're used to. For one thing, an operating system kernel's main loop never returns. Where would it return to? By convention, programs return an error code when they exit, to an operating system. But operating systems don't have an operating system to provide an exit code to. Secondly, starting a program at `main()` is also a convention. But that convention also doesn't exist for operating system kernels.

To start an operating system kernel, we require some software to talk directly to the CPU, called a bootloader.

The linker will expect to see one symbol defined, `_start`. `_start` is as the program's `_entry point`. It links `_start` to a function that's defined by your source code. The `_start()` function has three jobs. Its first is to reset the system. On an embedded system, for example, `_start()` might clear registers and reset memory to 0. Its second job is to call `main()`. Its third is to call `_exit()`, which cleans up after `main()`.

11.1.7 Being power conscious by interacting with the CPU directly

Before proceeding, FledgeOS needs to address one major shortcoming. It is extremely power hungry. The `_start()` function from [Listing 11.8](#) actually runs a CPU core at 100%. It's possible to avoid this by issuing the halt instruction to the CPU. The halt instruction, referred to as HLT in the technical literature, notifies the CPU that there's no more work to be done. The CPU will resume operating when an interrupt triggers new action.

Making use of the `x86_64` crate will allow us to issue instructions directly to the CPU.

Listing 11.13. New dependency section of the FledgeOS Cargo.toml (ch11/ch11-fledgeos-1/Cargo.toml)

```
[dependencies]
bootloader = "0.9"
x86_64 = "0.12"
```

Listing 11.14. Making use of the `x86_64` crate to access the `hlt` instruction. The instruction is given to the CPU during the main loop of `_start()` to prevent excessive power consumption. (ch11/ch11-fledgeos-1/src/main.rs)

```
use x86_64::instructions::{hlt};
```

```
#[no_mangle]
pub extern "C" fn _start() -> ! {
    let mut framebuffer = 0xb8000 as *mut u8;
```

```
unsafe {  
    framebuffer.offset(1).write_volatile(0x30);  
}  
loop {  
    hlt();  
}  
}
```

11.1.8 Handling exceptions properly, almost

We need to do some more housekeeping before moving on. FledgeOS cannot manage any exceptions generated from the CPU when it detects abnormal operation.

To handle exceptions, our program needs to define an “exception handling personality” function.

Personality functions are called on each stack frame as the stack is unwound after an exception. That is, the call stack is traversed, invoking the personality function at each stage. The personality function’s role is to determine whether the current stack frame is able to handle the exception. Exception handling is also known as catching an exception.



Note

What is stack “unwinding”? When functions are called, stack frames accumulate. Traversing the stack in reverse is called “unwinding”. Eventually, the unwinding the stack will hit `_start()`.

Since handling exceptions in a rigorous way is not necessary for FledgeOS, we’ll implement only the bare minimum. [Listing 11.15](#) provides a snippet of code with the minimal handler. Inject it into the `main.rs`. An empty function implies that any exception is fatal, as none will be marked as the handler.

Listing 11.15. Minimalist exception handling personality routine. When an exception occurs, do nothing. (ch11/ch11-fledgeos-2/src/main.rs)

```
#![feature(lang_items)]
```

```
#[lang = "eh_personality"]  
#[no_mangle]  
pub extern "C" fn eh_personality() { }
```

What is a “language item”?

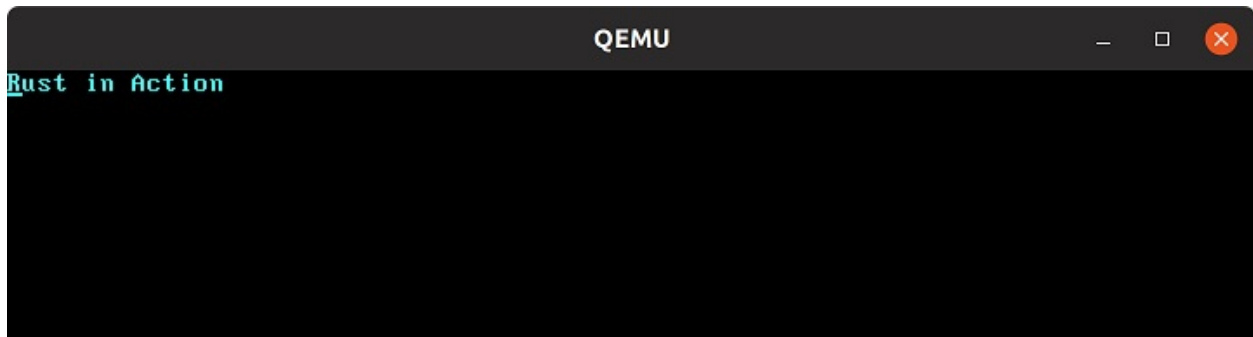
Language items are elements of Rust implemented as libraries outside of the compiler itself. As we’re stripping away the standard library with `#[no_std]`, we’ll need to implement some of its functionality ourselves.

Admittedly, that’s a lot of work to do nothing. But, at least we can be comforted by knowing that we’ve done nothing in the right way.

11.2 NICE OUTPUT

Let’s write some text to the screen. That way, if we really do encounter a panic, we can report it properly.

Figure 11.2. Output produced by [Listing 11.19](#)



To start, we’ll create a type for the color numeric constants that are used later at [Listing 11.19](#). Using an enum, rather than defining a series of `const` values provides enhanced type safety. In some sense, it adds a semantic relationship between the values. They are all treated as members of the same group.

Listing 11.16. Representing related numeric constants as an enum. This code creates an enum that represents the “VGA-

compatible text mode” color palette. The mapping between bit patterns and colors is defined by the standard and our code should comply with it. (ch11/ch11-fledgeos-3/src/main.rs)

```
#[allow(unused)]           ①
#[derive(Clone, Copy)]    ②
#[repr(u8)]               ③
enum Color {
    Black = 0x0,    White = 0xF,
    Blue = 0x1,     BrightBlue = 0x9,
    Green = 0x2,    BrightGreen = 0xA,
    Cyan = 0x3,     BrightCyan = 0xB,
    Red = 0x4,      BrightRed = 0xC,
    Magenta = 0x5,  BrightMagenta = 0xD,
    Brown = 0x6,    Yellow = 0xE,
    Gray = 0x7,     DarkGray = 0x8
}
```

① We won’t be using every color variant in our code, so we silence warnings.

② Opt into “copy semantics”

③ Instruct the compiler to use a single byte to represent the values.

11.2.1 Controlling the in-memory representation of enums

We’ve usually been content to allow the compiler to determine how an enum is represented. But, there are times when we need to pull on the reins. External systems often demand that our data matches their requirements.

[Listing 11.16](#) provides an example of fitting the colors from the “VGA-compatible text mode” palette enum into a single u8. It removes any discretion from the compiler about which bit pattern (formally called the *discriminant*) to associate with particular variants.

To prescribe a representation, add `repr` attribute. You are able to specify any integer type (`i32`, `u8`, `i16`, `u16`, ...), as well as some special cases.

Using a prescribed representation has some disadvantages. In particular, it reduces your flexibility. It also prevents Rust from making space optimizations. Some enums, those with a single variant, require no representation. They appear

in source code, but occupy zero space in the running program.

11.2.2 Why use enums?

You could model colors differently. For instance, it's possible to create numeric constants that look identical in memory:

```
const BLACK: u8 = 0x0;  
const BLUE: u8 = 0x1;  
// ...
```

Using an enum adds an extra guard rail. It becomes much more difficult to use an illegal value in our code than if we were using an `u8` directly. You will see this demonstrated when the `Cursor` struct is introduced shortly at [Listing 11.17](#) in [Section 11.2.3](#).

11.2.3 Creating a type that can “print” to the VGA frame buffer

To print to the screen, we'll use a `Cursor` struct that handles the raw memory manipulation and can convert between our `Color` type and what is expected by VGA.

Listing 11.17. Definition and methods for the `cursor`. This type manages the interface between our code and the VGA framebuffer. (ch11/ch11-fledgeos-3/src/main.rs)

```
struct Cursor {  
    position: isize,  
    foreground: Color,  
    background: Color,  
}  
  
impl Cursor {  
    fn color(&self) -> u8 {  
        let fg = self.foreground as u8;           ①  
        let bg = (self.background as u8) << 4;    ①  
        fg | bg                                     ①  
    }  
  
    fn print(&mut self, text: &[u8]) { ②
```

```

    let color = self.color();

    let framebuffer = 0xb8000 as *mut u8;

    for &character in text {
        unsafe {
            framebuffer.offset(self.position).write_volatile(character);
            framebuffer.offset(self.position + 1).write_volatile(color);
        }
        self.position += 2;
    }
}
}

```

- ① Use the foreground color as a base. It occupies the lower four bits. Shift the background color left to occupy the higher bits. Then merge them together.
- ② For expediency, the input will use a raw byte stream, rather than a type that guarantees the correct encoding.

11.2.4 Printing to the screen

Making use of `Cursor` involves setting its position and then sending a reference to the `Cursor.print()`.

Listing 11.18. New `_start()` function demonstrating printing to the screen. (ch11/ch11-fledgeos-3/src/main.rs)

```

#[no_mangle]
pub extern "C" fn _start() -> ! {
    let text = b"Rust in Action";

    let mut cursor = Cursor {
        position: 0,
        foreground: Color::BrightCyan,
        background: Color::Black,
    };
    cursor.print(text);

    loop {
        hlt();
    }
}

```

11.2.5 Full code listing of FledgeOS with printing enabled

Listing 11.19. FledgeOS now prints text to the screen (ch11/ch11-fledgeos-3/src/main.rs)

```
#![feature(core_intrinsics)]
#![feature(lang_items)]
#![no_std]
#![no_main]

use core::intrinsics;
use core::panic::PanicInfo;

use x86_64::instructions::{hlt};

#[allow(unused)]
#[derive(Clone, Copy)]
#[repr(u8)]
enum Color {
    Black = 0x0,    White = 0xF,
    Blue = 0x1,     BrightBlue = 0x9,
    Green = 0x2,    BrightGreen = 0xA,
    Cyan = 0x3,     BrightCyan = 0xB,
    Red = 0x4,      BrightRed = 0xC,
    Magenta = 0x5,  BrightMagenta = 0xD,
    Brown = 0x6,    Yellow = 0xE,
    Gray = 0x7,     DarkGray = 0x8
}

struct Cursor {
    position: isize,
    foreground: Color,
    background: Color,
}

impl Cursor {
    fn color(&self) -> u8 {
        let fg = self.foreground as u8;
        let bg = (self.background as u8) << 4;
        fg | bg
    }

    fn print(&mut self, text: &[u8]) {
        let color = self.color();

        let framebuffer = 0xb8000 as *mut u8;

        for &character in text {
            unsafe {
                framebuffer.offset(self.position).write_volatile(character);
                framebuffer.offset(self.position + 1).write_volatile(color);
            }
            self.position += 2;
        }
    }
}

#[panic_handler]
#[no_mangle]
```

```

pub fn panic(_info: &PanicInfo) -> ! {
    unsafe {
        intrinsics::abort();
    }
}

#[lang = "eh_personality"]
#[no_mangle]
pub extern "C" fn eh_personality() { }

#[no_mangle]
pub extern "C" fn _start() -> ! {
    let text = b"Rust in Action";

    let mut cursor = Cursor {
        position: 0,
        foreground: Color::BrightCyan,
        background: Color::Black,
    };
    cursor.print(text);

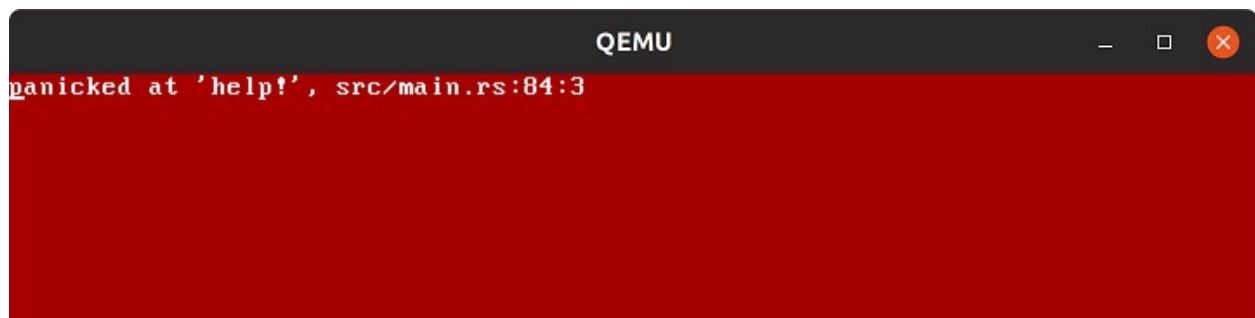
    loop {
        hlt();
    }
}

```

11.3 IMPLEMENTING A PANIC HANDLER THAT REPORTS THE ERROR TO THE USER

For the benefit of anyone doing embedded development or who wants to execute Rust on microcontrollers, it's important to learn how to report where a panic occurs. A good place to start is `core::fmt::Write`. That trait can be associated with the panic handler to display a message.

Figure 11.3. Displaying a message when a panic occurs.



11.3.1 Re-implementing `panic()` by making use

of `core::fmt::Write`

[Figure 11.3](#) displays the new panic handler. `panic()` now goes through a two stage process. In the first stage, `panic()` clears the screen. The second stage involves the `core::write!` macro. `core::write!` takes a destination object as its first argument (cursor) that implements the `core::fmt::Write` trait.

Listing 11.20. Panic handler that reports that an error has occurred by clearing the screen and printing the message (ch11/ch11-fledgeos-4/src/main.rs)

```
pub fn panic(info: &PanicInfo) -> ! {
    let mut cursor = Cursor {
        position: 0,
        foreground: Color::White,
        background: Color::Red,
    };
    for _ in 0..(80*25) { ①
        cursor.print(b" "); ①
    } ①
    cursor.position = 0; ②
    write!(cursor, "{}", info).unwrap(); ③

    loop {} ④
}
```

- ① Clear the screen by filling it with red
- ② Reset the position of the cursor
- ③ Print the `PanicInfo` to the screen
- ④ Spin in an infinite loop, allowing the user to read the message and restart the machine manually.

11.3.2 Implementing `core::fmt::Write`

Implementing `core::fmt::Write` involves implementing one method, `write_str()`. The trait defines several others, but they are able to be auto-generated by the compiler once an implementation of `write_str()` is available.

Listing 11.21. Implementing `core::fmt::Write` for the cursor type. The implementation re-uses the `print()` method and converts the

**UTF-8 encoded &str into &[u8] with the to_bytes() method.
(ch11/ch11-fledgeos-4/src/main.rs)**

```
impl fmt::Write for Cursor {
    fn write_str(&mut self, s: &str) -> fmt::Result {
        self.print(s.as_bytes());
        Ok(())
    }
}
```

11.3.3 Full code listing for FledgeOS with user-friendly panic handling

Listing 11.22. Full code listing of FledgeOS with complete panic handling (ch11/ch11-fledgeos-4/src/main.rs)

```
#![feature(core_intrinsics)]
#![feature(lang_items)]
#![no_std]
#![no_main]

use core::fmt;
use core::panic::PanicInfo;
use core::fmt::Write;

use x86_64::instructions::{hlt};

#[allow(unused)]
#[derive(Copy, Clone)]
#[repr(u8)]
enum Color {
    Black = 0x0,    White = 0xF,
    Blue = 0x1,     BrightBlue = 0x9,
    Green = 0x2,    BrightGreen = 0xA,
    Cyan = 0x3,     BrightCyan = 0xB,
    Red = 0x4,      BrightRed = 0xC,
    Magenta = 0x5,  BrightMagenta = 0xD,
    Brown = 0x6,    Yellow = 0xE,
    Gray = 0x7,     DarkGray = 0x8
}

struct Cursor {
    position: isize,
    foreground: Color,
    background: Color,
}

impl Cursor {
    fn color(&self) -> u8 {
        let fg = self.foreground as u8;
        let bg = (self.background as u8) << 4;
    }
}
```

```

    fg | bg
}

fn print(&mut self, text: &[u8]) {
    let color = self.color();

    let framebuffer = 0xb8000 as *mut u8;

    for &character in text {
        unsafe {
            framebuffer.offset(self.position).write_volatile(character);
            framebuffer.offset(self.position + 1).write_volatile(color);
        }
        self.position += 2;
    }
}

impl fmt::Write for Cursor {
    fn write_str(&mut self, s: &str) -> fmt::Result {
        self.print(s.as_bytes());
        Ok(())
    }
}

#[panic_handler]
#[no_mangle]
pub fn panic(info: &PanicInfo) -> ! {
    let mut cursor = Cursor {
        position: 0,
        foreground: Color::White,
        background: Color::Red,
    };
    for _ in 0..(80*25) {
        cursor.print(b" ");
    }
    cursor.position = 0;
    write!(cursor, "{}", info).unwrap();

    loop { unsafe { hlt(); }}
}

#[lang = "eh_personality"]
#[no_mangle]
pub extern "C" fn eh_personality() { }

#[no_mangle]
pub extern "C" fn _start() -> ! {
    panic!("help!");
}

```

11.4 SUMMING UP

You have learned:

- Avoiding the `std` module with the `#[no_std]` attribute
- Defining a program's entry point manually, in conjunction with the `[no_main]` and `[no_mangle]` attributes
- Using the `core` module to program for freestanding targets where you do not have access to an operating system
- Implementing a panic handler using a type that implements `core::fmt::Write`
- Avoiding raw pointer manipulation with methods, including `offset()`
- Writing to raw memory without the risk of the compiler re-writing or re-ordering your code with `write_volatile()`
- Accessing compiler intrinsics, such as `intrinsics::abort()`, to provide functionality to the program that's ordinarily inaccessible
- Accessing raw machine instructions, such as HTL, via the `x86_64` crate

12 SIGNALS, INTERRUPTS AND EXCEPTIONS

In this chapter:

- Learn how the outside world, such as the network and the keyboard, lets an application know that it has data ready for processing
- Disambiguating between the terms interrupt, exception, trap and fault
- Send and receive signals between running applications

This chapter describes the process by which the outside world communicates with your operating system. Programs' execution is constantly being interrupted by the network when bytes are ready to be delivered. That is, after connecting to a database—or at any other time—the operating system could demand that your application deals with a message. This chapter describes how this process operates and how to prepare your programs for it.

In the last chapter, you learned a digital clock is periodically notifying the operating system that time has progressed. This chapter explains how those notifications occur. It also introduces the concept of multiple applications running at the same time via the concept of signals.

Signals emerged as part of the UNIX operating system tradition and can be used to send messages between different running programs.

We'll be addressing both concepts—signals and interrupts—together as the programming model is similar but it's simpler to get started with signals.

This chapter tends to focus on the Linux operating system running on x86 CPUs. That's not to say that users of other operating systems are unable to follow

along.

12.1 DISAMBIGUATING SEVERAL RELATED TERMS

This computing sub-field contains related terms. For newcomers, they all look similar and it's not helped that they are used interchangeably. Examples of confusing terms include:

- Trap
- Exception
- Signal
- Fault
- Interrupt
- Abort
- Software interrupt

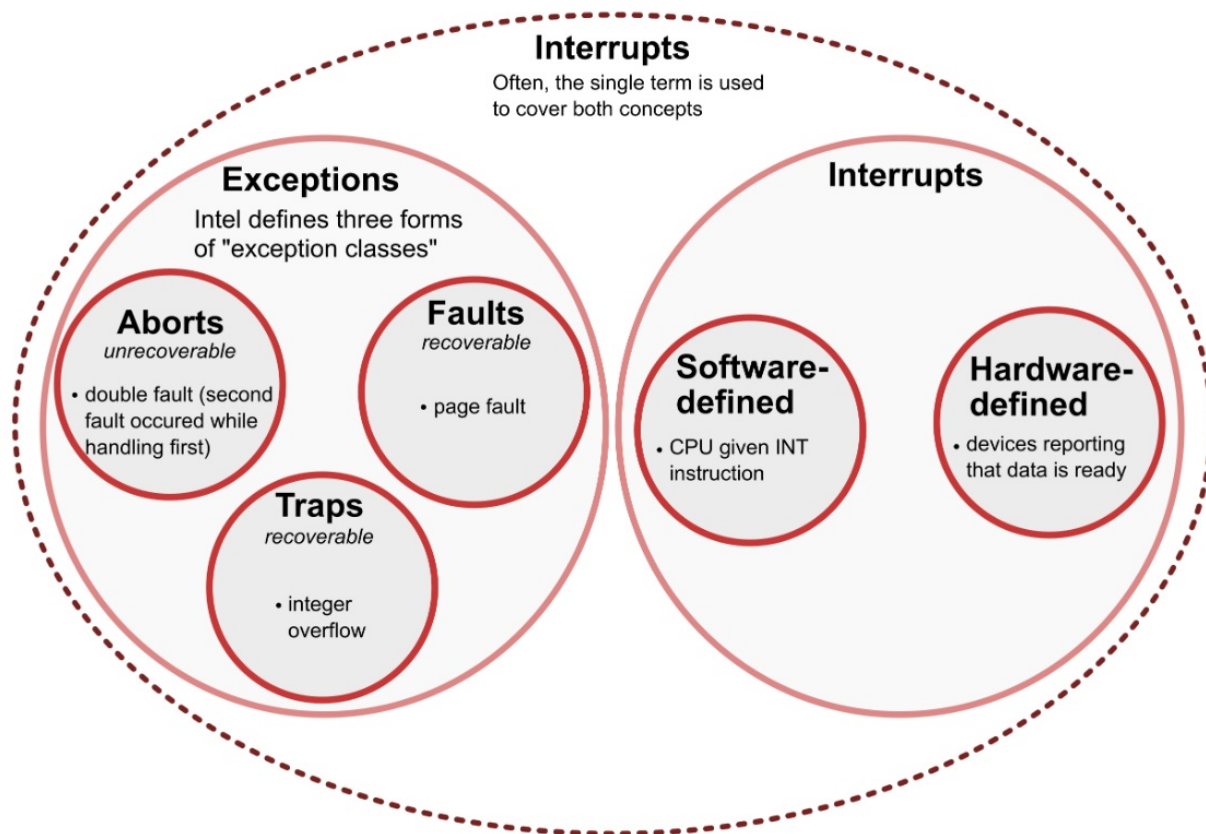
The two concepts that are most important to distinguish are *signals* and *interrupts*. A signal is a software-level abstraction that is associated with an operating system. An interrupt is a CPU-related abstraction and is closely associated with the system's hardware.

Signals are a form of limited inter-process communication. They don't contain content, but their presence indicates something. They're analogous to a physical, audible buzzer. The buzzer isn't providing any content, but the person who set the alarm still knows what's intended when it's making noise. To add confusion to the mix, signals are often described as software interrupts, although this chapter will attempt to avoid the use of the term interrupt to refer to a signal.

There are two forms of interrupts, which differ in their origin. One form of interrupts occur within the CPU during its processing. They are the result of attempting to process illegal instructions and attempting to access invalid memory addresses. This first form is known technically as *synchronous interrupts*, but you will tend to hear them referred to by their more common name: *exceptions*. The second form of interrupts are generated by hardware

devices, such as keyboards and accelerometers. They are what's commonly implied by the term interrupt. They can occur at any time and are formally known as *asynchronous interrupts*. Just like signals, they're also able to be generated within software. [Figure 12.1](#) illustrates how they inter-relate.

Figure 12.1. A visual taxonomy of how the terms interrupt, exception, trap and fault interact within Intel CPUs.



Interrupts can be specialized. A *trap* is an error detected by the CPU that it gives the operating system a chance to recover from. A *fault* is another form of recoverable problem. If the CPU is given a memory address that it can't read from, it notifies the operating system and asks for an updated address.

Interrupts will force an application's control flow to change, whereas many signals can be ignored when desired. Upon receiving an interrupt, the CPU will jump to handler code, irrespective of the current state of the program. The location of the handler code is pre-defined by the BIOS and operating system

during a system's bootup process.

Figure 12.2. Using addition to demonstrate signal handling control flow

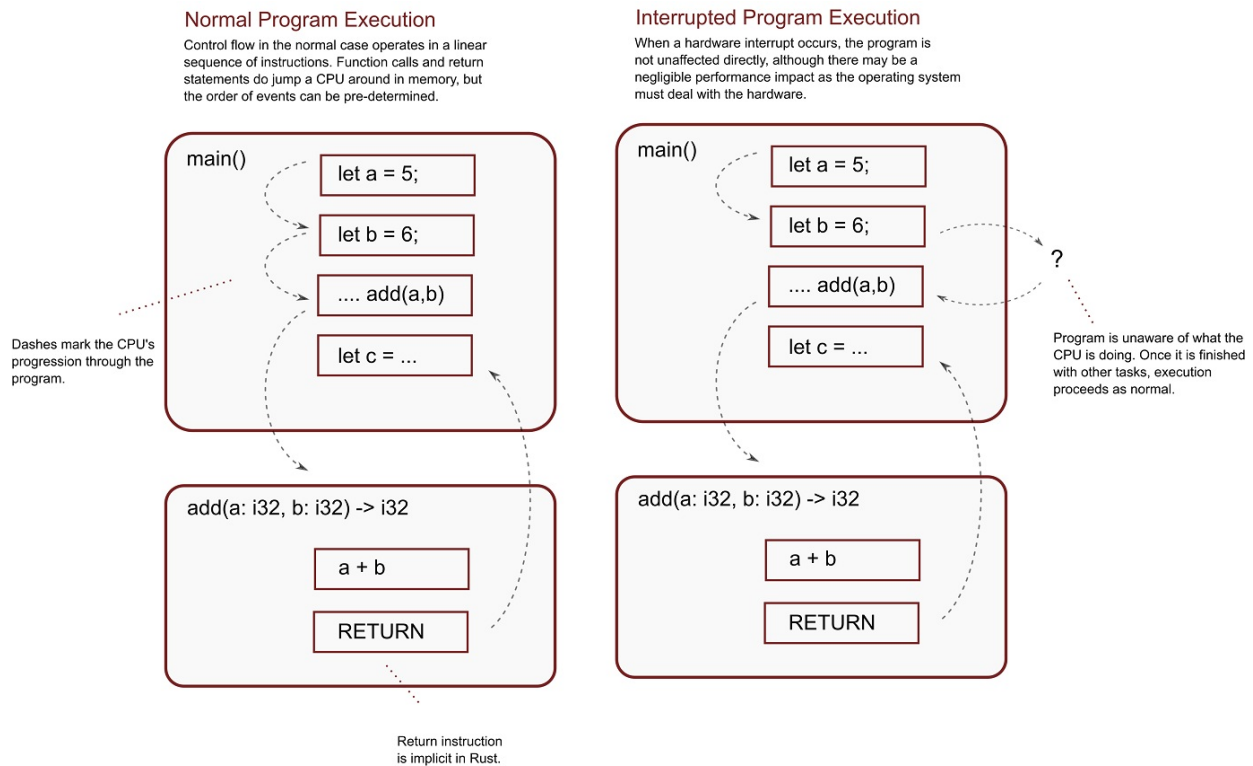
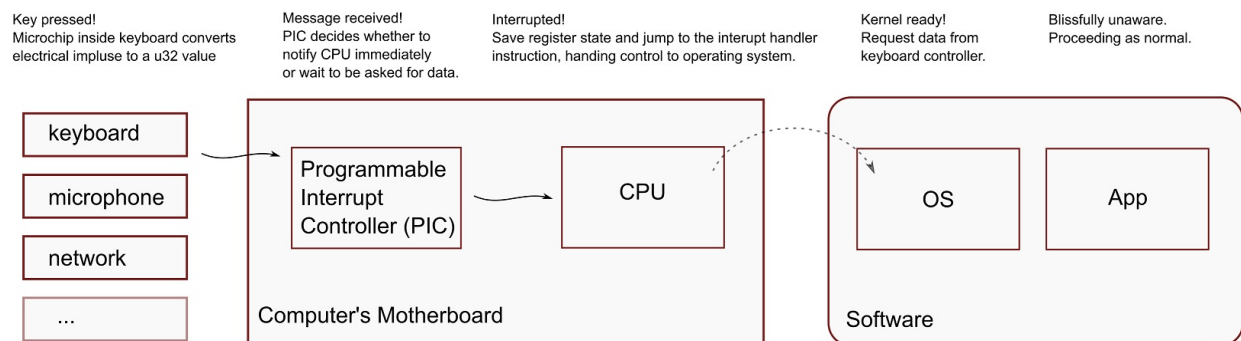


Figure 12.3. Lifetime of an interrupt generated from a hardware device. Once the operating system has been notified that data is ready, it will when communicate with the device (in this case, the keyboard) directly to read the data into its own memory.



To make things more difficult for learners, applications tend to be notified of

interrupts as signals. This makes it even easier to conflate the two.

Listing 12.1. Source code for [Figure 12.2](#)

```
fn add(a: i32, b:i32) -> i32 {  
    a + b  
}  
  
fn main() {  
    let a = 5;  
    let b = 6;  
    let c = add(a,b);  
}
```

Treating Signals as Interrupts

Handling interrupts directly means manipulating the operating system kernel. Because we would prefer not to do that in a learning environment, we'll play fast and loose with the terminology. For the rest of this chapter, treats signals as if they are interrupts.

Why simplify things? Writing operating system components involves tweaking the kernel. Breaking things there means that our system could become completely unresponsive without a clear way to fix things.

From a more pragmatic perspective, avoiding tweaks to the kernel means that we can avoid learning a whole new compiler toolchain.

We also have the advantage that the programming model is quite similar between signal handling and interrupt handling. That allows to keep any errors within our code to be constrained to our application, rather than risk bringing the whole system down.

The general pattern:

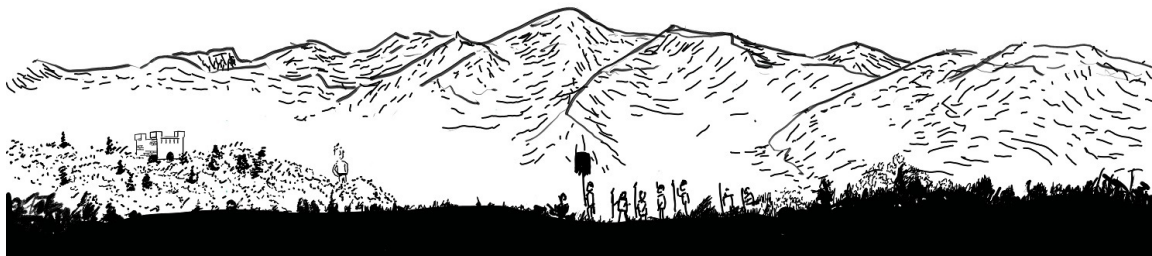
- Model your application's standard control flow
- Model the interrupted control flow—identify resources that would need to be cleanly shut down if required
- Write the interrupt/signal handler—be fast, update some state and return quickly. You will typically delegate time-consuming operations by only modifying a global variable that is regularly checked by the main loop of the program
- Modify your application's standard control flow to look for the GO/NO GO flag that may have been modified by a signal handler

12.2 AVOIDING WRITING CODE BY RELYING ON THE DEFAULT SIGNAL HANDLING BEHAVIOR

Sometimes the best approach is to let the system's defaults do the work. Code that you don't need to write is code that's free from bugs that you've caused.

The default behavior for most signals is shutting down the application. When an application does not provide a special handler function—we'll learn how to do that in this chapter—the operating system considers the signal to be an abnormal condition. When an operating system detects an abnormal condition within an application, things don't end well for the application. It terminates the application.

Figure 12.4. An application defending itself from maurauding hoards of unwanted signals. Signal handlers are the friendly giants of the computing world. They stay out of the way, but are there when your application needs to defend its castle. Although not part of everyday control-flow, they're extremely useful when the time is right. Note: Not all signals can be handled. SIGKILL is particularly vicious.



Here are three common signals that your application may receive and their intended action:

- SIGINT - Terminate the program, usually generated by person.
- SIGTERM - Terminate the program, usually generated by another program.
- SIGKILL - Immediately terminate the program, without the ability to recover.

There are many others. A fuller list is provided at [Listing 12.5](#). You'll notice that these three examples are heavily associated with terminating a running program.

That's not necessarily the case.

12.2.1 Using SIGSTOP and SIGCONT to suspend and resume a program's operation

In the case of the SIGSTOP signal, the application is suspended until it receives SIGCONT. This is used by UNIX systems for job control. It's also useful to know about if you wish to manually intervene and halt a running application, but would like the ability to recover some time in the future.

[Listing 12.4](#) is a minimal environment that allows us to invoke this stop, then continue functionality. What Rust will you learn? You'll discover the “inclusive range” syntax.

Listing 12.2. Overview of the files within the `ch12-sixty` project

```
ch12-sixty
├── src
│   ├── main.rs
└── Cargo.lock
```

Listing 12.3. Package description file (`ch12/ch12-sixty/Cargo.toml`)

```
[package]
name = "ch12-sixty"
version = "0.1.0"
authors = ["Tim McNamara <author@rustinaction.com>"]

[dependencies]
```

Listing 12.4. Basic application that lives for 60 seconds, printing out its progress along the way. Happ to receive signals `SIGSTOP` and `SIGCONT`. (`ch12/ch12-sixty/src/main.rs`)

```
use std::time;
use std::process;
use std::thread::{sleep};

fn main() {
    let delay = time::Duration::from_secs(1);①
```



```

let pid = process::id();
println!("{}", pid);

for i in 1..=60 {
    sleep(delay);
    println!("{}", i);
}

```

②

- ① This application is intended to be run in conjunction with second terminal that is generating signals.
- ② The `..=` syntax is an inclusive range. `1..60` counts from 1 to 59, `1..=60` counts from 1 to 60.

Once the code from [Listing 12.4](#) is saved to disk, open two consoles. In the first, execute `cargo run`. A 3-5 digit number will appear, followed by a counter incrementing per second. The first line's number is the “pid”, or *process id*.

Console 1: Compiles and executes ch12-sixty	Console 2: Sends signals to the ch12-sixty	Comments
... \$ cd ch12/ch12-sixty \$ cargo run 23221 . 1 . 2 . 3 . 4 ...		23221 appears as output because it is the <i>process identifier</i> (pid) for this invocation of ch10-sixty.
	... \$ kill -SIGSTOP 23221 ...	If you are unfamiliar with the shell command <code>kill</code> , its role is to send signals. It's named after its most common role, to send SIGTERM.
... [1]+ Stopped cargo run \$...		This console window will be returned to the prompt, as there is no longer anything running in the foreground.
	... \$ kill -SIGCONT 23221 ...	
... . 5 . 6 . 7 . 8 ...		

The default handling of SIGSTOP and SIGCONT provides utility without further tweaks. So what are the other signals and what are their default handlers?

12.2.2 Listing all signals supported by the operating system

We can ask `kill` for that information:

Listing 12.5. Generating a list of signals that are supported on your operating system with the `kill` command

```
$ kill -l ①
1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
6) SIGABRT     7) SIGEMT     8) SIGFPE     9) SIGKILL    10) SIGBUS
11) SIGSEGV    12) SIGSYS    13) SIGPIPE    14) SIGALRM    15) SIGTERM
16) SIGURG     17) SIGSTOP   18) SIGTSTP    19) SIGCONT    20) SIGCHLD
21) SIGTTIN    22) SIGTTOU   23) SIGIO      24) SIGXCPU    25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF   28) SIGWINCH   29) SIGPWR     30) SIGUSR1
31) SIGUSR2    32) SIGRTMAX
```

① -l stands for “list”

That’s a lot, Linux! To make matters worse, few signals have standardized behavior.

Thankfully, most applications don’t need to worry about setting handlers for many of them (if any). Here is a much tighter list of signals that are more likely to be encountered in day-to-day programming:

Table 12.1. List of common signals, their default actions and tips for sending them from the command line

Signal	Read as	Default Action	Comment	Shortcut
SIGHUP	Hung Up	Terminate	Its origins are from telephone-based digital communications. Now often sent to background applications (daemons/services) to request that they re-read their configuration files. Sent to running programs when you logout from a shell.	kbd: [CTRL+D]
SIGINT	Interrupt (or perhaps	Terminate	User-generated signal to terminate a running application.	kbd: [Ctrl+C]

	interactive)			
SIGTERM	Terminate	Terminate	Asks application to “gracefully” terminate.	
SIGKILL	Kill	Terminate (unstoppable)		
SIGQUIT	Quit	Write memory to disk as a <i>core dump</i> , then terminate.		kbd:[Ctrl+\]
SIGTSTP	Terminal Stop	Pause execution	Application was requested to stop from the terminal	kbd:[Ctrl+Z]
SIGSTOP	Stop	Pause execution (unstoppable)		
SIGCONT	Continue	Resume execution when paused		

Looking at the “Default Action” column, we can see that SIGKILL and SIGSTOP have special status. Programs are able to avoid the others. Those two cannot be handled or blocked by the application.

12.3 HANDLING SIGNALS WITH CUSTOM ACTIONS

The default actions for signals are fairly limited. Receiving a signal tends to end badly for applications by default. If external resources, such as a database connection, are left open, they might not be cleaned up properly when the application ends.

The most common use case for signal handlers is to allow an application to cleanly shutdown. Some common tasks that might be necessary when an application shuts down:

- flushing the hard disk drive to ensure that pending data has been written to disk
- closing any network connections

- de-registering from any distributed scheduler or work queue

To stop the current workload and shut down, a signal handler is required. To set up a signal handler, we need to create a function with the signature `f(i32) → ()`. That is, the function needs to accept an `i32` integer as its sole argument and return no value.

This poses some software engineering issues. The signal handler isn't able to access any information from the application, except which signal was sent. Therefore it doesn't know what state anything is in and so can't know what needs shutting down beforehand.

There are some additional constraints, in addition to the architectural one. Signal handlers must also act very quickly, with a subset of functionality available to general code. They're constrained in time and scope.

Signal handlers must act quickly for two main reasons:

1. They're blocking other signals of the same type from being handled
2. Moving fast reduces the likelihood that they are operating alongside another signal handler of another type

Signal handlers have reduced scope in what they're permitted to do. For example, they must avoid executing any code that might itself generate signals.

To wriggle out of this constrained environment, the ordinary approach to this is to use a Boolean flag as a global variable that is regularly checked during a program's execution.

If the flag is set, then call a function to cleanly shutdown the application within the context of the application.

For this pattern to work, there are two requirements:

- the signal handler's sole responsibility is to mutate the flag
- the application must regularly check the flag to detect whether the

flag has been modified

12.3.1 Global variables in Rust

Rust facilitates global variables (variables accessible anywhere within the program) by declaring a variable with the `static` keyword in global scope. Suppose we wanted to create a global value `SHUT_DOWN` that we set to `true` when a signal handler believes it's time to urgently shut down. We use this declaration:

```
static mut SHUT_DOWN: bool = false;
```

`static mut` is read as “mutable static”, irrespective of how grammatically contorted that is.

Global variables presents an issue for Rust programmers. Accessing them—even for reading only—is unsafe. That can mean that the code can become quite cluttered, as it's wrapped in unsafe blocks. This ugliness is a signal—avoid global state where possible.

[Listing 12.8](#) presents a toy example of reading from (line 12) and writing to (lines 7-9) a `static mut` variable. The call to `rand::random()` on line 8

produces Boolean values. Output is a series of dots. 50% [\[44\]](#) of the time, you'll receive an output that looks like this:

Listing 12.6. Typical output produced by [Listing 12.8](#)

```
.
```

Listing 12.7. Crate metadata for [Listing 12.8](#) (ch12/ch12-toy-global/Cargo.toml)

```
[package]
name = "ch12-toy-global"
version = "0.1.0"
authors = ["Tim McNamara <author@rustinaction.com>"]
edition = "2018"

[dependencies]
```

```
rand = "0.6"
```

Listing 12.8. Accessing global variables (mutable statics) in Rust (ch12/ch12-toy-global/src/main.rs)

```
use rand;

static mut SHUT_DOWN: bool = false;

fn main() {
    loop {
        unsafe {
            SHUT_DOWN = rand::random(); ① ②
        }
        print!(".");

        if unsafe { SHUT_DOWN } { ③
            break
        };
    }
    println!()
}
```

- ① Reading from and writing to a static `mut` variable requires an `unsafe` block
- ② `rand::random()` is a short-cut that calls `rand::thread_rng().gen()` to produce a random value. The required type is inferred from the type of `SHUT_DOWN`.
- ③ Because blocks evaluate to their last result, it's possible to use a block within

12.3.2 Using a global variable to indicate that shutdown has been initiated

Given that signal handlers must be quick and simple, we'll do the minimal work possible. We'll set a variable to indicate that the program needs to shut down. This technique is demonstrated by [Listing 12.11](#).

[Listing 12.11](#) is structured into three functions:

- `register_signal_handlers()` - Communicates to the operating system (via `libc`) what the signal handler for each signal is. This function makes use of a *function pointer*, which treats a function as data.

Function pointers are explained at [Section 12.4.1](#).

- `handle_signals()` - Handles incoming signals. This function is agnostic as to which signal is sent, although we'll only be dealing with `SIGTERM`.
- `main()` - Initializes the program and iterates through a “main loop”.

When run, the resulting executable produces a trace of where it is:

Listing 12.9. Intended output from [Listing 12.11](#)

```
1
SIGUSR1
2
SIGUSR1
3
SIGTERM
4
* ①
```

① I hope that you will forgive the cheap ASCII art explosion.



Note

If the signal handler was not correctly registered, the word “Terminated” may appear. Make sure that you’ve added a call to `register_signal_handler()` early within `main()`. [Listing 12.11](#) does this on line 38.

Listing 12.10. Crate setup for [Listing 12.11](#) (ch12/ch12-basic-handler/Cargo.toml)

```
[package]
name = "ch12-handler"
version = "0.1.0"
authors = ["Tim McNamara <author@rustinaction.com>"]

[dependencies]
libc = "0.2"
```

Listing 12.11. Creating an basic signal handler that modifies a global variable when executed (ch12/ch12-basic-

handler/src/main.rs

```
#![cfg(not(windows))] ①

extern crate libc; ②

use std::time; ③
use std::thread::{sleep}; ④
use libc::{SIGTERM, SIGUSR1}; ⑤

static mut SHUT_DOWN: bool = false; ⑥

fn main() {
    register_signal_handlers();

    let delay = time::Duration::from_secs(1);

    for i in 1_usize.. { ⑦
        println!("{}", i);
        unsafe {
            if SHUT_DOWN {
                println!("{}", i);
                return;
            }

            sleep(delay);

            let signal = if i > 2 {
                SIGTERM
            } else {
                SIGUSR1
            };
            unsafe {
                libc::raise(signal);
            }
        }
        unreachable!();
    }

    fn register_signal_handlers() {
        unsafe {
            libc::signal(SIGTERM, handle_sigterm as usize); ⑦
            libc::signal(SIGUSR1, handle_sigusr1 as usize); ⑧
        }
    }

    #[allow(dead_code)] ⑩
    fn handle_sigterm(_signal: i32) {
        register_signal_handlers();

        println!("SIGTERM");

        unsafe {
            SHUT_DOWN = true;
        }
    }
}
```



```
#[allow(dead_code)] ⑩
fn handle_sigusr1(_signal: i32) {
    register_signal_handlers();

    println!("SIGUSR1");
}
```

- ① Indicates to cargo/rustc that this code won't run on MS Windows.
- ② Imports the libc crate, enabling access to the kernel's functionality, such as signals.
- ③ Brings the Duration type into scope, enabling us to represent spans of time.
- ④ Enables our program to pause its execution.
- ⑤ Brings the two signal constants into local scope.
- ⑥ Initialize a Boolean mutable static to false.
- ⑦ Calling functions within libc requires an unsafe block, as it's outside of Rust's control.
- ⑧ libc::signal takes a signal name (technically, an integer) and an address of a function (a *function pointer*, albeit untyped) as arguments and associates the signal with the function
- ⑨ Why usize? libc::signal() requires an integer as its second argument. As *function pointers*. handle_sigterm and handle_sigusr1 have the type fn(i32) → ().
- ⑩ Without this attribute, rustc warns that this function is never run.

Understanding the difference between const and static

Static and constant seem similar. Here is the main difference:

- static values appear in a single location in memory
- const values may be duplicated in locations where they're accessed

Duplicating const values can be a CPU-friendly optimization. It allows for data locality and improved cache performance.

Why use confusingly similar names for two different things? It could be considered a historical accident. The word static refers to the segment of the address space that the variables live in. static values live outside of the stack space. They live within the region where string literals are held, near the bottom of the address space. That means, accessing a static variable almost certainly implies a dereferencing a pointer. The constant in const values refers to the value itself. When accessed from code, the data might get duplicated to every location that it's needed, if the compiler believes that will result in faster access.

[44] Assuming a fair random number generator, which Rust uses by default. This assumption holds as long as you trust your operating system's random number generator.

12.4 SENDING APPLICATION-DEFINED SIGNALS

Signals can be used as a limited form of messaging. Within your business rules, create definitions for SIGUSR1 and SIGUSR2. They're unallocated by design. In [Listing 12.11](#), we use SIGUSR1 to do very little work.

12.4.1 Understanding function pointers and their syntax

[Listing 12.11](#) includes some syntax that might be confusing:

```
handle_sigterm as usize
```

What is happening here? The address that the function is stored at is being converted to an integer. In Rust, the `fn` keyword creates a *function pointer*.

Readers who have worked through chapter 5 will understand that functions are just data. That is to say they're sequences of bytes that make sense to the CPU. A function pointer is a pointer to the start of that sequence.

A pointer is a data type that acts as a stand-in for its referent. Within an application's source code, pointers contain both the address of the value referred to as well as its type. The type information is something that's stripped away in the compiled binary. Their internal representation is an integer of `usize`. That makes pointers very cheap to pass around. In C, making use of function pointers can feel like arcane magic. In Rust, they're hiding in plain sight.

Every `fn` declaration is actually a function pointer. That means, that [Listing 12.13](#) is legal code and should print out something similar to the following line:

Listing 12.12. Expected output from [Listing 12.13](#)

```
noop as usize: 0x5620bb4af530 ①
```

① 0x5620bb4af530 is the memory address (in hexadecimal notation) of the start of the `noop()` function. This number will be different on your machine.

Listing 12.13. Casting a function to `usize` to demonstrate that it is being used as a function pointer.

```
fn noop() {}

fn main() {
    let fn_ptr = noop as usize;

    println!("noop as usize: 0x{:x}", fn_ptr);
}
```

But what is the type of the function pointer created from `fn noop()`? To describe function pointers, Rust re-uses its function signature syntax. In the case of `fn noop()`, the type is `*const fn() -> ()`. This type is read as “a *const* pointer to a function that takes no arguments and returns *unit*”. A *const pointer* is immutable and *unit* is Rust’s stand-in value for nothingness.

[Listing 12.15](#) adds a second type cast. Its output should two lines that should be mostly identical:

Listing 12.14. Expected output from [Listing 12.15](#)

```
noop as usize:      0x55ab3fdb05c0 ①
noop as *const T: 0x55ab3fdb05c0 ①
```

① These two numbers will be different on your machine, but they should match each other.

Listing 12.15. Casting a function to `usize` to demonstrate that it is being used as a function pointer.

```
fn noop() {}
```

```
fn main() {
    let fn_ptr = noop as usize;
    let typed_fn_ptr = noop as *const fn() -> ();

    println!("noop as usize:    0x{:x}", fn_ptr);
    println!("noop as *const T: {:p}", typed_fn_ptr); ①
}
```

① Note the use of the pointer format modifier `{:p}`

12.5 IGNORING SIGNALS

As illustrated earlier by [Table 12.1](#), most signals terminate the running program by default. This can be somewhat disheartening for the running program attempting to get its work done. Sometimes the application knows best. For those cases, many signals can be ignored.

`SIGSTOP` and `SIGKILL` aside, the constant `SIG_IGN` can be provided to `libc::signal()`, instead of a function pointer. An example of its usage is provided by [Listing 12.17](#), which prints the following line to the console:

Listing 12.16. Output from [Listing 12.17](#)

```
ok
```

Listing 12.17. Ignoring signals with `libc::SIG_IGN`, then resetting them to their default handling behavior.

```
extern crate libc;

use libc::{signal,raise};
use libc::{SIG_DFL, SIG_IGN, SIGTERM};

fn main() {
    unsafe {
        signal(SIGTERM, SIG_IGN); ②
        raise(SIGTERM);           ③
    }
    println!("ok");

    unsafe {
        signal(SIGTERM, SIG_DFL); ④
        raise(SIGTERM);           ⑤
    }
}
```

```
}  
println!("not ok");  
}  
⑥
```

- ① An unsafe block is required because Rust does not control what happens beyond the function boundaries.
- ② Set the SIGTERM signal to ignore.
- ③ `libc::raise()` allows code to make a signal, in this case to itself.
- ④ Reset the SIGTERM signal to its default.
- ⑤ Program terminates here.
- ⑥ This code is never reached, and therefore this string is never printed.

12.6 SHUTTING DOWN FROM DEEPLY NESTED CALL STACKS

What if our program is deep in the middle of a call stack and can't afford to unwind? When receiving a signal, the program might want to execute some clean up code before terminating (or being forcefully terminated). This is sometimes referred to as *nonlocal control transfer*.

UNIX-based operating systems provide some tooling to enable you to make use of that machinery via two system calls: `setjmp` and `longjmp`.

- `setjmp` sets a marker location
- `longjmp` jumps back to the previously marked location

Why bother with such programming gymnastics? Sometimes using low-level techniques like these is the only way out of a tight spot. They approach the Dark Arts of systems programming. To quote the manpage:

`setjmp()` and `longjmp()` are useful for dealing with errors and interrupts encountered in a low-level subroutine of a program.

-- Linux Documentation Project: `setjmp(3)`

These two tools circumvent normal control flow and allow programs to teleport themselves through the code. Occasionally an error occurs deep within a call stack. If our program takes too long to respond to the error, the operating system may simply abort the program and the program's data may be left in an inconsistent state. To avoid this, `longjmp` can be used to shift control directly to the error-handling code.

To understand what significance of this is, consider what happens in an ordinary program's callstack during several calls to a recursive function as shown in [Listing 12.18](#). Each call to `dive()` adds another place that control will eventually return to. `longjmp` is used to bypass them.

Figure 12.5. Control flow graph of the code from [Listing 12.23](#). The program's control flow can be intercepted via a signal and then resume from point of `setjmp()`

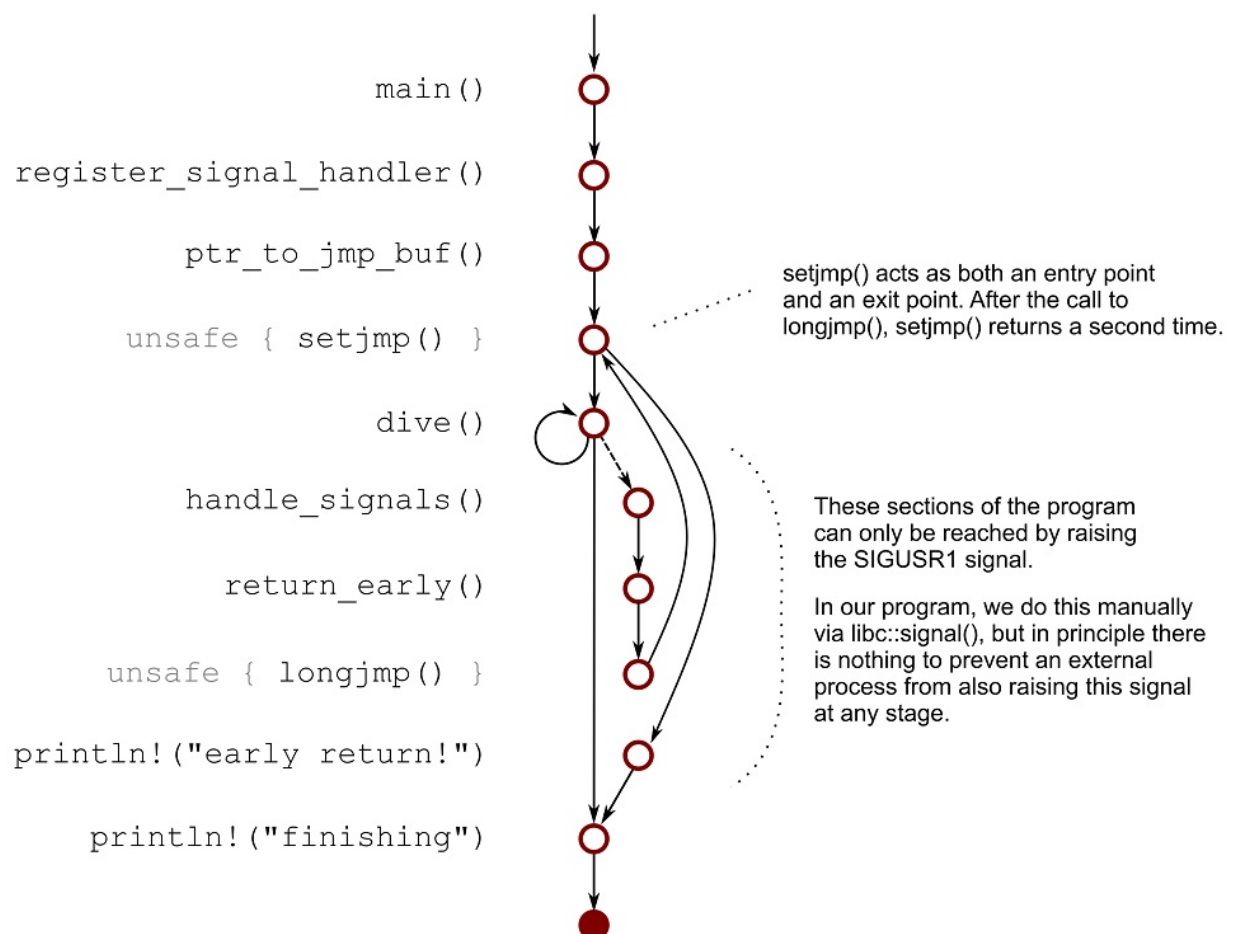


Table 12.2. Comparing the intended output from [Listing 12.18](#) and [Listing 12.23](#).

Listing 12.18 produces a symmetrical pattern. Each level is caused by a nested call to <code>dive()</code> , which is removed when the calls return.	Listing 12.23 produces a much different pattern. After a few calls to <code>dive()</code> , control teleports back to <code>main()</code> without the calls to <code>dive()</code> returning.
<pre># ## ### #### ##### #### ### ## #</pre>	<pre># ## ### early return! finishing!</pre>

On the right-hand side, the code jumps directly from the third call to the top to the call stack.

Listing 12.18. Illustrating how the callstack operates by printing its progress as the program executes (ch10/ch10-callstack/src/main.rs)

```
fn print_depth(depth: usize) {
    for _ in 0..depth {
        print!("#");
    }
    println!("\n");
}

fn dive(depth: usize, max_depth: usize) {
    print_depth(depth);
    if depth >= max_depth {
        return;
    } else {
        dive(depth+1, max_depth);
    }
    print_depth(depth);
}

fn main() {
    dive(0, 5);
}
```

Now... there's a lots of work to do to make this work.

The Rust language itself doesn't have the tools to enable this control flow trickery. It needs to access some provided by its compiler toolchain. Compilers provide special functions known as *intrinsics* to application programs. Using an *intrinsic function* from Rust takes some ceremony to set up, but operates a standard function once that set up is in-place.

12.6.1 Setting up intrinsics in a program

[Listing 12.23](#) uses two intrinsics, `setjmp()` and `longjmp()`. To enable them in our programs, the crate must be annotated with the following attribute:

Listing 12.19. Crate-level attribute required in `main.rs` (ch12/ch12-sjlj/src/main.rs)

```
#![feature(link_llvm_intrinsics)]
```

This raises two immediate questions that will be answered fully shortly:

1. What is an intrinsic function?
2. What is LLVM?

Additionally, we need to tell Rust about the functions that are being provided by LLVM. Rust won't know anything about them apart from their type signatures, which means that any use of them must occur within an `unsafe` block.

Listing 12.20. Declaring the LLVM intrinsic functions within [Listing 12.23](#) (ch12/ch12-sjlj/src/main.rs)

```
extern "C" {  
    #[link_name = "llvm.eh.sjlj.setjmp"] ①  
    pub fn setjmp(_: *mut i8) -> i32;      ②  
  
    #[link_name = "llvm.eh.sjlj.longjmp"] ①  
    pub fn longjmp(_: *mut i8);  
}
```

- ① Provide specific instructions to the linker about where it should look to find the function definitions

- ② As we're not using the argument's name, using an underscore (`_`) makes that explicit

This small section of code contains a fair amount of complexity.

- `extern "C"` means, "this block of code should obey C's conventions, rather than Rust's own ones"
- The `link_name` attribute tells the linker where to find the two functions that we're declaring
- The "eh" in `llvm.eh.sjlj.setjmp` stands exception handling and the "sjlj" stands for `setjmp/longjmp`.
- `*mut_i8` is a pointer to a signed byte. For those with C programming experience, you may recognize this as the pointer to the beginning of a string, e.g. a `*char` type.

WHAT IS AN INTRINSIC FUNCTION?

Intrinsic functions, generally referred to as *intrinsics*, are functions made available via the compiler, rather than as part of the language. Whereas Rust is largely target agnostic, the compiler has access to the target environment and this access can facilitate extra functionality. For example, a compiler understands the characteristics of the CPU that the to-be-compiled program will run on. The compiler can make that CPU's instructions available to the program via intrinsics.

Some examples of intrinsic functions:

- Atomic operations - Many CPUs provide specialist instructions for optimized certain workloads. For example, the CPU may guarantee that updating an integer is an *atomic operation*. Atomic here is meant in the sense of being indivisible. This can be extremely important when dealing with concurrent code.
- Exception handling - The facilities provided by CPUs for managing exceptions differ. These facilities can be used by programming language designers to create custom control flow. The `setjmp` and

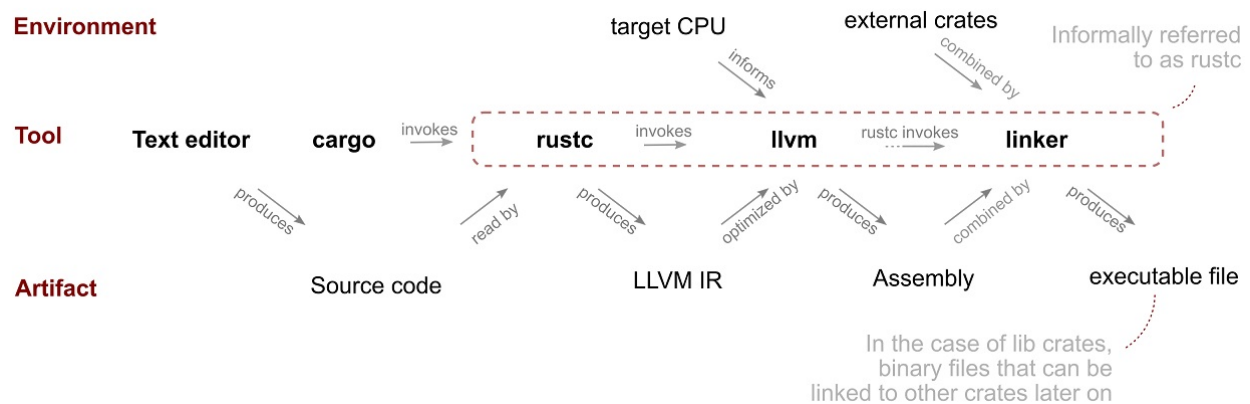
longjmp intrinsics, introduced later in this chapter, fall into this camp.

WHAT IS LLVM?

From the point of view of Rust programmers, LLVM can be considered as a subcomponent of `rustc`, the Rust compiler. It's an external tool that's bundled with `rustc`. Rust programmers can draw from tools that it provides. One set of tools that LLVM provides is intrinsic functions.

LLVM is itself a compiler. Its role is illustrated at [Figure 12.6](#).

Figure 12.6. Some of the major steps required to generate an executable from Rust source code. LLVM is an essentially part of the process, but one that is not user-facing.



It translates code produced by `rustc`, which produces LLVM IR (intermediate language), into machine-readable assembly language. To make matters more complicated, another tool, called a linker, is required to stitch multiple crates together. On MS Windows, Rust uses `link.exe`, a program provided by Microsoft, as its linker. On other operating systems, the GNU linker `ld` is used.

Understanding more detail about LLVM implies learning more about `rustc` and compilation in general. Like many things, getting closer to the truth requires exploring through a fractal-like domain. Learning every sub-system seems to require learning about another set of sub-systems. Explaining more here would

be a fascinating—but ultimately distracting—diversion.

12.6.2 Casting a pointer to another type

One of the more arcane parts of Rust’s syntax is how to cast between pointer types. You’ll encounter this as you make your way through [Listing 12.23](#). Problems arise because of the type signature of `setjmp()` and `longjmp()`:

```
extern "C" {  
    #[link_name = "llvm.eh.sjlj.setjmp"]  
    pub fn setjmp(_: *mut i8) -> i32;  
  
    #[link_name = "llvm.eh.sjlj.longjmp"]  
    pub fn longjmp(_: *mut i8);  
}
```

`setjmp()` and `longjmp()` both require a `*mut i8` as an input argument, yet the type that our Rust code is working with `&jmp_buf` [\[45\]](#). The `jmp_buf` type is defined like this:

```
const JMP_BUF_WIDTH: usize = mem::size_of::<usize>() * 8; ①  
type jmp_buf = [i8; JMP_BUF_WIDTH]; ②
```

- ① This constant will be 64 bits (8 bytes) in 64-bit machines and 32 bits (4 bytes) on 32-bit machines.
- ② We need enough storage available to store 8 or so integers. These bytes will be enough to for `setjmp()` to store a snapshot of the program’s current state.

There is only 1 `jmp_buf` value within our program, a global mutable static called `RETURN_HERE`:

```
static mut RETURN_HERE: jmp_buf = [0; JMP_BUF_WIDTH];
```

Thus arises our requirement to learn how to cast pointer types. Within the Rust code, we refer to `RETURN_HERE` as a reference, e.g. `&RETURN_HERE`. LLVM expects those bytes to be presented as a `*mut i8`.

To perform the conversion, there are multiple steps:

```
unsafe { &RETURN_HERE as *const i8 as *mut i8 }
```

Unpacking the conversion steps leads to this sequence:

- Start with `&RETURN_HERE`, a read-only reference to a global static variable of type `[i8; 8]` (or `[i8; 4]` on 32-bit machines)
- Convert that reference to a `*const i8`. Casting between pointer types is considered safe Rust, but dereferencing that pointer requires an `unsafe` block.
- Convert the `*const i8` to a `*mut i8`. This declares the memory as mutable (read/write).
- Wrap the conversion in an `unsafe` block, because it is dealing with access to a global variable

Why not use something like `&mut RETURN_HERE as *mut i8`? The Rust compiler becomes quite concerned about giving LLVM access to its data. The approach provided here, starting with a read-only reference, puts it at ease.

12.6.3 Running the Linux-specific code from [Listing 12.23](#) in other operating systems via Docker

The code in [Listing 12.23](#) does not run on MS Windows. To minimize this issue, we can use virtualization. Docker is a virtualization technology for Linux that enables you to create isolated environments. Docker Inc. also provides interfaces to enable macOS and MS Windows users to make use of Docker from within host operating system.

[Listing 12.21](#) provides a fairly complex Dockerfile. Docker will use it to create two Docker containers that result in a static binary that can be copied into any Linux-based operating system.

Listing 12.21. Dockerfile for [Listing 12.23](#) (ch12/ch12-

sjlj/Dockerfile)

```
FROM rust:latest as intermediate
RUN rustup toolchain install nightly
RUN rustup default nightly
RUN rustup target add x86_64-unknown-linux-musl

ENV PATH $PATH:/root/.cargo/bin
ENV PKG_CONFIG_ALLOW_CROSS=1

# fetch dependencies using a minimal project,
# enabling the docker image to be cached with dependencies installed
RUN USER=root cargo new --bin project
WORKDIR /project

COPY ./Cargo.lock ./Cargo.lock
COPY ./Cargo.toml ./Cargo.toml

# build actual project
COPY ./src ./src
RUN cargo +nightly -v check
RUN cargo +nightly -v build --target x86_64-unknown-linux-musl
RUN ls -R /project/target

FROM alpine
COPY --from=intermediate /project/target/x86_64-unknown-linux-musl/debug/ch12-sjlj /

CMD /ch12-sjlj
```

12.6.4 Compiling the code

We're now in a position where possible points of confusion within [Listing 12.23](#) should be minor. As it has been several pages, here again is the behavior that we're attempting to replicate:

Listing 12.22. Expected output from [Listing 12.23](#)

```
#
##
###
early return!
finishing!
```

One final note. [Listing 12.23](#) requires that `rustc` is on the nightly channel to compile correctly. If you encounter the following error, use `rustup` via the command line to install it `rustup install nightly`.

```

error[E0554]: #![feature] may not be used on the stable release channel
--> src/main.rs:1:1
   |
1 | #![feature(link_llvm_intrinsics)]
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
error: aborting due to previous error

For more information about this error, try `rustc --explain E0554`.

```

Once installed, you can request that cargo invokes the nightly compiler by adding `+nightly` to your command line scripts, e.g. `cargo +nightly build` and `cargo +nightly run`.

Listing 12.23. Using LLVM's internal compiler machinery ("intrinsics") to access the operating system's `longjmp` facilities. `longjmp` allows programs to escape their stack frame and jump anywhere within their address space. (ch12/ch12-sjlj/src/main.rs)

```

#![feature(link_llvm_intrinsics)]
#![allow(non_camel_case_types)]
#![cfg(not(windows))] ①

extern crate libc;

use libc::{SIGUSR1, SIGALRM, SIGHUP, SIGQUIT, SIGTERM}; ②
use std::mem;

const JMP_BUF_WIDTH: usize = mem::size_of::<usize>() * 8;
type jmp_buf = [i8; JMP_BUF_WIDTH];

static mut SHUT_DOWN: bool = false; ③
static mut RETURN_HERE: jmp_buf = [0; JMP_BUF_WIDTH]; ④
const MOCK_SIGNAL_AT: usize = 3; ⑤

extern "C" {
    #[link_name = "llvm.eh.sjlj.setjmp"]
    pub fn setjmp(_: *mut i8) -> i32;

    #[link_name = "llvm.eh.sjlj.longjmp"]
    pub fn longjmp(_: *mut i8);
}

#[inline] ⑥
fn ptr_to_jmp_buf() -> *mut i8 {
    unsafe { &RETURN_HERE as *const i8 as *mut i8 }
}

#[inline] ⑥
fn return_early() {
    let franken_pointer = ptr_to_jmp_buf();
    unsafe { longjmp(franken_pointer) }; ⑦
}

```

```

}

fn register_signal_handler() {
    unsafe {
        libc::signal(SIGUSR1, handle_signals as usize);
    }
}

#[allow(dead_code)]
fn handle_signals(_signal: i32) {
    register_signal_handler();

    unsafe {
        SHUT_DOWN = true;
    }

    return_early();
}

fn print_depth(depth: usize) {
    for _ in 0..depth {
        print!("#");
    }
    println!();
}

fn dive(depth: usize, max_depth: usize) {
    unsafe {
        if SHUT_DOWN {
            println!("!");
            return;
        }
    }
    print_depth(depth);

    if depth >= max_depth {
        return;
    } else if depth == MOCK_SIGNAL_AT {
        unsafe {
            libc::raise(SIGUSR1);
        }
    } else {
        dive(depth + 1, max_depth);
    }
    print_depth(depth);
}

fn main() {
    const JUMP_SET: i32 = 0;

    register_signal_handler();

    let return_point = ptr_to_jump_buf();
    let rc = unsafe { setjmp(return_point) };
    if rc == JUMP_SET {
        dive(0, 10);
    } else {
        println!("early return!");
    }
}

```

Annotations on the right side of the code block:

- ⑧
- ⑦
- ⑨
- ⑩
- ⑪
- ⑫

```
println!("finishing!")
}
```

- ① This attribute prevents the code from compiling on MS Windows, which has different low-level internals. If you're using an MS Windows machine, then consider using the Docker route outlined at [Section 12.6.3](#).
- ② Our code only uses a single signal, `SIGUSR1`, but it's possible to write a signal handler that handles more than one. Bringing them into scope aids readability later on.
- ③ When this flag is `true`, the program will initiate its shut down code.
- ④ A buffer to hold a checkpoint of the program's current register state when `setjmp()` is called
- ⑤ Only allow a recursion depth of 3
- ⑥ An `#[inline]` attribute marks the function as being available for *inlining*, which is a compiler optimization technique for eliminating the cost of function calls.
- ⑦ Calling LLVM intrinsics requires an `unsafe` block. Rust can't make any guarantees about what happens on the other side of the function barrier.
- ⑧ This function only registers one of the many signals that have been pulled into local scope for brevity
- ⑨ Ask `libc` to associate the `handle_signals` function with the `SIGUSR1` signal.
- ⑩ `#[allow(dead_code)]` silences a warning from the Rust compiler. During normal operation, a signal handler would never be reached and so Rust marks it as unreachable.
- ⑪ Immediately re-register signal handlers minimizes the chances of a missing a signal while handling this one.
- ⑫ A recursive function that returns once `max_depth` (or `MOCK_SIGNAL_AT`) has been reached.

[45] `jmp_buf` is the conventional name for this buffer, which might be useful for any readers who wish to dive deeper themselves.

12.7 A NOTE ON APPLYING THESE TECHNIQUES TO PLATFORMS WITHOUT

SIGNALS

Signals are a UNIX-ism. In other platforms, messages from the operating system are handled differently.

On MS Windows, for example, command-line applications need to provide a handler function via `SetConsoleCtrlHandler` to the kernel. That handler function is then invoked from a when a “signal” is sent to the application.

Regardless of the specific mechanism, the high-level approach demonstrated in this chapter should be fairly portable.

Here is the pattern:

- Your CPU generates interrupts that it requires the operating system to respond to
- Operating systems often delegate responsibility for handling interrupts via some sort of callback system
- A callback system means creating function pointer

12.8 REVISING EXCEPTIONS

At the start of the chapter, we discussed the distinction between signals, interrupts and exceptions. There has been very little coverage of exceptions directly. We have treated them as a special class of interrupts. Interrupts themselves have been modelled as signals.

12.9 SUMMARY

On the Rust side, we have spent lots of time interacting with `libc` and `unsafe` blocks, unpacking function pointers, tweaking global variables and beginning to explore some of the features available in `rustc` and `LLVM`. The bulk of the chapter though has been utilising these features to work with signals. Within Linux, signals are the main mechanism that the operating system communicates with applications.

You have learned:

- the differences between traps, exceptions and interrupts are
- that hardware devices, such as the network, notify applications about data to be processed via interrupting the CPU
- what a signal is
- what the effects of delegating signal handling to the system's defaults are
- how to set up a signal handler
- how UNIX manages job control via `SIGSTOP` and `SIGCONT`
- what a function pointer is and how they can be defined and shared
- how to ignore signals
- how to escape the general function call/return procedure via `setjmp` and `longjmp`
- what LLVM is and how the Rust compiler uses it to build software